A Survey of Non-Euclidean Geometric Learning from Hyperbolic to Mixed-Curvature Spaces

Menglin Yang, Zihao Liu, Jiahong Liu, Jindong Li, Linxiao Cao, Buze Zhang, Irwin King
yangmenglin001@gmail.com

Abstract

Euclidean space has long served as the default representation space in machine learning. This choice rests on the implicit assumption that the geometry of the data itself is flat, yet many datasets carry hierarchies, cyclical and directional patterns, or a mixture of these structures. Non-Euclidean geometric learning addresses this limitation by introducing representation spaces whose curvature reflects the underlying structure of the data. In recent years, hyperbolic geometry has attracted substantial attention because its negative curvature and exponentially expanding volume align naturally with hierarchical and scale-free structures. At the same time, spherical geometry serves the complementary case, where positive curvature and a closed, compact structure suit cyclical, directional, and angular data. Mixed-curvature or generalized curvature models combine hyperbolic, spherical, and Euclidean components within a unified framework for representing heterogeneous geometric patterns. In this work, we present a comprehensive survey of geometric learning, from hyperbolic to mixed-curvature spaces. We review their mathematical foundations, geometric operators, neural architectures, learning strategies, and applications spanning natural language processing, computer vision, speech, multimodal learning, knowledge graphs, recommendation, robotics, the sciences, and trustworthiness, covering developments from shallow embeddings to convolutional, recurrent, and graph neural networks, Transformers, and generative models. We trace how geometry now reaches foundation-model pipelines through pretraining, adaptation, retrieval, and analysis. In addition, we examine practical considerations involving numerical stability, optimization cost, geometric model selection, and unified geometric frameworks. A continuously updated collection of papers and resources is maintained at https://github.com/marlin-codes/Awesome-Hyperbolic-Representation-and-Deep-Learning/.

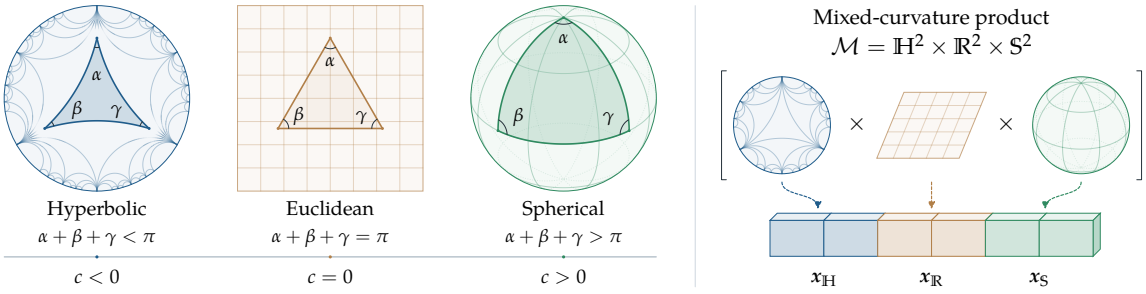


Figure 1 Constant-curvature geometries and their mixed-curvature product. **Left**: Geodesic triangles in hyperbolic ($c < 0$), Euclidean ($c = 0$), and spherical ($c > 0$) geometry, whose interior angles sum to less than, equal to, and greater than π, respectively. The hyperbolic triangle is drawn in the Poincaré disk over part of an ideal-triangle tessellation, and the spherical triangle is an octant of the sphere. **Right**: The product manifold $\mathcal{M} = \mathbb{H}^2 \times \mathbb{R}^2 \times \mathbb{S}^2$ combines three two-dimensional factors (Section 2.2.3). A representation in $\mathcal{M}$ concatenates the components $x_{\mathbb{H}}$, $x_{\mathbb{R}}$, and $x_{\mathbb{S}}$, each occupying two cells that correspond to the intrinsic dimensions of its factor. Euclidean space is the default representation space for most models and algorithms, yet it has clear limitations in modeling complex structures. Incorporating non-Euclidean geometry aligns representations with the underlying structure of the data, yielding more expressive embeddings, stronger downstream performance, and improved interpretability.

Contents

1 Introduction **3**

2 Preliminaries **5**
2.1 Mathematical Preliminaries . . . 5
2.2 Geometric Models . . . 8
2.3 Neural Modules . . . 15
2.4 Metrics . . . 23

3 From Hyperbolic to Mixed-Curvature Geometric Learning **28**
3.1 Shallow Model . . . 28
3.2 Neural Network . . . 31
3.3 Convolutional Neural Network . . . 33
3.4 Recurrent Neural Network . . . 35
3.5 Graph Neural Network . . . 35
3.6 Transformer . . . 42
3.7 Generative Model . . . 47
3.8 Foundation Model . . . 51
3.9 Learning Paradigms . . . 58

4 Applications **66**
4.1 Natural Language Processing . . . 66
4.2 Computer Vision . . . 69
4.3 Speech and Audio Processing . . . 73
4.4 Multimodal Learning . . . 74
4.5 Knowledge Graph Completion . . . 77
4.6 Recommender Systems . . . 80
4.7 Robotics . . . 84
4.8 AI for Science . . . 86
4.9 Trustworthiness . . . 88
4.10 Tools and Resources . . . 90

5 Challenges and Discussion **91**
5.1 Numerical Instability . . . 92
5.2 Optimization Cost . . . 92
5.3 Comparing Geometric Models . . . 93
5.4 Geometric Model Selection . . . 93
5.5 Unified Geometric Framework . . . 93

6 Conclusion **94**

A Technical constructions **112**
A.1 Hyperbolic batch normalization . . . 112
A.2 MoSLoRA coordinate construction . . . 113

1 Introduction

From classical machine learning [19, 234, 264] to recent foundation models [21, 91, 97, 263], Euclidean space has served as the de facto representation space. This choice carries an implicit assumption: representations are typically learned in Euclidean vector spaces and compared using inner-product similarities or Euclidean distances. However, real-world data often contain complex structures that are difficult to preserve in low-dimensional Euclidean representations. For example, **hierarchies** organize taxonomies [81, 203], complex networks [3, 20, 52, 139, 211], the semantic relations that tie an image to the categories and captions describing it [64, 66, 209], and user–item interactions [48, 256, 282]. **Cyclic and directional patterns** appear in molecular conformations [84, 178] and in the spherical domain of 360° imagery [55, 255, 296]. **Mixed structure** appears in knowledge graphs such as WordNet and Freebase, which place taxonomic relations that branch hierarchically alongside symmetric relations that close into cycles [284]. Such structures arise across natural language [46, 106, 266], computer vision [6, 10, 129], neuroscience [31, 77, 179, 246], and computational biology [128, 135], among others. Non-Euclidean geometry therefore provides an alternative foundation for data whose organization departs from flat geometry [13, 28, 29].

Hyperbolic geometry provides a negative-curvature setting suited to hierarchical data. Its volume grows exponentially with radius, matching the branching growth of trees. Euclidean embeddings of tree metrics can require distortion that grows with the size and structure of the tree [25], while low-dimensional hyperbolic constructions preserve these distances with substantially lower distortion [235, 238, 305]. Poincaré and Lorentz-model embeddings established this connection for taxonomies [203, 205]. Later work developed fully hyperbolic neural networks [45], Lorentz-model operations [144, 227], residual architectures [110], graph convolution [39], and efficient attention in *Hypformer* [307]. These architectures have improved recommendation [256, 301, 302, 310], vision and vision–language modeling [10, 64, 129], molecular and drug representation [317, 318], and the fine-tuning of large language models [186, 308].

Spherical geometry provides a complementary positive-curvature setting. Its compact, closed structure suits cyclical, directional, and angular data. Representative uses include normalized embeddings on the unit hypersphere [146, 180, 279, 286], spatial rotations [32, 167], three-dimensional point clouds [150], and molecular graphs [178]. Across these studies, representation quality improves when the curvature of the space matches the intrinsic structure of the data.

Mixed-curvature geometry provides a setting for data whose structure varies within a single dataset. Such data combine substructures that favor different curvatures. Many graphs are tree-like in some regions and cycle-rich in others, so one constant curvature fits part of the graph closely and the remainder less so [51, 257, 287]. In knowledge graphs, relation types can favor different geometries, and models that assign a curvature per relation type capture more of them [36, 121, 284]. Molecular datasets often combine directional three-dimensional interactions, in which bond angles and torsions carry angular structure [84, 178], with hierarchical relations among drugs, such as the Anatomical Therapeutic Chemical (ATC) classification [318]. A space that carries several curvatures at once can therefore match more of this structure than any single constant curvature, as Figure 1 illustrates. Models realize this by combining constant-curvature components into one space [94, 250] and, more recently, by treating curvature as a learnable parameter rather than fixing it in advance [13, 93, 102].

This geometric perspective extends from task-specific models throughout foundation-model pipelines, including pretraining, adaptation, retrieval, multimodal alignment, and analysis. **Pretraining** places the backbone itself in curved space, through hyperbolic language representations [46], mixture-of-curvature LLM backbones in *HELM* [106], hyperspherical Transformers [180], and hierarchy-encoding mRNA language models [274]. **Adaptation** introduces curvature into a model that is already pre-

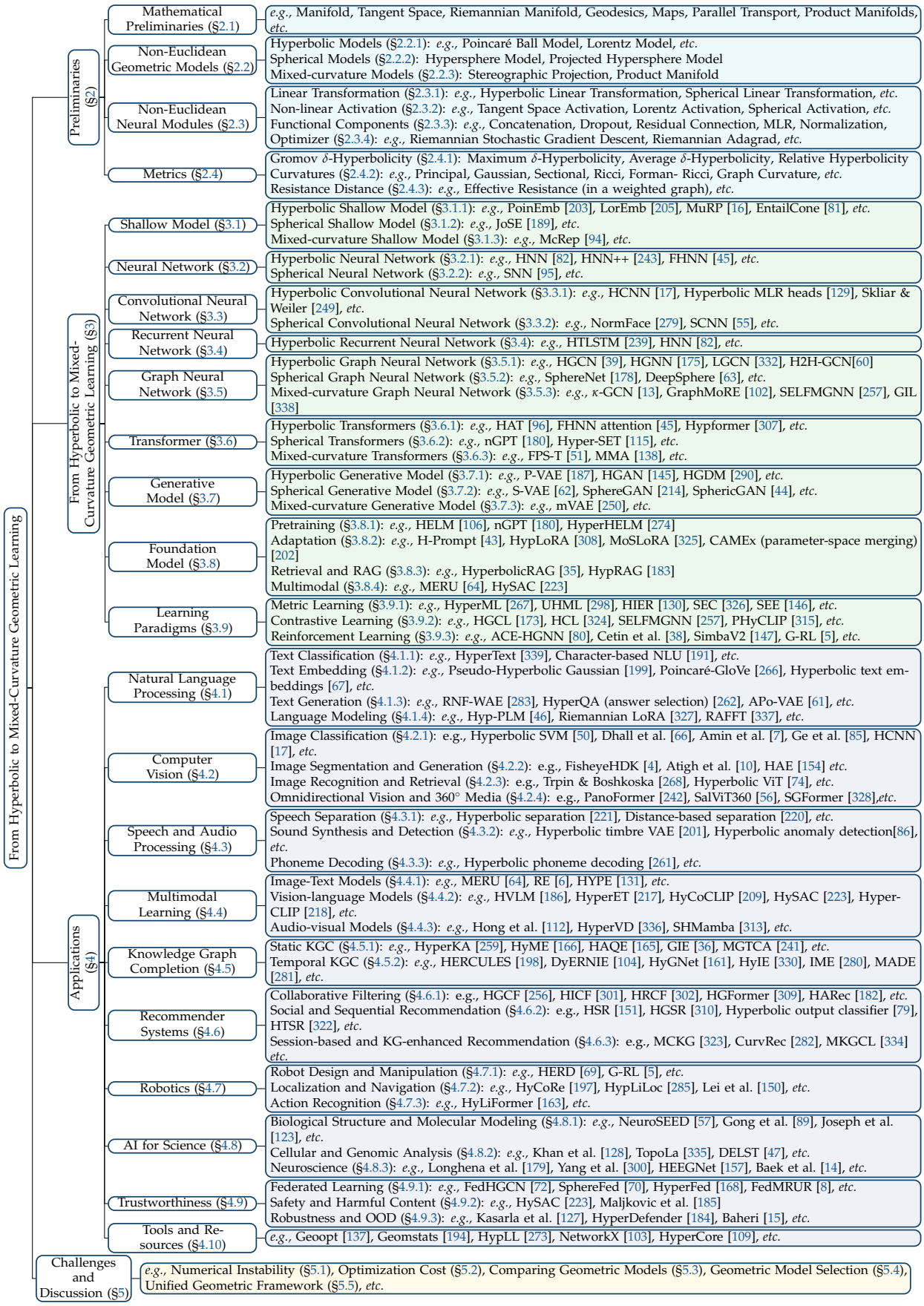


Figure 2 Structure of this survey with representative works.

trained, through hyperbolic prompt representations [43, 289], Riemannian-preconditioned and fully hyperbolic LoRA [308, 327], federated fine-tuning with dynamic rank [337], and mixture-of-space experts [325]. **Retrieval** applies geometry outside the model, embedding the index in hyperbolic space to use hierarchical relations when selecting evidence [35, 183]. **Multimodal alignment** uses curved shared representations and, in entailment-based methods, expresses semantic inclusion geometrically. Examples include hyperbolic multimodal LLMs [186, 217], compositional vision–language models [209], and safety-aware vision–language models [223]. **Analysis** complements these by measuring the geometry that Euclidean models already carry, probing hierarchy in contextual embeddings [42, 54] and in image–text representations [6]. Libraries including Geoopt [137], Geomstats [194], HypLL [273], Hyperlib [117], and HyperCore [109] make these constructions practical.

Research on geometric representation learning now spans machine learning, graph mining, computer vision, natural language processing, and the sciences. Existing surveys focus on hyperbolic neural architectures [215], hyperbolic graph representation learning [304], computer vision [76, 190], knowledge graph embedding [34, 170], and foundation models [108]. Table 1 organizes these surveys by geometric, architectural, and application focus. We connect these perspectives through three recurring questions: what structure a geometry represents, which model operations use it, and what benefits and costs those operations introduce. The mathematical preliminaries explain the available spaces and operators. The method comparisons show how architectures use them, and the applications connect those choices to learning tasks. This organization gives newcomers a starting point and helps practitioners compare techniques for a concrete problem.

Structure of the Survey. Figure 2 provides a reading map. Section 2 introduces the geometry, neural operators, and structural diagnostics used throughout the survey. Section 3 compares model families and learning objectives, with attention to representation spaces, transformations, and aggregation. Section 4 connects these designs to tasks and closes with tools and resources. Section 5 discusses the numerical, computational, and model-selection questions that follow from these comparisons.

2 Preliminaries

Geometry enters a learning model through the distances between representations, the operations on features, and the constraints on trainable parameters. We introduce these three ingredients before discussing diagnostics for choosing a representation space. Table 2 summarizes the notation.

2.1 Mathematical Preliminaries

We summarize the mathematical concepts used throughout the survey. Standard differential-geometry texts provide more detailed treatments [68, 148, 149].

Manifold. An n-dimensional manifold $\mathcal{M}$ locally has coordinates in $\mathbb{R}^n$: each point $\boldsymbol{p} \in \mathcal{M}$ has a neighborhood U and a homeomorphism $\varphi : U \to V$ onto an open subset $V \subset \mathbb{R}^n$. We assume the usual Hausdorff and second-countability conditions. A smooth manifold additionally has smoothly compatible coordinate charts, which allow differentiation on $\mathcal{M}$.

Tangent Space. The tangent space $\mathcal{T}_p\mathcal{M}$ consists of the velocities of smooth curves passing through $\boldsymbol{p}$. It is an n-dimensional vector space and gives a first-order approximation of $\mathcal{M}$ near $\boldsymbol{p}$. Neural layers can use linear operations in this space and then map their outputs back to the manifold, as illustrated in Figure 3.

Riemannian Manifold. A Riemannian manifold $(\mathcal{M}, g)$ is a smooth manifold equipped with a smoothly varying positive-definite symmetric inner product $g_p : \mathcal{T}_p\mathcal{M} \times \mathcal{T}_p\mathcal{M} \to \mathbb{R}$. The metric measures

Table 1 Overview of relevant surveys

Survey	Geometry	Model focus	Application focus	Foundation models
Lin et al. (2021) [170]	Hyperbolic	Hyperbolic knowledge graph embedding models	Knowledge graph completion	NO
Peng et al. (2022) [215]	Hyperbolic	Hyperbolic operators, shallow embeddings, and neural networks	Graphs, vision, and natural language processing	NO
Yang et al. (2022) [304]	Hyperbolic	Graph embeddings and HGNNs	Graph learning, knowledge graphs, recommendation, and bioinformatics	LIMITED
Fang et al. (2023) [76]	Hyperbolic	Hyperbolic operators and vision backbones	Computer vision	NO
Xiong et al. (2023) [294]	Vector, region-based, and manifold spaces	Geometric embeddings for relational data	Knowledge graphs, ontologies, and hierarchies	NO
Cao et al. (2024) [34]	Euclidean, complex, hyperbolic, and spherical	Knowledge graph embedding models organized by representation space	Knowledge graph completion and downstream reasoning	NO
Mettes et al. (2024) [190]	Hyperbolic	Supervised and unsupervised vision models	Computer vision	NO
He et al. (2025) [108]	Hyperbolic	Building blocks and foundation models	Language, vision, and multimodal learning	YES
This work	Hyperbolic, spherical, and mixed-curvature	Shallow embeddings, CNNs, RNNs, GNNs, Transformers, generative models, and foundation models, with metric, contrastive, and reinforcement learning	NLP, vision, speech, multimodal learning, knowledge graph completion, recommendation, robotics, AI for science, and trustworthiness, including federated learning, safety, robustness, and OOD detection	YES

tangent-vector lengths and angles; integrating these lengths along curves defines distance on the manifold.

Geodesics and Distance. A geodesic on $(\mathcal{M}, g)$ is a smooth curve γ satisfying the geodesic equation $\nabla_{\dot\gamma}\dot\gamma = 0$. A minimizing geodesic is a geodesic that realizes the distance between two points on the relevant interval. The distance between $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{M}$ is given by $d_{\mathcal{M}}(\boldsymbol{x}, \boldsymbol{y}) = \inf_\gamma L(\gamma)$, where the infimum is over piecewise smooth curves satisfying $\gamma(\alpha) = \boldsymbol{x}$ and $\gamma(\beta) = \boldsymbol{y}$. For any initial point $\boldsymbol{p} \in \mathcal{M}$ and initial velocity $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{p}}\mathcal{M}$, there exists a unique maximal geodesic determined by these initial conditions.

Maps and Parallel Transport. The exponential map $\exp_{\boldsymbol{p}}$ sends a tangent vector $\boldsymbol{v}$ to $\gamma(1)$, where $\gamma(0) = \boldsymbol{p}$ and $\gamma'(0) = \boldsymbol{v}$. Its domain consists of vectors whose geodesics exist up to time 1; on a geodesically complete manifold, this is the entire tangent space $\mathcal{T}_{\boldsymbol{p}}\mathcal{M}$. The logarithmic map $\log_{\boldsymbol{p}}$ is the local inverse of $\exp_{\boldsymbol{p}}$ on a normal neighborhood of $\boldsymbol{p}$, where the minimizing geodesic is unique. The relationships between the exponential map, logarithmic map, and tangent space are illustrated in Figure 3a. Parallel transport $\mathrm{PT}_\gamma : \mathcal{T}_{\boldsymbol{x}}\mathcal{M} \to \mathcal{T}_{\boldsymbol{y}}\mathcal{M}$ carries tangent vectors along a curve γ while preserving their inner products. It generally depends on the path. Our notation $\mathrm{PT}_{\boldsymbol{x}\to\boldsymbol{y}}$ uses the unique minimizing geodesic when one exists, as in Figure 3b.

Table 2 Summary of notation used throughout this survey.

Notation	Description
Manifolds and Tangent Spaces	
$\mathcal{M}$	n-dimensional smooth manifold.
$\boldsymbol{p}, \boldsymbol{x}, \boldsymbol{y}$	Points on the manifold $\mathcal{M}$.
$\mathcal{T}_{\boldsymbol{p}}\mathcal{M}$	Tangent space of $\mathcal{M}$ at $\boldsymbol{p}$.
$g_{\boldsymbol{x}}$	Riemannian metric at $\boldsymbol{x}$.
$d_{\mathcal{M}}(\boldsymbol{x}, \boldsymbol{y})$	Geodesic distance between $\boldsymbol{x}$ and $\boldsymbol{y}$ on $\mathcal{M}$.
γ	Geodesic or smooth curve.
$\exp_{\boldsymbol{x}}^{c}(\cdot)$	Exponential map at $\boldsymbol{x}$.
$\log_{\boldsymbol{x}}^{c}(\cdot)$	Logarithmic map at $\boldsymbol{x}$.
$\mathrm{PT}_{\boldsymbol{x}\to\boldsymbol{y}}^{c}$	Parallel transport from $\mathcal{T}_{\boldsymbol{x}}\mathcal{M}$ to $\mathcal{T}_{\boldsymbol{y}}\mathcal{M}$.
$\mathcal{M}'$	Product manifold with factorwise metric.
Specific Non-Euclidean Models	
c	Signed sectional curvature: $c<0$ hyperbolic, $c=0$ Euclidean, and $c>0$ spherical.
$\mathbb{P}_c^n$	n-dimensional Poincaré ball model.
$\mathbb{L}_c^n$	n-dimensional Lorentz model.
$\boldsymbol{o}_c$	Canonical Lorentz basepoint $(1/\sqrt{-c}, 0, \ldots, 0)^T$ for $c<0$.
$x_t, \boldsymbol{x}_s$	First coordinate $x_t = x_1$ (Lorentz time or sphere polar) and remaining vector $(x_2, \ldots, x_{n+1})^T$.
$\mathbb{S}_c^n$	n-dimensional hypersphere model.
$\mathbb{S}_c'^n$	Projected hypersphere model.
$\Pi_{\cdot\to\cdot}^{c}$	Coordinate map between geometric models, stereographic where applicable.
Core Curvature Quantities	
κ_1, κ_2	Principal curvatures.
K	Gaussian curvature.
H	Mean curvature.

Notation	Description
Algebraic Operations and Norms	
$\langle\cdot,\cdot\rangle$	Euclidean inner product.
$\|\cdot\|$	Euclidean norm.
$\langle\cdot,\cdot\rangle_{\mathcal{L}}$	Ambient Lorentz bilinear form $\boldsymbol{x}^T J_{\mathcal{L}} \boldsymbol{y}$.
$J_{\mathcal{L}}$	Ambient signature matrix $\mathrm{diag}(-1, 1, \ldots, 1)$.
$\|\cdot\|_{\mathcal{L}}$	Positive-definite norm on Lorentz tangent spaces.
$\|\cdot\|_{\mathcal{L}}$	Timelike ambient magnitude, $\sqrt{-\langle\cdot,\cdot\rangle_{\mathcal{L}}}$.
$\lambda_{\boldsymbol{x}}^{c}$	Conformal factor in the Poincaré or projected-spherical stereographic chart.
$\oplus_c$	Möbius addition in a gyrovector space.
$\otimes_c$	Möbius scalar or matrix multiplication.
$\mathrm{gyr}[\cdot,\cdot]\boldsymbol{v}$	Gyration operator applied to $\boldsymbol{v}$.
Local and Graph Metrics	
$K_{\boldsymbol{x}}(\boldsymbol{v}, \boldsymbol{w})$	Sectional curvature at $\boldsymbol{x}$ in the plane spanned by $\boldsymbol{v}$ and $\boldsymbol{w}$.
$\mathrm{Ric}_{\boldsymbol{x}}(\boldsymbol{v}, \boldsymbol{v})$	Ricci curvature at $\boldsymbol{x}$ along tangent direction $\boldsymbol{v}$.
$\delta_{\mathrm{4pt}}, \delta_{\mathrm{slim}}$	Four-point and slim-triangle hyperbolicity constants.
$\delta_{\max}, \delta_{\mathrm{avg}}$	Maximum and average four-point hyperbolicity summaries.
$W_1(\cdot,\cdot)$	Wasserstein distance.
ξ_G	Graph sectional curvature.
L	Graph Laplacian matrix.
L^W	Weighted Laplacian matrix.
Ω_{ij}	Resistance distance between nodes i and j.

Curvature and Sectional Curvature. Curvature measures departure from flat geometry [68, 149]. For a plane curve, the curvature magnitude is $1/R$, where R is the radius of its osculating circle. For a Riemannian manifold, intrinsic curvature is determined by the metric and its derivatives [200]. Sectional curvature assigns a value to each two-dimensional tangent subspace $\sigma_p \subset \mathcal{T}_p\mathcal{M}$. Positive and negative values describe different local geodesic-separation behavior, which we make precise in Section 2.4.2.

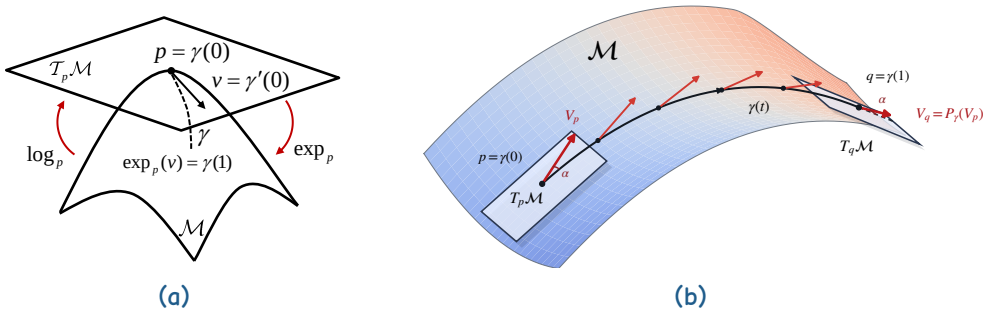


Figure 3 Key operations on a Riemannian manifold $\mathcal{M}$, where $T_z\mathcal{M}$ denotes the tangent space at $z \in \mathcal{M}$ and $\dot{\gamma} = \gamma'$ denotes the geodesic velocity. (a) Given $p \in \mathcal{M}$ and $v \in T_p\mathcal{M}$, the exponential map follows the geodesic γ satisfying $\gamma(0) = p$ and $\dot{\gamma}(0) = v$ to $q = \exp_p(v) = \gamma(1)$; conversely, $\log_p(q) = v$. (b) The black curve is the geodesic from $p = \gamma(0)$ to $q = \gamma(1)$. A vector $V_p \in T_p\mathcal{M}$ is carried to $V_q = \mathrm{PT}_\gamma(V_p) \in T_q\mathcal{M}$, where $\mathrm{PT}_\gamma : T_p\mathcal{M} \to T_q\mathcal{M}$ is the parallel-transport map. The red arrows sample the transported vector field $V(t)$, which satisfies $\nabla_{\dot{\gamma}} V = 0$. The dashed black segment shows the terminal tangent direction $\dot{\gamma}(1)$; parallel transport preserves both $\|V(t)\|$ and the marked angle $\alpha = \angle(V(t), \dot{\gamma}(t))$.

Product Manifolds. A product manifold $\mathcal{M}'$ of dimension $n_1 + n_2 + \ldots + n_k$ is constructed as the Cartesian product $\times_{i=1}^{k} \mathcal{M}_i = \mathcal{M}_1 \times \mathcal{M}_2 \times \ldots \times \mathcal{M}_k$ of multiple smooth manifolds $\mathcal{M}_i$ with dimensions n_i. A point $\boldsymbol{p} \in \mathcal{M}'$ is expressed as $\boldsymbol{p} = (\boldsymbol{p}_1, \boldsymbol{p}_2, \ldots, \boldsymbol{p}_k)$, where $\boldsymbol{p}_i \in \mathcal{M}_i$. A tangent vector $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{p}}\mathcal{M}'$ decomposes as $\boldsymbol{v} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k)$, where $\boldsymbol{v}_i \in \mathcal{T}_{\boldsymbol{p}_i}\mathcal{M}_i$. With the Riemannian product metric, the squared distance between points $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{M}'$ is $d^2_{\mathcal{M}'}(\boldsymbol{x}, \boldsymbol{y}) = \sum_{i=1}^{k} d^2_{\mathcal{M}_i}(\boldsymbol{x}_i, \boldsymbol{y}_i)$, where $\boldsymbol{x}_i$ and $\boldsymbol{y}_i$ denote the vectors on $\mathcal{M}_i$.

2.2 Geometric Models

We introduce hyperbolic, spherical, and mixed-curvature models together with their metrics, distances, exponential and logarithmic maps, and parallel transport. We use the notation $\|\cdot\|$ to denote the Euclidean norm, $\langle \cdot, \cdot \rangle$ to denote the Euclidean inner product, and $\langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathcal{L}} = -x_1 y_1 + \sum_{i=2}^{n+1} x_i y_i = \boldsymbol{x}^T J_{\mathcal{L}} \boldsymbol{y}$ for the ambient Lorentz (or Minkowski) bilinear form, where $J_{\mathcal{L}} = \mathrm{diag}(-1, 1, \ldots, 1)$. We write $x_t = x_1$ for the time coordinate and $\boldsymbol{x}_s = (x_2, \ldots, x_{n+1})^T$ for the spatial coordinates. For a Lorentz tangent vector, $\|\boldsymbol{v}\|_{\mathcal{L}} := \sqrt{\langle \boldsymbol{v}, \boldsymbol{v} \rangle_{\mathcal{L}}}$ is real, since every nonzero tangent vector is spacelike. For a timelike ambient vector, we use $|\boldsymbol{z}|_{\mathcal{L}} := \sqrt{-\langle \boldsymbol{z}, \boldsymbol{z} \rangle_{\mathcal{L}}}$. More generally, the Riemannian norm is $\|\boldsymbol{u}\|_{\boldsymbol{x}} := \sqrt{g_{\boldsymbol{x}}(\boldsymbol{u}, \boldsymbol{u})}$. Throughout this section, $c \in \mathbb{R}$ denotes signed sectional curvature, with $c = 0$ for Euclidean geometry.

2.2.1 Hyperbolic Models

Hyperbolic geometry is a Riemannian geometry with constant negative sectional curvature $c < 0$. Hyperbolic geometry can be represented through several analytic models. We focus on the Lorentz (or hyperboloid) model and the Poincaré ball model, as these are the most widely used models in hyperbolic machine learning.

Poincaré Ball Model. The n-dimensional Poincaré ball model with negative curvature c ($c < 0$) is defined as a Riemannian manifold $\mathbb{P}^n_c = (\mathcal{P}^n_c, g^{\mathcal{P}}_{\boldsymbol{x}})$, where $\mathcal{P}^n_c = \{\boldsymbol{x} \in \mathbb{R}^n : -c\|\boldsymbol{x}\|^2 < 1\}$ is an open ball with radius $1/\sqrt{-c}$. We use the standard stereographic coordinate convention throughout the survey. The metric $g^{\mathcal{P}}_{\boldsymbol{x}} = (\lambda^c_{\boldsymbol{x}})^2 g^E$ is conformal to $g^E = \mathbb{I}_n$ with

$$\lambda^c_{\boldsymbol{x}} = \frac{2}{1 + c\|\boldsymbol{x}\|^2}.$$

The conformal factor λ_x^c induces the inner product $\langle \boldsymbol{u}, \boldsymbol{v} \rangle_x^c = (\lambda_x^c)^2 \langle \boldsymbol{u}, \boldsymbol{v} \rangle$ and norm $\|\boldsymbol{v}\|_x^c = \lambda_x^c \|\boldsymbol{v}\|$ for $\boldsymbol{u}, \boldsymbol{v} \in \mathcal{T}_x \mathbb{P}_c^n$, both of which change with $\boldsymbol{x}$.

For any two points $\boldsymbol{x}$, $\boldsymbol{y} \in \mathbb{P}_c^n$, the geodesic distance between $\boldsymbol{x}$ and $\boldsymbol{y}$ is:

$$d_{\mathbb{P}}^c(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{-c}} \operatorname{arcosh}\left(1 - \frac{2c\|\boldsymbol{x} - \boldsymbol{y}\|^2}{(1 + c\|\boldsymbol{x}\|^2)(1 + c\|\boldsymbol{y}\|^2)}\right). \tag{1}$$

To support the vectorial structure operations in hyperbolic space, the framework of gyrovector spaces is introduced [82, 269]. The framework of gyrovector spaces provides an algebraic formalism for studying hyperbolic geometry, just as vector spaces serve as the algebraic setting for Euclidean geometry [270]. In the Möbius gyrovector space, the Möbius addition $\oplus_c : \mathbb{P}_c^n \times \mathbb{P}_c^n \to \mathbb{P}_c^n$ is defined as:

$$\boldsymbol{x} \oplus_c \boldsymbol{y} = \frac{(1 - 2c\langle \boldsymbol{x}, \boldsymbol{y} \rangle - c\|\boldsymbol{y}\|^2)\boldsymbol{x} + (1 + c\|\boldsymbol{x}\|^2)\boldsymbol{y}}{1 - 2c\langle \boldsymbol{x}, \boldsymbol{y} \rangle + c^2\|\boldsymbol{x}\|^2\|\boldsymbol{y}\|^2}. \tag{2}$$

This operation is generally noncommutative. The Möbius subtraction is defined using the following notation: $\boldsymbol{x} \ominus_c \boldsymbol{y} := \boldsymbol{x} \oplus_c (-\boldsymbol{y})$. The Möbius scalar multiplication of $\boldsymbol{x} \in \mathbb{P}_c^n \setminus \{\mathbf{0}\}$ by $r \in \mathbb{R}$ is defined as

$$r \otimes_c \boldsymbol{x} = \frac{1}{\sqrt{-c}} \tanh\left(r \operatorname{artanh}(\sqrt{-c}\|\boldsymbol{x}\|)\right) \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|}, \tag{3}$$

and $r \otimes_c \mathbf{0} = \mathbf{0}$. The Möbius scalar multiplication recovers the Euclidean scalar multiplication when curvature c goes to zero: $\lim_{c \to 0} r \otimes_c \boldsymbol{x} = r\boldsymbol{x}$. For a matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{x} \in \mathbb{P}_c^n$, the Möbius matrix multiplication $\mathbf{M} \otimes_c \boldsymbol{x} \in \mathbb{P}_c^m$ is

$$\mathbf{M} \otimes_c \boldsymbol{x} = \begin{cases} \frac{1}{\sqrt{-c}} \tanh\left(\frac{\|\mathbf{M}\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \operatorname{artanh}(\sqrt{-c}\|\boldsymbol{x}\|)\right) \frac{\mathbf{M}\boldsymbol{x}}{\|\mathbf{M}\boldsymbol{x}\|}, & \boldsymbol{x} \neq \mathbf{0},\ \mathbf{M}\boldsymbol{x} \neq \mathbf{0}, \\ \mathbf{0}, & \boldsymbol{x} = \mathbf{0} \text{ or } \mathbf{M}\boldsymbol{x} = \mathbf{0}. \end{cases} \tag{4}$$

Based on the Möbius addition, the distance of $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{P}_c^n$ is formulated as:

$$d_{\mathbb{P}}^c(\boldsymbol{x}, \boldsymbol{y}) = \frac{2}{\sqrt{-c}} \operatorname{artanh}(\sqrt{-c}\| - \boldsymbol{x} \oplus_c \boldsymbol{y}\|). \tag{5}$$

The two distance formulas in Equations (1) and (5) are equivalent [250]. Under this stereographic coordinate convention, $d_{\mathbb{P}}^c(\boldsymbol{x}, \boldsymbol{y}) \xrightarrow{c \to 0^-} 2\|\boldsymbol{x} - \boldsymbol{y}\|$, and $g_{\boldsymbol{x}}^{\mathcal{P}} \to 4g^E$. This fixed coordinate scaling is useful when matching hyperbolic, Euclidean, and spherical factors in a common parameterization.

To apply Euclidean neural operations in hyperbolic geometry, the exponential and logarithmic maps are used to move between hyperbolic space and its Euclidean tangent space. As derived in studies [82], the exponential map and logarithm map at $\mathbb{P}_c^n$ based on Möbius addition are:

$$\exp_{\boldsymbol{x}}^c(\boldsymbol{v}) = \boldsymbol{x} \oplus_c \left(\tanh\left(\sqrt{-c}\frac{\lambda_{\boldsymbol{x}}^c\|\boldsymbol{v}\|}{2}\right)\frac{\boldsymbol{v}}{\sqrt{-c}\|\boldsymbol{v}\|}\right), \tag{6}$$

$$\log_{\boldsymbol{x}}^c(\boldsymbol{y}) = \frac{2}{\sqrt{-c}\lambda_{\boldsymbol{x}}^c} \operatorname{artanh}\left(\sqrt{-c}\| - \boldsymbol{x} \oplus_c \boldsymbol{y}\|\right)\frac{-\boldsymbol{x} \oplus_c \boldsymbol{y}}{\| - \boldsymbol{x} \oplus_c \boldsymbol{y}\|}. \tag{7}$$

The continuous extensions $\exp_{\boldsymbol{x}}^c(\mathbf{0}) = \boldsymbol{x}$ and $\log_{\boldsymbol{x}}^c(\boldsymbol{x}) = \mathbf{0}$ are understood whenever the displayed expressions contain a removable zero denominator.

The gyration operation $\operatorname{gyr}[\boldsymbol{x}, \boldsymbol{y}]\boldsymbol{v}$ gives the following closed form for parallel transport $\mathrm{PT}_{\boldsymbol{x} \to \boldsymbol{y}}^c$:

$\mathcal{T}_{\boldsymbol{x}}\mathbb{P}_c^n \to \mathcal{T}_{\boldsymbol{y}}\mathbb{P}_c^n$ of $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{P}_c^n$:

$$\mathrm{PT}^c_{\boldsymbol{x}\to\boldsymbol{y}}(\boldsymbol{v}) = \frac{\lambda_{\boldsymbol{x}}^c}{\lambda_{\boldsymbol{y}}^c}\,\mathrm{gyr}[\boldsymbol{y}, -\boldsymbol{x}]\boldsymbol{v}, \tag{8}$$

where $\mathrm{gyr}[\boldsymbol{x}, \boldsymbol{y}]\boldsymbol{z} = -(\boldsymbol{x} \oplus_c \boldsymbol{y}) \oplus_c (\boldsymbol{x} \oplus_c (\boldsymbol{y} \oplus_c \boldsymbol{z}))$ for $\boldsymbol{z}$ in the ball. Gyration extends linearly to the full tangent-coordinate space, which defines its action on arbitrary tangent vectors $\boldsymbol{v}$.

Lorentz Model. The n-dimensional Lorentz model with curvature $c < 0$ is $\mathbb{L}_c^n = (\mathcal{L}_c^n, g^{\mathcal{L}})$, where $\mathcal{L}_c^n = \{\boldsymbol{x} \in \mathbb{R}^{n+1} : c\langle \boldsymbol{x}, \boldsymbol{x}\rangle_{\mathcal{L}} = 1,\ x_t > 0\}$ is the upper sheet of a hyperboloid. Its Riemannian metric is the restriction of the ambient bilinear form to tangent vectors: $g^{\mathcal{L}}_{\boldsymbol{x}}(\boldsymbol{u}, \boldsymbol{v}) = \langle \boldsymbol{u}, \boldsymbol{v}\rangle_{\mathcal{L}}$ for $\boldsymbol{u}, \boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{L}_c^n$. This restriction is positive definite, although the ambient matrix $J_{\mathcal{L}}$ is indefinite. Writing $\boldsymbol{x} = [x_t, \boldsymbol{x}_s]^T$, the constraint gives $x_t = \sqrt{\|\boldsymbol{x}_s\|^2 - 1/c}$. This identity lets neural layers construct a valid Lorentz point from any spatial output [17, 45].

For any two points $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{L}_c^n$, $\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}} = 1/c$ if and only if $\boldsymbol{x} = \boldsymbol{y}$; otherwise, $\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}} < 1/c$ [144]. For the Lorentz model, the tangent space $\mathcal{T}_{\boldsymbol{x}}\mathbb{L}_c^n$ of a point $\boldsymbol{x} \in \mathbb{L}_c^n$ consists of all vectors orthogonal to $\boldsymbol{x}$:

$$\mathcal{T}_{\boldsymbol{x}}\mathbb{L}_c^n = \{\boldsymbol{v} \in \mathbb{R}^{n+1} : \langle \boldsymbol{v}, \boldsymbol{x}\rangle_{\mathcal{L}} = 0\}. \tag{9}$$

For any two points $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{L}_c^n$, the geodesic distance between $\boldsymbol{x}$ and $\boldsymbol{y}$ can be computed as:

$$d^c_{\mathbb{L}}(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{-c}}\,\mathrm{arcosh}(c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}})\,. \tag{10}$$

The ambient Lorentz chord surrogate is instead defined by [144, 231]:

$$\widetilde{d}^2_{\mathbb{L},c}(\boldsymbol{x}, \boldsymbol{y}) := \frac{2}{c} - 2\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}}. \tag{11}$$

It equals $\frac{2}{-c}\big(\cosh(\sqrt{-c}\, d^c_{\mathbb{L}}(\boldsymbol{x}, \boldsymbol{y})) - 1\big)$. Thus, the chord surrogate preserves geodesic-distance rankings, while its different scale changes losses and attention weights.

For a tangent vector $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{L}_c^n$ such that $\boldsymbol{v} \neq 0$, the exponential map and logarithmic map of the Lorentz model are defined as follows [39, 205]:

$$\exp^c_{\boldsymbol{x}}(\boldsymbol{v}) = \cosh\left(\sqrt{-c}\|\boldsymbol{v}\|_{\mathcal{L}}\right)\boldsymbol{x} + \frac{1}{\sqrt{-c}}\sinh\left(\sqrt{-c}\|\boldsymbol{v}\|_{\mathcal{L}}\right)\frac{\boldsymbol{v}}{\|\boldsymbol{v}\|_{\mathcal{L}}}, \tag{12}$$

$$\log^c_{\boldsymbol{x}}(\boldsymbol{y}) = d^c_{\mathbb{L}}(\boldsymbol{x}, \boldsymbol{y})\frac{\boldsymbol{y} - c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}}\boldsymbol{x}}{\|\boldsymbol{y} - c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}}\boldsymbol{x}\|_{\mathcal{L}}}. \tag{13}$$

Here $\exp^c_{\boldsymbol{x}}(\boldsymbol{0}) = \boldsymbol{x}$ and $\log^c_{\boldsymbol{x}}(\boldsymbol{x}) = \boldsymbol{0}$ are defined by continuity.

The Lorentz model admits the following expression for transporting $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{L}_c^n$ to $\mathcal{T}_{\boldsymbol{y}}\mathbb{L}_c^n$ [250]:

$$\mathrm{PT}^c_{\boldsymbol{x}\to\boldsymbol{y}}(\boldsymbol{v}) = \boldsymbol{v} - \frac{c\langle \boldsymbol{y}, \boldsymbol{v}\rangle_{\mathcal{L}}}{1 + c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}}}(\boldsymbol{x} + \boldsymbol{y}). \tag{14}$$

Connection between the hyperbolic geometry models. The Poincaré ball and Lorentz models are isometric representations of the same hyperbolic space, connected by stereographic projection [39, 250]. A point in $\mathbb{L}_c^n$ is projected to $\mathbb{P}_c^n$ by

$$\Pi^c_{\mathbb{L}\to\mathbb{P}}(\boldsymbol{x}) = \frac{\boldsymbol{x}_s}{\sqrt{-c}\, x_t + 1}. \tag{15}$$

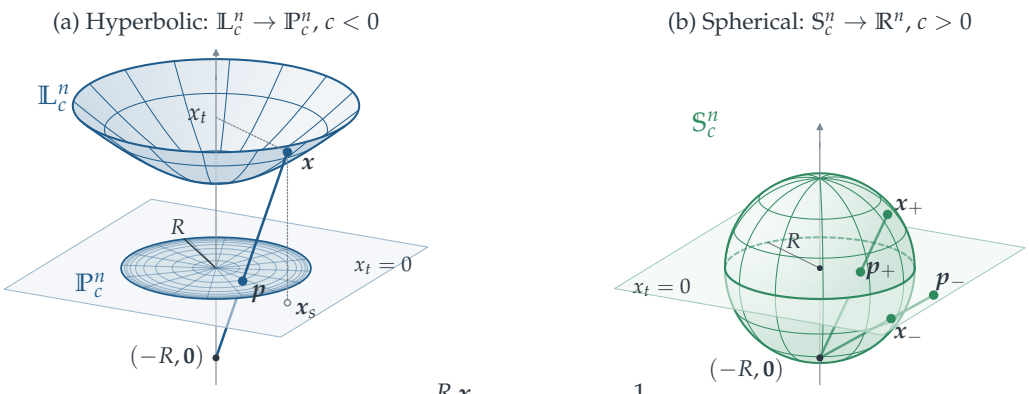


Figure 4 Stereographic projections of the hyperboloid and sphere, shown for $n = 2$. Write $\boldsymbol{x} = (x_t, \boldsymbol{x}_s)^T$, with x_t the vertical coordinate and $\boldsymbol{x}_s \in \mathbb{R}^n$. Both panels project from $(-R, \mathbf{0})$ onto the shaded plane $x_t = 0$ using the displayed map. **(a)** For $c < 0$, the Lorentz upper sheet $\mathbb{L}_c^n$ maps onto the Poincaré ball $\mathbb{P}_c^n$ of radius R; dotted guides mark the coordinates x_t and $\boldsymbol{x}_s$ of $\boldsymbol{x}$. **(b)** For $c > 0$, upper and lower hemisphere points $\boldsymbol{x}_+$ and $\boldsymbol{x}_-$ map inside and outside the radius-R equator circle, respectively. The spherical chart covers all of $\mathbb{R}^n$, with $\|\boldsymbol{p}\| \to \infty$ as $\boldsymbol{x}$ approaches the excluded south pole.

The point at $\mathbb{P}_c^n$ can be projected to $\mathbb{L}_c^n$ by:

$$\Pi^c_{\mathbb{P}\to\mathbb{L}}(\boldsymbol{x}) = \left[\frac{1 - c\|\boldsymbol{x}\|^2}{\sqrt{-c}\,(1 + c\|\boldsymbol{x}\|^2)}, \frac{2\boldsymbol{x}}{1 + c\|\boldsymbol{x}\|^2}\right]^T. \tag{16}$$

Figure 4a illustrates this isometry. The ball uses bounded coordinates and a position-dependent conformal metric; the hyperboloid uses unbounded coordinates and an ambient bilinear form. Both represent an intrinsically unbounded space. These coordinate choices affect numerical representation and optimization differently [37]. Under 64-bit arithmetic, Mishne et al. [195] find a larger correctly representable region in the Poincaré ball, together with an optimization advantage for the Lorentz model in their gradient analysis. Lorentz coordinates also support direct neural constructions [45], while the Poincaré ball provides the Möbius operations widely used in representation learning [81, 82, 203].

The Poincaré half-space, Klein, and hemisphere models are equivalent representations of hyperbolic geometry [33]. For example, the Klein model is defined as $\mathbb{K}_c^n = \{\boldsymbol{k} \in \mathbb{R}^n : -c\|\boldsymbol{k}\|^2 < 1\}$ with metric

$$g^{\mathcal{K}}_{\boldsymbol{k}} = \frac{1}{1 + c\|\boldsymbol{k}\|^2}\mathbb{I}_n + \frac{-c}{(1 + c\|\boldsymbol{k}\|^2)^2}\boldsymbol{k}\boldsymbol{k}^T.$$

For $\boldsymbol{k} \in \mathbb{K}_c^n$ and $\boldsymbol{x} \in \mathbb{P}_c^n$, the projections are

$$\Pi^c_{\mathbb{K}\to\mathbb{P}}(\boldsymbol{k}) = \frac{\boldsymbol{k}}{1 + \sqrt{1 + c\|\boldsymbol{k}\|^2}}, \qquad \Pi^c_{\mathbb{P}\to\mathbb{K}}(\boldsymbol{x}) = \frac{2\boldsymbol{x}}{1 - c\|\boldsymbol{x}\|^2}.$$

A Lorentz point maps to the Klein model through

$$\Pi^c_{\mathbb{L}\to\mathbb{K}}(\boldsymbol{x}) = \frac{\boldsymbol{x}_s}{\sqrt{-c}\,x_t}.$$

Consequently, the geodesic lines in the Klein model are Euclidean segments, and this model has been

used for midpoint-based aggregation operations [96]. The n-dimensional Poincaré half-space model is $\mathbb{U}_c^n = \{\boldsymbol{x} \in \mathbb{R}^n : x_n > 0\}$ with metric $g_{\boldsymbol{x}}^{\mathcal{U}} = g^E/(-cx_n^2)$. At $c = -1$, this is the usual unit-curvature normalization. The ball and half-space are conformal, so their tangent-vector angles equal Euclidean angles. Their geodesics are circle arcs or straight lines meeting the domain boundary orthogonally. Based on these properties, some studies [58, 266] have linked hyperbolic embeddings to Gaussian distributions through the relationship between Fisher distance and the distance in $\mathbb{U}^2$.

2.2.2 Spherical Models

The n-dimensional hypersphere $\mathbb{S}_c^n = (\mathcal{S}_c^n, g^{\mathcal{S}})$ has constant positive curvature $c > 0$ and radius $1/\sqrt{c}$, with $\mathcal{S}_c^n = \{\boldsymbol{x} \in \mathbb{R}^{n+1} : c\langle \boldsymbol{x}, \boldsymbol{x}\rangle = 1\}$ [291]. Its tangent space is $\mathcal{T}_{\boldsymbol{x}}\mathbb{S}_c^n = \{\boldsymbol{v} \in \mathbb{R}^{n+1} : \langle \boldsymbol{x}, \boldsymbol{v}\rangle = 0\}$, and its metric is the restriction of the Euclidean inner product to this space. We write $\boldsymbol{x} = (x_t, \boldsymbol{x}_s)^T$ with the polar coordinate x_t first. Figure 4b shows the stereographic projection used in the shared parameterization below. The angle excess of a spherical geodesic triangle is illustrated in Figure 1.

The hypersphere provides the positive-curvature counterpart of the Lorentz model: its closed-form operations use the circular functions sin and cos together with the Euclidean inner product $\langle \cdot, \cdot \rangle$, corresponding to the hyperbolic functions sinh and cosh and the Lorentz inner product $\langle \cdot, \cdot \rangle_{\mathcal{L}}$. For two points $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{S}_c^n$, the distance function between $\boldsymbol{x}$ and $\boldsymbol{y}$ is:

$$d_{\mathrm{S}}^c(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{c}} \arccos\left(c\langle \boldsymbol{x}, \boldsymbol{y}\rangle\right). \tag{17}$$

As in [250], for a tangent vector $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{S}_c^n$ and points $\boldsymbol{x}, \boldsymbol{y} \in \mathrm{S}_c^n$, the exponential and logarithmic maps are:

$$\exp_{\boldsymbol{x}}^c(\boldsymbol{v}) = \cos\left(\sqrt{c}\|\boldsymbol{v}\|\right)\boldsymbol{x} + \frac{1}{\sqrt{c}} \sin\left(\sqrt{c}\|\boldsymbol{v}\|\right) \frac{\boldsymbol{v}}{\|\boldsymbol{v}\|}, \tag{18}$$

$$\log_{\boldsymbol{x}}^c(\boldsymbol{y}) = \arccos(c\langle \boldsymbol{x}, \boldsymbol{y}\rangle) \frac{\boldsymbol{y} - c\langle \boldsymbol{x}, \boldsymbol{y}\rangle \boldsymbol{x}}{\sqrt{1 - (c\langle \boldsymbol{x}, \boldsymbol{y}\rangle)^2}}. \tag{19}$$

The parallel transport of $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{S}_c^n$ to $\mathcal{T}_{\boldsymbol{y}}\mathbb{S}_c^n$ is

$$\mathrm{PT}_{\boldsymbol{x}\to\boldsymbol{y}}^c(\boldsymbol{v}) = \boldsymbol{v} - \frac{c\langle \boldsymbol{y}, \boldsymbol{v}\rangle}{1 + c\langle \boldsymbol{x}, \boldsymbol{y}\rangle}(\boldsymbol{x} + \boldsymbol{y}). \tag{20}$$

The exponential-map formula is extended by $\exp_{\boldsymbol{x}}^c(\boldsymbol{0}) = \boldsymbol{x}$, while $\log_{\boldsymbol{x}}^c(\boldsymbol{x}) = \boldsymbol{0}$. The logarithmic map and the displayed parallel transport use the unique minimizing geodesic and therefore require $\boldsymbol{y} \neq -\boldsymbol{x}$; the antipodal case lies on the cut locus and has no unique minimizing geodesic.

2.2.3 Mixed-curvature Model

Mixed-curvature models combine factors with different curvature signs to represent different relations. Riemannian products provide the general construction and also include products of several hyperbolic factors, which we group with hyperbolic models in our method comparisons. We first describe product geometry, then introduce stereographic coordinates that allow a factor's curvature to vary continuously through zero. Pseudo-Riemannian models provide a related approach based on metric signature, discussed in Section 2.2.4.

Product manifolds. Let $\mathcal{M}_{c_i}^{n_i}$ denote an n_i-dimensional Euclidean, hyperbolic, or spherical factor with curvature c_i. A product space $\mathcal{M}' = \times_{i=1}^k \mathcal{M}_{c_i}^{n_i}$ is the Cartesian product of k such component

manifolds [94]. The product space $(\mathcal{M}', g')$ has the block-diagonal metric

$$\mathcal{M}' := \mathcal{M}_{c_1}^{n_1} \times \cdots \times \mathcal{M}_{c_k}^{n_k}, \quad g' := \begin{bmatrix} g^1 & & \\ & \ddots & \\ & & g^k \end{bmatrix}. \tag{21}$$

The tangent space at $\boldsymbol{x} = (\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(k)})$ is given by $\mathcal{T}_{\boldsymbol{x}}\mathcal{M}' = \bigoplus_{i=1}^{k} \mathcal{T}_{\boldsymbol{x}^{(i)}}\mathcal{M}_{c_i}^{n_i}$. The induced distance function d' on $\mathcal{M}'$ is given by $d'(\boldsymbol{x}, \boldsymbol{y})^2 = \sum_{i=1}^{k} d^i(\boldsymbol{x}^i, \boldsymbol{y}^i)^2$, where d^i is the distance in $\mathcal{M}_{c_i}^{n_i}$. The sectional curvature depends on the tangent plane. A 2-plane contained in the i-th factor has curvature c_i, while a plane spanned by one direction from each of two distinct factors is flat. Thus, a product containing both hyperbolic and spherical factors of dimension at least two has negative, positive, and zero sectional curvatures at the same point. The exponential map, logarithmic map, and parallel transport act separately on each factor. This makes component selection and dimensional allocation explicit modeling choices: different coordinates can represent hierarchical, angular, and approximately flat relations.

A shared stereographic parameterization. To learn a factor's curvature across both signs, we use operators that extend continuously through $c = 0$. The Poincaré ball provides the negative-curvature chart. Its positive-curvature counterpart is the hypersphere projected from the south pole:

$$\mathbb{S}_c'^n := \Pi_{\mathbb{S}\to\mathbb{S}'}^c(\mathbb{S}_c^n \setminus \{-\boldsymbol{\mu}_0\}), \qquad \boldsymbol{\mu}_0 = \left(\frac{1}{\sqrt{c}}, 0, \ldots, 0\right)^T \in \mathbb{S}_c^n.$$

For $\boldsymbol{z} = (z_t, \boldsymbol{z}_s)^T \in \mathbb{S}_c^n \setminus \{-\boldsymbol{\mu}_0\}$, this projection is $\Pi_{\mathbb{S}\to\mathbb{S}'}^c(\boldsymbol{z}) = \boldsymbol{z}_s/(1 + \sqrt{c}\, z_t)$, as illustrated in Figure 4b. The chart covers $\mathbb{R}^n$. Its conformal factor is

$$\lambda_{\boldsymbol{x}}^c = \frac{2}{1 + c\|\boldsymbol{x}\|^2}.$$

The coordinate formula for the metric is shared with the Poincaré ball, while the sign of c determines the geometry and the coordinate domain. For any two points $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{S}_c'^n$, the distance function is:

$$d_{\mathbb{S}'}^c(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{c}} \arccos\left(1 - \frac{2c\|\boldsymbol{x} - \boldsymbol{y}\|^2}{(1 + c\|\boldsymbol{x}\|^2)(1 + c\|\boldsymbol{y}\|^2)}\right). \tag{22}$$

On the projected hypersphere, Möbius addition uses the same algebraic expression as in the Poincaré chart [13, 250]. Where $-\boldsymbol{x} \oplus_c \boldsymbol{y}$ is finite, the distance is equivalently

$$d_{\mathbb{S}'}^c(\boldsymbol{x}, \boldsymbol{y}) = \frac{2}{\sqrt{c}} \arctan(\sqrt{c}\| - \boldsymbol{x} \oplus_c \boldsymbol{y}\|), \tag{23}$$

Antipodal sphere points have distance $\pi/\sqrt{c}$, obtained by continuity in this expression. In projected coordinates, the antipode of a nonzero $\boldsymbol{x}$ is $-\boldsymbol{x}/(c\|\boldsymbol{x}\|^2)$; the antipode of the origin is the excluded pole.

The exponential map and logarithm map at $\mathbb{S}_c'^n$ based on Möbius addition are:

$$\exp_{\boldsymbol{x}}^c(\boldsymbol{v}) = \boldsymbol{x} \oplus_c \left(\tan\left(\sqrt{c}\frac{\lambda_{\boldsymbol{x}}^c\|\boldsymbol{v}\|}{2}\right)\frac{\boldsymbol{v}}{\sqrt{c}\|\boldsymbol{v}\|}\right), \tag{24}$$

Table 3 Hyperbolic models and operations for signed curvature $c < 0$. The Lorentz and Poincaré models are isometric; their ambient coordinates and metric expressions differ. The Lorentz time coordinate is $x_t = x_1$. Removable zero denominators use continuous extensions, including $\exp_{\boldsymbol{x}}^c(\mathbf{0}) = \boldsymbol{x}$ and $\log_{\boldsymbol{x}}^c(\boldsymbol{x}) = \mathbf{0}$.

Space	Lorentz Model $\mathbb{L}_c^n$	Poincaré Ball Model $\mathbb{P}_c^n$
Manifold	$\mathcal{L}_c^n = \{\boldsymbol{x} = [x_t, \boldsymbol{x}_s]^T \in \mathbb{R}^{n+1} : c\langle \boldsymbol{x}, \boldsymbol{x}\rangle_{\mathcal{L}} = 1,\ x_t > 0\}$	$\mathcal{P}_c^n = \{\boldsymbol{x} \in \mathbb{R}^n : -c\|\boldsymbol{x}\|^2 < 1\}$
Metric	$g_{\boldsymbol{x}}^{\mathcal{L}}(\boldsymbol{u}, \boldsymbol{v}) = \langle \boldsymbol{u}, \boldsymbol{v}\rangle_{\mathcal{L}}, \quad \boldsymbol{u}, \boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{L}_c^n$ positive-definite tangent restriction	$g_{\boldsymbol{x}}^{\mathcal{P}} = (\lambda_{\boldsymbol{x}}^c)^2 g^E, \quad g^E = \mathbb{I}_n, \quad \lambda_{\boldsymbol{x}}^c = \frac{2}{1 + c\|\boldsymbol{x}\|^2}$
Distance function	$d_{\mathbb{L}}^c(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{-c}} \operatorname{arcosh}(c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}})$, $\widetilde{d}_{\mathbb{L},c}^2(\boldsymbol{x}, \boldsymbol{y}) = \frac{2}{c} - 2\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}}$ (ambient chord surrogate)	$d_{\mathbb{P}}^c(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{-c}} \operatorname{arcosh}\left(1 - \frac{2c\|\boldsymbol{x} - \boldsymbol{y}\|^2}{(1 + c\|\boldsymbol{x}\|^2)(1 + c\|\boldsymbol{y}\|^2)}\right)$ $= \frac{2}{\sqrt{-c}} \operatorname{artanh}\left(\sqrt{-c}\|-\boldsymbol{x} \oplus_c \boldsymbol{y}\|\right)$
Exponential map	$\exp_{\boldsymbol{x}}^c(\boldsymbol{v}) = \cosh(\sqrt{-c}\|\boldsymbol{v}\|_{\mathcal{L}})\, \boldsymbol{x} + \frac{\sinh(\sqrt{-c}\|\boldsymbol{v}\|_{\mathcal{L}})}{\sqrt{-c}} \frac{\boldsymbol{v}}{\|\boldsymbol{v}\|_{\mathcal{L}}}$	$\exp_{\boldsymbol{x}}^c(\boldsymbol{v}) = \boldsymbol{x} \oplus_c \left(\tanh\left(\sqrt{-c}\, \frac{\lambda_{\boldsymbol{x}}^c \|\boldsymbol{v}\|}{2}\right) \frac{\boldsymbol{v}}{\sqrt{-c}\|\boldsymbol{v}\|}\right)$
Logarithmic map	$\log_{\boldsymbol{x}}^c(\boldsymbol{y}) = d_{\mathbb{L}}^c(\boldsymbol{x}, \boldsymbol{y}) \frac{\boldsymbol{y} - c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}} \boldsymbol{x}}{\|\boldsymbol{y} - c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}} \boldsymbol{x}\|_{\mathcal{L}}}$	$\log_{\boldsymbol{x}}^c(\boldsymbol{y}) = \frac{2}{\sqrt{-c}\lambda_{\boldsymbol{x}}^c} \operatorname{artanh}(\sqrt{-c}\|-\boldsymbol{x} \oplus_c \boldsymbol{y}\|) \frac{-\boldsymbol{x} \oplus_c \boldsymbol{y}}{\|-\boldsymbol{x} \oplus_c \boldsymbol{y}\|}$
Parallel transport	$\mathrm{PT}_{\boldsymbol{x} \to \boldsymbol{y}}^c(\boldsymbol{v}) = \boldsymbol{v} - \frac{c\langle \boldsymbol{y}, \boldsymbol{v}\rangle_{\mathcal{L}}}{1 + c\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathcal{L}}} (\boldsymbol{x} + \boldsymbol{y})$	$\mathrm{PT}_{\boldsymbol{x} \to \boldsymbol{y}}^c(\boldsymbol{v}) = \frac{\lambda_{\boldsymbol{x}}^c}{\lambda_{\boldsymbol{y}}^c} \operatorname{gyr}[\boldsymbol{y}, -\boldsymbol{x}]\boldsymbol{v}$

$$\log_{\boldsymbol{x}}^c(\boldsymbol{y}) = \frac{2}{\sqrt{c}\lambda_{\boldsymbol{x}}^c} \arctan\left(\sqrt{c}\| -\boldsymbol{x} \oplus_c \boldsymbol{y}\|\right) \frac{-\boldsymbol{x} \oplus_c \boldsymbol{y}}{\| -\boldsymbol{x} \oplus_c \boldsymbol{y}\|}. \tag{25}$$

The continuous extensions $\exp_{\boldsymbol{x}}^c(\mathbf{0}) = \boldsymbol{x}$ and $\log_{\boldsymbol{x}}^c(\boldsymbol{x}) = \mathbf{0}$ are understood. The logarithmic map and parallel transport use the unique minimizing geodesic for non-antipodal points. We evaluate the displayed chart formulas where their trigonometric terms and Möbius denominators are finite and regular.

Similarly to the Poincaré ball, the parallel transport $\mathrm{PT}_{\boldsymbol{x} \to \boldsymbol{y}}^c : \mathcal{T}_{\boldsymbol{x}}\mathbb{S}_c'^n \to \mathcal{T}_{\boldsymbol{y}}\mathbb{S}_c'^n$ in the projected hypersphere is:

$$\mathrm{PT}_{\boldsymbol{x} \to \boldsymbol{y}}^c(\boldsymbol{v}) = \frac{\lambda_{\boldsymbol{x}}^c}{\lambda_{\boldsymbol{y}}^c} \operatorname{gyr}[\boldsymbol{y}, -\boldsymbol{x}]\boldsymbol{v}. \tag{26}$$

As $c \to 0$ from either sign, Möbius addition converges to vector addition and the conformal factor converges to 2. Consequently, the limiting metric is $4g^E$ and the limiting distance is $2\|\boldsymbol{x} - \boldsymbol{y}\|$. A Euclidean factor expressed in the same chart must use this scale; the coordinate change $\boldsymbol{z} = 2\boldsymbol{x}$ recovers the usual Euclidean metric. For points $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{P}_c^n$ or $\mathbb{S}_c'^n$ and a tangent vector $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{P}_c^n$ or $\mathcal{T}_{\boldsymbol{x}}\mathbb{S}_c'^n$, the maps and parallel transport converge to their Euclidean variants:

$$\lim_{c \to 0} \exp_{\boldsymbol{x}}^c(\boldsymbol{v}) = \boldsymbol{x} + \boldsymbol{v}, \quad \lim_{c \to 0} \log_{\boldsymbol{x}}^c(\boldsymbol{y}) = \boldsymbol{y} - \boldsymbol{x},$$

$$\lim_{c \to 0} \mathrm{PT}_{\boldsymbol{x} \to \boldsymbol{y}}^c(\boldsymbol{v}) = \boldsymbol{v}. \tag{27}$$

Therefore, the Poincaré ball $\mathbb{P}_c^n$ and the projected hypersphere $\mathbb{S}_c'^n$ are selected as the hyperbolic and spherical components of the mixed-curvature space $\mathcal{M}'$ in works [13, 250, 284]. Tables 3 and 4 summarize the four hyperbolic and spherical models, together with their distances, maps, and parallel transport.

Table 4 Spherical models and operations for signed curvature $c > 0$. The projected model is the stereographic chart of the radius-$1/\sqrt{c}$ sphere with one pole removed. Logarithmic maps and parallel transport assume non-antipodal points. In ambient sphere coordinates the antipode is $-\boldsymbol{x}$; in projected coordinates it is $-\boldsymbol{x}/(c\|\boldsymbol{x}\|^2)$ for $\boldsymbol{x} \neq \mathbf{0}$. Removable zero denominators use continuous extensions, and chart expressions are evaluated where finite and regular.

Space	Hypersphere Model $\mathbb{S}^n_c$	Projected Hypersphere Model $\mathbb{S}'^n_c$
Manifold	$\mathcal{S}^n_c = \{\boldsymbol{x} \in \mathbb{R}^{n+1} : c\langle \boldsymbol{x}, \boldsymbol{x}\rangle = 1\}$	$\mathbb{S}'^n_c = \Pi^c_{\mathbb{S}\to\mathbb{S}'}(\mathbb{S}^n_c \setminus \{-\boldsymbol{\mu}_0\}), \quad \boldsymbol{\mu}_0 = (1/\sqrt{c}, 0, \ldots, 0)^T \in \mathbb{S}^n_c$
Metric	$g^{\mathcal{S}}_{\boldsymbol{x}} = \mathbb{I}_{n+1}\big\vert_{\mathcal{T}_{\boldsymbol{x}}\mathbb{S}^n_c}$	$g^{\mathcal{S}'}_{\boldsymbol{x}} = (\lambda^c_{\boldsymbol{x}})^2 g^E, \quad g^E = \mathbb{I}_n, \quad \lambda^c_{\boldsymbol{x}} = \dfrac{2}{1+c\|\boldsymbol{x}\|^2}$
Distance function	$d^c_{\mathbb{S}}(\boldsymbol{x},\boldsymbol{y}) = \dfrac{1}{\sqrt{c}}\arccos(c\langle \boldsymbol{x}, \boldsymbol{y}\rangle)$	$d^c_{\mathbb{S}'}(\boldsymbol{x},\boldsymbol{y}) = \dfrac{1}{\sqrt{c}}\arccos\left(1 - \dfrac{2c\|\boldsymbol{x}-\boldsymbol{y}\|^2}{(1+c\|\boldsymbol{x}\|^2)(1+c\|\boldsymbol{y}\|^2)}\right) = \dfrac{2}{\sqrt{c}}\arctan\left(\sqrt{c}\|-\boldsymbol{x}\oplus_c \boldsymbol{y}\|\right)$
Exponential map	$\exp^c_{\boldsymbol{x}}(\boldsymbol{v}) = \cos\left(\sqrt{c}\|\boldsymbol{v}\|\right)\boldsymbol{x} + \dfrac{\sin\left(\sqrt{c}\|\boldsymbol{v}\|\right)}{\sqrt{c}}\dfrac{\boldsymbol{v}}{\|\boldsymbol{v}\|}$	$\exp^c_{\boldsymbol{x}}(\boldsymbol{v}) = \boldsymbol{x}\oplus_c\left(\tan\left(\sqrt{c}\,\dfrac{\lambda^c_{\boldsymbol{x}}\|\boldsymbol{v}\|}{2}\right)\dfrac{\boldsymbol{v}}{\sqrt{c}\|\boldsymbol{v}\|}\right)$
Logarithmic map	$\log^c_{\boldsymbol{x}}(\boldsymbol{y}) = \arccos(c\langle \boldsymbol{x}, \boldsymbol{y}\rangle)\dfrac{\boldsymbol{y} - c\langle \boldsymbol{x}, \boldsymbol{y}\rangle \boldsymbol{x}}{\sqrt{1-(c\langle \boldsymbol{x}, \boldsymbol{y}\rangle)^2}}$	$\log^c_{\boldsymbol{x}}(\boldsymbol{y}) = \dfrac{2}{\sqrt{c}\lambda^c_{\boldsymbol{x}}}\arctan\left(\sqrt{c}\|-\boldsymbol{x}\oplus_c \boldsymbol{y}\|\right)\dfrac{-\boldsymbol{x}\oplus_c \boldsymbol{y}}{\|-\boldsymbol{x}\oplus_c \boldsymbol{y}\|}$
Parallel transport	$\mathrm{PT}^c_{\boldsymbol{x}\to\boldsymbol{y}}(\boldsymbol{v}) = \boldsymbol{v} - \dfrac{c\langle \boldsymbol{y}, \boldsymbol{v}\rangle}{1+c\langle \boldsymbol{x}, \boldsymbol{y}\rangle}(\boldsymbol{x}+\boldsymbol{y})$	$\mathrm{PT}^c_{\boldsymbol{x}\to\boldsymbol{y}}(\boldsymbol{v}) = \dfrac{\lambda^c_{\boldsymbol{x}}}{\lambda^c_{\boldsymbol{y}}}\mathrm{gyr}[\boldsymbol{y}, -\boldsymbol{x}]\boldsymbol{v}$

2.2.4 Pseudo-Riemannian Models

Pseudo-Riemannian models change the metric signature within a single space. For p spatial and $q+1$ time coordinates, define

$$\mathcal{Q}^{p,q}_{\beta} = \{\boldsymbol{x} \in \mathbb{R}^{p+q+1} : \langle \boldsymbol{x}, \boldsymbol{x}\rangle_q = -\beta\}, \tag{28}$$

where $\beta > 0$ and $\langle \boldsymbol{x}, \boldsymbol{y}\rangle_q = -\sum_{i=1}^{q+1} x_i y_i + \sum_{i=q+2}^{p+q+1} x_i y_i$. Restricting this form to the tangent space gives signature (p, q), with p positive and q negative directions [143]. At $q = 0$, the upper sheet $x_1 > 0$ is $\mathbb{L}^p_{-1/\beta}$. At $p = 0$, the point set is a sphere of radius $\sqrt{\beta}$; reversing the sign of its negative-definite metric gives $\mathbb{S}^q_{1/\beta}$.

For intermediate signatures, spacelike, timelike, and null tangent directions produce hyperbolic, circular, and linear geodesic formulas, respectively. This behavior supports hierarchical and cyclic relations without a product decomposition. The pseudo-hyperboloid itself has constant sectional curvature $-1/\beta$ on nondegenerate tangent planes, so metric signature and mixed sectional curvature describe distinct constructions. The modeling choice is between factor dimensions and curvatures in a Riemannian product, or positive and negative metric directions in a pseudo-Riemannian space.

An indefinite metric requires its own dissimilarity and optimization rules: nonzero vectors can have zero length, and some point pairs have no connecting geodesic. Building on the initial geometric construction, later work develops neural layers [142], graph convolution [293], and knowledge graph embeddings [292] on these spaces.

2.3 Neural Modules

Neural modules must produce valid manifold points while transforming their features. Two common constructions apply Euclidean operations in a tangent space or transform ambient coordinates and enforce the manifold constraint. We compare these choices for linear layers, activations, functional

components, and optimization.

2.3.1 Linear Transformation

A Euclidean affine layer combines matrix multiplication with bias addition. Its geometric counterparts differ in where they apply the matrix and how they produce a valid output point.

For **hyperbolic linear transformations**, tangent-space layers apply a matrix between a logarithmic map and an exponential map [39, 82]. Given a stereographic hyperbolic embedding $\boldsymbol{x} \in \mathbb{P}^n_{c_1}$ and a tangent-space map $f : \mathcal{T}_{\mathbf{0}}\mathbb{P}^n_{c_1} \to \mathcal{T}_{\mathbf{0}}\mathbb{P}^m_{c_2}$, the hyperbolic linear transformation $\mathrm{LT}_{\mathbb{P}} : \mathbb{P}^n_{c_1} \to \mathbb{P}^m_{c_2}$ is

$$\mathrm{LT}_{\mathbb{P}}(\boldsymbol{x}; c_1, c_2, f) = \exp_{\mathbf{0}}^{c_2}\left(f\left(\log_{\mathbf{0}}^{c_1}(\boldsymbol{x})\right)\right) \oplus_{c_2} \boldsymbol{b}, \tag{29}$$

where f is linear, $\boldsymbol{b} \in \mathbb{P}^m_{c_2}$, and $c_1, c_2 < 0$ may differ. The Möbius bias acts in the output ball. Lorentz layers instead use the tangent or ambient constructions in Equations (31) and (32).

Shimizu et al. [243] construct each output coordinate from a signed hyperbolic-hyperplane score, giving a Poincaré layer $\mathrm{LT}_{\mathbb{P}} : \mathbb{P}^n_c \to \mathbb{P}^m_c$. Given an input $\boldsymbol{x} \in \mathbb{P}^n_c$ and nonzero parameters $\boldsymbol{Z} = \{\boldsymbol{z}_k \in \mathcal{T}_{\mathbf{0}}\mathbb{P}^n_c \setminus \{\mathbf{0}\}\}_{k=1}^m$, $\boldsymbol{r} = \{r_k \in \mathbb{R}\}_{k=1}^m$, let $\widehat{\boldsymbol{z}}_k = \boldsymbol{z}_k / \|\boldsymbol{z}_k\|$. The Poincaré linear transformation is defined as:

$$\mathrm{LT}_{\mathbb{P}}(\boldsymbol{x}; \boldsymbol{Z}, \boldsymbol{r}, c) = \boldsymbol{w}\left(1 + \sqrt{1 - c\|\boldsymbol{w}\|^2}\right)^{-1}, \tag{30}$$

where

$$\boldsymbol{w} = \left((-c)^{-1/2}\sinh\left(\sqrt{-c}\, v_k(\boldsymbol{x})\right)\right)_{k=1}^m,$$
$$v_k(\boldsymbol{x}) = \frac{2}{\sqrt{|c|}}\|\boldsymbol{z}_k\| \operatorname{arsinh}\left(\lambda_{\boldsymbol{x}}^c \langle \sqrt{|c|}\boldsymbol{x}, \widehat{\boldsymbol{z}}_k\rangle \cosh\left(2\sqrt{|c|}r_k\right) - (\lambda_{\boldsymbol{x}}^c - 1)\sinh\left(2\sqrt{|c|}r_k\right)\right).$$

Tangent-space Lorentz layers set the time coordinate to zero, which is the tangent constraint at the canonical basepoint [303, 332]. For $\boldsymbol{x} \in \mathbb{L}^n_c$, let $\boldsymbol{o}_c = (1/\sqrt{-c}, 0, \ldots, 0)^T$ be the Lorentz basepoint. The linear transformation is formulated as:

$$\mathrm{LT}_{\mathbb{L}}(\boldsymbol{x}; c, f) = \exp_{\boldsymbol{o}_c}^c\left(\left[0, f\left(\log_{\boldsymbol{o}_c}^c(\boldsymbol{x})_{[2:n+1]}\right)\right]\right). \tag{31}$$

To avoid repeated logarithmic and exponential mappings between hyperbolic spaces and tangent spaces, Chen et al. [45] proposed a direct Lorentz-valued linear transformation that constructs the time component from the transformed spatial coordinates. Given $\boldsymbol{x} \in \mathbb{L}^n_c$, the hyperbolic linear transformation $\mathrm{LT}_{\mathbb{L}} : \mathbb{L}^n_c \to \mathbb{L}^m_c$ consists of the operations on time component and spatial component, and is defined as:

$$\mathrm{LT}_{\mathbb{L}}(\boldsymbol{x}) = \left[\sqrt{\|\boldsymbol{u}(\boldsymbol{x})\|_2^2 - \frac{1}{c}},\ \boldsymbol{u}(\boldsymbol{x})\right]^T, \tag{32}$$

where $\boldsymbol{u}(\boldsymbol{x}) = \lambda\sigma(\mathbf{v}^T\boldsymbol{x} + b')\frac{\mathbf{W}\boldsymbol{h}(\boldsymbol{x})+\boldsymbol{b}}{\max\{\|\mathbf{W}\boldsymbol{h}(\boldsymbol{x})+\boldsymbol{b}\|_2, \varepsilon\}} \in \mathbb{R}^m$, $\mathbf{v} \in \mathbb{R}^{n+1}$, $\mathbf{W} \in \mathbb{R}^{m\times(n+1)}$, and $\boldsymbol{b} \in \mathbb{R}^m$. Here, $\boldsymbol{h} : \mathbb{R}^{n+1} \to \mathbb{R}^{n+1}$ is the chosen Euclidean feature map, σ denotes the sigmoid function, $b' \in \mathbb{R}$ is a scalar bias, $\lambda > 0$ controls the scaling range, and $\varepsilon > 0$ prevents division by zero. Yang et al. [307] generalize the Lorentz transformation by allowing its input and output curvatures to differ. Given a point $\boldsymbol{x} \in \mathbb{L}^n_{c_1}$, transformation matrix $\mathbf{W} \in \mathbb{R}^{(n+1)\times m}$ and bias $\boldsymbol{b} \in \mathbb{R}^m$, the hyperbolic transformation

with curvatures $\mathrm{LT}_{\mathbb{L}} : \mathbb{L}^n_{c_1} \to \mathbb{L}^m_{c_2}$ is given as the following equation:

$$\mathrm{LT}_{\mathbb{L}}(\boldsymbol{x}; f_t, c_1, c_2) := \left[\sqrt{\frac{c_1}{c_2}\|f_t(\boldsymbol{x})\|^2 - \frac{1}{c_2}}, \sqrt{\frac{c_1}{c_2}} f_t(\boldsymbol{x}) \right]^T, \tag{33}$$

where the $f_t(\boldsymbol{x}) = \mathbf{W}^T\boldsymbol{x} + \boldsymbol{b}$ represents the linear transformation with bias.

For **spherical linear transformations**, Gu et al. [95] map through the tangent space using the exponential and logarithmic maps. For $\boldsymbol{x} \in \mathbb{S}^{n-1}_c$, the transform $\mathrm{LT}_{\mathbb{S}} : \mathbb{S}^{n-1}_c \to \mathbb{S}^{m-1}_c$ is

$$\mathrm{LT}_{\mathbb{S}}(\boldsymbol{x}; c, g) = \exp^c_{\boldsymbol{x_2}}\left(g\left(\log^c_{\boldsymbol{x_1}}(\boldsymbol{x})\right)\right), \tag{34}$$

where $g : \mathcal{T}_{\boldsymbol{x_1}}\mathbb{S}^{n-1}_c \to \mathcal{T}_{\boldsymbol{x_2}}\mathbb{S}^{m-1}_c$ is a linear transformation on Euclidean space, $\exp^c_{\boldsymbol{x_2}}$ and $\log^c_{\boldsymbol{x_1}}$ denote the spherical exponential and logarithmic maps, respectively. Here, $\boldsymbol{x_1} = (1/\sqrt{c}, 0, \ldots, 0) \in \mathbb{R}^n$ and $\boldsymbol{x_2} = (1/\sqrt{c}, 0, \ldots, 0) \in \mathbb{R}^m$ represent the north poles of $\mathbb{S}^{n-1}_c$ and $\mathbb{S}^{m-1}_c$. Liu et al. [177] define a hyperspherical convolutional operator through the similarity between a local patch $\boldsymbol{x}$ and a filter $\boldsymbol{w}$. For nonzero $\boldsymbol{x}$ and $\boldsymbol{w}$, their angle $\theta(\boldsymbol{w}, \boldsymbol{x})$ is the geodesic distance between their normalized directions on the unit sphere. The hyperspherical convolution replaces the usual dot-product response with an angular score $g(\theta)$, such as $g(\theta) = -2\theta/\pi + 1$ or $g(\theta) = \cos\theta$. Thus, tangent-space spherical layers transform manifold points, while angular convolutions use spherical similarity to generate filter responses.

2.3.2 Non-linear Activation

Activations introduce nonlinearity while respecting the output geometry. As with linear layers, they can act on tangent coordinates or directly on manifold coordinates with an appropriate constraint-preserving step.

For **hyperbolic activations**, a common construction applies a Euclidean activation in a reference tangent space and maps back [39, 75, 82, 303]. In the Poincaré model, the reference point is the coordinate origin $\mathbf{0}$:

$$\sigma^{c_1 \to c_2}_{\mathbb{P}}(\boldsymbol{x}) = \exp^{c_2}_{\mathbf{0}}\left(\sigma\left(\log^{c_1}_{\mathbf{0}}(\boldsymbol{x})\right)\right), \qquad \sigma^{c_1 \to c_2}_{\mathbb{P}} : \mathbb{P}^n_{c_1} \to \mathbb{P}^n_{c_2}. \tag{35}$$

For a Lorentz tangent-space activation, the transformed vector must remain tangent. At $\boldsymbol{o}_c$, this means retaining a zero time coordinate and applying the activation to the spatial coordinates [332]:

$$\sigma_{\mathbb{L}}(\boldsymbol{x}) = \exp^{c_2}_{\boldsymbol{o}_{c_2}}\left([0, \sigma((\log^{c_1}_{\boldsymbol{o}_{c_1}}(\boldsymbol{x}))_{[2:n+1]})]^T\right). \tag{36}$$

Bdeir et al. [17] avoid logarithmic and exponential maps by activating the spatial component $\boldsymbol{x}_s$ and reconstructing the time coordinate:

$$\sigma_{\mathbb{L}}(\boldsymbol{x}) = \left[\sqrt{\|\sigma(\boldsymbol{x_s})\|^2 - 1/c},\ \sigma(\boldsymbol{x_s})\right]^T. \tag{37}$$

For variable curvature, Yang et al. [307] apply the activation to the spatial component $\boldsymbol{x_s}$. For $\boldsymbol{x} \in \mathbb{L}^n_{c_1}$, the resulting map $\sigma_{\mathbb{L}} : \mathbb{L}^n_{c_1} \to \mathbb{L}^n_{c_2}$ is

$$\sigma_{\mathbb{L}}(\boldsymbol{x}) = \left[\sqrt{\frac{c_1}{c_2}\|\sigma(\boldsymbol{x_s})\|^2 - \frac{1}{c_2}}, \sqrt{\frac{c_1}{c_2}}\,\sigma(\boldsymbol{x_s})\right]^T. \tag{38}$$

ReLU and leaky ReLU with negative-side slope in $[0, 1]$ preserve the Poincaré ball because they do not increase the Euclidean norm. Applied to all Lorentz coordinates, they generally violate the hyperboloid constraint [60, 175]. A coordinate-conversion approach therefore projects to the ball, applies the activation, and maps back:

$$\sigma_{\mathbb{L}}(\boldsymbol{x}) = \Pi^{c}_{\mathbb{P}\to\mathbb{L}}(\sigma(\Pi^{c}_{\mathbb{L}\to\mathbb{P}}(\boldsymbol{x}))). \tag{39}$$

For **spherical activations**, Gu et al. [95] define spherical ReLU on $\mathbb{S}^{n-1}_c$. For $\boldsymbol{x} \in \mathbb{S}^{n-1}_c$, it is

$$\mathrm{ReLU}_{\mathbb{S}}(\boldsymbol{x}) = \begin{cases} \dfrac{1}{\sqrt{c}} \dfrac{\mathrm{ReLU}(\boldsymbol{x})}{\|\mathrm{ReLU}(\boldsymbol{x})\|}, & \mathrm{ReLU}(\boldsymbol{x}) \neq \mathbf{0}, \\ \boldsymbol{x}, & \mathrm{ReLU}(\boldsymbol{x}) = \mathbf{0}, \end{cases} \tag{40}$$

The fallback branch is a convention for the case in which the normalized ReLU expression is undefined. Liao et al. [167] define a spherical exponential activation that maps unconstrained logits $\boldsymbol{x} \in \mathbb{R}^{n+1}$ to the positive orthant of the unit sphere $\mathbb{S}^{n}_1$:

$$\sigma_{\mathbb{S}}(x_i; \boldsymbol{x}) = \frac{e^{x_i}}{\sqrt{\sum_k (e^{x_k})^2}}. \tag{41}$$

Their regression model predicts signs separately to obtain signed spherical outputs.

2.3.3 Functional Components

Concatenation, dropout, and residual addition change feature coordinates and therefore need a rule for producing a valid manifold output. Normalization additionally requires a geometric center and spread, while multinomial logistic regression (MLR) requires class scores. We introduce representative hyperbolic constructions for each component.

Concatenation. One way to combine two inputs is to transform each separately and add the results. Given $\boldsymbol{x}_1 \in \mathbb{P}^{n}_c$, $\boldsymbol{x}_2 \in \mathbb{P}^{p}_c$, $\boldsymbol{M}_1 \in \mathbb{R}^{m\times n}$, and $\boldsymbol{M}_2 \in \mathbb{R}^{m\times p}$, Ganea et al. [82] use the product-input notation $\boldsymbol{M} \otimes_c \boldsymbol{x}$ for

$$\boldsymbol{M} \otimes_c \boldsymbol{x} = \boldsymbol{M}_1 \otimes_c \boldsymbol{x_1} \oplus_c \boldsymbol{M_2} \otimes_c \boldsymbol{x_2}. \tag{42}$$

This combines concatenation with a learned projection: its Euclidean limit is multiplication by the block matrix $[\boldsymbol{M}_1\ \boldsymbol{M}_2]$,

$$\lim_{c\to 0} \boldsymbol{M} \otimes_c \boldsymbol{x} = \lim_{c\to 0} ((\boldsymbol{M}_1 \otimes_c \boldsymbol{x}_1) \oplus_c (\boldsymbol{M}_2 \otimes_c \boldsymbol{x}_2)) = \boldsymbol{M}_1\boldsymbol{x}_1 + \boldsymbol{M}_2\boldsymbol{x}_2.$$

Shimizu et al. [243] introduce tangent-space β-concatenation for the Poincaré ball $\mathbb{P}^{n}_c$, with dimension-dependent scaling designed to preserve the expected representation norm. The construction combines tangent blocks before a final exponential map, avoiding sequential Möbius additions. In the β-concatenation process, the input $\{\boldsymbol{x_i} \in \mathbb{P}^{n_i}_c\}_{i=1}^{N}$ is first scaled and concatenated in the tangent space:

$$\boldsymbol{v_i} = \log^{c}_{\mathbf{0}}(\boldsymbol{x_i}) \in \mathcal{T}_{\mathbf{0}}\mathbb{P}^{n_i}_c, \tag{43}$$

$$\boldsymbol{v} = (\beta_n\beta^{-1}_{n_1}\boldsymbol{v_1}^T, \ldots, \beta_n\beta^{-1}_{n_N}\boldsymbol{v_N}^T)^T, \tag{44}$$

where $\beta_n = \mathbf{B}\left(\frac{n}{2}, \frac{1}{2}\right)$, and $\mathbf{B}$ denotes the Beta function. The vector v is then projected back to the Poincaré ball $\mathbb{P}^{n}_c$ using the exponential map $\exp^{c}_{\mathbf{0}}(v)$ at the origin, with $\sum_{i=1}^{N} n_i = n$. Conversely, for $\boldsymbol{x} \in \mathbb{P}^{n}_c$, write $\log^{c}_{\mathbf{0}}(\boldsymbol{x}) = (\boldsymbol{v}_1^T, \ldots, \boldsymbol{v}_N^T)^T$ with $\boldsymbol{v}_i \in \mathbb{R}^{n_i}$. The β-split outputs are manifold points

$\boldsymbol{x}_i = \exp_{\mathbf{0}}^c(\beta_{n_i}\beta_n^{-1}\boldsymbol{v}_i) \in \mathbb{P}_c^{n_i}$.

Qu & Zou [227, 228] define direct Lorentz concatenation by combining spatial components and completing the time component. Given inputs $\{\boldsymbol{x_i} \in \mathbb{L}_c^{n_i}\}_{i=1}^N$, $\mathrm{HCat}(\{\boldsymbol{x_i}\}_{i=1}^N)$ produces $\boldsymbol{y} \in \mathbb{L}_c^n$:

$$\boldsymbol{y} = \mathrm{HCat}(\{\boldsymbol{x}_i\}_{i=1}^N) = \left[\sqrt{\textstyle\sum_{i=1}^N x_{i_t}^2 + \frac{N-1}{c}}, \boldsymbol{x}_{1_s}^T, \ldots, \boldsymbol{x}_{N_s}^T\right]^T, \tag{45}$$

where $\sum_{i=1}^N n_i = n$, x_{i_t} and $\boldsymbol{x_{i_s}}$ are the temporal and spatial components of $\boldsymbol{x}_i \in \mathbb{L}_c^{n_i}$.

Dropout. Dropout masks activations, whereas DropConnect masks connection weights [278]. Chami et al. [39] apply DropConnect to the Euclidean weight matrix of a tangent-space transformation. Direct Lorentz layers can instead mask features before completing the time coordinate. In the direct Lorentz-valued linear layer of FHNN [45], dropout for $\boldsymbol{x} \in \mathbb{L}_c^n$ is defined as

$$\boldsymbol{y} = \left[\sqrt{\|\phi(\boldsymbol{x}; \mathbf{W})\|^2 - \frac{1}{c}},\ \phi(\boldsymbol{x}; \mathbf{W})\right]^T, \tag{46}$$

where $\mathbf{W} \in \mathbb{R}^{m\times(n+1)}$ and, for a single realization of the dropout mask,

$$\phi(\boldsymbol{x}; \mathbf{W}) = \mathbf{W}\,\mathrm{dropout}(\boldsymbol{x}).$$

The transformed feature becomes the spatial output, and reconstructing the time coordinate restores the Lorentz constraint. Yang et al. [307] extend this dropout operation to variable-curvature Lorentz layers. For points $\boldsymbol{x} \in \mathbb{L}_{c_1}^n$ and $\boldsymbol{y} \in \mathbb{L}_{c_2}^n$, the hyperbolic dropout operation is defined as:

$$\boldsymbol{y} = \left[\sqrt{\frac{c_1}{c_2}\|\mathrm{dropout}(\boldsymbol{x_s})\|^2 - \frac{1}{c_2}}, \sqrt{\frac{c_1}{c_2}}\,\mathrm{dropout}(\boldsymbol{x_s})\right]^T, \tag{47}$$

where both appearances of $\mathrm{dropout}(\boldsymbol{x}_s)$ use the same sampled mask, ensuring that the time coordinate matches the spatial output.

Residual Connections. Residual connections can alleviate the vanishing gradient problem in deep neural networks [105]. Geometric residual connections can reuse the addition operations developed for bias terms, including parallel-transport addition [39] and Möbius addition [82]. Bdeir et al. [17] add the spatial components and reconstruct the time component to preserve the Lorentz constraint.

Similarly, Chen et al. [45] implement a residual-like operation in the Lorentz model, where residual operations are integrated into the preceding Lorentz linear layer of each computational block. Given the input $\boldsymbol{x} \in \mathbb{L}_c^n$ of the computational block and the output $\boldsymbol{o} = f(\boldsymbol{x})$ before the final Lorentz linear layer, $\boldsymbol{x}$ is used as the bias for the Lorentz linear layer:

$$\boldsymbol{y} = \left[\sqrt{\|\phi(\boldsymbol{o}; \mathbf{W}, \mathbf{v}, \boldsymbol{x})\|^2 - 1/c},\ \phi(\boldsymbol{o}; \mathbf{W}, \mathbf{v}, \boldsymbol{x})\right]^T, \tag{48}$$

$$\phi(\boldsymbol{o}; \mathbf{W}, \mathbf{v}, \boldsymbol{x}) = \lambda\sigma(\mathbf{v}^T\boldsymbol{o})\frac{\mathbf{W}\boldsymbol{h}(\boldsymbol{o}) + \boldsymbol{x}_s}{\max\{\|\mathbf{W}\boldsymbol{h}(\boldsymbol{o}) + \boldsymbol{x}_s\|_2, \varepsilon\}}, \tag{49}$$

where the residual requires matching dimensions: $\boldsymbol{x}, \boldsymbol{o}, \boldsymbol{y} \in \mathbb{L}_c^n$, $\mathbf{v} \in \mathbb{R}^{n+1}$, $\mathbf{W} \in \mathbb{R}^{n\times(n+1)}$, and $\boldsymbol{h} : \mathbb{R}^{n+1} \to \mathbb{R}^{n+1}$. The constants $\lambda > 0$ and $\varepsilon > 0$ control the output scale and guard the normalization, respectively.

He et al. [110] generalize the Lorentzian centroid [144] into a residual connection that avoids exponential and logarithmic maps. Let $\boldsymbol{x} \in \mathbb{L}_c^n$ and $f(\boldsymbol{x}) \in \mathbb{L}_c^n$ denote the layer input and output. The residual connection is

$$\boldsymbol{x} \oplus_{\mathbb{L}} f(\boldsymbol{x}) = \frac{w_x \boldsymbol{x} + w_y f(\boldsymbol{x})}{\sqrt{c \left\langle w_x \boldsymbol{x} + w_y f(\boldsymbol{x}),\, w_x \boldsymbol{x} + w_y f(\boldsymbol{x}) \right\rangle_{\mathcal{L}}}}, \tag{50}$$

where $w_x, w_y > 0$ are weights that can be either learned or fixed.

MLR. For non-Euclidean MLR, Ganea et al. [82] define a multiclass classifier on the Poincaré ball $\mathbb{P}_c^n$. Given K classes and $k \in \{1, \ldots, K\}$, $\boldsymbol{p}_k \in \mathbb{P}_c^n$, and $\boldsymbol{a}_k \in \mathcal{T}_{\boldsymbol{p}_k}\mathbb{P}_c^n \setminus \{\boldsymbol{0}\}$, for every $\boldsymbol{x} \in \mathbb{P}_c^n$, the hyperbolic MLR used to predict the probability of class k is defined as:

$$p(y = k \mid \boldsymbol{x}) = \frac{\exp(v_k(\boldsymbol{x}))}{\sum_{j=1}^{K} \exp(v_j(\boldsymbol{x}))}, \tag{51}$$

$$v_k(\boldsymbol{x}) = \frac{\lambda_{\boldsymbol{p}_k}^c \|\boldsymbol{a}_k\|}{\sqrt{-c}} \operatorname{arsinh}\left(\frac{2\sqrt{-c}\langle -\boldsymbol{p}_k \oplus_c \boldsymbol{x}, \boldsymbol{a}_k \rangle}{(1 + c\| -\boldsymbol{p}_k \oplus_c \boldsymbol{x}\|^2)\|\boldsymbol{a}_k\|} \right), \tag{52}$$

where the conformal chart identifies each tangent space with $\mathbb{R}^n$ when evaluating the Euclidean inner product. As $c \to 0^-$, the score converges to $4\langle \boldsymbol{x} - \boldsymbol{p}_k, \boldsymbol{a}_k \rangle$ under this coordinate convention, recovering Euclidean MLR after absorbing the constant factor into the class parameters. The corresponding hyperbolic hyperplane in $\mathbb{P}_c^n$ is

$$\widetilde{H}_{\boldsymbol{a},\boldsymbol{p}}^c := \{\boldsymbol{x} \in \mathbb{P}_c^n : \langle -\boldsymbol{p} \oplus_c \boldsymbol{x}, \boldsymbol{a} \rangle = 0\}. \tag{53}$$

Figure 5 shows a hyperbolic hyperplane in the three-dimensional Poincaré ball. However, the hyperbolic MLR in HNN requires almost twice the number of parameters compared to its Euclidean MLR. To reduce the number of parameters, Shimizu et al. [243] reformulated the scalar score $v_k(\boldsymbol{x}) = \langle \boldsymbol{a}_k, \boldsymbol{x} \rangle - b_k$ and proposed a unidirectional Poincaré MLR in the Poincaré ball $\mathbb{P}_c^n$ without increasing the parameter count. Given $r_k \in \mathbb{R}$ and nonzero $\boldsymbol{z}_k \in \mathcal{T}_{\boldsymbol{0}}\mathbb{P}_c^n$, let $\widehat{\boldsymbol{z}}_k = \boldsymbol{z}_k / \|\boldsymbol{z}_k\|$. Then, for each class $k \in \{1, \ldots, K\}$,

$$v_k(\boldsymbol{x}) = \frac{2}{\sqrt{|c|}} \|\boldsymbol{z}_k\| \operatorname{arsinh}\left(s_k(\boldsymbol{x}) - m_k(\boldsymbol{x})\right), \tag{54}$$

$$s_k(\boldsymbol{x}) = \lambda_{\boldsymbol{x}}^c \langle \sqrt{|c|}\boldsymbol{x}, \widehat{\boldsymbol{z}}_k \rangle \cosh\left(2\sqrt{|c|} r_k\right), \tag{55}$$

$$m_k(\boldsymbol{x}) = (\lambda_{\boldsymbol{x}}^c - 1) \sinh\left(2\sqrt{|c|} r_k\right), \tag{56}$$

where v_k is the same scalar score as that used in Equation (30).

Batch Normalization. Euclidean batch normalization (BN) centers and rescales channels using mini-batch statistics [119]. Geometric variants use a manifold center and squared geodesic distances to measure spread. Lorentz BN uses a closed-form Lorentz centroid and transports centered tangent features for rescaling and mapping to a learned center [17]. This centroid construction differs from computing the Fréchet mean, which minimizes summed squared geodesic distances. Poincaré BN uses Möbius centering, scaling, and translation [249]. Closed-form gyromidpoints provide surrogate centers [270, 272], while iterative methods approximate the Fréchet mean [181]. Appendix A.1 gives the complete recipes and the equivalence of the two midpoint formulas.

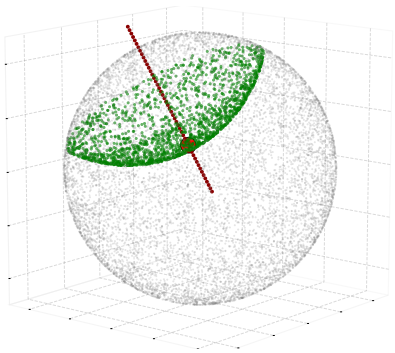


Figure 5 A hyperbolic hyperplane in $\mathbb{P}^3_{-1}$ plotted by sampling. The green points sample the hyperplane, and the red line shows its normal direction $\boldsymbol{a}$; their intersection is the base point $\boldsymbol{p}$.

2.3.4 Optimizer

Riemannian optimization applies when trainable parameters themselves lie on a manifold. Euclidean matrices inside a manifold-valued layer can still use standard optimizers. For a smooth loss $f : \mathcal{M} \to \mathbb{R}$, Riemannian gradient descent moves along a tangent gradient and returns to the manifold through an exponential map [23]:

$$\theta_{t+1} = \exp_{\theta_t}(-\eta \operatorname{grad} f(\theta_t)), \tag{57}$$

where $\eta > 0$ is the learning rate and $\operatorname{grad} f(\theta_t)$ is the Riemannian gradient. Riemannian stochastic gradient descent (RSGD) uses a stochastic estimate of this gradient. A retraction can replace the exponential map when a cheaper local approximation is available.

For **hyperbolic optimization**, Nickel & Kiela [205] develop RSGD on the Lorentz model. The ambient Euclidean gradient is first converted using the Lorentz bilinear form:

$$h_t = J_{\mathcal{L}}^{-1} \nabla f(\theta_t) = J_{\mathcal{L}} \nabla f(\theta_t), \tag{58}$$

where $J_{\mathcal{L}}$ is the ambient signature matrix defined above. Lorentz-orthogonal projection onto the tangent space then gives the Riemannian gradient:

$$\operatorname{grad} f(\theta_t) = h_t - c\langle \theta_t, h_t \rangle_{\mathcal{L}} \theta_t. \tag{59}$$

For Poincaré ball embeddings, Nickel & Kiela [203] adapted RSGD by rescaling the Euclidean gradient by the inverse of the metric $g^{\mathcal{P}}_{\theta_t}$ to compute the Riemannian gradient $\operatorname{grad} f(\theta_t)$. Since the metric $g^{\mathcal{P}}_{x} = (\lambda^c_x)^2 g^E$ is conformal to the Euclidean metric, the Riemannian gradient $\operatorname{grad} f(\theta_t)$ can be computed using the following simple formulation:

$$\operatorname{grad} f(\theta_t) = \left(1/\lambda^c_{\theta_t}\right)^2 \nabla f(\theta_t). \tag{60}$$

For **spherical optimization**, Meng et al. [189] apply Riemannian optimization to text embeddings on the unit hypersphere. Since the sphere inherits its metric from Euclidean space, only a tangent projection is required. For the radius-$1/\sqrt{c}$ sphere $\mathbb{S}^n_c$, it is

$$\operatorname{grad} f(\theta_t) = \left(I_{n+1} - c\,\theta_t \theta_t^T\right) \nabla f(\theta_t). \tag{61}$$

Riemannian Adagrad was introduced for Cartesian products of Riemannian manifolds, which include **mixed-curvature product spaces** $\mathcal{M}' = \times_{i=1}^{k} \mathcal{M}^{n_i}_{c_i}$ as a special case [18]. Let $g^i_t = (\operatorname{grad} f(\theta_t))^i \in \mathcal{T}_{\theta^i_t} \mathcal{M}^{n_i}_{c_i}$

Table 5 Representative neural operators across geometries. Entries distinguish tangent-space constructions from direct coordinate formulas; the final column locates the corresponding discussions in Section 2. A dash indicates that no separate construction is developed here. For product models, geometric primitives act factorwise, while neural architectures specify interactions between factors.

Operator family	Poincaré ball	Lorentz model	Spherical	Mixed / product	Discussed in
Distance, maps, and parallel transport	Möbius coordinate formulas (table 3)	Ambient Lorentz formulas (table 3)	Trigonometric formulas (table 4)	Componentwise over factors; stereographic chart shared with $\mathbb{S}^n_c$ [13, 250]	Section 2.2
Linear transformation	Tangent map at the origin [82]; hyperplane-score layer [243]	Basepoint tangent map [332]; direct Lorentz layers [45, 307]	Tangent map at reference poles [95]; angular convolution [177]	—	Section 2.3.1
Non-linear activation	Tangent activation at the origin [39, 82]	Basepoint tangent activation [332]; activation on spatial parts [17, 307]	Spherical ReLU [95]; spherical exponential [167]	—	Section 2.3.2
Functional components	Tangent β-concatenation [243]; Möbius addition and MLR [82]; gyromidpoint normalization [249, 272]	Spatial concatenation [227]; tangent-layer DropConnect [39]; feature dropout with time-coordinate completion [45]; Lorentzian residual [110]; transport-based normalization [17]	—	—	Section 2.3.3
Riemannian optimization	RSGD with conformal rescaling [203]	RSGD with Lorentz projection [205]	RSGD with tangent projection [189]	Riemannian Adagrad over factors [18]	Section 2.3.4

denote the i-th component of the Riemannian gradient of a smooth objective $f : \mathcal{M}' \to \mathbb{R}$. The update treats each manifold factor as an adaptive coordinate:

$$\theta^i_{t+1} \leftarrow \exp^i_{\theta^i_t}\left(-\eta \frac{g^i_t}{\sqrt{\varepsilon + \sum_{m=1}^{t} \|g^i_m\|^2_{\theta^i_m}}}\right), \tag{62}$$

where $\varepsilon > 0$ prevents a zero denominator and $\|g^i_m\|^2_{\theta^i_m} = g^i_{\theta^i_m}(g^i_m, g^i_m)$. Each factor shares one adaptive scale; one-dimensional Euclidean factors recover coordinatewise Adagrad. The projected version has a regret guarantee for geodesically convex objectives on complete factors with lower-bounded nonpositive curvature and compact geodesically convex feasible sets. Its update additionally projects onto the feasible set. The accompanying WordNet experiments evaluate adaptive optimizers on a single Poincaré ball.

Table 5 compares the operator families across geometries. Tangent-space constructions reuse Euclidean operations, while direct coordinate constructions enforce the manifold constraint within each layer. This distinction is separate from intrinsic geometry: an ambient formula can compute an intrinsic distance, and a valid manifold-valued layer need not preserve distances. In a product model, geometric primitives act componentwise, while neural architectures determine how information passes between factors, as discussed in Section 3.

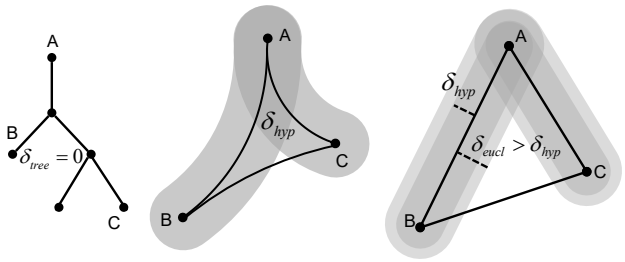


Figure 6 Schematic slim-triangle comparison. Tree triangles are 0-slim (left), curvature--1 hyperbolic triangles admit a uniform finite slimness bound (center), and Euclidean triangles can become arbitrarily thick under scaling (right).

2.4 Metrics

Geometric diagnostics connect data structure to representation geometry. Gromov δ-hyperbolicity measures global tree-likeness, sectional and Ricci curvature describe intrinsic geometry, and their discrete counterparts support graph analysis. Resistance distance adds a global graph metric derived from electrical networks. We introduce these quantities below and explain how to interpret them together.

2.4.1 Gromov Hyperbolicity

Gromov δ-hyperbolicity, introduced by Gromov [92], quantifies the degree to which an arbitrary metric space resembles a hyperbolic space. For a 4-tuple $(x, y, z, w) \in X^4$ of points in an arbitrary metric space X with distance function d, let $\delta_{4\text{pt}}(x, y, z, w)$ be half the difference between the two largest values among the following sums:

$$d(x, y) + d(z, w),\ d(x, z) + d(y, w),\ d(x, w) + d(y, z). \tag{63}$$

The four-point hyperbolicity of a metric space X is [27]

$$\delta_{\max}(X) := \sup_{(x,y,z,w)\in X^4} \delta_{4\text{pt}}(x, y, z, w).$$

We use δ_{avg} for the corresponding average over sampled or enumerated 4-tuples.

As shown in Figure 6, a complementary geodesic-triangle definition calls a geodesic space δ_{slim}-hyperbolic when every side of every geodesic triangle lies in the δ_{slim}-neighborhood of the other two sides [27, 92]. The four-point and slim-triangle definitions are equivalent up to universal constant factors, although their numerical constants need not coincide. The Euclidean space $\mathbb{R}^n$ for $n \geq 2$ has no finite global slim-triangle constant, whereas the curvature--1 Poincaré ball $\mathbb{P}^n_{-1}$ admits $\delta_{\text{slim}} = \log(1 + \sqrt{2})$. In the graph setting below, δ refers to the four-point convention, so a graph G is δ-hyperbolic when $\delta_{\max}(G) \leq \delta$. For a tree, $\delta_{\max}(G) = \delta_{\text{avg}}(G) = 0$. For a finite graph with a precomputed distance matrix, direct enumeration of all vertex 4-tuples requires $O(n^4)$ evaluations; $\delta_{\text{avg}}(G)$ can instead be estimated by sampling. Define $d_{\text{avg}}(G)$ and $d_{\max}(G)$ as the average and maximal pairwise distances, respectively. For $d_{\max}(G) > 0$, the diameter-normalized worst-case score is

$$\delta_{\text{rel}}(G) = \frac{2\delta_{\max}(G)}{d_{\max}(G)}.$$

This is the normalization used by Khrulkov et al. [129], who estimate scores on random subsets and

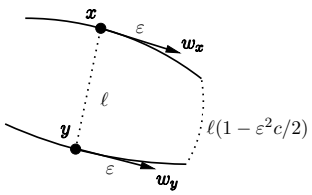


Figure 7 Sectional curvature through geodesic separation. Points initially ℓ apart move by ε in parallel-transported directions. Their separation contracts for positive sectional curvature c and expands for negative c, to leading order.

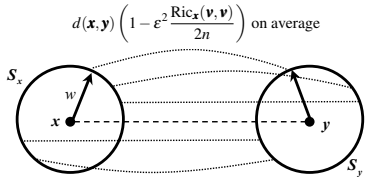


Figure 8 Ricci curvature through average geodesic separation. Centers are ℓ apart, and the spheres have radius ε. Parallel transport pairs directions w; averaging their endpoint separation measures Ricci curvature along v to leading order.

average the resulting values. An average-based alternative is

$$\bar{\delta}^{(\star)}_{\mathrm{rel}}(G) = \frac{2\delta_{\mathrm{avg}}(G)}{d_\star(G)}, \qquad \star \in \{\mathrm{avg}, \max\}.$$

The maximum measures the largest four-point departure, while the average summarizes typical sampled tuples. Small normalized values motivate evaluating hyperbolic embeddings; they do not guarantee an isometric embedding in a fixed-dimensional hyperbolic space.

2.4.2 Curvature

Curvature can describe either the bending of an embedded surface or the intrinsic behavior of distances on a manifold [200]. We distinguish these meanings before introducing graph analogues. A curved surface can sit inside a Euclidean ambient space while retaining a non-Euclidean intrinsic metric.

Principal, Gaussian, and Mean Curvature. For a smooth surface $S \subset \mathbb{R}^3$, the two *principal curvatures* describe extremal bending directions at a point p. Choose a unit normal N at p [149]. The principal curvatures of S at p, denoted by κ_1 and κ_2, are the minimum and maximum normal curvatures over unit tangent directions. Equivalently, they are the extremal curvatures of the normal-section curves obtained by intersecting S with planes through p that contain the normal vector N. Based on the principal curvatures, we define the *mean curvature* H and the *Gaussian curvature* K as follows:

$$H = \frac{\kappa_1 + \kappa_2}{2}, \quad K = \kappa_1 \cdot \kappa_2. \tag{64}$$

Reversing N changes the signs of the principal and mean curvatures while preserving their product. Gaussian curvature is intrinsic; it is positive for spherical surfaces and negative for hyperbolic surfaces. Mean curvature is extrinsic and depends on the surface's embedding [200].

Riemannian Curvatures. Sectional curvature resolves individual tangent planes, while Ricci curvature combines the planes containing a chosen direction [149, 200, 237]. We use a geodesic-separation interpretation [206, 207] on an n-dimensional Riemannian manifold $(\mathcal{M}^n, g)$ with distance d. Given orthonormal tangent vectors v and w_x at $x \in \mathcal{M}^n$, with $\varepsilon, \ell > 0$, let $y = \exp_x(\ell v)$, and let w_y be the parallel transport of w_x from x to y. As $(\varepsilon, \ell) \to 0$, the following equation holds:

$$d\left(\widehat{w}_x, \widehat{w}_y\right) = \ell\left(1 - \frac{\varepsilon^2}{2}K_x(v, w_x) + O(\varepsilon^3 + \varepsilon^2\ell)\right), \tag{65}$$

where $\widehat{w}_x = \exp_x(\varepsilon w_x)$, $\widehat{w}_y = \exp_y(\varepsilon w_y)$, and $K_x(v, w_x)$ is the sectional curvature of their spanning

plane, as shown in Figure 7. This indicates that positive sectional curvature leads to convergence of geodesics (shorter distance), while negative sectional curvature causes divergence (longer distance). Sectional curvature generalizes Gaussian curvature to higher dimensions by assigning a scalar to each two-dimensional tangent subspace. These values determine the Riemann curvature tensor through standard identities.

For a unit vector v and an orthonormal basis $\{v, e_2, \ldots, e_n\}$ of $\mathcal{T}_x\mathcal{M}^n$, the Ricci curvature is

$$\mathrm{Ric}_x(v, v) = \sum_{a=2}^{n} K_x(v, e_a),$$

or equivalently $(n-1)$ times the average over the orthogonal directions. For the full-sphere average in Figure 8, let w range uniformly over all unit vectors in $\mathcal{T}_x\mathcal{M}$. Pair $\exp_x(\varepsilon w)$ with $\exp_y(\varepsilon\, \mathrm{PT}_{x\to y} w)$, both of which are points on the manifold. As $(\varepsilon, \ell) \to 0$, their average distance satisfies [207]

$$\mathbb{E}_w\, d\Big(\exp_x(\varepsilon w), \exp_y(\varepsilon\, \mathrm{PT}_{x\to y} w)\Big) = \ell\left(1 - \frac{\varepsilon^2}{2n}\,\mathrm{Ric}_x(v, v) + O(\varepsilon^3 + \varepsilon^2\ell)\right). \tag{66}$$

The denominator is n because this average includes directions with a component along v. Restricting w to the unit sphere in $v^\perp$ instead gives the coefficient $1/[2(n-1)]$. On a two-dimensional Riemannian manifold, the Ricci tensor satisfies $\mathrm{Ric}_x = K(x) g_x$.

Discrete Curvature. *Ollivier-Ricci curvature* compares neighborhood measures through optimal transport, while *Forman-Ricci curvature* uses local incidence and weight information [200, 237, 253]. These constructions transfer different aspects of curvature to graphs. The definition of Ollivier-Ricci curvature is applicable in metric spaces. Given a metric space (X, d) equipped with a family of probability measures m_x, $x \in X$, the Ollivier-Ricci curvature of (X, d, m) for distinct points $x, y \in X$ is defined as:

$$\kappa(x, y) = 1 - \frac{W_1(m_x, m_y)}{d(x, y)}, \tag{67}$$

where $W_1(m_x, m_y)$ denotes transportation distance or Wasserstein distance of order 1 between the probability measures m_x and m_y, also known as the W_1 distance. Exact discrete optimal transport can be computationally expensive; its complexity depends on the solver and problem structure. Entropically regularized transport can instead be approximated with Sinkhorn iterations [59, 222, 248]. For an unweighted simple graph with m_x uniform over the neighbors of x and no mass retained at x, degree- and triangle-count bounds give a local surrogate [124, 265]. Once the endpoint degrees and common-neighbor count are available, the surrogate takes constant time per edge:

$$\widehat{\kappa}(x, y) := \frac{\kappa_{\mathrm{lb}}(x, y) + \kappa_{\mathrm{ub}}(x, y)}{2} \tag{68}$$

where

$$\kappa_{\mathrm{lb}}(x, y) := -\left(1 - \frac{1}{d_x} - \frac{1}{d_y} - \frac{\#(x, y)}{d_x \wedge d_y}\right)_+ - \left(1 - \frac{1}{d_x} - \frac{1}{d_y} - \frac{\#(x, y)}{d_x \vee d_y}\right)_+ + \frac{\#(x, y)}{d_x \vee d_y}, \tag{69}$$

$$\kappa_{\mathrm{ub}}(x, y) := \frac{\#(x, y)}{d_x \vee d_y}, \tag{70}$$

and $\kappa_{\mathrm{lb}}(x, y) \le \kappa(x, y) \le \kappa_{\mathrm{ub}}(x, y)$. Here $\#(x, y)$ denotes the number of triangles containing the edge (x, y), $a \wedge b = \min(a, b)$, $a \vee b = \max(a, b)$, and d_x denotes the degree of node x.

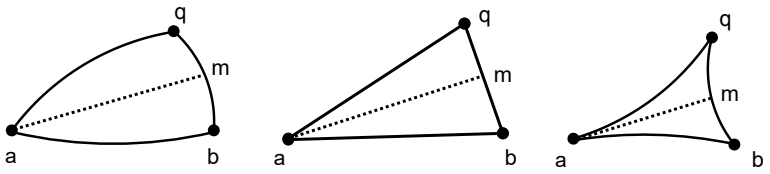


Figure 9 Graph sectional curvature from triangle medians [94]. For the midpoint m of bq, the median am is relatively longer in a cycle-like space (left), satisfies the Euclidean median relation in a flat space (center), and is relatively shorter in a tree-like space (right), corresponding respectively to positive, zero, and negative values of $\xi_G(a, b, q)$.

The definition of Forman-Ricci curvature is applicable within the framework of weighted CW complexes (or cellular complexes). In the one-dimensional case, i.e., for weighted graphs or networks, the Forman-Ricci curvature takes the following form:

$$F(e) = w_e \left(\frac{w_{v_1}}{w_e} + \frac{w_{v_2}}{w_e} - \sum_{\substack{e' \sim v_1 \\ e' \neq e}} \frac{w_{v_1}}{\sqrt{w_e w_{e'}}} - \sum_{\substack{e'' \sim v_2 \\ e'' \neq e}} \frac{w_{v_2}}{\sqrt{w_e w_{e''}}} \right), \tag{71}$$

where e is the edge between nodes v_1 and v_2, w_e is the weight of edge e, w_{v_1} and w_{v_2} are the node weights, and $e' \sim v_1$ and $e'' \sim v_2$ denote incident edges excluding e itself. With unit node and edge weights, this formula reduces to $F(e) = 4 - d_{v_1} - d_{v_2}$, making its sensitivity to endpoint degrees explicit. Ollivier-Ricci curvature additionally depends on the transport between neighborhoods. Their different structural sensitivities motivate joint analysis [288].

Gu et al. [94] introduce a graph sectional-curvature statistic motivated by triangle comparison. For a geodesic triangle Δabq with midpoint m of bq, compare its median with the Euclidean median relation, as illustrated in Figure 9:

$$\xi_G(a, b, q) = d_{a,m}^2 + \frac{d_{b,q}^2}{4} - \frac{d_{a,b}^2 + d_{a,q}^2}{2}, \tag{72}$$

where $d_{a,b}$ denotes geodesic distance. Within a suitable convex neighborhood, curvature bounded below (respectively, above) by zero gives a non-negative (respectively, non-positive) comparison value. For a connected unweighted graph $G = (V, E)$, choose a node m and distinct neighbors b, q. The graph statistic averages the same expression over reference nodes:

$$\xi_G(m; b, q) = \frac{1}{|V| - 1} \sum_{a \neq m} \frac{\xi_G(a, b, q)}{2 d_{a,m}}, \tag{73}$$

where $d_{x,y}$ is the graph shortest-path distance. The midpoint interpretation applies directly when b, m, q form a shortest path. The statistic vanishes on a line, is positive for simple cycles, and becomes negative at branching nodes in a tree. It is a distance-based diagnostic, expressed at the scale of the graph metric.

2.4.3 Resistance Distance

Resistance distance measures the availability of alternative paths between graph nodes. For an unweighted graph, assign unit resistance to each edge; the resistance distance Ω_{ij} is the effective electrical resistance between i and j [12]. On a tree it equals shortest-path distance, since current

follows a unique path. Additional paths can lower effective resistance, which makes it sensitive to connectivity beyond the shortest path. Resistance distance defines a graph metric [311], while effective resistance has also been used to construct measures of global graph cyclicity [312]. For a connected unweighted graph $G = (V, E)$ with $N = |V|$, the resistance distance Ω_{ij} can be calculated from the Laplacian matrix L, defined as:

$$L = D - A, \tag{74}$$

where D is the degree matrix, a diagonal matrix with D_{ii} representing the degree of node i; and A is the adjacency matrix, with $A_{ij} = 1$ if vertices i and j are connected, and 0 otherwise. The effective resistance Ω_{ij} of nodes i and j is given by:

$$\Omega_{ij} = (F^{-1})_{ii} - 2(F^{-1})_{ij} + (F^{-1})_{jj}, \tag{75}$$

where $F = L + J/N$, and $J \in \mathbb{R}^{N\times N}$ has all entries equal to 1. The connectedness assumption makes F invertible. Equivalently, $\Omega_{ij} = L^+_{ii} - 2L^+_{ij} + L^+_{jj}$, where L^+ is the Moore–Penrose pseudoinverse. The same expression extends to connected weighted graphs when the edge resistance is $r_{ij} = 1/w_{ij}$ and L is replaced by the weighted Laplacian L^W [73]. The weighted Laplacian L^W is defined as $L^W = S - W$, where W is the weight matrix $W = (w_{ij})$ and S is the diagonal matrix of strengths with elements $S_{ii} = \sum_{j=1}^{N} w_{ij}$. The effective resistance Ω_{ij} between nodes i and j in a weighted graph is then given by:

$$\Omega_{ij} = (L^W)^+_{ii} - 2(L^W)^+_{ij} + (L^W)^+_{jj}. \tag{76}$$

2.4.4 Structural Diagnostics

These diagnostics address different modeling questions. Hyperbolicity summarizes metric tree-likeness, discrete curvature compares local or neighborhood structure, and resistance measures alternative connectivity. We relate them to the applications reviewed in Section 4.

Hierarchical and tree-like structure. Gromov δ-hyperbolicity measures how closely four-point distances satisfy the tree-metric relation. Normalizing by the same pairwise-distance statistic supports comparisons across scales. The mean describes typical sampled quadruples, while the maximum captures the largest departure. Negative graph sectional curvature complements this summary by identifying branching neighborhoods. Embedding distortion and dimension also depend on the metric class and target-space constraints [22, 235, 277].

Metric and hyperbolic embedding theory. Classical metric embedding results characterize the distortion incurred when finite metric spaces are mapped into normed spaces [26, 171]. For hyperbolic targets, Bonk & Schramm [22] study embeddings of Gromov-hyperbolic spaces, while tree- and graph-specific constructions and precision–dimension trade-offs are developed in [235, 238, 277]. These results explain why hyperbolicity informs geometry selection without fixing an embedding dimension or distortion guarantee by itself.

Cyclic or periodic structure. Positive graph sectional-curvature values and resistance-based cyclicity summaries provide complementary evidence for cyclic relations [94, 312]. Spherical factors offer one way to represent such relations within a product model [250]. Periodic variables also call for attention to topology: a circle or a product of circles represents periodicity directly, so periodicity alone does not determine a positive sectional curvature.

Directional structure. Direction and angle are structural signals rather than quantities returned by the curvature estimators above. They are read from the data representation itself, for example

from normalized embeddings whose informative content lies in angle rather than magnitude [189], and from geometric descriptors such as the angles and torsions of three-dimensional molecular graphs [178].

Heterogeneous local curvature. Edgewise Ollivier-Ricci and Forman-Ricci values, together with node-centered sectional-curvature statistics, show how geometry varies across a graph. Their distributions reveal variation that a single mean can hide, and their different sensitivities make joint inspection useful [237, 288]. A broad or multimodal distribution motivates evaluating models with several geometric factors [13, 94].

Curvature, hyperbolicity, and resistance distance provide complementary evidence about data structure. These metrics can inform model selection and embedding analysis when interpreted jointly with downstream performance, dimensionality, and optimization behavior.

3 From Hyperbolic to Mixed-Curvature Geometric Learning

A hierarchy of concepts, a panoramic image, and a graph containing both branches and cycles call for different geometric assumptions. Hyperbolic representations accommodate hierarchical expansion, spherical methods encode directions or process signals on a sphere, and mixed-curvature methods combine complementary geometric factors. We organize the methods by architecture, from shallow embeddings and neural layers to GNNs, Transformers, generative models, and foundation models, followed by metric, contrastive, and reinforcement learning. Within each family, we compare where geometry enters the computation, which structural property it represents, and what operations or constraints it introduces.

Table 6 provides a navigation map across architecture, foundation-model intervention, learning paradigm, and curvature. A method can occupy several rows when these roles overlap. The spherical column includes both curved representations and spherical signal domains; the distinction is made explicit in the corresponding subsections. Mixed-curvature entries combine different curvature signs, whereas products or mixtures containing only negatively curved components are grouped in the hyperbolic category. Methods that impose geometry on model parameters are identified separately from methods that curve learned representations.

3.1 Shallow Model

Shallow models learn an embedding for each object directly, as in word representation [125, 193, 219, 266] and multi-relational modeling [24, 204]. Their central choice is the relation that the embedding should preserve: proximity and hierarchical order, angular similarity, or a combination of graph-distance patterns. This direct parameterization makes the geometric objective explicit; representing an unseen object requires estimating an additional embedding or introducing an encoder.

3.1.1 Hyperbolic Shallow Model

Hyperbolic shallow models differ in how they represent a relation. PoinEmb and LorEmb learn distance-based proximity, MuRP adds relation-specific transformations, and EntailCone represents directed order through nested regions. The first pair mainly differs in its coordinates and optimization, while the latter two change the relation being scored. **Distance-based hierarchy.** PoinEmb [203] learns d-dimensional representations of hierarchical data in the unit Poincaré ball $\mathbb{P}^d$ using RSGD. For an object set V, the model learns $\Theta = \{\boldsymbol{\theta}_v \in \mathbb{P}^d : v \in V\}$ and brings semantically related objects close under the Poincaré distance $d_{\mathbb{P}}$ with curvature $c = -1$. For instance, the loss function in the hierarchy

Table 6 Reading map for geometric learning methods by architecture, foundation-model intervention, learning paradigm, and curvature. Methods may appear in more than one row; a dash means no example is listed.

Group	Model or intervention family	Hyperbolic	Spherical	Mixed-curvature
Architecture families	Shallow embedding	PoinEmb [203], LorEmb [205], MuRP [16], EntailCone [81]	JoSE [189]	McRep [94]
	Feed-forward layers	HNN [82], HNN++ [243], FHNN [45]	SNN [95]	—
	CNNs and geometric heads	Hyperbolic MLR heads [129], Poincaré convolution [243, 249], HCNN [17]	NormFace [279], SCNN [55]	—
	Recurrent network	Hyperbolic RNN [82], HTLSTM [239]	—	—
	Graph neural network	HGCN [39], HGNN [175], LGCN [332], H2H-GCN [60]	SphereNet [178], DeepSphere [63]	κ-GCN [13], SELFMGNN [257], GIL [338], Spectro-Riemannian GNN [93], GraphMoRE [102]
	Attention and Transformers	HAT [96], FHNN attention [45], Hypformer [307]	nGPT [180], Hyper-SET [115]	FPS-T [51], MMA [138] (mixed-geometry relation maps)
	Generative model	P-VAE [187], HGAN [145], HGDM [290]	S-VAE [62], vMF-VAE [295], SphereGAN [214], SphericGAN [44]	mVAE [250]
Foundation-model intervention points	Backbone and pretraining	HELM-D [106], HyperHELM [274]	nGPT [180]	—
	Prompt tuning	Hyperbolic prompt tuning [43]	—	—
	Parameter-efficient tuning	HypLoRA [308]	—	MoSLoRA [325]
	Mixture-of-experts	HELM-MiCE [106]	—	—
Parameter-space merging	Expert merging	CAMEx [202] uses Riemannian parameter geometry; see Table 9.		
Learning paradigms	Metric learning	HyperML [267], UHML [298], hyperbolic ViT embeddings [74], HIER [130]	SEC [326], SEE [146]	—
	Contrastive learning	HGCL [173], HCL [324], object–scene contrast [85], HiHPQ [226] (product-hyperbolic)	—	SELFMGNN [257], ProGDM [287]
	Reinforcement learning	Hyperbolic latent RL [38], ACE-HGNN [80]	Hyperspherical normalization [147], G-RL [5]	—

embedding task is defined as:

$$L(\Theta) = -\sum_{(u,v)\in D} \log \frac{\exp(-d_{\mathbb{P}}(\boldsymbol{\theta}_u, \boldsymbol{\theta}_v))}{\exp(-d_{\mathbb{P}}(\boldsymbol{\theta}_u, \boldsymbol{\theta}_v)) + \sum_{v'\in N(u)} \exp(-d_{\mathbb{P}}(\boldsymbol{\theta}_u, \boldsymbol{\theta}_{v'}))}, \tag{77}$$

where D is the set of hypernymy relations and $N(u)$ is a set of negative samples for u; the positive term is included explicitly in the denominator. For lexical entailment, the plausibility of a hierarchical relation (u, v) is measured by

$$\text{score}(u, v) = -\left(1 + \alpha(\|\boldsymbol{\theta}_v\| - \|\boldsymbol{\theta}_u\|)\right) d_{\mathbb{P}}(\boldsymbol{\theta}_u, \boldsymbol{\theta}_v), \tag{78}$$

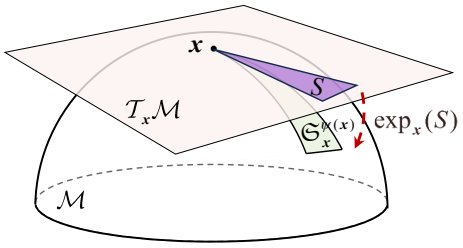


Figure 10 Entailment cone at x on a manifold $\mathcal{M}$ [81]. The tangent-space cone $S_x^{\psi(x)} \subseteq \mathcal{T}_x\mathcal{M}$ has generatrix angle $\psi(x)$, and the exponential map sends it to the geodesic cone $\exp_x(S_x^{\psi(x)})$ on the manifold. Points inside the mapped cone are represented as descendants of x.

where the term $\alpha(\|\boldsymbol{\theta}_v\| - \|\boldsymbol{\theta}_u\|)$ penalizes cases where v is lower in the embedding hierarchy, i.e., has a larger embedding norm than u.

LorEmb [205] retains the distance-based approach and moves learning to the Lorentz model $\mathbb{L}^n$, where tangent gradients and exponential-map updates admit simple expressions. Its practical benefit is a better-conditioned optimization formulation for the evaluated hierarchy tasks. Poincaré coordinates remain useful for visualization, and the isometries $\Pi^c_{\mathbb{L}\to\mathbb{P}}$ and $\Pi^c_{\mathbb{P}\to\mathbb{L}}$ allow learning and display to use different coordinate models. Thus, the coordinate change preserves the underlying hyperbolic geometry while changing the numerical computation.

Relation transformations and order regions. MuRP [16] scores multi-relational knowledge graphs by allowing each relation to transform the entity embeddings. In the Poincaré ball $\mathbb{P}^n_c$, it combines a diagonal tangent transformation with a hyperbolic translation:

$$\phi(e_s, r, e_o) = -d^c_{\mathbb{P}}\left(\exp^c_{\mathbf{0}}\left(\mathbf{R}\log^c_{\mathbf{0}}(\boldsymbol{h}_s)\right), \boldsymbol{h}_o \oplus_c \boldsymbol{r}_h\right)^2 + b_s + b_o, \tag{79}$$

where $\boldsymbol{h}_s, \boldsymbol{h}_o \in \mathbb{P}^n_c$ are entity embeddings, $\mathbf{R}$ is a relation-specific diagonal matrix, $\boldsymbol{r}_h$ is the relation translation vector, and b_s, b_o are entity biases. This parameterization supports different relational patterns with a compact diagonal operator, while learning the entity states directly from triples.

EntailCone [81] makes directional order explicit: a tangent-space cone $S \subseteq \mathcal{T}_x\mathcal{M}$ maps to a geodesic region $\exp_x(S) \subset \mathcal{M}$ that contains descendants of a node in a hierarchical directed acyclic graph. Cone inclusion encodes transitivity, whereas proximity scores require a separate rule to recover direction. Figure 10 illustrates the construction in the unit Poincaré ball $\mathbb{P}^n$. Given a dataset $\mathcal{X}$ with entailment relations $(x, y) \in \mathcal{X}$, where each pair indicates that x entails y, the S-cone at x is defined as:

$$\mathfrak{S}_x^{\psi(x)} = \{x\} \cup \{y \in \mathbb{P}^n \setminus \{x\} \mid \Xi(x, y) \le \psi(x)\}, \tag{80}$$

where $\Xi(x,y) = \arccos\left(\frac{\langle x,y\rangle(1+\|x\|^2) - \|x\|^2(1+\|y\|^2)}{\|x\|\cdot\|x-y\|\sqrt{1+\|x\|^2\|y\|^2-2\langle x,y\rangle}}\right)$, and $\psi(x) = \arcsin\left(\eta_{\text{cone}}\frac{1-\|x\|^2}{\|x\|}\right)$ is the generatrix angle with cone parameter $\eta_{\text{cone}} > 0$. For distinct x and y, this expression is defined for $x \neq 0$ and $0 \le \eta_{\text{cone}}(1 - \|x\|^2)/\|x\| \le 1$. The relation (x, y) is considered valid if $y \in \mathfrak{S}_x^{\psi(x)}$.

3.1.2 Spherical Shallow Model

Spherical shallow models make direction the shared basis for training and retrieval. JoSE [189] jointly learns word and paragraph embeddings on the unit hypersphere $\mathbb{S}^n$, aligning its training objective with cosine similarity at inference. Fixing the norm removes magnitude from the representation, which suits directional similarity while leaving hierarchy depth to be expressed through other

structure. JoSE models a center word u from paragraph d and a context word v from u as

$$p(u|d) \propto \exp(\cos(u,d)), \quad p(v|u) \propto \exp(\cos(v,u)), \tag{81}$$

where $\|u\| = \|v\| = \|d\| = 1$, and $\cos(\cdot,\cdot)$ denotes the cosine of the angle between two vectors on $\mathbb{S}^n$. For a positive tuple (u,v,d), JoSE replaces the center word with a sampled negative u' and optimizes the following margin loss with spherical RSGD:

$$L(u,v,d) = \max\left(0, L_s(u,v,d)\right), \tag{82}$$

$$L_s(u,v,d) = m - \cos(v,u) - \cos(u,d) + \cos(v,u') + \cos(u',d), \tag{83}$$

where $m > 0$ is the margin.

3.1.3 Mixed-curvature Shallow Model

Mixed-curvature shallow models distribute distance structure across several factors instead of committing every relation to one curvature sign. McRep [94] uses the product space $\mathcal{M}' = \times_{i=1}^{k} \mathcal{M}_{c_i}^{n_i}$, where each factor belongs to $\{\mathbb{E}^{n_i}, \mathbb{S}_{c_i}^{n_i}, \mathbb{L}_{c_i}^{n_i}\}$. Its graph-curvature diagnostic in Equation (73) guides the choice of product signature, while learned component curvatures adjust the distance scale within that signature. The graph-distance objective is

$$\mathcal{L}(x) = \sum_{1 \le i < j \le n} \left| \frac{d'(\boldsymbol{x}_i, \boldsymbol{x}_j)^2}{d_G(X_i, X_j)^2} - 1 \right|, \tag{84}$$

where $d'(\boldsymbol{x_i}, \boldsymbol{x_j})^2 = \sum_{m=1}^{k} d^m(\boldsymbol{x_i^m}, \boldsymbol{x_j^m})^2$ sums squared factor distances. Embedding coordinates and curvature scales are optimized jointly, while the number, type, and dimension of the factors remain architectural choices. Given $p, q \in \mathbb{S}_c^n$ or $p, q \in \mathbb{L}_c^n$ with radius $R = 1/\sqrt{|c|}$, the distance scales as $d_c(p,q) = R\, d_{\mathrm{sgn}(c)}(p/R, q/R)$. Here p/R and q/R lie on the corresponding unit-curvature manifold, and $d_{\mathrm{sgn}(c)}$ denotes its geodesic distance. Thus, the curvature can be optimized through the radius while normalized coordinates are stored on unit manifolds.

Choosing a shallow representation. Distance-based hyperbolic embeddings suit relations organized around a dominant hierarchy; relation operators and cones add different forms of directionality. Spherical embeddings suit objectives defined by angular similarity. Product embeddings provide separate factors for heterogeneous distance patterns, with the additional choice of how to allocate dimensions and curvature. The task's relation structure therefore determines both the geometry and the score used to train it.

3.2 Neural Network

Neural networks replace directly optimized object embeddings with a learned feature map. Their geometric design must specify both the space of hidden states and the transformations between them. We first compare hyperbolic feed-forward layers and residual connections, then examine spherical layers that preserve a fixed-radius state space.

3.2.1 Hyperbolic Neural Network

Hyperbolic layers differ in the operation that replaces a Euclidean affine map. HNN lifts tangent-space transformations to the Poincaré ball, HNN++ builds outputs from hyperplane-distance scores, and FHNN parameterizes transformations directly in Lorentz coordinates. Residual networks

Table 7 Comparison of hyperbolic neural layer designs. Each row identifies the geometric operation, its modeling role, and the computation or constraint it introduces. LResNet provides a residual operator that can be combined with Lorentz feature layers.

Method	Geometric operation	Modeling role	Computation or constraint
HNN [82]	Tangent linear map and Möbius bias	Reuses Euclidean transforms with hyperbolic states	Logarithmic and exponential maps around the linear transform
HNN++ [243]	Hyperplane-distance FC layer	Geometric score interpretation with a vector and scalar per output	Hyperbolic score evaluation and norm-aware concatenation
FHNN [45]	Relaxed Lorentz transform	Direct parameterization of manifold-valued features	Spatial normalization and reconstruction of the time coordinate
LResNet [110]	Weighted Lorentzian centroid	Combines skip and transformed branches on the manifold	Positive weights, Lorentz normalization, and compatible branches

introduce a further choice: how to combine two manifold-valued branches. Table 7 compares these choices and their computational consequences. **Tangent transformations and hyperplane scores.** HNN [82] generalizes multinomial logistic regression (MLR) and feed-forward neural networks (FFNNs) to $\mathbb{P}_c^n$. Its layer combines the tangent-based transformation $\mathrm{LT}_{\mathbb{P}}$ in Equation (29), the activation in Equation (35), and, when needed, Möbius concatenation in Equation (42). For $f_{in} \in \mathbb{P}_c^n$, it produces $f_{out} \in \mathbb{P}_c^n$ through

$$f_{out} = \sigma_{\mathbb{P}}^{c \to c} \left(\mathrm{LT}_{\mathbb{P}} \left(f_{in} \right) \right). \tag{85}$$

This construction reuses Euclidean matrix operations between logarithmic and exponential maps. For the MLR in Equation (51), a class normal at $\boldsymbol{p}_k$ can be optimized through an origin-tangent parameter: $\boldsymbol{a}_k = \mathrm{PT}_{\mathbf{0} \to \boldsymbol{p}_k}^c(\boldsymbol{a}_k')$, with $\boldsymbol{a}_k' \in \mathcal{T}_{\mathbf{0}} \mathbb{P}_c^n$.

HNN++ [243] instead starts from the signed distance to a discriminative hyperplane. Its MLR uses a direction vector and a scalar offset r_k, removing the redundant vector-valued bias parameterization. The resulting scores in Equation (54) define the fully connected layer in Equation (30), whose output coordinates preserve their hyperplane-distance interpretation. This changes the layer's contour geometry as well as its parameterization. Its β-concatenation in Equation (44) controls the expected norm when combining feature vectors and supports the convolutional layers reviewed below.

Lorentz transformations and residual combination. FHNN [45] constructs layers on $\mathbb{L}_c^n$ by relaxing Lorentz transformations, including rotations and boosts, into trainable manifold-preserving maps. The authors characterize origin-tangent linear maps as a relaxation of Lorentz rotations; an arbitrary tangent matrix is therefore not itself a Lorentz isometry. The transformations in Equations (32) and (46) learn spatial coordinates and enforce the Lorentz constraint through the time coordinate. Their residual-like operation inserts the skip input into the final linear layer of a block, as shown in Equation (48).

LResNet [110] treats residual combination as a separate operation. It forms a positively weighted ambient sum of the input and transformed branch and normalizes it onto the Lorentz hyperboloid. This weighted-centroid construction avoids exponential maps, logarithmic maps, and parallel transport within the skip operation, and supports Lorentz CNNs, GNNs, and graph Transformers. Both branches must have matching dimensions and curvature; the weights control their contribution, while an optional scaling operation controls the distance from the origin.

3.2.2 Spherical Neural Network

Spherical neural layers constrain hidden-state magnitude and learn through direction and angular separation. In adversarial domain adaptation, Gu et al. [95] construct a spherical MLP on $\mathbb{S}_c^{n-1}$. Their perceptron combines the transformation in Equation (34) with the activation in Equation (40). For $f_{in} \in \mathbb{S}_c^{n_1-1}$ and $f_{out} \in \mathbb{S}_c^{n_2-1}$,

$$f_{out} = \mathrm{ReLU}_{\mathbb{S}}\left(\mathrm{LT}_{\mathbb{S}}\left(f_{in}\right)\right). \tag{86}$$

The classifier uses

$$p(y=k \mid z) = \frac{\exp(w_k^T z + b_k)}{\sum_{j=1}^{K} \exp(w_j^T z + b_j)}, \qquad k = 1, 2, \ldots, K, \tag{87}$$

where $w_k \in \mathbb{R}^n$, $\|w_k\| = 1$, and $b_k \in [-r, r]$. The equation $w_k^T z + b_k = 0$ defines the zero-level hypersurface of the kth class score on $\mathbb{S}_c^{n-1}$. Multiclass boundaries occur where two maximal scores are equal, satisfying $(w_k - w_j)^T z + (b_k - b_j) = 0$. Here $r = 1/\sqrt{c}$ is the spherical radius, so the constraint can be enforced by $b_k = r \tanh(b_k')$, where $b_k' \in \mathbb{R}$ is learned. To obtain a proper estimate of r, the following bound is employed:

$$r \geq \frac{K-1}{K} \ln\left(\frac{(K-1)P_w}{1-P_w}\right), \tag{88}$$

where P_w is a hyperparameter indicating the expected minimal classification probability of the class center.

Choosing neural operations. Tangent-based layers provide a direct route from existing Euclidean transformations to curved hidden states. Hyperplane-based layers give classifier scores and feature coordinates a shared geometric interpretation. Direct Lorentz layers and centroid residuals reduce chart changes in the operations they replace, while spherical layers make angular separation and radius the relevant design variables. Layer choice therefore depends on the required transformation and state constraint, alongside the curvature of the representation.

3.3 Convolutional Neural Network

Convolutional neural networks (CNNs) introduce geometry at two distinct locations: the learned feature space and the domain on which a signal is sampled. Hyperbolic feature maps target hierarchical visual representations. Spherical methods either normalize learned features or define rotation-equivariant operators for signals on a sphere.

3.3.1 Hyperbolic Convolutional Neural Network

The main distinction among hyperbolic CNNs is how early curved features enter the network. A hyperbolic output head changes the final embedding and classifier; hyperbolic convolutions make geometry part of feature extraction at every receptive field. The latter additionally require compatible normalization, concatenation, and residual operations. **Output geometry.** Khrulkov et al. [129] retain a conventional CNN backbone and map its final features into $\mathbb{P}_c^n$ with an exponential map. The Poincaré MLR in Equation (51) then provides the classification head. This approach permits reuse of established image encoders and reports gains on the evaluated image tasks, while concentrating the geometric intervention in the final representation.

Geometry throughout the encoder. HNN++ [243] combines features in a receptive field with β-concatenation and applies its Poincaré fully connected transformation. For a $K \times K$ field X_{kl} contain-

ing features in $\mathbb{P}_c^{C_{\text{in}}}$, the output at (k, l) is

$$h_{kl} = \mathrm{LT}_{\mathbb{P}}\left(\mathrm{Concat}_{\beta}(X_{kl}); \boldsymbol{Z}, \boldsymbol{r}, c\right), \tag{89}$$

where $\mathrm{LT}_{\mathbb{P}}(\boldsymbol{x}; \boldsymbol{Z}, \boldsymbol{r}, c)$ denotes the linear transformation in Equation (30) from $\mathbb{P}_c^{K^2 C_{\text{in}}}$ to $\mathbb{P}_c^{C_{\text{out}}}$, and Concat_{β} denotes the β-concatenation in Equation (44) of the K^2 vectors in X_{kl}. Poincaré ResNet [272] builds these components into a deep visual encoder. It addresses depth through identity-based initialization, midpoint batch normalization, and manually derived backward operations that reduce the size of the computation graph. This illustrates the engineering cost of extending a geometric layer into a trainable deep network.

Skliar & Weiler [249] retain a standard Euclidean convolution kernel and change its input and output coordinates. For a hyperbolic feature field $f : \mathbb{R}^2 \to \mathbb{P}_c^n$, the convolution is

$$\mathrm{conv2d}_{\otimes}^{c}(f) = \exp_{\mathbf{0}}^{c}\left(\mathrm{conv2d}\left(\log_{\mathbf{0}}^{c}(f)\right)\right), \tag{90}$$

where conv2d represents the standard Euclidean 2D convolution operation. This construction exposes existing convolution implementations through logarithmic and exponential maps. In contrast, the Lorentz CNN of Bdeir et al. [17] combines receptive-field features directly through Lorentz concatenation and a manifold-preserving affine layer. Given a hyperbolic input feature map $x = \{x_{h,w} \in \mathbb{L}_c^n\}_{h,w=1}^{H,W}$ representing image pixels, the receptive field starting at (h', w') is $\{x_{h'+\tilde{h}-1, w'+\tilde{w}-1}\}_{\tilde{h}=1,\tilde{w}=1}^{\tilde{H},\tilde{W}}$. The Lorentz convolutional layer is defined as:

$$y_{h,w} = \mathrm{LT}_{\mathbb{L}}\left(\mathrm{HCat}\left(\{x_{h'+\tilde{h}-1, w'+\tilde{w}-1}\}_{\tilde{h}=1,\tilde{w}=1}^{\tilde{H},\tilde{W}}\right)\right), \tag{91}$$

where HCat is the Lorentz concatenation from Equation (45) [228], and $\mathrm{LT}_{\mathbb{L}}$ adapts the FHNN transformation in Equation (32) [45]. The layer omits the original spatial normalization and uses a separate Lorentz batch-normalization module. Thus, manifold-preserving convolution and feature-scale control are implemented as distinct operations.

3.3.2 Spherical Convolutional Neural Network

Spherical feature normalization and spherical signal processing solve different problems. The former aligns learned representations with angular similarity; the latter accounts for rotations of the input domain. **Normalized features.** NormFace [279] normalizes feature vectors and classifier weights for face recognition. Comparisons then depend on direction, and gradients through feature normalization are tangent to the sphere. This changes the representation and classification objective while retaining conventional image convolutions.

Spherical signal domains. SCNN [55] defines correlations for signals on $\mathbb{S}^2$ and the rotation group $\mathrm{SO}(3)$. For $R \in \mathrm{SO}(3)$, rotation acts on a spherical signal as $[L_R f](x) = f(R^{-1}x)$. The first correlation maps a K-channel spherical signal to a function of rotation:

$$[\psi \star f](R) = \langle L_R \psi, f \rangle = \int_{\mathbb{S}^2} \sum_{k=1}^{K} \psi_k(R^{-1}x) f_k(x)\, dx. \tag{92}$$

Similarly, for signals $f, \psi : \mathrm{SO}(3) \to \mathbb{R}^K$, and $R, Q \in \mathrm{SO}(3)$, the correlation of signals is:

$$[\psi \star f](R) = \langle L_R \psi, f \rangle = \int_{\mathrm{SO}(3)} \sum_{k=1}^{K} \psi_k\left(R^{-1}Q\right) f_k(Q)\, dQ. \tag{93}$$

These correlations are rotation equivariant:

$$[\psi \star [L_Q f]](R) = [L_Q [\psi \star f]](R). \tag{94}$$

The output retains orientation as a group coordinate, allowing subsequent layers to process how patterns transform under rotation. Compared with feature normalization alone, this requires sampling and filtering functions over the spherical or rotation-group domain.

Choosing a convolutional design. A curved output head is a direct choice when the task mainly requires a geometric embedding or classifier. Curved encoders make that assumption part of local feature extraction and require compatible operations throughout the network. For panoramic or other spherical signals, rotation-equivariant correlation addresses the input domain itself. Hierarchical organization concerns feature geometry, while rotation equivariance concerns how a signal changes under transformations, so the two can be evaluated independently.

3.4 Recurrent Neural Network

Recurrent neural networks (RNNs) must preserve a geometric hidden state across repeated temporal updates. The main design choice is whether to use a direct recurrent transition or an additional gated memory. Hyperbolic RNNs generalize the transition with Möbius operations; hyperbolic LSTMs also specify where the gates and cell state are computed.

In a standard RNN, the update rule of the hidden state h_t is typically defined as $h_{t+1} = \sigma(Wh_t + Ux_t + b)$, where σ denotes a non-linear activation. Ganea et al. [82] replace the affine transition with Poincaré operations. Let $h_t \in \mathbb{P}_c^m$, $x_t \in \mathbb{P}_c^d$, $\mathbf{W} \in \mathbb{R}^{m\times m}$, $\mathbf{U} \in \mathbb{R}^{m\times d}$, and $\mathbf{b} \in \mathbb{P}_c^m$. The recurrent update is

$$h_{t+1} = \sigma_{\mathbb{P}}^{c\to c}\left((\mathbf{W} \otimes_c h_t \oplus_c \mathbf{U} \otimes_c x_t) \oplus_c \mathbf{b}\right) \in \mathbb{P}_c^m, \tag{95}$$

where $\otimes_c$ and $\oplus_c$ denote Möbius matrix-vector multiplication and Möbius addition, respectively. For a Euclidean input $x_t^E \in \mathbb{R}^d$, the same update uses $x_t = \exp_{\mathbf{0}}^c(x_t^E) \in \mathbb{P}_c^d$. Sawhney et al. [239] introduce the Hierarchical Time-Aware Hyperbolic LSTM (HTLSTM) for irregular streams of online financial text. The model combines time-aware hyperbolic recurrence within a day with a recurrent stage across days. Its released implementation[1] forms recurrent input combinations with Möbius operations, computes sigmoid gates after an origin logarithmic map, and adjusts memory using timestamps before combining it through Möbius operations. This adds two choices to the basic hyperbolic RNN: how elapsed time changes retained information and how recurrent modules organize different temporal scales. The comparison therefore concerns both the geometry of the transition and the treatment of temporal information. A basic hyperbolic RNN evolves one manifold-valued state through Möbius operations. HTLSTM adds time-dependent memory adjustment and hierarchical temporal modeling, while using tangent coordinates to evaluate gates. HTLSTM thus combines a hierarchical representation space with the irregular timing of a sequence.

3.5 Graph Neural Network

Graph neural networks (GNNs) learn by transforming node features and aggregating messages along edges, supporting node classification [133], recommendation [83, 314], and molecular and biological modeling [88, 254]. Geometry can enter the message descriptors, the aggregation rule, or the learned node states. Hyperbolic and mixed-curvature GNNs adapt the representation and aggregation to

[1] Author implementation: TimeLSTMHyp and HTLSTM.

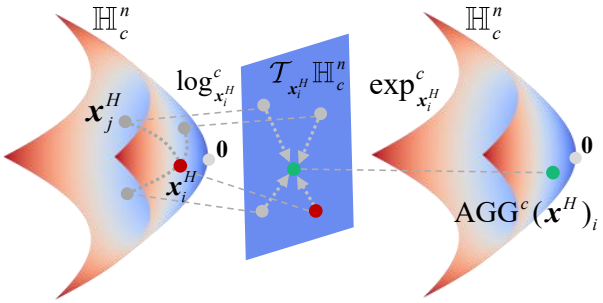


Figure 11 HGCN neighborhood aggregation [39]. Neighbor embeddings around the center node x_i are mapped by $\log^c_{x_i}$ to the tangent space $\mathcal{T}_{x_i}\mathbb{P}^n_c$, where attention-weighted aggregation is performed. The exponential map $\exp^c_{x_i}$ returns the aggregate $\mathrm{AGG}^c(x)_i$ to the Poincaré ball.

graph structure [13, 39]; spherical constructions additionally include models of spatial molecular arrangements and signals sampled on a sphere.

3.5.1 Hyperbolic Graph Neural Network

Hyperbolic GNN aggregation has two main forms: a weighted sum in a tangent chart, or a manifold-valued centroid. Within tangent aggregation, HGCN centers the chart at the receiving node, while HGNN shares a reference chart across nodes. LGCN and H2H-GCN use centroid constructions in Lorentz and Klein coordinates. Their expressive and computational differences also depend on the feature transformation and message weights, as compared in Table 8. **Node-centered and shared tangent aggregation.** HGCN [39] learns negative curvature for its layers and uses attention to weight neighbors. The model is implemented in Lorentz coordinates; the equivalent computation below uses the isometric Poincaré chart $\mathbb{P}^n_c$ and the operators in Equations (29) and (35). Figure 11 illustrates the node-centered aggregation. In the hyperbolic attention-based aggregation, the embeddings $(x_i^{\mathbb{P}}, x_j^{\mathbb{P}})$ are first mapped to the tangent space at the Poincaré origin by $\log^c_{\mathbf{0}}$ to compute the attention weights w_{ij} with concatenation and a Euclidean MLP:

$$w_{ij} = \mathrm{softmax}_{j\in\mathcal{N}(i)}\left(\mathrm{MLP}\left(\mathrm{Concat}\left(\log^c_{\mathbf{0}}(x_i^{\mathbb{P}}), \log^c_{\mathbf{0}}(x_j^{\mathbb{P}})\right)\right)\right). \tag{96}$$

Then, hyperbolic aggregation is employed to average the representations of the nodes:

$$\mathrm{AGG}^c(x^{\mathbb{P}})_i = \exp^c_{x_i^{\mathbb{P}}}\left(\sum_{j\in\mathcal{N}(i)} w_{ij}\log^c_{x_i^{\mathbb{P}}}(x_j^{\mathbb{P}})\right). \tag{97}$$

The attention scores use a common origin chart, while the weighted messages use the chart at the receiving node $x_i^{\mathbb{P}}$. This adapts aggregation to the receiver's location and requires computing neighbor logarithms at different basepoints. The reported HGCN results associate larger gains with graphs having lower δ-hyperbolicity.

HGNN [175] applies its message transform and aggregation in a common tangent space, with implementations in the Lorentz model $\mathbb{L}^n_c$ and Poincaré ball $\mathbb{P}^n_c$. For each node u, propagation is

$$h_u^{k+1} = \sigma_{\mathcal{M}}\left(\exp^c_{x_0}\left(\sum_{v\in\mathcal{I}(u)}\hat{\mathbf{A}}_{uv}\mathbf{W}^k\log^c_{x_0}(h_v^k)\right)\right), \tag{98}$$

where $h_v^k \in \mathcal{M}$ represents the node embedding at layer k, $v \in \mathcal{I}(u)$ denotes a neighbor of u, $\hat{\mathbf{A}}$ and $\mathbf{W}^k$ are the normalized adjacency matrix and trainable parameters, respectively. Here $\boldsymbol{x}_0 \in \mathcal{M}$ is a fixed reference point (often the origin). The node embedding $h_v \in \mathcal{M}$ is first mapped to $\mathcal{T}_{x_0}\mathcal{M}$ using $\log_{x_0}^c$, and $\exp_{x_0}^c$ maps the transformed tangent vector back to $\mathcal{M}$. Finally, $\sigma_{\mathcal{M}} : \mathcal{M} \to \mathcal{M}$ denotes a manifold-preserving activation applied after the exponential map. Using one reference chart permits shared tangent-coordinate computation across nodes. HGNN then converts manifold states into Euclidean prediction features through distances to learned centroids $\mathcal{C} = \{\mathbb{C}_i\}_{i=1}^{|\mathcal{C}|}$. With $\psi_{ij} = d(\mathbb{C}_i, h_j^K)$, node regression uses

$$\hat{y}_j = w_o^T(\psi_{1j}, \ldots, \psi_{|\mathcal{C}|j}), \tag{99}$$

where $w_o \in \mathbb{R}^{|\mathcal{C}|}$. For C_{cls}-class node classification,

$$p(y_j) = \text{softmax}\left(\mathbf{W}_o(\psi_{1j}, \ldots, \psi_{|\mathcal{C}|j})\right), \tag{100}$$

where $\mathbf{W}_o \in \mathbb{R}^{C_{\text{cls}} \times |\mathcal{C}|}$. For graph-level predictions, average pooling is used to aggregate the distances across different nodes, resulting in $(\psi_1, \ldots, \psi_{|\mathcal{C}|})$, where $\psi_i = \frac{1}{|V|}\sum_{j=1}^{|V|}\psi_{ij}$. The aggregated vector is then fed into fully connected networks.

Centroid aggregation. LGCN [332] replaces a tangent-space average with the Lorentzian centroid, while retaining tangent-coordinate feature transformations. Given an input feature $\boldsymbol{h} \in \mathbb{R}^n$, its augmentation $[0, \boldsymbol{h}^T]^T$ lies in the tangent space at the Lorentz basepoint $\boldsymbol{o}_c = (1/\sqrt{-c}, 0, \ldots, 0)^T$. The manifold feature is

$$\boldsymbol{h}_c = \exp_{\boldsymbol{o}_c}^c\left([0, \boldsymbol{h}^T]^T\right) \in \mathbb{L}_c^n. \tag{101}$$

For an existing feature $\boldsymbol{h}_{c'} \in \mathbb{L}_{c'}^n$, a curvature change uses the common spatial tangent coordinates: $\boldsymbol{h}_c = \exp_{\boldsymbol{o}_c}^c\left(\mathcal{I}_{c'\to c}(\log_{\boldsymbol{o}_{c'}}^{c'}(\boldsymbol{h}_{c'}))\right)$, where $\mathcal{I}_{c'\to c}([0, \boldsymbol{v}^T]^T) = [0, \boldsymbol{v}^T]^T$. The linear transformation $\text{LT}_{\mathbb{L}}$ and non-linear activation $\sigma_{\mathbb{L}}$ of LGCN are described in Equations (31) and (36). In the neighborhood aggregation of LGCN, a self-attention mechanism is used to learn the aggregation weight w_{ij}. For two node features $\mathbf{h}_i, \mathbf{h}_j \in \mathbb{L}_c^n$, the attention coefficient μ_{ij}, which indicates the importance of node j to node i, is computed using the ambient Lorentz chord surrogate $\widetilde{d}_{\mathbb{L},c}^2$ given by Equation (11):

$$\mu_{ij} = -\widetilde{d}_{\mathbb{L},c}^2\left(\text{LT}_{\mathbb{L}}(\mathbf{h}_i; c, f_{\text{att}}), \text{LT}_{\mathbb{L}}(\mathbf{h}_j; c, f_{\text{att}})\right), \tag{102}$$

where $f_{\text{att}} : \mathbb{R}^n \to \mathbb{R}^d$ is the tangent-coordinate map $f_{\text{att}}(\boldsymbol{v}) = \mathbf{M}_{\text{att}}\boldsymbol{v}$ with $\mathbf{M}_{\text{att}} \in \mathbb{R}^{d\times n}$. Thus, $\text{LT}_{\mathbb{L}}(\cdot; c, f_{\text{att}})$ maps both arguments into the same Lorentz model $\mathbb{L}_c^d$ before their chord surrogate is evaluated. For all neighbors $\mathcal{N}[i]$ of node i (including node i), the softmax function is used to compute the aggregation weight w_{ij}:

$$w_{ij} = \frac{\exp(\mu_{ij})}{\sum_{t\in\mathcal{N}[i]}\exp(\mu_{it})}. \tag{103}$$

H2H-GCN [60] uses an orthogonal spatial transformation in Lorentz coordinates and an Einstein midpoint in the Klein chart. The feature transformation at layer ℓ is

$$h_i^\ell = \mathbf{W}^\ell h_i^{\ell-1} \text{ s.t. } \mathbf{W}^\ell = \begin{bmatrix} 1 & \mathbf{0}^T \\ \mathbf{0} & \widehat{\mathbf{W}}^\ell \end{bmatrix}, \widehat{\mathbf{W}}^{\ell T}\widehat{\mathbf{W}}^\ell = I, \tag{104}$$

where $\mathbf{0}$ is a column vector of zeros. The orthogonality constraint preserves the Lorentz inner product

Table 8 Comparison of representative hyperbolic GNN aggregation designs. The averaging coordinates and feature transformations distinguish methods even when they use the same underlying hyperbolic space.

Method	Where messages are averaged	Weighting and aggregation	Distinguishing design choice
HGCN [39]	Tangent space at the receiving node	Learned attention; weighted tangent sum mapped back to the node manifold	Aggregation chart changes with the receiving node
HGNN [175]	Tangent space at a common reference point	Normalized adjacency; linear message transform and tangent sum	Nodes share one chart for message computation
LGCN [332]	Lorentz model	Attention from Lorentz chord scores; Lorentzian centroid	Centroid aggregation combined with tangent-coordinate feature transforms
H2H-GCN [60]	Klein chart, with Lorentz input and output	Einstein midpoint weighted by Lorentz factors	Orthogonal spatial transform and Lorentz–Klein coordinate projections

under this transformation and restricts its spatial action to an isometry. Aggregation projects the transformed features to the Klein model $\mathbb{K}_c^n$, computes their Einstein midpoint, and returns to Lorentz coordinates:

$$\begin{cases} h_j^{\ell,\mathcal{K}} = \Pi^c_{\mathbb{L}\to\mathbb{K}}(h_j^{\ell,\mathcal{L}}) \\ m_i^{\ell,\mathcal{K}} = \sum_{j\in\mathcal{N}(i)} \gamma_j h_j^{\ell,\mathcal{K}} \,/\, \sum_{j\in\mathcal{N}(i)} \gamma_j, \\ m_i^{\ell,\mathcal{L}} = \Pi^c_{\mathbb{K}\to\mathbb{L}}(m_i^{\ell,\mathcal{K}}) \end{cases} \tag{105}$$

where $\mathcal{N}(i)$ is the set of nodes comprising the i-th node and its neighbors, $\gamma_j = 1/\sqrt{1 + c\|h_j^{\ell,\mathcal{K}}\|^2}$ denotes the Lorentz factor in the curvature-c Klein chart, and m_i^{ℓ} represents the hyperbolic average.

The centroid constructions provide closed-form manifold-valued aggregation, while tangent aggregation provides a Euclidean vector space for combining messages. Their costs occur at different operations: receiver-specific maps in HGCN, shared maps in HGNN, centroid normalization in LGCN, and coordinate projections plus an orthogonality constraint in H2H-GCN. The aggregation rule and its cost therefore characterize a hyperbolic GNN more precisely than the coordinate model alone.

3.5.2 Spherical Graph Neural Network

Spherical graph methods use angular geometry in two different ways. SphereNet constructs local geometric descriptors for molecular messages. DeepSphere builds a graph approximation of a spherical signal domain. In both cases, the relevant spherical structure comes from the input geometry, rather than a general requirement that all hidden node vectors lie on a hypersphere. **Local geometric messages.** SphereNet [178] represents a 3D molecular arrangement through distances, angles, and torsions. These descriptors distinguish spatial configurations beyond pairwise distance and require messages to incorporate higher-order geometric relationships. Its spherical message passing (SMP) acts on $G = (u, V, E, P)$, where $u \in \mathbb{R}^{d_u}$ is the global feature vector, $V = \{v_i\}_{i=1}^n$ contains atom features, $P = \{p_i \in \mathbb{R}^3\}_{i=1}^n$ contains atomic positions, and $E = \{(e_k, r_k, s_k)\}_{k=1}^m$ contains edge features and receiver/sender indices. The SMP is defined in a local spherical coordinate system, with s_k

serving as the origin and the direction of the message e_k naturally aligned with the z-axis:

$$e'_k = \phi^e \left(e_k, v_{r_k}, v_{s_k}, E_{s_k}, \rho^{p \to e} \left(\{p_h\}_{h \in \{r_k, s_k\} \cup \mathcal{N}(s_k)} \right) \right), \tag{106}$$

$$v'_i = \phi^v \left(v_i, \rho^{e \to v} \left(E'_i \right) \right), \quad u' = \phi^u \left(u, \rho^{v \to u} \left(V' \right) \right), \tag{107}$$

where ϕ^e, ϕ^v, and ϕ^u are the information update functions for edges, atoms, and the entire graph, respectively. $\rho^{e \to v}$ and $\rho^{v \to u}$ aggregate information between different geometric elements, and $\rho^{p \to e}$ is the position aggregation function used for message updates. **Graph discretization of spherical signals.** DeepSphere [63] represents samples on $\mathbb{S}^2$ as vertices of a sparse graph. Polynomial graph filters provide localized processing without explicit spherical harmonic transforms. The graph resolution and neighborhood size control a tradeoff between computational cost and approximate rotation equivariance. Let $f : \mathbb{S}^2 \to \mathbb{R}$ be a spherical signal and let $\boldsymbol{f} \in \mathbb{R}^n$ collect its samples at the graph vertices. Graph convolutions can be defined as:

$$h(\mathbf{L})\boldsymbol{f} = \left(\sum_{i=0}^{P} \alpha_i \mathbf{L}^i \right) \boldsymbol{f}, \tag{108}$$

where $\mathbf{L} \in \mathbb{R}^{n \times n}$ is the combinatorial graph Laplacian, P is the polynomial order (which corresponds to the filter size), and α_i are the coefficients to be optimized during training. Given a rotation $g \in \mathrm{SO}(3)$, the normalized equivariance error, for $\mathbf{L}\boldsymbol{f} \neq 0$, is defined as:

$$E_{\mathbf{L}}(\boldsymbol{f}, g) = \left(\frac{\| R_V(g)\mathbf{L}\boldsymbol{f} - \mathbf{L}R_V(g)\boldsymbol{f} \|}{\| \mathbf{L}\boldsymbol{f} \|} \right)^2, \tag{109}$$

where $R_V(g) = T_V R(g) T_V^{-1}$, T_V is the sampling operator, T_V^{-1} denotes reconstruction on the chosen signal class, and $R(g)$ is the rotation operator. More generally, for a signal class C equipped with a sampling distribution ν_C, and for a prescribed rotation distribution $\mu_{\mathrm{SO}(3)}$, the mean equivariance error is given by:

$$E_{\mathbf{L},C} = \mathbb{E}_{f \sim \nu_C, g \sim \mu_{\mathrm{SO}(3)}} [E_{\mathbf{L}}(\boldsymbol{f}, g)]. \tag{110}$$

For a graph constructed using uniform random sampling, the specific weight w_{ij} can be defined as follows:

$$w_{ij} = \exp \left(-\frac{1}{4t} \| x_i - x_j \|^2 \right), \tag{111}$$

where $x_i \in \mathbb{S}^2$ is a sampled vertex. Under the corresponding sampling, bandwidth, and scaling assumptions, the graph Laplacian approaches the rotation-equivariant Laplace–Beltrami operator $\Delta_{\mathbb{S}^2}$. At finite resolution, the kernel width t is selected by minimizing the mean equivariance error. Thus, graph construction is part of the geometric approximation and determines both filter cost and equivariance quality.

3.5.3 Mixed-curvature Graph Neural Network

Mixed-curvature GNNs differ in the level at which geometry adapts. A product architecture assigns geometric factors to every node; component attention learns their relative contribution; expert routing adapts mixtures to local topology; and interaction networks exchange features between geometric channels. Each mechanism adds flexibility beyond learning the magnitude of a single negative curvature. **Shared operators across curvature signs.** Product embeddings [94] motivate learning separate geometric factors for heterogeneous graphs. The κ-stereographic GCN, or κ-GCN [13], provides a common graph-convolution algebra for negative, zero, and positive curvature. Using our

signed curvature notation c, its stereographic model is

$$\mathfrak{st}_c^d = \{\boldsymbol{x} \in \mathbb{R}^d : 1 + c\|\boldsymbol{x}\|_2^2 > 0\}, \qquad g_{\boldsymbol{x}}^c = (\lambda_{\boldsymbol{x}}^c)^2 \mathbb{I}_d, \quad \lambda_{\boldsymbol{x}}^c = \frac{2}{1 + c\|\boldsymbol{x}\|_2^2}.$$

For a graph $G = (V, \mathbf{A}, \mathbf{X})$, with features $\mathbf{X} \in \mathbb{R}^{n \times d}$ and adjacency $\mathbf{A} \in \mathbb{R}^{n \times n}$, negative curvature imposes a ball-domain constraint. The input features can be placed inside the stereographic domain by

$$\mathbf{X} \mapsto \begin{cases} \dfrac{\mathbf{X}}{\max\{1, \sqrt{-c}\|\mathbf{X}\|_{\max}/(1-\varepsilon)\}}, & c < 0, \\ \mathbf{X}, & c \geq 0, \end{cases} \tag{112}$$

where $\|\mathbf{X}\|_{\max}$ denotes the maximal row norm and $\varepsilon \in (0, 1)$ keeps projected points strictly inside the ball. The $(l+1)$-th layer of κ-GCN is given by:

$$\mathbf{H}^{(l+1)} = \sigma_c \left(\hat{\mathbf{A}} \boxtimes_c \left(\mathbf{H}^{(l)} \otimes_c \mathbf{W}^{(l)} \right) \right), \tag{113}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d \times e}$ denotes the weight parameters of the l-th layer, and

$$(X \otimes_c \mathbf{W})_{i\bullet} = \exp_{\mathbf{0}}^c \left((\log_{\mathbf{0}}^c(X)\mathbf{W})_{i\bullet} \right)$$

is the right-matrix-multiplication at the stereographic origin, and

$$(\mathbf{A} \boxtimes_c X)_{i\bullet} = \left(\sum_j \mathbf{A}_{ij} \right) \otimes_c m_c(\{X_{j\bullet}\}_{j=1}^n; \mathbf{A}_{i\bullet})$$

is the left-matrix-multiplication with the gyromidpoint operation m_c. A sign-compatible row-wise activation is

$$\sigma_c(X) = \exp_{\mathbf{0}}^c(\sigma(\log_{\mathbf{0}}^c(X))),$$

where the maps are evaluated on their regular domains, and $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$ with $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\tilde{\mathbf{D}}_{ii} = \sum_k \tilde{\mathbf{A}}_{ik}$. The shared algebra recovers Euclidean GCN as $c \to 0$. A common-tangent classifier, $p(y_i) = \mathrm{softmax}(\mathbf{W}_o \log_{\mathbf{0}}^c(\mathbf{H}_i^{(L)}) + \boldsymbol{b}_o)$, can read out any of these component types; the Poincaré MLR in Equation (51) offers a geometric alternative for $c < 0$. Products of such components extend the same architecture to mixed curvature, with separate factor dimensions and curvature parameters.

Spectral filtering and adaptive mixtures. CUSP, the Spectro-Riemannian GNN [93], adds a spectral design choice to product geometry. It combines a curvature-aware Laplacian with a bank of Riemannian filters and attention-based pooling, allowing different frequency bands and geometric components to contribute to the prediction. This targets both homophilic and heterophilic graphs and adds curvature estimation and multiple filter evaluations to the computation.

GraphMoRE [102] makes the geometry mixture node dependent. A topology-aware gate weights Riemannian experts using local graph structure, and a pairwise alignment rule permits comparison of nodes with different expert weights. This provides finer adaptation than a single shared product metric, with additional computation for local topology sampling, expert encoding, and alignment.

SELFMGNN [257] instead couples product representations with hierarchical attention and self-supervision. Intra-component attention weights neighbors, inter-component attention fuses geometric factors, and single-view and cross-view contrastive objectives use the factors as complementary views. For component a with curvature c_a, write $z_i^{(a)}$ for the representation of node i. The intra-

component weights are

$$\hat{\mathbf{A}}_{ij}^{(a)} = \frac{\exp(\mathrm{att}_{\mathrm{intra}}^{(a)}(z_i^{(a)}, z_j^{(a)}))}{\sum_{k\in\mathcal{N}_i}\exp(\mathrm{att}_{\mathrm{intra}}^{(a)}(z_i^{(a)}, z_k^{(a)}))}, \tag{114}$$

where $\mathcal{N}_i$ is the neighborhood of node i and the score concatenates transformed tangent coordinates:

$$\mathrm{att}_{\mathrm{intra}}^{(a)}(z_i^{(a)}, z_j^{(a)}) = \sigma\Big((\theta_{\mathrm{intra}}^{(a)})^T \,\mathrm{Concat}\Big(\mathbf{W}_{\mathrm{intra}}^{(a)} \log_{\mathbf{0}}^{c_a}(z_i^{(a)}), \mathbf{W}_{\mathrm{intra}}^{(a)} \log_{\mathbf{0}}^{c_a}(z_j^{(a)})\Big)\Big).$$

Here σ is a sigmoid activation. With self-loops, $\mathbf{A}^{(a)} = \mathbf{I} + \hat{\mathbf{A}}^{(a)}$, and $\mathbf{A}^{(a)} \boxtimes_{c_a} Z^{(a)}$ performs attentional aggregation. Inter-component attention maps each node's component embeddings to a common tangent-coordinate dimension and computes their mean across the K components:

$$\boldsymbol{u}_i^{(a)} = \log_{\mathbf{0}}^{c_a}(\mathbf{W}_a \otimes_{c_a} z_i^{(a)}), \qquad \boldsymbol{\mu}_i = \frac{1}{K}\sum_{a=1}^{K} \boldsymbol{u}_i^{(a)}. \tag{115}$$

Each component is scored relative to this node-specific centroid:

$$s_{ia} = \sigma\Big(\boldsymbol{\theta}_{\mathrm{inter}}^T(\boldsymbol{u}_i^{(a)} - \boldsymbol{\mu}_i)\Big). \tag{116}$$

These scores are normalized with a softmax to obtain attention weights:

$$w_{ia} = \frac{\exp(s_{ia})}{\sum_{b=1}^{K}\exp(s_{ib})}. \tag{117}$$

The node representation is the product tuple $z_i = \mathrm{Concat}_{a=1}^{K}(w_{ia} \otimes_{c_a} z_i^{(a)})$. This preserves separate geometric factors while learning their relative importance; training additionally evaluates contrastive relations within and across components.

Interaction between geometric channels. GIL [338] learns Euclidean and hyperbolic node representations in parallel and exchanges information between them during propagation. This differs from weighting independently learned factors only at the output. Related knowledge graph completion methods also combine geometric components [36, 241], with relation scoring providing the interaction context. Given row-wise hyperbolic embeddings $\boldsymbol{H}_{\mathbb{P}} \in (\mathbb{P}_c^F)^N$ and Euclidean embeddings $\boldsymbol{H}_{\mathbb{R}} \in \mathbb{R}^{N\times F}$, the maps, distances, and Möbius operations below are applied independently to each row:

$$\boldsymbol{H}'_{\mathbb{P}} = \boldsymbol{H}_{\mathbb{P}} \oplus_c \left(\beta\, \boldsymbol{d}_{\mathbb{P}} \otimes_c \exp_{\mathbf{0}}^c(\boldsymbol{H}_{\mathbb{R}})\right), \tag{118}$$

$$\boldsymbol{H}'_{\mathbb{R}} = \boldsymbol{H}_{\mathbb{R}} + \beta'\, \boldsymbol{d}_{\mathbb{R}} \odot \log_{\mathbf{0}}^c(\boldsymbol{H}_{\mathbb{P}}), \tag{119}$$

where $(\boldsymbol{d}_{\mathbb{P}})_i = d_{\mathbb{P}}^c((\boldsymbol{H}_{\mathbb{P}})_i, \exp_{\mathbf{0}}^c((\boldsymbol{H}_{\mathbb{R}})_i))$ and $(\boldsymbol{d}_{\mathbb{R}})_i = d_{\mathbb{R}}((\boldsymbol{H}_{\mathbb{R}})_i, \log_{\mathbf{0}}^c((\boldsymbol{H}_{\mathbb{P}})_i))$ are row-wise scalar distances broadcast across features. The trainable scalars are $\beta, \beta' \in \mathbb{R}$.

Choosing geometry for graph learning. Local coordinates are useful when messages must distinguish physical arrangements, and graph discretization is useful when the input is a spherical signal. For learned node representations, a shared hyperbolic space provides a direct hierarchy prior. Product factors support several structural patterns, component attention selects their contribution, and expert gates condition that choice on local topology. GIL adds explicit cross-space exchange, while CUSP additionally adapts spectral filtering. A useful comparison therefore fixes both the graph task and the operation being changed: message description, averaging, curvature assignment, or communication between components.

3.6 Transformer

Transformers combine pairwise attention scores with weighted value aggregation, followed by projection, normalization, and residual updates [275]. This architecture supports language [65] and vision [71], while leaving several choices for introducing geometry. Hyperbolic designs change distance-based matching and manifold-valued aggregation, spherical designs constrain directions and norms, and mixed-curvature designs assign different geometries to attention heads or relation maps. We compare both the attention rule and the operations needed to keep successive hidden states in the chosen space.

3.6.1 Hyperbolic Transformers

HAT, FHNN, and Hypformer differ in how they compute attention and maintain hyperbolic states. HAT combines geodesic matching with a Klein-coordinate midpoint, FHNN uses a Lorentz chord score and direct Lorentz aggregation, and Hypformer computes linear attention on spatial features before restoring the Lorentz constraint. Hyperbolic Attention Networks (*HAT*) [96] instantiated attention directly in the Lorentz model $\mathbb{L}_c^n$. Given queries $q_i \in \mathbb{L}_c^n$ and keys $k_j \in \mathbb{L}_c^n$, HAT defines attention weights by a distance-based kernel on the Lorentz geodesic distance $d_{\mathbb{L}}^c(q_i, k_j)$ (defined in table 3):

$$\alpha_{ij} = \frac{\exp\big(-\beta\, d_{\mathbb{L}}^c(q_i, k_j)\big)}{\sum_\ell \exp\big(-\beta\, d_{\mathbb{L}}^c(q_i, k_\ell)\big)}, \tag{120}$$

where $\beta > 0$ is an inverse-temperature. For aggregation, *HAT* computes a geometry-consistent weighted mean via the Einstein midpoint in Klein coordinates: for any Lorentz point $\boldsymbol{x} = [x_t, \boldsymbol{x}_s]^\top \in \mathbb{L}_c^n$, define its curvature-aware Klein coordinate as $\boldsymbol{v} = \Pi_{\mathbb{L}\to\mathbb{K}}^c(\boldsymbol{x}) = \boldsymbol{x}_s/(\sqrt{-c}\, x_t)$, and aggregate values $\{\boldsymbol{v}_{ij}\}_j$ as

$$m_i\big(\{\alpha_{ij}\}_j, \{v_{ij}\}_j\big) = \sum_j \left[\frac{\alpha_{ij}\,\gamma_c(v_{ij})}{\sum_\ell \alpha_{i\ell}\,\gamma_c(v_{i\ell})}\right] v_{ij}, \tag{121}$$

$$\gamma_c(\boldsymbol{v}) = \frac{1}{\sqrt{1 + c\|\boldsymbol{v}\|_2^2}}, \tag{122}$$

where $\|\cdot\|_2$ is the Euclidean norm in the curvature-aware Klein coordinates and $\gamma_c(\cdot)$ is the corresponding Lorentz factor. This couples Lorentz-distance matching with Einstein-midpoint aggregation, yielding a hyperbolic attention block that is curvature-consistent in both similarity and pooling.

Fully Hyperbolic Neural Networks (*FHNN*) [45] compute attention and aggregation directly in Lorentz coordinates. Their neural layers relax Lorentz rotations and boosts, while the attention kernel uses the squared Lorentz chord surrogate $\widetilde{d}^2_{\mathbb{L},c}$ from Equation (11). The corresponding centroid minimizes the weighted sum of this surrogate and has the closed form:

$$\begin{aligned}\boldsymbol{\mu} &= \mathrm{Centroid}(\nu_1, \ldots, \nu_{|\mathcal{P}|}, \mathbf{x}_1, \ldots, \mathbf{x}_{|\mathcal{P}|}) \\ &= \frac{\sum_{j=1}^{|\mathcal{P}|} \nu_j \mathbf{x}_j}{\sqrt{c\left\langle \sum_{i=1}^{|\mathcal{P}|} \nu_i \mathbf{x}_i, \sum_{i=1}^{|\mathcal{P}|} \nu_i \mathbf{x}_i \right\rangle_{\mathcal{L}}}},\end{aligned}$$

where $\mathcal{P}$ denotes the set of points being aggregated, ν_j represents the weight of the j-th point, and $\mathbf{x}_j$ denotes its hyperbolic embedding. Specifically, given query set $\mathcal{Q} = \{\mathbf{q}_1, \ldots, \mathbf{q}_{|\mathcal{Q}|}\}$, key set $\mathcal{K} = \{\mathbf{k}_1, \ldots, \mathbf{k}_{|\mathcal{K}|}\}$, and value set $\mathcal{V} = \{\mathbf{v}_1, \ldots, \mathbf{v}_{|\mathcal{V}|}\}$, the hyperbolic attention mechanism computes

distance-based attention

$$\mathrm{Attn}(\mathcal{Q},\mathcal{K},\mathcal{V}) = \{\mu_1,\cdots,\mu_{|\mathcal{Q}|}\}$$

using:

$$\mu_i = \frac{\sum_{j=1}^{|\mathcal{K}|} \nu_{ij}\mathbf{v}_j}{\sqrt{c\left\langle \sum_{k=1}^{|\mathcal{K}|} \nu_{ik}\mathbf{v}_k, \sum_{k=1}^{|\mathcal{K}|} \nu_{ik}\mathbf{v}_k \right\rangle_{\mathcal{L}}}}, \quad i \in \{1,\cdots,|\mathcal{Q}|\}, \tag{123}$$

$$\nu_{ij} = \frac{\exp\left(-\widetilde{d}^2_{\mathbb{L},c}(\mathbf{q}_i,\mathbf{k}_j)/\sqrt{d}\right)}{\sum_{k=1}^{|\mathcal{K}|}\exp\left(-\widetilde{d}^2_{\mathbb{L},c}(\mathbf{q}_i,\mathbf{k}_k)/\sqrt{d}\right)}, \tag{124}$$

where d is the attention-head dimension, $c<0$ is the signed curvature, and ν_{ij} denotes the attention weight. The chord surrogate and geodesic distance order pairs identically, but their softmax kernels assign different weights. The closed-form centroid also avoids iterative Fréchet-mean computation. The multi-headed attention is then defined as:

$$\mathrm{MHAttn}(\mathcal{Q},\mathcal{K},\mathcal{V}) = \{\boldsymbol{\mu}_i\}_{i=1}^{|\mathcal{Q}|}, \tag{125}$$

$$\boldsymbol{\mu}_i = \mathrm{HL}\left(\mathrm{Concat}_{h=1}^{H}\boldsymbol{\mu}_i^{(h)}\right), \tag{126}$$

$$\boldsymbol{\mu}_i^{(h)} = \mathrm{Attn}_i^{(h)}\left(\mathrm{HL}_h^{Q}(\mathcal{Q}),\mathrm{HL}_h^{K}(\mathcal{K}),\mathrm{HL}_h^{V}(\mathcal{V})\right). \tag{127}$$

Here H is the number of heads and h indexes a head, while i indexes a query token. *Hypformer* [307] extends Lorentz-valued processing to linear attention and curvature changes between blocks. It introduces two operators: HTC for curvature-aware hyperbolic linear transformations and HRC for applying Euclidean feature operators on the space-like part, followed by Lorentz re-normalization. For a Lorentz point $\boldsymbol{x} = [x_t, \boldsymbol{x}_s] \in \mathbb{L}^n_{c_1}$ with time-like coordinate x_t and space-like coordinate $\boldsymbol{x}_s \in \mathbb{R}^n$, HTC first applies an affine map on ambient coordinates $f_t(\boldsymbol{x}) = W^\top \boldsymbol{x} + b$ and then projects to curvature c_2:

$$\mathrm{HTC}(x;c_1,c_2) = \left(\sqrt{\frac{c_1}{c_2}\|f_t(x)\|_2^2 - \frac{1}{c_2}},\ \sqrt{\frac{c_1}{c_2}}\,f_t(x)\right), \tag{128}$$

where $c_1, c_2 < 0$ are source/target curvatures. HRC instantiates non-linear blocks (e.g., LayerNorm, Dropout, activation) by operating only on the space-like features x_s via a Euclidean operator $f_r(\cdot)$ and recalibrating the time-like coordinate to satisfy the Lorentz constraint:

$$\mathrm{HRC}(x;c_1,c_2) = \left(\sqrt{\frac{c_1}{c_2}\|f_r(x_s)\|_2^2 - \frac{1}{c_2}},\ \sqrt{\frac{c_1}{c_2}}\,f_r(x_s)\right). \tag{129}$$

Based on HTC/HRC, Hypformer further proposes a **hyperbolic *linear* self-attention** that computes attention in the space-like dimensions using the associative rearrangement $Q(K^\top V)$ (instead of $(QK^\top)V$) and then restores a valid Lorentz point. For a sequence of N tokens, let $Q, K, V \in (\mathbb{L}^{d'}_{c_2})^N$ be obtained by HTC. The nonnegative, focused space-like features are defined row-wise by

$$\mathcal{Q}_s = \phi(Q_{[:,s]}), \qquad \mathcal{K}_s = \phi(K_{[:,s]}), \qquad \mathcal{V}_s = \phi(V_{[:,s]}),$$

where $[:,s]$ selects the d' space-like coordinates and ϕ is the focus map used by Hypformer. The

space-like output is

$$Z_s = \mathcal{Q}_s(\mathcal{K}_s^\top \mathcal{V}_s) \oslash \left(\mathcal{Q}_s(\mathcal{K}_s^\top \mathbf{1}) + \varepsilon_{\text{att}}\mathbf{1}\right), \qquad \tilde{Z}_s = Z_s + \psi(\mathcal{V}_s), \tag{130}$$

followed by time-like recalibration and re-concatenation:

$$Z_t = \sqrt{\frac{c_2}{c_3}\|\tilde{Z}_s\|_2^2 - \frac{1}{c_3}}, \qquad Z = \left(Z_t, \sqrt{\frac{c_2}{c_3}}\,\tilde{Z}_s\right), \tag{131}$$

where $\oslash$ denotes elementwise row-wise division, $\mathbf{1}$ is an all-ones vector of the required dimension, $\varepsilon_{\text{att}} > 0$ guards the normalization, ψ is a learnable linear map, and $c_2, c_3 < 0$ control the curvature before and after attention. The norm and square root in the time-coordinate completion are evaluated separately for each token. Hypformer's hyperbolic positional-encoding module combines a Lorentz feature with a learned feature transform and normalizes the result onto the manifold. For $\boldsymbol{x} \in \mathbb{L}_c^n$, the feature update is:

$$\boldsymbol{z} = \boldsymbol{x} + \epsilon \boldsymbol{p}, \qquad \boldsymbol{x}' = \frac{\boldsymbol{z}}{\sqrt{c\langle \boldsymbol{z}, \boldsymbol{z}\rangle_{\mathcal{L}}}}, \qquad \boldsymbol{p} := \mathrm{HTC}(\boldsymbol{x}),$$

where $\boldsymbol{p}$ is the learned Lorentz feature produced by HTC and ϵ is its mixing scale. With shared HTC parameters, this pointwise update depends only on the input feature; token order must be represented in that input or provided by another position-dependent component. The normalization is defined when $\boldsymbol{z}$ is future-directed timelike; because $c < 0$ and $\langle \boldsymbol{z}, \boldsymbol{z}\rangle_{\mathcal{L}} < 0$, its denominator is real and enforces $\langle \boldsymbol{x}', \boldsymbol{x}'\rangle_{\mathcal{L}} = 1/c$.

Beyond attention, *Hypformer* completes the Transformer stack by defining standard building blocks directly via the HRC. In particular, Hyperbolic LayerNorm, Dropout, pointwise activations, and concatenation are instantiated as $\mathrm{Hyp}_f(X) = \mathrm{HRC}(X, f)$, where f can be Euclidean

$$\text{LayerNorm}, \quad \text{Dropout}, \quad \text{Activation}, \quad \text{Concat}$$

functions, enabling a fully hyperbolic Transformer block without frequent tangent-space mappings.

3.6.2 Spherical Transformers

nGPT and Hyper-SET both normalize token representations, but derive their updates from different principles. nGPT constrains parameters and states to control angular updates, while Hyper-SET derives attention and feedforward blocks from constrained energy minimization. In the spherical Transformers below, hyperspherical constraints act on token states or their projections and shape attention and normalization. This embedding-space construction serves a different purpose from processing a spherical input domain, such as panoramic imagery.

Loshchilov et al. [180] proposed a normalized Transformer *nGPT* that explicitly performs representation learning on the unit hypersphere by enforcing unit-norm constraints for all vectors forming token embeddings, projection matrices, and hidden states. The key point is to replace the standard residual updates with hyperspherical interpolation steps followed by a retraction (normalization) back to the manifold. Given hidden states $h \in \mathbb{R}^d$ and block outputs $h_A = \mathrm{Norm}(\mathrm{ATTN}(h))$, $h_M = \mathrm{Norm}(\mathrm{MLP}(h))$, nGPT updates:

$$h \leftarrow \mathrm{Norm}(h + \alpha_A \odot (h_A - h)) \tag{132}$$

$$h \leftarrow \mathrm{Norm}(h + \alpha_M \odot (h_M - h)), \tag{133}$$

where $\mathrm{Norm}(x) = x/\|x\|_2$ is a unit-norm projection (interpretable as a retraction step), and $\alpha_A, \alpha_M \in \mathbb{R}^d_{\geq 0}$ are learnable eigen learning rates that control the per-dimension step size of the attention and MLP displacements. This interprets each layer as an optimization step along the hypersphere, while eliminating the need for LayerNorm/RMSNorm within the block. To make attention compatible with hyperspherical constraints, nGPT normalizes the attention projection matrices along their embedding dimension and additionally normalizes queries and keys after projection:

$$q \leftarrow \mathrm{Norm}(q) \odot s_{qk}, \qquad k \leftarrow \mathrm{Norm}(k) \odot s_{qk}, \tag{134}$$

with trainable scaling $s_{qk} \in \mathbb{R}^{d_k}$, so that subsequent dot products behave as controlled cosine similarities. Under this normalization, the usual softmax temperature is adjusted by changing the scaling from $1/\sqrt{d_k}$ to $\sqrt{d_k}$ to restore unit-variance score statistics. A trainable logit scaling s_z (element-wise on the vocabulary dimension) is also introduced to recover appropriate softmax confidence. The authors report $4\times$–$20\times$ fewer training steps to reach the same accuracy in their evaluated settings, depending on sequence length.

Hyper-SET [115] derives Transformer components from iterative hyperspherical energy minimization by casting token representations $X = [x_1^T; \ldots; x_N^T] \in \mathbb{R}^{N\times d}$ as a joint maximum-likelihood problem on the hypersphere that balances distributional uniformity of low-dimensional projections and semantic alignment in the ambient space:

$$\min_X \sum_{h=1}^{H} D_{\mathrm{KL}}\Big(p(z) \parallel p_\phi(z^h \mid X)\Big) \; - \; \log p_\theta(X), \tag{135}$$

where H is the number of subspaces (heads), $Z^h = XW_h \in \mathbb{R}^{N\times p}$ is the p-dimensional row-wise projection induced by $W_h \in \mathbb{R}^{d\times p}$, and $z_i^h = W_h^T x_i$. The distribution $p(z)$ is uniform on the radius-$\sqrt{p}$ hypersphere, $p_\phi(z^h \mid X)$ models projected tokens, and $p_\theta(X)$ encodes semantic alignment. Instantiating these objectives via extended Hopfield-style energies yields a repulsive attention energy over subspaces:

$$E_{\mathrm{ATTN}} = \sum_{h=1}^{H} \beta^{-1} \sum_{i=1}^{N} \log\left(\sum_{j=1}^{N} \exp\big(\beta\, (z_i^h)^\top z_j^h\big) \right), \tag{136}$$

$$\text{s.t.} \;\; \|W_h^\top x_i\|_2 = \sqrt{p}, \tag{137}$$

where $\beta > 0$ is an inverse-temperature controlling the sharpness of interactions, and the hyperspherical constraint drives the projected tokens z_i^h to spread on the sphere (uniformity). In parallel, Hyper-SET introduces an attractive feedforward energy:

$$E_{\mathrm{FF}} = -\frac{1}{2} \sum_{i=1}^{N} \sum_{m=1}^{M} \mathrm{ReLU}\big(d_m^\top x_i\big)^2, \quad \text{s.t.} \;\; \|D^\top x_i\|_2 = \sqrt{M}, \tag{138}$$

where $D = [d_1, \ldots, d_M] \in \mathbb{R}^{d\times M}$ collects M semantic directions that pull tokens toward informative half-spaces (alignment). Discretizing the corresponding gradient flows under these spherical constraints recovers a symmetric multi-head attention update and an RMS-normalized feedforward update, with $\mathrm{RMSNorm}(XW_h)$ and $\mathrm{RMSNorm}(XD)$ acting as row-wise retraction-like normalization steps that enforce the hyperspherical constraints up to scale. *Hyper-SET* also shares parameters across recurrent iterations and learns iteration-dependent step sizes α_t and γ_t, making the forward pass an explicit energy-descent process on the hypersphere.

3.6.3 Mixed-curvature Transformers

FPS-T assigns geometry to the values and aggregation of each attention head, while MMA combines relation maps computed in different geometries. The distinction is whether the model maintains a product-space representation or uses multiple geometric descriptors to form attention scores. Cho et al. [51] propose the Fully Product-Stereographic Transformer (*FPS-T*), whose heads operate in a product of constant-curvature spaces. Each attention head in *FPS-T* resides in a learnable stereographic model st_c^d, parameterized by sectional curvature c, allowing the architecture to interpolate between hyperbolic ($c < 0$), Euclidean ($c = 0$), and spherical ($c > 0$) geometries. Given input embeddings $X \in (\mathrm{st}_c^d)^n$, one head first computes

$$Q = XW^Q \in \mathcal{T}_V(\mathrm{st}_c^{d'})^n, \qquad K = XW^K \in \mathcal{T}_V(\mathrm{st}_c^{d'})^n, \qquad V = \mathrm{Linear}^c(X; W^V) \in (\mathrm{st}_c^{d'})^n.$$

Thus $Q_i \in \mathcal{T}_{V_i}\mathrm{st}_c^{d'}$ and $K_j \in \mathcal{T}_{V_j}\mathrm{st}_c^{d'}$, although their coordinate arrays lie in $\mathbb{R}^{d'}$. The attention score is evaluated only after transporting these tangent vectors to the common origin:

$$\alpha_{ij} = \left\langle \mathrm{PT}^c_{V_i \to \mathbf{0}}(Q_i), \mathrm{PT}^c_{V_j \to \mathbf{0}}(K_j) \right\rangle_{\mathbf{0}}.$$

The paper first expresses aggregation using these inner-product scores. Its linearized variant, used in the experiments, instead applies the elementwise positive feature map $\phi(u) = \mathrm{ELU}(u) + 1$ to the transported query and key coordinates. Writing $\widetilde{q}_i = \mathrm{PT}^c_{V_i \to \mathbf{0}}(Q_i)$ and $\widetilde{k}_j = \mathrm{PT}^c_{V_j \to \mathbf{0}}(K_j)$, its weights are

$$a_{ij} = \phi(\widetilde{q}_i)^\top \phi(\widetilde{k}_j) > 0.$$

We use these kernel weights in the normalized aggregation below; they need no separate row normalization. For a product model, these operations are applied componentwise and the results are concatenated. Value aggregation uses a curvature-aware weighted Einstein midpoint in place of the Euclidean weighted sum. The aggregation at point i is given by:

$$\mathrm{Aggregate}_c(V, a)_i := \frac{1}{2} \otimes_c \left(\sum_{j=1}^n \frac{a_{ij}\lambda^c_{V_j}}{\sum_{k=1}^n a_{ik}(\lambda^c_{V_k} - 1)} V_j \right), \tag{139}$$

where $\lambda^c_{V_j} = \frac{2}{1+c\|V_j\|^2}$ is the conformal factor at V_j in the standard stereographic coordinate convention. At $c = 0$, the aggregation becomes the Euclidean weighted average $\sum_j a_{ij}V_j / \sum_j a_{ij}$. For a positive-curvature head, define $D_i = \sum_j a_{ij}(\lambda^c_{V_j} - 1)$. The displayed principal branch agrees with normalized ambient spherical averaging when $D_i > 0$, which is guaranteed if $c\|V_j\|^2 < 1$ for every positively weighted value. Full-sphere evaluation additionally requires a consistent spherical branch convention. Finally, the multi-head attention output is formed by concatenating the results across H independently curved heads:

$$\mathrm{MHA}_{\otimes c}(X) = \mathrm{Concat}_{h=1}^H \mathrm{Aggregate}_{c_h}(V^{(h)}, a^{(h)}), \tag{140}$$

where each attention head h is assigned with its own learnable curvature c_h.

Konstantinidis et al. [138] pursued a complementary mixed-geometry design for Vision Transformers by fusing relation maps defined on Euclidean space, the Symmetric Positive Definite (SPD) manifold, and the Grassmann manifold. Given patch tokens $X \in \mathbb{R}^{L \times D}$, each attention head forms projections $Q, K, V \in \mathbb{R}^{L \times d}$, where $d = D/H$. MMA constructs one $L \times L$ pairwise relation map in each geometry: a Euclidean map from query–key relations, an SPD map from local covariance descriptors, and a Grassmann map from orthogonalized subspace descriptors. In early fusion, the three maps are stacked and batch-normalized, and a learned convolution produces entrywise mixture weights

$W \in \mathbb{R}^{3\times L\times L}$ with a softmax over the geometry axis. The fused attention logits and head output are

$$A = \sum_{r\in\{E,\mathrm{SPD},G\}} W_r \odot D_r, \qquad V' = \mathrm{softmax}(A)V. \tag{141}$$

Here, D_r is the relation map for geometry r, and W_r contains its entrywise mixture weights.

These attention designs introduce different computational requirements. HAT converts values to Klein coordinates for aggregation; FHNN computes a centroid in Lorentz coordinates; Hypformer rearranges spatial attention to avoid an explicit token-by-token attention matrix. nGPT and HyperSET require normalization during hidden-state updates. FPS-T additionally learns head curvatures, while MMA constructs and fuses several relation maps. Matching a Transformer to a task therefore involves choosing both a similarity model and the geometric operations used at every layer.

3.7 Generative Model

Variational autoencoders (VAEs) learn latent distributions [132], generative adversarial networks (GANs) learn through adversarial objectives [90], and diffusion models learn stochastic generation dynamics [111, 252]. Their geometric extensions differ in where curvature enters generation: the latent distribution, the network transformation, the training discrepancy, or the sampling dynamics.

3.7.1 Hyperbolic Generative Models

P-VAE, HGAN, and HGDM use hyperbolic geometry at different stages of generation. P-VAE uses hyperbolic prior and posterior families, HGAN inserts hyperbolic layers into an adversarial network, and HGDM learns diffusion dynamics after constructing a hyperbolic graph latent space. Mathieu et al. [187] introduced a variational auto-encoder *P-VAE* whose latent variable lives in the d-dimensional Poincaré ball $\mathbb{P}_c^d$ with signed curvature $c < 0$, equipped with the Poincaré metric $g_p^c(z) = (\lambda_z^c)^2 g^E$ where $\lambda_z^c = 2/(1 + c\|z\|^2)$. The generative model keeps a standard decoder likelihood $p_\theta(x \mid z)$ but replaces the Euclidean prior and posterior by hyperbolic normal families on $\mathbb{P}_c^d$, yielding an ELBO defined with respect to the Riemannian volume element $d\mathcal{M}(z) = (\lambda_z^c)^d\, dz$:

$$\log p_\theta(x) \;\geq\; \mathcal{L}(x;\theta,\phi) \tag{142}$$

$$= \int_{\mathbb{P}_c^d} \log \frac{p_\theta(x \mid z)\, p(z)}{q_\phi(z \mid x)}\, q_\phi(z \mid x)\, d\mathcal{M}(z), \tag{143}$$

where θ, ϕ are decoder and encoder parameters. To make $q_\phi(z \mid x)$ reparameterizable on $\mathbb{P}_c^d$, they derive sampling procedures and densities for two canonical Gaussian generalizations: the maximum-entropy (Riemannian) normal and the wrapped normal. The Riemannian normal has density:

$$\mathcal{N}^R_{\mathbb{P}_c^d}(z \mid \mu, \sigma^2) = \frac{1}{Z_R} \exp\Big(-\frac{d^c_{\mathbb{P}}(\mu, z)^2}{2\sigma^2}\Big), \tag{144}$$

where $\mu \in \mathbb{P}_c^d$ is the Fréchet mean, $\sigma > 0$ is a dispersion parameter, $d^c_{\mathbb{P}}$ is the Poincaré geodesic distance, and Z_R is the normalizer. The wrapped normal is defined as the push-forward of a Euclidean Gaussian on the tangent space via the exponential map; with $v \sim \mathcal{N}(0, \Sigma)$ and $z = \exp^c_\mu(v/\lambda^c_\mu)$, its density admits the closed form:

$$\mathcal{N}_{\mathbb{P}_c^d}(z \mid \mu, \Sigma) = \mathcal{N}\Big(\lambda^c_\mu \log^c_\mu(z) \,\Big|\, 0, \Sigma\Big) \left(\frac{\sqrt{-c}\, d^c_{\mathbb{P}}(\mu, z)}{\sinh(\sqrt{-c}\, d^c_{\mathbb{P}}(\mu, z))}\right)^{d-1}, \tag{145}$$

where $\exp_\mu^c$ and $\log_\mu^c$ are the Poincaré exponential and logarithm maps. P-VAE also replaces the decoder's first affine layer with a hyperbolic *gyroplane* operator $f_{\boldsymbol{a},\boldsymbol{p}}^c : \mathbb{P}_c^d \to \mathbb{R}$ based on signed distance to a gyroplane $\widetilde{H}_{\boldsymbol{a},\boldsymbol{p}}^c$, providing a decoder that is sensitive to geodesic structure rather than relying on Euclidean linearization. Empirically, the resulting hyperbolic-latent VAE yields improved held-out likelihood and more faithful recovery of hierarchical latent organization when the data exhibit tree-like structure.

Hyperbolic Generative Adversarial Networks (*HGAN*) [145] place geometry in the generator or discriminator transformations. Hyperbolic layers model semantic groupings, while exponential and logarithmic maps connect them to Euclidean network blocks. Their affine transformation combines Möbius matrix-vector multiplication and bias addition:

$$M \otimes_c v := \exp_{\mathbf{0}}^c \left(M \cdot \log_{\mathbf{0}}^c(v) \right),$$

$$v \oplus_c b := \exp_v^c \left(\frac{\lambda_{\mathbf{0}}^c}{\lambda_v^c} \log_{\mathbf{0}}^c(b) \right),$$

where $c < 0$ and $\lambda_u^c = \frac{2}{1+c\|u\|^2}$ is the conformal factor at u. These operations enter standard GAN architectures by positioning exponential/logarithmic maps around hyperbolic layers, forming configurations such as EHE (Euclidean–Hyperbolic–Euclidean) blocks within the generator and discriminator.

HGDM [290] separates representation learning from score-based sampling: a hyperbolic VAE first encodes a graph, and diffusion then generates latent node features and edges. The model consists of two stages. First, a hyperbolic VAE is trained to encode node features into latent embeddings using the same wrapped normal distributions in [187]. After learning the latent representation, HGDM diffuses node embeddings in hyperbolic space and adjacency matrices in Euclidean space. The reverse process uses a hyperbolic node score and a Euclidean edge score, both conditioned on the noisy graph:

$$s_{\theta,t}(X_t, A_t) \approx \operatorname{grad}_{X_t} \log p_t(X_t, A_t), \qquad s_{\phi,t}(X_t, A_t) \approx \nabla_{A_t} \log p_t(X_t, A_t), \tag{146}$$

where X_t and A_t are the noisy node embeddings and adjacency matrix, and p_t is their joint distribution. Denoising score matching provides conditional perturbation-score targets during training. The node network combines features from successive HGAT layers, maps its output to the tangent spaces at X_t, and rescales it using a learned function of time t and the conformal factors $\lambda_{X_t}^c$. The edge network predicts a Euclidean adjacency score from the noisy node and edge information. Node scores thus lie in hyperbolic tangent spaces, while edge scores remain Euclidean.

HGDM builds its score network with a Hyperbolic Graph Attention Layer (HGAT), which performs neighborhood aggregation in the tangent space and projects results back to the manifold using exponential maps:

$$\mathbf{h}_i^{\text{out}} = \sigma_{\mathbb{H}} \left(\exp_{\mathbf{h}_i}^c \left(\mathrm{PT}_{\mathbf{0} \to \mathbf{h}_i}^c \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W \log_{\mathbf{0}}^c(\mathbf{h}_j) \right) \right) \right), \tag{147}$$

where $\mathrm{PT}_{\mathbf{0} \to \mathbf{h}_i}^c$ denotes parallel transport from the origin to point $\mathbf{h}_i$, and the neighbor features are first mapped into the origin tangent space. The transport places the aggregated tangent vector at the receiving node before the exponential map returns it to the manifold. After reverse sampling, the pretrained VAE decoder uses the generated hyperbolic embeddings and adjacency to reconstruct

node features; the sampled adjacency gives the graph edges.

3.7.2 Spherical Generative Models

Spherical VAEs represent latent direction separately from concentration. Davidson et al.'s Spherical Variational Autoencoder (*S-VAE*) [62] learns the concentration of a von Mises–Fisher (vMF) posterior, while Xu and Durrett's text model fixes it. S-VAE places the latent variable on $\mathbb{S}^{d-1} \subset \mathbb{R}^d$ and uses a vMF posterior with a uniform spherical prior. The vMF distribution has mean direction $\boldsymbol{\mu} \in \mathbb{S}^{d-1}$, concentration $\kappa_{\text{vMF}} \geq 0$, and density

$$q(z \mid \boldsymbol{\mu}, \kappa_{\text{vMF}}) = C_d(\kappa_{\text{vMF}}) \exp(\kappa_{\text{vMF}} \boldsymbol{\mu}^\top z), \quad z \in \mathbb{S}^{d-1}, \tag{148}$$

where $C_d(\kappa) = \kappa^{d/2-1} / ((2\pi)^{d/2} I_{d/2-1}(\kappa))$, and I_ν denotes the modified Bessel function of the first kind of order ν. The vMF distribution supports the uniform prior at $\kappa_{\text{vMF}} = 0$ and produces samples on the manifold. The uniform prior makes the KL divergence independent of the posterior mean direction. Its dependence on κ_{vMF} is

$$\mathcal{D}_{\text{KL}}(q \parallel p) = \kappa_{\text{vMF}} \frac{I_{d/2}(\kappa_{\text{vMF}})}{I_{d/2-1}(\kappa_{\text{vMF}})} + \log C_d(\kappa_{\text{vMF}}) + \text{const.}, \tag{149}$$

where the constant accounts for the uniform prior. S-VAE learns both mean direction and concentration, using a reparameterized rejection sampler followed by a Householder reflection to align samples with the mean direction.

Xu & Durrett [295] use a fixed κ_{vMF} in their spherical VAE for text generation. Fixing concentration makes the KL term constant, removing its gradient contribution and retaining the reconstruction objective's dependence on the mean direction. On document and neural language-modeling benchmarks, Xu and Durrett report lower negative log-likelihood and higher KL divergence than the compared Gaussian models. Their analysis also examines sensitivity to word order and the information retained in the latent code.

SphereGAN and SphericGAN place spherical geometry in different components: SphereGAN matches discriminator-feature moments, whereas SphericGAN organizes latent samples around class prototypes. *SphereGAN* [214] defines integral probability metrics (IPMs) on the hypersphere $\mathbb{S}^n$ by mapping discriminator outputs through the inverse stereographic projection:

$$\Pi^{-1}(p) = \left(\frac{2p}{\|p\|^2 + 1}, \frac{\|p\|^2 - 1}{\|p\|^2 + 1} \right), \quad p \in \mathbb{R}^n, \tag{150}$$

which maps Euclidean feature vectors onto $\mathbb{S}^n$ with the north pole $N = (0, \ldots, 0, 1)$ removed. SphereGAN places the polar coordinate last and uses N as its projection pole. It then defines central moment distances $d_r^s(N, x) = d_{\mathbb{S}^n}(N, x)^r$, where $r \in \mathbb{N}$. The discriminator maximizes, while the generator minimizes, the following moment matching objective:

$$\min_G \max_D \sum_{r=1}^{R} \left(\mathbb{E}_{x \sim P_{\text{real}}} d_r^s(N, D(x)) - \mathbb{E}_{z \sim P_z} d_r^s(N, D(G(z))) \right), \tag{151}$$

which matches a finite set of geometric moments. The paper relates convergence under its spherical IPM to Wasserstein convergence. The compact geometry bounds these moment functions and avoids the additional Lipschitz constraints or gradient penalties used by WGAN-GP and MMD-GAN. Higher-order moments increase the magnitude of the corresponding differentials and therefore

require controlled optimization.

SphericGAN [44] uses spherical latent prototypes for semi-supervised fine-grained image generation. It projects prior samples and real image embeddings onto a hypersphere, then aligns their clusters to preserve class-conditional diversity with limited labels. A latent vector $z^\circ \in \mathbb{S}^n$ is associated with learned prototype centers $\{P_k\}_{k=1}^K$, and cluster assignment is based on cosine similarity:

$$\phi(z^\circ)_k = \frac{\exp(\cos(z^\circ, P_k))}{\sum_{j=1}^K \exp(\cos(z^\circ, P_j))}. \tag{152}$$

Real image embeddings are projected via a mapping network $F(x) = z_x^\circ$, and real-fake alignment is enforced using a prototypical discriminator D_{prt} and a cycle-consistent regularizer:

$$\mathcal{L}_{\text{cons}} = \mathbb{E}_x \left[\| z_x^\circ - F(G(z_x^\circ, y)) \|_1 \right], \tag{153}$$

which encourages latent vectors of real images and reconstructions to be close. The full adversarial loss integrates three discriminators: a latent discriminator distinguishing z° and z_x°, a prototypical discriminator enforcing cluster-wise alignment, and a class-conditional discriminator D_{class} matching image-label pairs. Compared to SphereGAN, SphericGAN explicitly models both *class-dependent* and *class-independent* variation by introducing structured latent prototypes and cluster matching. Its architecture also incorporates label-guided synthesis and unlabeled data through pseudo-labeling; Chen et al. report improvements on the evaluated fine-grained datasets under semi-supervised constraints.

3.7.3 Mixed-curvature Generative Models

Mixed-curvature VAEs (mVAEs) [250] extend the single-manifold latent distributions of P-VAE and S-VAE to a product of factors. Each factor has its own dimension and curvature, so latent components can represent hyperbolic, spherical, or Euclidean relations within one decoder. The latent manifold is

$$\mathcal{M} = \prod_{i=1}^k \mathcal{M}_{c_i}^{n_i}, \qquad \sum_{i=1}^k n_i = n, \tag{154}$$

where $\mathcal{M}_{c_i}^{n_i}$ is an n_i-dimensional component with constant curvature $c_i \in \mathbb{R}$ (spherical $c_i > 0$, Euclidean $c_i = 0$, hyperbolic $c_i < 0$). Distances and geometric operators decompose componentwise; in particular:

$$d^2_{\mathcal{M}}(x, y) = \sum_{i=1}^k d^2_{\mathcal{M}_{c_i}^{n_i}} \left(x^{(i)}, y^{(i)} \right), \tag{155}$$

with $x^{(i)}$ the subvector of x assigned to the i-th factor. For each nonzero-curvature factor, the stereographic chart uses the conformal projection ρ_c and its inverse:

$$\rho_c\left(\begin{bmatrix}\xi \\ x\end{bmatrix}\right) = \frac{x}{1 + \sqrt{|c|}\,\xi}, \qquad \rho_c^{-1}(y) = \begin{bmatrix} \frac{1}{\sqrt{|c|}} \frac{1 - c\|y\|_2^2}{1 + c\|y\|_2^2} \\ \frac{2y}{1 + c\|y\|_2^2} \end{bmatrix}, \qquad c \neq 0, \tag{156}$$

Here $(\xi; x) \in \mathbb{R}^{n_i+1}$ is an ambient-space coordinate and $y \in \mathbb{R}^{n_i}$ is the projected coordinate, making the metric conformal with factor $\lambda_y^c = 2/(1 + c\|y\|_2^2)$. The Euclidean factor $c = 0$ is defined directly in projected coordinates with Euclidean operators; ρ_c^{-1} itself has no finite ambient-coordinate limit as $c \to 0$ because the curvature radius diverges. For variational inference on $\mathcal{M}$, mVAE instantiates

manifold-valued priors/posteriors using a Wrapped Normal family on each factor and concatenates samples across factors. For a component $\mathcal{M}_{c_i}^{n_i}$ with origin μ_0, sampling is reparameterized by tangent-space Gaussian noise transported and exponentiated ($v \sim \mathcal{N}(0, \Sigma) \in T_{\mu_0}\mathcal{M}_{c_i}^{n_i}$):

$$u = \mathrm{PT}_{\mu_0 \to \mu}^{c_i}(v) \in T_\mu \mathcal{M}_{c_i}^{n_i}, \quad z = \exp_\mu^{c_i}(u) \in \mathcal{M}_{c_i}^{n_i}, \tag{157}$$

where $\mu \in \mathcal{M}_{c_i}^{n_i}$ is the component mean, $\exp^{c_i}$ is the exponential map, and PT^{c_i} denotes parallel transport; the log-density follows from the change of variables through $f = \exp_\mu^{c_i} \circ \mathrm{PT}_{\mu_0 \to \mu}^{c_i}$. The resulting ELBO over the product manifold uses the Riemannian volume element and Monte Carlo KL estimation:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - D_{\mathrm{KL}}(q \| p), \tag{158}$$

$$D_{\mathrm{KL}}(q \| p) \approx \frac{1}{L} \sum_{\ell=1}^{L} \log \frac{q(z^{[\ell]})}{p(z^{[\ell]})}, \tag{159}$$

where $z^{[\ell]} \sim q_\phi(\cdot \mid x)$ denotes a complete product-space latent sample, θ, ϕ are decoder and encoder parameters, and $z = \big(z^{(1)}, \ldots, z^{(k)}\big) \in \mathcal{M}$ lists the factor coordinates of one sample. The distributions $p(z), q_\phi(z \mid x)$ factorize across components as Wrapped Normals (or Gaussian when $c_i = 0$). A further contribution is learning the curvature constants $\{c_i\}_{i=1}^k$ (equivalently radii $R_i = 1/\sqrt{|c_i|}$) by differentiating the ELBO with respect to curvature and optimizing them alongside network weights, yielding a mixed-curvature latent geometry that adapts to data while recovering the Euclidean VAE as $c_i \to 0$ after the fixed Euclidean coordinate rescaling described in Section 2.2.3.

Factorizing the prior and conditional posterior gives separate KL penalties for the latent components, while their concatenated samples enter a shared decoder for joint reconstruction. Distribution families remain part of the choice: a spherical factor can use mVAE's default wrapped-normal pair or S-VAE's vMF posterior with a uniform prior, while a Poincaré factor can also use P-VAE's Riemannian normal family [62, 187]. Factor counts and dimensions specify the allocation; ordinary curvature learning adjusts the selected factors, and the universal variant permits sign changes within a prescribed partition [250]. These choices are evaluated at a fixed total intrinsic latent dimension. In the six-dimensional MNIST comparison with diagonal covariance, the single learned spherical factor has a higher mean log-likelihood than the displayed mixed products, while the Euclidean–hyperbolic–spherical product remains close to the Euclidean baseline.

HGAN introduces hyperbolic transformations into an adversarial generator or discriminator, whereas HGDM learns score-based sampling dynamics after constructing a hyperbolic latent representation. Geometry also occupies different roles within spherical GANs: SphereGAN defines the discrepancy used by the discriminator on a hypersphere, while SphericGAN organizes latent variation through angular prototypes and cluster alignment. The relevant design decision is therefore which part of generation benefits from geometry: the latent distribution, the network transformation, the training discrepancy, or the sampling dynamics.

3.8 Foundation Model

Foundation models are general-purpose architectures pretrained on broad data at scale to learn reusable representations that can be adapted to diverse downstream tasks [21]. Geometric interventions change different parts of this pipeline. HELM and nGPT train geometric backbones, HypLoRA and MoSLoRA introduce geometric adaptation branches, retrieval methods change evidence selection, and multimodal methods constrain alignment between modalities. We distinguish geometry applied to representations from geometry used to update or merge parameters.

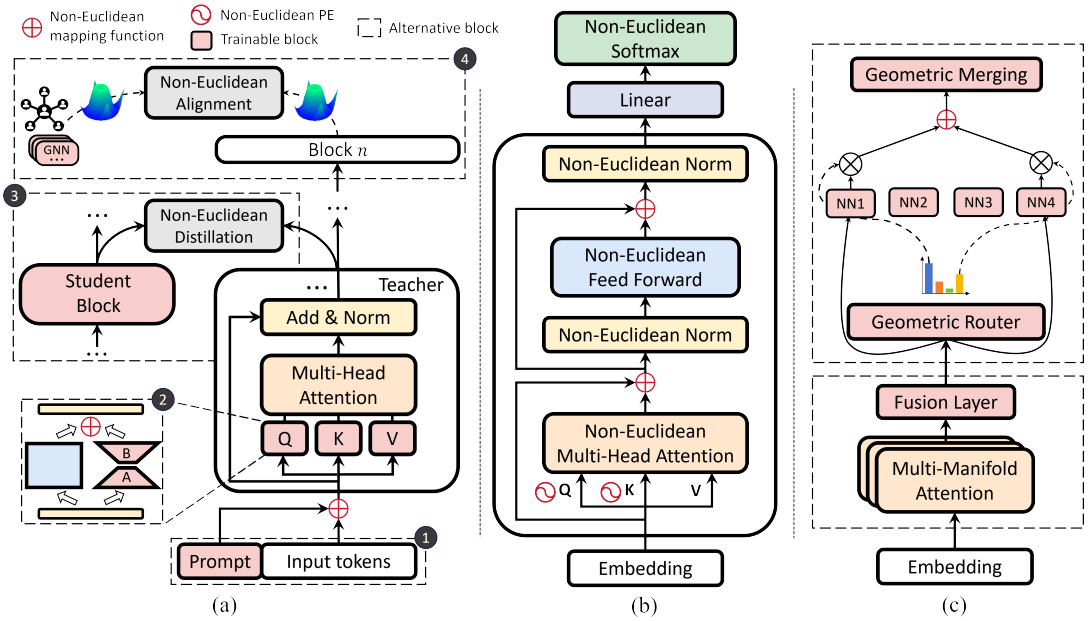


Figure 12 Geometry-aware foundation-model designs, adapted from [107]. (a) Fine-tuning inserts non-Euclidean mappings or trainable geometric blocks into an existing Euclidean backbone. (b) Geometry-native pretraining defines the embedding, attention, normalization, feed-forward, output, and softmax components in the target geometry. (c) Hybrid architectures route representations through geometry-specific branches and merge their outputs for downstream prediction.

3.8.1 Pretraining

Backbone pretraining. HELM and nGPT carry geometric constraints through successive backbone layers during both training and inference, as illustrated in Figure 12(b). *HELM* [106] uses Lorentz-valued token states, whereas nGPT uses hyperspherical normalization. HELM's dense variant, HELM-D, instantiates its core primitives in the Lorentz model $\mathbb{L}_c^n = \{\boldsymbol{x} \in \mathbb{R}^{n+1} : \langle \boldsymbol{x}, \boldsymbol{x} \rangle_{\mathcal{L}} = 1/c,\ x_t > 0\}$ with $c < 0$, where $\boldsymbol{x} = [x_t, \boldsymbol{x}_s]^\top$ and $\langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathcal{L}} = -x_t y_t + \boldsymbol{x}_s^\top \boldsymbol{y}_s$. A curvature-aware Lorentz linear transform maps $\mathbb{L}_{c_1}^n \to \mathbb{L}_{c_2}^m$ as:

$$\mathrm{HLT}(x; W, b) = \left[\sqrt{\frac{c_1}{c_2} \|W^\top x + b\|_2^2 - \frac{1}{c_2}},\ \sqrt{\frac{c_1}{c_2}} (W^\top x + b) \right]^\top, \tag{160}$$

with parameters $W \in \mathbb{R}^{(n+1)\times m}$ and $b \in \mathbb{R}^m$. Self-attention uses the Lorentz chord surrogate for matching and a Lorentz centroid for aggregation: for tokens $x_i \in \mathbb{L}_c^n$, queries/keys/values are $Q = \mathrm{HLT}(X; W_Q, b_Q)$, $K = \mathrm{HLT}(X; W_K, b_K)$, $V = \mathrm{HLT}(X; W_V, b_V)$, and the i-th output is the Lorentzian weighted centroid:

$$\mathrm{Att}_{\mathcal{L}}(x)_i = \frac{\sum_{j=1}^T \nu_{i,j} v_j}{\sqrt{c \left\langle \sum_{\ell=1}^T \nu_{i,\ell} v_\ell, \sum_{\ell=1}^T \nu_{i,\ell} v_\ell \right\rangle_{\mathcal{L}}}}, \tag{161}$$

$$\nu_{i,j} = \frac{\exp\left(-\tilde{d}^2_{\mathbb{L},c}(q_i, k_j)/\sqrt{m}\right)}{\sum_{\ell=1}^T \exp\left(-\tilde{d}^2_{\mathbb{L},c}(q_i, k_\ell)/\sqrt{m}\right)}, \tag{162}$$

Table 9 Geometric interventions in foundation-model pipelines. Comparing the affected component and geometric object separates representation-space designs from parameter-space optimization and merging.

Representative method	Component changed	Geometric object	Core operation
HELM-D / HELM-MiCE [106]	Pretrained backbone	Lorentz token states; expert-specific negative curvature in MiCE	Manifold-preserving layers and curvature-aligned expert routing
nGPT [180]	Pretrained backbone	Normalized token states and parameter vectors	Hyperspherical normalization and interpolated residual updates
HypLoRA [308]	Low-rank adapter	Token representations within a Lorentz adaptation branch	Hyperbolic low-rank transform with a Euclidean output interface
MoSLoRA [325]	Geometric adapter experts	Hyperbolic, spherical, and Euclidean expert representations	Token routing, learnable curvature, and projected output fusion
Riemannian LoRA [327]	Adapter optimization	Low-rank parameter factors	Riemannian preconditioning of parameter updates
CAMEx [202]	Expert merging	Neural network parameter space	Curvature-aware adaptation of expert merge updates
HyperbolicRAG [35]	Graph retriever	Passage and entity representations	Geometry-dependent seeding, shared-graph relevance propagation, and ranking fusion
HypRAG [183]	Dense retriever	Lorentz query and document embeddings	Geometric encoding, radius-weighted pooling, and contrastive training
MERU [64]; HyCoCLIP [209]	Multimodal alignment	Image, text, and compositional representations	Cross-modal matching with entailment constraints

where $\widetilde{d}^2_{\mathbb{L},c}$ is the squared Lorentz chord surrogate in Equation (11) and m is the head dimension. To match modern LLM positional inductive biases, HELM introduces Hyperbolic Rotary Positional Encoding (HoPE), a Lorentz rotation that applies Euclidean block rotations $R_{i,\Theta}$ only to the space-like component while preserving manifold validity:

$$\mathrm{HoPE}(z_i) = \left[\sqrt{\|R_{i,\Theta}(z_i)_s\|_2^2 - \frac{1}{c}},\ R_{i,\Theta}(z_i)_s\right]^\top \tag{163}$$

$$= \begin{pmatrix} 1 & 0 \\ 0 & R_{i,\Theta} \end{pmatrix} z_i, \tag{164}$$

where z_i is a query or key, $\Theta = \{\theta_\ell\}_{\ell=1}^{m/2}$ for even m, and each R_{i,θ_ℓ} rotates a 2D subspace by angle $i\theta_\ell$; the resulting attention score $-\widetilde{d}^2_{\mathbb{L},c}(\mathrm{HoPE}(q_a), \mathrm{HoPE}(k_b))$ depends on embeddings and relative offset $(a-b)$, enabling RoPE-like relative encoding properties.

Multi-curvature hyperbolic MoE. HELM-MiCE introduces a Mixture-of-Curvature Experts block in which every routed or shared expert remains a Lorentz expert with its own negative curvature, denoted by $c_{r,i} < 0$ or $c_{s,i} < 0$. The method is therefore a multi-curvature hyperbolic MoE rather than a mixed-sign representation model. For token $x_t \in \mathbb{L}^n_c$, top-K_r routed gates are computed by:

$$g_{t,i} = \frac{g'_{t,i}}{\sum_{j=1}^{N_r} g'_{t,j}}, \quad s_{t,j} = \mathrm{act}\big((x_t)_s^\top y_j\big), \tag{165}$$

$$g'_{t,j} = \begin{cases} s_{t,j}, & s_{t,j} \in \mathrm{Top}_k(\{s_{t,k}\}_{k \le N_r}, K_r) \\ 0, & \text{otherwise,} \end{cases} \tag{166}$$

where y_j is the j-th expert centroid and $\mathrm{act}(\cdot) \geq 0$ is a gating nonlinearity for which at least one selected score is positive. Tokens are curvature-aligned by scaling $\sqrt{c/c_{(\cdot)}}$ before expert processing, expert outputs are scaled back, merged via a Lorentzian centroid, and added with a Lorentz residual, yielding:

$$\mathrm{MiCE}(x_t) = x_t \oplus_{\mathcal{L}} \frac{S_t}{\sqrt{c\langle S_t, S_t\rangle_{\mathcal{L}}}}, \qquad S_t = \sum_{i=1}^{N_s} y_{t,i} + \sum_{i=1}^{N_r} g_{t,i} z_{t,i}, \tag{167}$$

where $y_{t,i}$ and $z_{t,i}$ denote (scaled-back) shared and routed expert outputs, and $\oplus_{\mathcal{L}}$ is the Lorentz residual operator. To address scaling bottlenecks, HELM further proposes Hyperbolic Multi-Head Latent Attention (HMLA), which projects tokens into low-dimensional latent (ℓ_t^Q, ℓ_t^{KV}) and reconstructs per-head $(q_{t,i}^C, k_{t,i}^C, v_{t,i}^C)$ while using a decoupled HoPE pathway $(q_{t,i}^R, k_t^R)$; for sequence length T, caching latent key-values reduces KV memory per layer from $O(2mHT)$ to $O((n_{kv} + n_r)T)$. Here H is the number of heads, m is their spatial width, and n_{kv} and n_r are the latent key-value and positional widths, respectively, with $n_{kv}, n_r \ll mH$. HELM also normalizes spatial features and recomputes the time coordinate through hyperbolic RMS normalization:

$$\mathrm{RMSN}_L(x) = \left[\sqrt{\|\mathrm{RMSN}(x_s)\|_2^2 - \frac{1}{c}},\ \mathrm{RMSN}(x_s) \right]^\top, \tag{168}$$

where $\mathrm{RMSN}(\cdot)$ is the Euclidean RMSNorm applied to the space-like part; the time-coordinate completion keeps the normalized output on $\mathbb{L}_c^n$. With these modules, HELM trains a dense hyperbolic variant (HELM-D) and a multi-curvature hyperbolic MoE variant (HELM-MiCE). Under the roughly 100M–1B parameter scales and 5B-token training budget evaluated by He et al., the authors report gains over matched Euclidean LLaMA/DeepSeek-style baselines on MMLU and ARC; they also report radial organization correlated with semantic specificity [106].

Pretraining also uses spherical backbones and domain-specific hyperbolic prediction heads. *nGPT* constrains token embeddings, projection matrices, and hidden states to the unit hypersphere during pretraining [180], while *HyperHELM* guides masked-codon pretraining of a Euclidean Transformer with fixed hyperbolic prototypes derived from the codon–amino-acid hierarchy [274]. These models are cross-listed under their architecture or application families and under the pretraining intervention point in Table 6 and Figure 2.

3.8.2 Adaptation

Hyperbolic prompt tuning. Prompt tuning introduces trainable input tokens, while hyperbolic adapters transform hidden representations through an additional branch. Hyperbolic prompt tuning models hierarchical relationships between prefix prompts and downstream tasks [43], as shown in Figure 12(a). Specifically, trainable prompt embeddings are prepended to the input token embeddings and passed through a pretrained model, following the standard prompt-tuning pipeline. The model outputs, such as the final hidden states, are then projected from Euclidean space to the Poincaré ball and optimized under hyperbolic geometry. In a complementary task-specific setting, Welz et al. [289] add a Poincaré layer to a soft-prompt-tuned T5 pipeline for multi-hop question answering and compare it with a parameter-matched Euclidean layer.

Hyperbolic PEFT. LoRA adapts a pretrained weight matrix with a low-rank update [114]. HypLoRA [308] applies a low-rank transformation to the spatial coordinates of Lorentz token rep-

resentations and reconstructs the time coordinate. This generally gives a nonlinear adapter update while retaining a Euclidean interface to the pretrained backbone:

$$\begin{aligned}\mathbf{z}^E &= \mathcal{T}_{\text{HypLoRA}}(\mathbf{x}^E) = \mathbf{W}\mathbf{x}^E + \Delta_{\text{HypLoRA}}(\mathbf{x}^E) \\ &= \mathbf{W}\mathbf{x}^E + \left[\log^c_{\mathbf{o}_c}\left(\text{LorentzTrans}_c(\mathbf{A},\mathbf{B},\exp^c_{\mathbf{o}_c}([0,(\mathbf{x}^E)^T]^T))\right)\right]_{[2:n+1]},\end{aligned} \tag{169}$$

Here Δ_{HypLoRA} denotes the nonlinear adapter update. For a same-width adapter with rank r, $\mathbf{x}^E, \mathbf{z}^E \in \mathbb{R}^n$, $\mathbf{W} \in \mathbb{R}^{n\times n}$, $\mathbf{A} \in \mathbb{R}^{r\times n}$, and $\mathbf{B} \in \mathbb{R}^{n\times r}$. For the lifted point $\mathbf{x}^H \in \mathbb{L}^n_c$, let $\mathbf{x}^H_s \in \mathbb{R}^n$ be its spatial coordinates. The transformation is

$$\text{LorentzTrans}_c(\mathbf{A},\mathbf{B},\mathbf{x}^H) = \left[\sqrt{\|\mathbf{B}\mathbf{A}\mathbf{x}^H_s\|_2^2 - \frac{1}{c}},\ (\mathbf{B}\mathbf{A}\mathbf{x}^H_s)^T\right]^T. \tag{170}$$

The completion enforces the Lorentz constraint for $c < 0$, and the logarithmic-map slice $[2 : n+1]$ returns an n-dimensional Euclidean update. The adapter can change both the direction and radial position of a token representation before returning an update to the Euclidean backbone. Geometry-aware low-rank adaptation also includes Riemannian preconditioning of LoRA factors [327] and dynamic-rank Riemannian adaptation in federated fine-tuning [337]. These methods use geometry to control the optimization of low-rank parameters. Riemannian preconditioning leaves a fixed low-rank update that can be merged into the pretrained weight after optimization. HypLoRA generally evaluates its geometric adapter branch during inference, adding projections and Lorentz transformations at the adapted layers.

Mixed-curvature PEFT. MoSLoRA [325] introduced a parameter-efficient fine-tuning framework that equips LLMs with heterogeneous geometric experts spanning hyperbolic ($c < 0$), spherical ($c > 0$), and Euclidean ($c = 0$) constant-curvature spaces, and learns both expert parameters and per-expert curvatures to match heterogeneous semantic subspaces.

For an ambient expert output $(\xi'; s')$, we denote its projection into the original feature space by ρ_c. For $c \neq 0$, this stereographic output projection requires $1 + \sqrt{|c|}\xi' \neq 0$; the $c = 0$ branch returns the spatial coordinates directly and is defined separately. Input and output scaling accompany the coordinate maps, which avoid repeated exponential and logarithmic maps. Appendix A.2 gives the construction, including the hyperbolic input domain and the spherical completion on the nonnegative-first-coordinate hemisphere. MoSLoRA instantiates N geometric LoRA experts in each layer with learnable expert-specific curvature c_i, routes each token x_j to a sparse top-K set through $R(x_j)_i$, and merges projected expert outputs in the original space as

$$o_j = \sum_{i=1}^{N} R(x_j)_i\, \rho_{c_i}\left(\begin{bmatrix}\xi_{j,i}\\ s'_{j,i}\end{bmatrix}\right), \tag{171}$$

where $(\xi_{j,i}; s'_{j,i})$ denotes the ambient-space output produced by applying the FFN transformation (with LoRA updates) within the i-th geometric expert. To stabilize mixed-curvature training and encourage effective exploration of geometry, the method separates curvature parameters from capacity parameters and updates them with distinct optimizers.

MoSLoRA extends the geometric adapter design from a Lorentz branch to token-routed experts with different curvature signs. Its relation to HELM-MiCE is more specific: both route tokens among geometric experts, but HELM-MiCE builds a pretrained backbone from negatively curved Lorentz experts, while MoSLoRA adapts a pretrained model using hyperbolic, spherical, and Euclidean experts.

Expert routing, curvature diversity, and the stage of training are therefore separate comparison axes.

Curvature-aware expert merging. For Sparse Mixture-of-Experts (SMoE) backbones, geometry also enters through expert merging. Nguyen et al. [202] propose CAMEx, which uses dynamic curvature adaptation to merge experts in the Riemannian geometry of neural network parameter spaces. CAMEx operates on parameter-space geometry and does not define a mixed-curvature representation backbone. Its merge rule uses a natural-gradient-inspired preconditioner:

$$\theta_{n+1} = \theta_n - \eta\, P(\theta_n)\nabla\mathcal{L}(\theta_n), \tag{172}$$

θ_n denotes parameters at step n, η is the step size, $\mathcal{L}$ is the training objective, and $P(\theta_n)$ is a metric-induced preconditioner, such as an inverse-Fisher approximation, that rescales gradients according to local geometry. Translating this principle to SMoE merging, let vec stack an expert matrix columnwise. CAMEx updates a merged expert at layer l as:

$$\mathrm{vec}(\hat{E}_m^l) = \mathrm{vec}(E_m^l) + \alpha \sum_{i=1}^{N-1} M_i\, \mathrm{vec}\left(s_i^l \odot \tau_i^l\right), \tag{173}$$

where $E_m^l \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the current merged expert, $\hat{E}_m^l$ is the curvature-adapted merged expert, N is the number of experts, $\alpha \in (0, 1]$ is a merge scale, $\tau_i^l = E_i^l - E_m^l$ is the domain vector of expert i, s_i^l is a router-derived coefficient broadcast to the parameter shape, and M_i is a linear operator on the $d_{\text{out}}d_{\text{in}}$-dimensional parameter vector. CAMEx learns M_i through backpropagation of the task loss. The natural-gradient interpretation motivates how this transform adapts an expert's merge direction to parameter-space geometry. A Kronecker factorization makes the learned transform practical for large expert matrices:

$$M_i \approx M_i^{\text{in}} \otimes M_i^{\text{out}}, \quad M_i^{\text{in}} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{in}}}, \;\; M_i^{\text{out}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{out}}}, \tag{174}$$

which acts consistently on $\mathrm{vec}(E)$ and reduces memory and compute. Equivalently, this operator applies the matrix transformation $M_i^{\text{out}} E (M_i^{\text{in}})^T$. The factors are learned during training to refine the merge transform.

3.8.3 Retrieval and Augmented Generation

Geometry-aware retrieval changes the representations used to select evidence for a generator. Compared with geometric backbone pretraining or adaptation, it places the geometric operation in the retriever and corpus index, allowing the generation backbone to remain unchanged.

For text, HyperbolicRAG augments a corpus graph, whereas HypRAG trains dense text encoders. HyperbolicRAG [35] builds passage and entity nodes and learns depth-aware Poincaré representations. It forms Euclidean- and hyperbolic-similarity seeds, propagates the two relevance signals separately over the same graph, and fuses their passage rankings. HypRAG [183] instead trains fully hyperbolic or hybrid dense encoders. Its Outward Einstein Midpoint weights Lorentz token representations by radius when pooling document embeddings. HyperbolicRAG thus relies on graph construction and ranking fusion, whereas HypRAG relies on encoder training and geometric pooling; both pass the retrieved passages to a generator.

Multimodal retrieval uses the same property across modalities, where a caption is more general than the item it describes and the resulting *is-a* relation is represented by containment in hyperbolic space. Image–text models such as MERU and HyCoCLIP train both modalities in a shared hyperbolic space using contrastive and entailment objectives. Retrieval ranks candidates by hyperbolic similarity,

while entailment shapes their hierarchical organization [64, 209]. *HLFormer* extends this to partially relevant video retrieval, where a query matches part of a video rather than the whole [152], and hierarchical alignment reasoning carries it to text–3D retrieval [158].

AMCAD applies mixed-curvature retrieval to query–item–advertisement interactions in a web-scale advertisement system [297]. It provides a deployment example in which adaptive geometry shapes candidate retrieval, complementing the evidence-retrieval role in augmented generation.

3.8.4 Multimodal Foundation Models

Hyperbolic multimodal alignment. An image and its caption describe overlapping content, while a crop and a short phrase can isolate one component of that content. Alignment must specify both which observations match and which contain additional context. MERU [64] trains image and text encoders in a shared Lorentz space, combining distance-based contrastive matching with a constraint that places the image inside its caption's entailment cone. HyCoCLIP [209] adds grounded crop–phrase pairs and within-modality containment. In its construction, an isolated crop or noun phrase is more general than the complete image or caption, which adds scene context. The image–text cone constraint applies at both scales; additionally, the complete image and caption lie inside their corresponding crop and phrase cones (Figures 10 and 19). Its contrastive objective also distinguishes whole inputs from components when selecting negatives, since similar objects can occur in different images. These additional relations require crop–phrase correspondences; the paper obtains them offline with noun-phrase extraction and a pretrained grounding model. The learned representation thus depends on these crop–phrase correspondences as well as the curvature.

Connecting visual evidence to a language generator requires a further interface. In the BLIP-2 architecture [153], a trainable Q-Former and projection translate features from a frozen vision encoder into the input space of a frozen language model. HVLM [186] introduces Poincaré mappings at the Q-Former's visual-query and text outputs, retaining this division between the learned bridge and pretrained backbones. Its working contrastive formulation uses cosine similarity on the mapped outputs. Random Query Selection changes which visual query receives the caption-alignment signal, while Random Text Pruning removes contiguous caption tokens during training. These choices shape the information distributed across the queries that are subsequently passed to the language model. The geometric alignment acts at the bridge; the generator consumes the projected query sequence through its existing token interface.

An existing multimodal system can also be adapted by changing its visual attention projections. HyperET [217] learns structured matrices that adjust frozen pretrained vision-attention weights within an MLLM training pipeline. Diagonal, block-diagonal, and banded forms control which coordinates can mix and how many parameters are learned. HyperCLIP [218] draws on hyperbolic fine-tuning [308] to learn block-diagonal adjustments in both the image and text branches of CLIP, while keeping the original CLIP weights frozen. Its adapted features feed a dense prediction decoder. The two designs therefore update different components: the visual branch of an MLLM, and the paired CLIP branches used for pixel–class comparisons.

The ideal map used to motivate these adjustments has a useful algebraic interpretation. For a tangent vector v at the Poincaré origin and a compatible matrix $\mathbf{M}$, standard Möbius matrix multiplication with matching origin and curvature gives $\log_{\mathbf{0}}^{c}(\mathbf{M} \otimes_c \exp_{\mathbf{0}}^{c}(v)) = \mathbf{M}v$ when the maps are unmodified [217, 218]. Thus the ideal round trip implements a structured matrix action between Euclidean interfaces. Uniform positive scaling preserves a direction while changing radius; a general diagonal, block, or banded matrix can change direction as well. Matrix structure and placement remain substantive adaptation choices, together with any constraints applied to intermediate representations.

For safety, HySAC learns entailment relations that separate unsafe and safe content in a vision–language model [223].

Table 9 compares these methods by the component they change and the object on which geometry acts. Backbone designs such as HELM and nGPT carry geometric constraints through successive layers; HypLoRA and MoSLoRA concentrate geometric processing in adaptation branches. Riemannian LoRA and CAMEx instead operate on parameter updates or expert merging. Retrieval and multimodal alignment use geometric relations to select evidence or couple modalities.

3.9 Learning Paradigms

The learning paradigms below place geometry in three complementary roles: a task-specific distance for metric learning, a manifold-aware similarity for contrastive learning, and a representation or control variable for reinforcement learning.

3.9.1 Metric Learning

Metric learning optimizes distances or similarities so that neighborhoods reflect task-specific relations [140]. Geometric variants differ in the source of those relations and the object being constrained: observed pairs, inferred hierarchies, learned proxies, or embedding directions.

Hyperbolic metric learning. HyperML learns from observed user-item pairs, UHML derives similarity levels through clustering, and HIER introduces latent ancestor proxies. Their common Poincaré representation therefore supports distinct supervision mechanisms. HyperML [267] uses $\mathbb{P}_c^n$ to represent hierarchical user-item relationships in recommender systems. Specifically, each user u_i and item v_j is represented by $\boldsymbol{u}_i, \boldsymbol{v}_j \in \mathbb{P}_c^n$, respectively. A pull-and-push loss function is introduced to encourage user embeddings to be closer to positively interacted items and farther from negative ones:

$$\mathcal{L}_P = \sum_{(i,j)\in\mathcal{S}} \sum_{(i,k)\notin\mathcal{S}} \left[m + d_{\mathbb{P}}^c(\boldsymbol{u}_i, \boldsymbol{v}_j)^2 - d_{\mathbb{P}}^c(\boldsymbol{u}_i, \boldsymbol{v}_k)^2 \right]_+, \tag{175}$$

where $\mathcal{S}$ is the set of observed (positive) user-item interactions, k denotes a negative item not interacted by user i, j is an item user i liked, m is the margin, and $[z]_+ = \max(0, z)$ is the hinge function. To preserve global structure when mapping from Euclidean to hyperbolic space, a distortion loss is introduced:

$$\mathcal{L}_D = \sum_{(i,j)\in\mathcal{S}} \frac{\left| d_{\mathbb{P}}^c(f(\boldsymbol{e}_{u_i}), f(\boldsymbol{e}_{v_j})) - d_{\mathbb{E}}(\boldsymbol{e}_{u_i}, \boldsymbol{e}_{v_j}) \right|}{d_{\mathbb{E}}(\boldsymbol{e}_{u_i}, \boldsymbol{e}_{v_j})} + \sum_{(i,k)\notin\mathcal{S}} \frac{|d_{\mathbb{P}}^c(f(\boldsymbol{e}_{u_i}), f(\boldsymbol{e}_{v_k})) - d_{\mathbb{E}}(\boldsymbol{e}_{u_i}, \boldsymbol{e}_{v_k})|}{d_{\mathbb{E}}(\boldsymbol{e}_{u_i}, \boldsymbol{e}_{v_k})}, \tag{176}$$

where $\boldsymbol{e}_{u_i}$ and $\boldsymbol{e}_{v_j}$ are Euclidean precursor embeddings, f maps them to the Poincaré ball, and the normalized terms are evaluated only for pairs with nonzero Euclidean distance. This loss encourages faithful distance preservation during embedding. Finally, the two components are unified into a multi-task learning framework:

$$\min \mathcal{L} = \mathcal{L}_P + \gamma \mathcal{L}_D, \tag{177}$$

where γ balances the metric learning objective and distortion minimization.

UHML [298] replaces observed pair labels with similarity levels inferred by hierarchical clustering. In their approach, features extracted by a Euclidean metric learning model are projected onto the hyperbolic space via the exponential map, enabling embeddings in the Poincaré ball $\mathbb{P}_c^n$. To guide the learning process without explicit labels, a hierarchical clustering module generates similarity measures $\mathcal{S} = \{s_{ij}\}_{i,j=1}^n$, where $s_{ij} \in \{1, \ldots, K\}$, reflecting different levels of similarity between

sample pairs. The training objective is defined by a log-ratio loss function that enforces consistency between coordinate-space distance ratios of Poincaré embeddings and hierarchical similarity ratios. For any selected triplet of embeddings $(\mathbf{z}_i, \mathbf{z}_j, \mathbf{z}_l)$, with corresponding entries s_{ij} and s_{il} in $\mathcal{S}$, the loss is formulated as:

$$\mathcal{L}(i,j,l) = \left(\log \frac{\|\mathbf{z}_i - \mathbf{z}_j\|_2}{\|\mathbf{z}_i - \mathbf{z}_l\|_2} - \log \Omega^{s_{ij} - s_{il}} \right)^2, \tag{178}$$

where $\Omega > 0$ is a scaling hyperparameter controlling the similarity ratio. The reported UHML objective uses Euclidean distances between Poincaré-ball coordinates. The target $\Omega^{s_{ij}-s_{il}}$ specifies a relative coordinate-distance ratio for each triplet, transferring the clustering hierarchy into the learned embeddings.

For supervised visual metric learning, Ermolov et al. [74] compare hyperbolic and cosine distances within a pairwise cross-entropy objective. A vision encoder and fully connected projection produce image features, which the hyperbolic variant maps to $\mathbb{P}_c^n$ through the exponential map. During each training iteration, a batch of $K = 2N$ samples is formed by selecting N different categories with two samples per category, creating N positive pairs. The cosine-distance baseline between embeddings z_i and z_j is defined as:

$$D_{\cos}(z_i, z_j) = \frac{1}{2} \left\| \frac{z_i}{\|z_i\|_2} - \frac{z_j}{\|z_j\|_2} \right\|_2^2 = 1 - \frac{z_i^T z_j}{\|z_i\|_2 \|z_j\|_2}. \tag{179}$$

The pairwise cross-entropy loss for a positive pair (i, j) is formulated as:

$$l_{i,j} = -\log \frac{\exp\left(-D(\mathbf{z}_i, \mathbf{z}_j)/\tau\right)}{\sum_{k=1, k\neq i}^{K} \exp\left(-D(\mathbf{z}_i, \mathbf{z}_k)/\tau\right)}, \tag{180}$$

where D denotes the chosen distance function (either hyperbolic distance $d_{\mathbb{P}}^c$ or cosine distance $D_{\cos}$), and τ is a temperature hyperparameter controlling the softness of the distribution. Changing D preserves the pair-selection rule while changing the geometry used to rank the candidates.

HIER [130] supplements sample-to-sample metric learning with learnable ancestor proxies in $\mathbb{P}_c^n$. As shown in Figure 13, these proxies represent intermediate concepts that connect related samples to a shared hierarchy. To construct the training triplets (x_i, x_j, x_k), where x_i and x_j are semantically related and x_k is unrelated, HIER uses reciprocal nearest neighbors rather than class labels:

$$T = \left\{ (x_i, x_j, x_k) \mid x_j \in R_K(x_i) \wedge x_k \notin R_K(x_i) \right\}, \tag{181}$$

$$R_K(x) = \{ x' \mid x' \in N_K(x) \wedge x \in N_K(x') \}, \tag{182}$$

where $N_K(x)$ denotes the set of K-nearest neighbors of x in the hyperbolic space. Each hierarchical proxy $\rho \in \mathcal{Q}$ serves as a potential least common ancestor (LCA) for a pair of related embeddings. An unnormalized positive score for ρ to serve as the LCA of x_i and x_j is

$$a_{ij}(\rho) = \exp\left(-\max\left(d_{\mathbb{P}}^c(x_i, \rho), d_{\mathbb{P}}^c(x_j, \rho)\right)\right). \tag{183}$$

An LCA proxy ρ_{ij} is sampled using the Gumbel-max trick:

$$\rho_{ij} = \arg\max_{\rho \in \mathcal{Q}} \left(\log a_{ij}(\rho) + g_\rho \right), \quad g_\rho \sim \text{Gumbel}(0, 1). \tag{184}$$

Similarly, the global LCA ρ_{ijk} for the entire triplet (x_i, x_j, x_k) is selected from $\mathcal{Q} \setminus \{\rho_{ij}\}$. The HIER loss

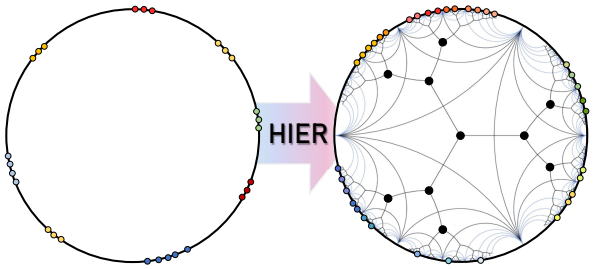


Figure 13 Hierarchical proxy learning in HIER [130]. The left panel shows embeddings concentrated near the boundary of the Poincaré ball. HIER introduces hierarchical proxies (larger black points) in the interior and connects samples through proxy-mediated geodesic relations, producing the tree-like organization shown on the right.

for a triplet $t = (x_i, x_j, x_k)$ is defined as a sum of three margin-based triplet losses:

$$\begin{aligned}\mathcal{L}_{\text{HIER}}(t) = &\left[d^c_{\mathbb{P}}(x_i, \rho_{ij}) - d^c_{\mathbb{P}}(x_i, \rho_{ijk}) + m_{\text{HIER}}\right]_+ \\ &+ \left[d^c_{\mathbb{P}}(x_j, \rho_{ij}) - d^c_{\mathbb{P}}(x_j, \rho_{ijk}) + m_{\text{HIER}}\right]_+ \\ &+ \left[d^c_{\mathbb{P}}(x_k, \rho_{ijk}) - d^c_{\mathbb{P}}(x_k, \rho_{ij}) + m_{\text{HIER}}\right]_+,\end{aligned} \tag{185}$$

where $[z]_+ = \max(z, 0)$ and $m_{\text{HIER}} > 0$ is the triplet margin. This objective encourages related samples (x_i, x_j) to be close to their shared ancestor ρ_{ij} and far from the global ancestor ρ_{ijk}, while the unrelated sample x_k is pushed closer to ρ_{ijk} and away from ρ_{ij}. Over time, this constructs a tree-like structure within $\mathbb{P}^n_c$, where proxies near the center represent higher-level concepts. The overall loss function combines the metric learning loss $\mathcal{L}_{\text{ML}}$ based on cosine distances or hyperbolic distances with the HIER regularization term:

$$\mathcal{L} = \mathcal{L}_{\text{ML}} + \lambda \sum_{t \in T_x \cup T_\rho} \mathcal{L}_{\text{HIER}}(t), \tag{186}$$

where λ is a balancing hyperparameter, T_x and T_ρ are sets of triplets among samples and proxies respectively.

Prototype-based formulations replace pairwise supervision with class prototypes: hyperbolic Busemann learning places ideal prototypes on the boundary of the Poincaré ball and trains with a penalized Busemann distance [87]. Rather than fixing one distance in advance, Li et al. [155] learn a geometry-aware distance in hyperbolic space that adapts to the diversity of hierarchical structures present in the data.

Spherical metric learning. SEC controls the radial variation of learned embeddings, whereas SEE synthesizes new embeddings through angular transformations. The spherical embedding constraint (SEC) of Zhang et al. [326] regularizes embedding norms. This constraint encourages embeddings to lie near a common, data-dependent Euclidean hyperspherical shell whose radius is determined by mini-batch statistics rather than by a fixed curvature parameter. Specifically, SEC penalizes deviations of individual embedding norms from the average norm within a mini-batch. Given an embedding f_i, the overall training objective is formulated as:

$$\mathcal{L} = \mathcal{L}_{\text{ML}} + \lambda \mathcal{L}_{\text{SEC}}, \tag{187}$$

where $\mathcal{L}_{\text{SEC}} = \frac{1}{N} \sum_{i=1}^{N} \left(\|f_i\|_2 - \mu\right)^2$ is the spherical regularization term, $\mu = \frac{1}{N} \sum_{j=1}^{N} \|f_j\|_2$ denotes the average embedding norm in the mini-batch, and λ is a balancing coefficient. By enforcing this soft

constraint, SEC encourages embeddings to have similar magnitudes, which in turn promotes more stable and balanced updates in angular space. Consequently, the directional changes of embeddings are primarily determined by their angular relationships with other samples, rather than being dominated by norm discrepancies. Spherical embedding expansion (SEE) [146] uses the angular degrees of freedom on $\mathbb{S}_c^n$ to augment embeddings while preserving their relation to a class proxy. Given a set of normalized class proxies $w = \{w_1, \ldots, w_C\}$ and an embedding vector z with label $y \in \mathcal{Y} = \{1, \ldots, C\}$, SEE synthesizes n_{aug} additional vectors $\{z_i^*\}$ such that each z_i^* preserves the angular distance to its class proxy w_y, while the mutual angular distances among $\{z_i^*\}$ are maximized. This ensures that the synthetic samples remain class-consistent yet diverse, enhancing the robustness of the learned metric space. To avoid potential optimization instability in early training, SEE adopts a curriculum learning strategy, selectively applying augmentation only to the top-k samples in each mini-batch with the smallest distances to their respective proxies. The SEE-specific loss is defined as:

$$\mathcal{L}_{\text{SEE}} = \mathbb{E}_{(z,y)\sim\widehat{P}_{\mathcal{A}_k}} \left[\sum_{i=1}^{n_{\text{aug}}} \ell(z_i^* \mid y, \boldsymbol{w}) \right], \tag{188}$$

where $\mathcal{A}_k$ is the set of selected labeled embeddings in the mini-batch, $\widehat{P}_{\mathcal{A}_k}$ is its empirical distribution, and ℓ is any proxy-based metric loss. The overall training objective combines the original metric loss and the auxiliary SEE loss:

$$\mathcal{L} = \mathcal{L}_{\text{ML}} + \lambda \mathcal{L}_{\text{SEE}}, \tag{189}$$

where λ is a balancing hyperparameter.

3.9.2 Contrastive Learning

Contrastive learning matches positive pairs against a set of alternatives [208]. With normalized embeddings, alignment and uniformity describe the balance between matching related samples and spreading representations on a hypersphere [286]. Geometric extensions change the similarity function, the views being matched, or the factors assigned to different relations. We distinguish these choices because the same contrastive objective can support several geometric constructions.

Table 10 compares the representations and relations matched by these objectives.

Hyperbolic contrastive learning. HGCL contrasts coordinate-model views of a graph, whereas HCL contrasts augmented image views within a Poincaré representation. HGCL [173] uses Poincaré and Lorentz GNN encoders for semi-supervised node classification. These are two coordinate models of hyperbolic geometry; the contrastive objective connects their learned representations. In HGCL, the input graph in Euclidean space is encoded by two HGCNs using the Poincaré ball $\mathbb{P}_c^n$ and the Lorentz model $\mathbb{L}_c^n$ as views α and β, respectively, to obtain two node embeddings, denoted as h^α and h^β. The method refines the embeddings with a hyperbolic position consistency (HPC) loss $\mathcal{L}_{\text{hpc}}$, which combines consistency and tolerance mutual information across the two views:

$$\mathcal{L}_{\text{HPC}} = -\frac{1}{2n} \sum_{i=1}^{n} \Bigg[\underbrace{\text{MI}\left(h_i^\alpha, t_1(h_i^\beta)\right) + \text{MI}\left(h_i^\beta, t_2(h_i^\alpha)\right)}_{\text{P1: Consistency}} + \underbrace{\text{MI}\left(h_i^\alpha, \mathcal{N}(h_i^\alpha)\right) + \text{MI}\left(h_i^\beta, \mathcal{N}(h_i^\beta)\right)}_{\text{P2: Tolerance}} \Bigg], \tag{190}$$

where MI is a local estimator based on the Jensen-Shannon divergence to distinguish between positive and negative sample pairs, defined as:

$$\text{MI}\left(h_i^\alpha, t_1(h_i^\beta)\right) = \mathbb{E}_P \left[\log D\left(h_i^\alpha, t_1(h_i^\beta)\right) \right] + \lambda_n \sum_{j=1}^{m} \mathbb{E}_{P\times\bar{P}} \left[\log\left(1 - D\left(h_i^\alpha, \bar{t}_1(h_j^\beta)\right)\right) \right], \tag{191}$$

Table 10 Geometric choices in contrastive learning. Each row lists the representations being matched and the relation imposed by the objective. Product-hyperbolic models use negative curvature in every factor; mixed-sign views include hyperbolic, spherical, and Euclidean geometry.

Method	Representations compared	Relation encoded
HGCL [173]	Poincaré and Lorentz graph-encoder views	Agreement for the same node and consistency with its neighborhood
HCL [324]	Augmented image embeddings in a Poincaré space	Augmentation invariance under hyperbolic similarity
Ge et al. [85]	Euclidean object and hyperbolic scene representations	Object-level matching and hierarchical scene relations
HiHPQ [226]	Quantized embeddings in hyperbolic product factors	Agreement between augmented views after quantization
PHyCLIP [315]	Vision–language embeddings in an ℓ_1-product of hyperbolic factors	Hierarchy within concept families and composition across factors
SELFMGNN [257]	Graph augmentations within and across mixed-sign geometric views	Node identity across augmentations and geometric views

$$\mathrm{MI}\left(h_i^\alpha, \mathcal{N}(h_i^\alpha)\right) = \sum_{h_j \in \mathcal{N}(h_i^\alpha)} \mathbb{E}_P\left[\log D\left(h_i^\alpha, h_j\right)\right] + \lambda_n \sum_{j=1}^{m} \mathbb{E}_{P\times\bar{P}}\left[\log\left(1 - D\left(h_i^\alpha, \bar{h}_j^\alpha\right)\right)\right]. \tag{192}$$

Here t_1 and t_2 map between the two coordinate models, and $\mathcal{N}$ denotes a node neighborhood. The discriminator $D(\cdot,\cdot) \in (0,1)$ is distance-aware and estimates sample similarity, P is the joint distribution of positive pairs, and $P \times \bar{P}$ forms negative pairs. The number of negative samples is m, and λ_n is their penalty coefficient.

HCL [324] keeps augmentation-based image pairs and changes the distance used to compare them. Its contrastive objective in the Poincaré ball $\mathbb{P}_c^n$ is

$$\mathcal{L}_{\text{hyp}}^{\text{self}} = -\sum_{i\in I} \log \frac{\exp\left(-D(z_i, z_{j(i)})/\tau\right)}{\sum_{a\in A(i)} \exp\left(-D(z_i, z_a)/\tau\right)} \tag{193}$$

where $\tau \in \mathbb{R}^+$ is a scalar temperature parameter, $A(i) \equiv I \setminus \{i\}$, and D is a dissimilarity, chosen as either cosine distance $1 - \cos(\cdot,\cdot)$ or the Poincaré distance $d_{\mathbb{P}}^c(\cdot,\cdot)$. For a batch of $2N$ augmented views, anchor i has one positive $j(i)$ and $2N-2$ negatives, giving $2N-1$ terms in the denominator.

Ge et al. [85] assign different geometries to object and scene relations. A Euclidean contrastive loss $\mathcal{L}_{\text{euc}}$ learns object representations, while a Poincaré loss $\mathcal{L}_{\text{hyp}}$ captures hierarchical scene structure. Both use a temperature-scaled pairwise objective of the form in Equation (193); the distinction lies in the semantic level and distance associated with each branch.

Product-hyperbolic contrastive quantization. HiHPQ [226] applies contrastive learning to quantized embeddings, adding a compact-code objective to geometric representation learning. It represents multilevel semantic relationships in the product space $\mathcal{P} = \times_{i=1}^{k} \mathbb{L}_{c_i}^{n_i}$, where every factor is a Lorentz model with negative curvature $c_i < 0$. HiHPQ is therefore product-hyperbolic even though its factors can have different negative curvatures. Given a mini-batch $\mathcal{B}$ of size N_B, two random augmentations are applied to each input x_i, resulting in $2N_B$ views $\{(x_i^{(1)}, x_i^{(2)})\}_{i=1}^{N_B}$. The encoder and quantization

module produce quantized embeddings $\{(\tilde{h}_{x_i}^{(1)}, \tilde{h}_{x_i}^{(2)})\}_{i=1}^{N_B}$, and the quantized contrastive loss is

$$\mathcal{L}_{\text{aug}} = \frac{1}{N_B} \sum_{x \in \mathcal{B}} \left(\ell^1(x) + \ell^2(x) \right), \tag{194}$$

where the partial loss $\ell^{(j)}(x)$, for $j \in \{1, 2\}$, is

$$\ell^{(j)}(x) = -\log \frac{S(\tilde{h}_x^{(j)}, \tilde{h}_x^{(3-j)})}{S(\tilde{h}_x^{(j)}, \tilde{h}_x^{(3-j)}) + \sum\limits_{\substack{t \in \mathcal{B} \setminus \{x\} \\ n=1,2}} S(\tilde{h}_x^{(j)}, \tilde{h}_t^{(n)})}. \tag{195}$$

For the query $\tilde{h}_x^{(j)}$, the other augmentation of x is the positive key and the $2N_B - 2$ views of other images are negative keys. The product-space similarity is

$$S(\tilde{h}_1, \tilde{h}_2) = \exp\left(-\frac{d_{\mathcal{P}}(\tilde{h}_1, \tilde{h}_2)}{\tau_{qc}} \right), \tag{196}$$

where $d_{\mathcal{P}}$ is the product-space distance and τ_{qc} is a temperature parameter.

The distribution of representations also matters within a fixed hyperbolic space. Zhang et al. [333] characterize and mitigate dimensional collapse in graph contrastive learning, where embeddings concentrate near a low-dimensional submanifold. Applications extend the pairwise objective to other relations: *Geo-Sign* regularizes skeletal representations for sign language translation in the Poincaré ball [78], while other methods target session-based recommendation [98] and multi-view hierarchical clustering [169].

PHyCLIP also uses a product of hyperbolic factors, but targets compositional vision–language alignment rather than quantized retrieval codes. Its ℓ_1-product metric combines relations across concept families, with hierarchical structure represented within each factor [315].

Mixed-curvature contrastive learning. SELFMGNN learns agreement across geometric views, while ProGDM uses contrastive geometric representations as the latent space for graph diffusion. In SELFMGNN [257], three stereographic views $\mathcal{R} = \{\mathbb{H}, \mathbb{S}, \mathbb{R}\}$ have signed curvatures c_r. Unlike HGCL's coordinate-model views, these views span different curvature signs. Let $z_i^{r,\alpha}$ denote node i in view $r \in \mathcal{R}$ under graph augmentation α, and let s_r be its within-view similarity score, such as a negative geodesic distance divided by a temperature. A fully indexed single-view InfoNCE term is

$$\mathcal{L}_S^r(\alpha, \beta; i) = -\log \frac{\exp s_r(z_i^{r,\alpha}, z_i^{r,\beta})}{\sum_{j=1}^{|V|} \exp s_r(z_i^{r,\alpha}, z_j^{r,\beta})}, \tag{197}$$

and its symmetric aggregate is

$$\mathcal{J}_S = \sum_{i=1}^{|V|} \sum_{r \in \mathcal{R}} \left(\mathcal{L}_S^r(\alpha, \beta; i) + \mathcal{L}_S^r(\beta, \alpha; i) \right). \tag{198}$$

For two distinct geometric views $r \neq s$, let $s_{r,s}$ denote the learned cross-view discriminator. The

corresponding cross-view term and aggregate are

$$\mathcal{L}_C^{r\to s}(\alpha;i) = -\log \frac{\exp s_{r,s}(\boldsymbol{z}_i^{r,\alpha}, \boldsymbol{z}_i^{s,\alpha})}{\sum_{j=1}^{|V|} \exp s_{r,s}(\boldsymbol{z}_i^{r,\alpha}, \boldsymbol{z}_j^{s,\alpha})}, \tag{199}$$

$$\mathcal{J}_C = \sum_{i=1}^{|V|} \sum_{\substack{r,s\in\mathcal{R} \\ r\neq s}} \left(\mathcal{L}_C^{r\to s}(\alpha;i) + \mathcal{L}_C^{r\to s}(\beta;i)\right). \tag{200}$$

The final optimization of SELFMGNN combines both contrastive mechanisms:

$$\mathcal{J}_{self} = \mathcal{J}_S + \gamma \mathcal{J}_C, \tag{201}$$

with γ controlling their relative contribution.

ProGDM [287] learns a product-space graph representation for subsequent diffusion. Its encoder uses contrastive learning in the κ-stereographic model to obtain component representations with learnable curvatures. Given graph-structured data $G = (X, A)$, with $X \in \mathbb{R}^{N\times F}$, the encoder maps each node into the product space

$$\mathcal{P} = \prod_{m=1}^{k} \mathcal{M}_{c_m}^{d_m}.$$

Specifically, a set of learnable curvature parameters $C = \{c_1, c_2, \ldots, c_k\}$ is optimized via a contrastive learning framework that incorporates Riemannian generators and discriminators. Adversarial training updates the curvature parameters together with the geometric representation, adapting the product space $\mathcal{P}$ to the graph data. The output is $Z = (z_1, \ldots, z_N) \in \mathcal{P}^N$, where

$$z_i = (z_i^{(1)}, \ldots, z_i^{(k)}), \qquad z_i^{(m)} \in \mathcal{M}_{c_m}^{d_m}.$$

A Euclidean factor is obtained by setting one of the c_m to zero, so the product space covers all three curvature signs within the same parameterization. ProGDM then reuses the learned curvatures in the wrapped-normal perturbations and tangent score fields for diffusion of each component's node embeddings. Adjacency diffusion remains Euclidean, with reverse node and edge scores conditioned on the noisy graph as in HGDM (Section 3.7.1).

3.9.3 Reinforcement Learning

Reinforcement learning (RL) learns policies from rewards collected through interaction [260], with applications in game playing [196, 244, 245], robotics [9, 216], and autonomous driving [134, 236]. Geometry can enter the policy representation, the action variables, or a curvature-selection policy for another model.

Hyperbolic reinforcement learning. Cetin et al. [38] place the policy and value representations in a Poincaré latent space, while ACE-HGNN uses RL to select the curvature of graph-network layers. In the former, a hyperbolic representation captures relations among evolving features, and encoder normalization controls the inputs to the exponential map. The hyperbolic representations are obtained by applying the exponential map $\boldsymbol{x}_H = \exp_{\mathbf{0}}^c(\boldsymbol{x}_E)$ to the Euclidean velocity vectors $\boldsymbol{x}_E = f_E(\boldsymbol{x})$ with input data $\boldsymbol{x}$ and Euclidean encoder sub-network f_E. The set of policy and value scalars $f_{\mathbb{H}}(\boldsymbol{x}_H) = \{f_i(\boldsymbol{x}_H)\}$ is generated by affine transformations $\{f_i\}$, where these transformations are

defined based on the signed distance from a gyroplane in $\mathbb{P}_c^n$:

$$f_i(\boldsymbol{x}) = \operatorname{sign}(\langle(-\boldsymbol{p}) \oplus_c \boldsymbol{x}, \boldsymbol{w}\rangle) \frac{2\|\boldsymbol{w}\|}{\sqrt{1+c\|\boldsymbol{p}\|^2}} d_{\mathbb{P}}^c\left(\boldsymbol{x}, \widetilde{H}_{\boldsymbol{w},\boldsymbol{p}}^c\right), \tag{202}$$

where $\widetilde{H}_{\boldsymbol{w},\boldsymbol{p}}^c = \{\boldsymbol{y} \in \mathbb{P}_c^n : \langle(-\boldsymbol{p}) \oplus_c \boldsymbol{y}, \boldsymbol{w}\rangle = 0\}$ is the gyroplane, with $\boldsymbol{p} \in \mathbb{P}_c^n$ and $\boldsymbol{w} \in \mathcal{T}_{\boldsymbol{p}}\mathbb{P}_c^n \setminus \{\boldsymbol{0}\}$. To address optimization challenges in RL with hyperbolic representations, spectral normalization (SN) is applied to the Euclidean encoder sub-network f_E. The output of f_E is rescaled by $1/\sqrt{n}$ so that dimensionality does not change the magnitude of $\boldsymbol{x}_E \in \mathbb{R}^n$ before the map to $\mathbb{P}_c^n$.

ACE-HGNN [80] makes curvature a learned control variable. Its ACE-Agent proposes layer curvatures, while the HGNN-Agent learns node representations and decides whether to accept the proposals. The ACE-Agent explores one signed curvature for each HGNN layer. Let N_{lay} be the number of layers and collect their curvatures as $\boldsymbol{c}^t = (c_1^t, \ldots, c_{N_{\text{lay}}}^t) \in \mathbb{R}_{<0}^{N_{\text{lay}}}$. The ACE-Agent state at epoch t is

$$S_{\text{ACE}}^t = \boldsymbol{c}^{t-1}. \tag{203}$$

To express the curvature update independently of a particular source parameterization, denote the layer-ℓ curvature proposal by

$$\widehat{c}_\ell^t := \mathcal{C}_\ell\left(G, \{\boldsymbol{h}_{\ell,i}^{t-1}\}_{i\in V}\right) < 0, \qquad \ell = 1, \ldots, N_{\text{lay}}, \tag{204}$$

where $\mathcal{C}_\ell$ is the curvature estimator used by the ACE-Agent and is evaluated from the graph and the layer-ℓ representations. This abstraction separates the estimator from the graph-distance quantity ξ_G in Equation (73). To smooth a signed-curvature proposal while preserving $c_\ell^t < 0$, define the radius $r_\ell^t = 1/\sqrt{-c_\ell^t}$ and $\widehat{c}_{\ell,-}^t = \min\{\widehat{c}_\ell^t, -\varepsilon_c\}$ with $\varepsilon_c > 0$:

$$r_\ell^t = (1-\gamma) r_\ell^{t-1} + \frac{\gamma}{\sqrt{-\widehat{c}_{\ell,-}^t}}, \qquad c_\ell^t = -\frac{1}{(r_\ell^t)^2}, \tag{205}$$

where $\gamma \in [0, 1]$. The embeddings are re-embedded componentwise through the canonical Lorentz basepoints:

$$\boldsymbol{h}_{\ell,i}^t \leftarrow \exp_{\boldsymbol{o}_{c_\ell^t}}^{c_\ell^t}\left(\mathcal{I}_{c_\ell^{t-1}\to c_\ell^t}\left(\log_{\boldsymbol{o}_{c_\ell^{t-1}}}^{c_\ell^{t-1}}\left(\boldsymbol{h}_{\ell,i}^{t-1}\right)\right)\right), \tag{206}$$

where $\boldsymbol{o}_c = (1/\sqrt{-c}, 0, \ldots, 0)^T$ and $\mathcal{I}_{c'\to c}([0, \boldsymbol{v}^T]^T) = [0, \boldsymbol{v}^T]^T$ identifies the spatial tangent coordinates at the two basepoints. Let $\boldsymbol{H}^t$ collect the resulting node representations. Under a downstream evaluation functional $\mathbf{M}$, the ACE-Agent reward is

$$R_{\text{ACE}}^t = \mathbf{M}(\boldsymbol{H}^t) - \mathbf{M}(\boldsymbol{H}^{t-1}). \tag{207}$$

Similarly, the HGNN-Agent uses $S_{\text{HGNN}}^t = \boldsymbol{c}^{t-1}$ and chooses whether to accept the proposed vector of layerwise curvatures:

$$A_{\text{HGNN}}^t = \left\{\boldsymbol{c}_{\text{HGNN}}^t \leftarrow \boldsymbol{c}_{\text{ACE}}^t,\ \boldsymbol{c}_{\text{HGNN}}^t \leftarrow \boldsymbol{c}_{\text{HGNN}}^{t-1}\right\}.$$

Its reward uses the same improvement form, $R_{\text{HGNN}}^t = \mathbf{M}(\boldsymbol{H}^t) - \mathbf{M}(\boldsymbol{H}^{t-1})$.

Spherical reinforcement learning. Hyperspherical normalization constrains network weights and features, whereas G-RL respects the geometry of physical action variables. Lee et al. [147] use

hyperspherical normalization to control weight and feature norms when scaling RL models and datasets. The hyperspherical normalization replaces layer normalization with ℓ_2-normalization and projects the weights onto the unit hypersphere $\mathbb{S}^{n-1}$ following each gradient update. G-RL [5] learns robotic manipulation skills from manifold-valued variables such as orientation and stiffness, including orientations on $\mathbb{S}^3$. In G-RL, the policy is learned in a constant tangent space to generate actions based on the current state and is then transferred to the local tangent space through parallel transport, where it is projected onto the Riemannian manifold. The local tangent space dynamically adapts over time to remain within the neighborhood of the current point, minimizing approximation errors. This construction keeps policy outputs consistent with the geometry of the action variables.

Synthesis across learning paradigms. The task relation determines where geometry is useful. Metric learning uses pair labels, inferred hierarchy levels, or proxies to define neighborhoods. Contrastive learning chooses which views and semantic levels should agree, then uses a geometric similarity to compare them. RL distinguishes learned feature geometry from the geometry of actions and from curvature treated as a control variable.

4 Applications

We compare how geometric models use the structure of their target data: semantic hierarchies in language and perception, heterogeneous relations in graphs, and manifold-valued variables in robotics and science. Within each domain, we distinguish the object represented geometrically, the operation that uses curvature, and the resulting modeling tradeoff. Table 11 maps these choices to representative methods. The section closes with trustworthiness applications and implementation resources.

4.1 Natural Language Processing

Language models use geometry to represent different relations: a hypernym connects a specific concept to a broader one, whereas semantic similarity compares words or sentences at the same level. Hyperbolic representations provide radial and angular coordinates for these two roles; spherical representations focus on direction. We compare these choices in classification, embedding, generation, and language modeling.

4.1.1 Text Classification

Hyperbolic text classifiers differ in the linguistic units they embed and the classifier built on those units. HyperText [339] replaces the Euclidean word embeddings and pooling of FastText [125] with Poincaré embeddings and an Einstein midpoint, followed by a hyperbolic classifier (Figure 14). It improves accuracy and reaches comparable performance with smaller parameter budgets. Micic & Chu [191] instead combine hyperbolic character skip-gram embeddings with a hyperbolic Transformer for Chinese NLP, making character composition an alternative to explicit word segmentation. HyperText therefore changes the geometry of a lightweight word-based classifier, while the Chinese character approach also changes the unit of representation.

4.1.2 Text Embedding

Placing the word dog close to the word animal captures a semantic association; identifying animal as the more general term requires a directional readout. Poincaré-GloVe [266] learns word and context-word points by fitting geometric distances to log co-occurrence counts. Its distance objective is unchanged by a common isometry, so raw coordinates alone leave the orientation of generality

Table 11 Application domains reviewed in this survey, the structural signal in the data that motivates a curved representation, the geometry adopted by the surveyed methods, and representative tasks and method families. Rows follow the order of the subsections in Section 4.

Application	Structural signal	Geometry	Representative tasks and methods
Natural language processing	Lexical and semantic hierarchies; directional similarity among embeddings	Hyperbolic, spherical, mixed-curvature	Classification and embedding: HyperText [339], Poincaré-GloVe [266], JoSE [189]. Language modeling and adaptation: Hyp-PLM [46], HypLoRA [308], HELM-MiCE [106] (multi-curvature hyperbolic), MoSLoRA [325] (mixed-curvature PEFT)
Computer vision	Visual-semantic hierarchies; spherical image domains with projection distortion	Hyperbolic, spherical	Classification and retrieval: hyperbolic embeddings [129], HCNN [17], hyperbolic ViT metric learning [74]. Segmentation and generation: hyperbolic segmentation [10], HAE [154]. Omnidirectional vision: PanoFormer [242], SGFormer [328]
Speech and audio	Source and time–frequency hierarchies; timbral and phonemic taxonomies	Hyperbolic	Separation: geometric source masks [221] and distance-group hierarchies [220]. Synthesis and anomaly detection: hyperbolic timbre VAE [201], radial anomaly detection [86]. Phoneme decoding: hyperbolic phoneme decoder [261]
Multimodal learning	Cross-modal partial orders; temporal and semantic correspondence across modalities	Hyperbolic	Image–text pretraining: MERU [64], HyCoCLIP [209]. Vision–language models: HVLM [186], HyperET [217], HySAC [223]. Audio–visual models: HyperVD [336], SHMamba [313]
Knowledge graph completion	Hierarchical relations alongside cyclic and compositional relational motifs; temporal evolution	Hyperbolic, spherical, mixed-curvature	Static completion: ATTH [40], FHRE [164], M^2GNN [284], GIE [36]. Temporal completion: DyERNIE [104], IME [280], MADE [281]
Recommender systems	Power-law and long-tail interaction hierarchies; heterogeneous auxiliary graphs	Hyperbolic, mixed-curvature	Collaborative filtering: HGCF [256], HICF [301]. Sequential and session-based: H^2SeqRec [162], HCGR [99]. Knowledge-graph-enhanced: MCKG [323], CurvRec [282]
Robotics	Part-to-whole and coarse-to-fine design hierarchies; orientations on compact manifolds	Hyperbolic, spherical	Design and manipulation: HERD [69], G-RL [5]. Localization and navigation: HyCoRe [197], HypLiLoc [285], spherical kernels [150]. Action recognition: HyLiFormer [163]
AI for science	Evolutionary, taxonomic, and scale-free organization of sequences, molecules, cells, and brain networks	Hyperbolic	Structure and molecules: NeuroSEED [57], hyperbolic drug embeddings [318]. Genomics and transcriptomics: HyperHELM [274], DELST [47]. Neuroscience: HEEGNet [157]
Trustworthiness	Client heterogeneity and class alignment; specificity-based safety boundaries; radial uncertainty and hierarchy-aware attack propagation	Hyperbolic, spherical	Federated learning: SphereFed [70], HyperFed [168], FedMRUR [8], FlatLand [174]. Safety: HySAC [223], harmful-prompt detection and sanitization [185]. Robustness and OOD: balanced hyperbolic embeddings [127], HyperDefender [184], hierarchy-aware backdoors [15]

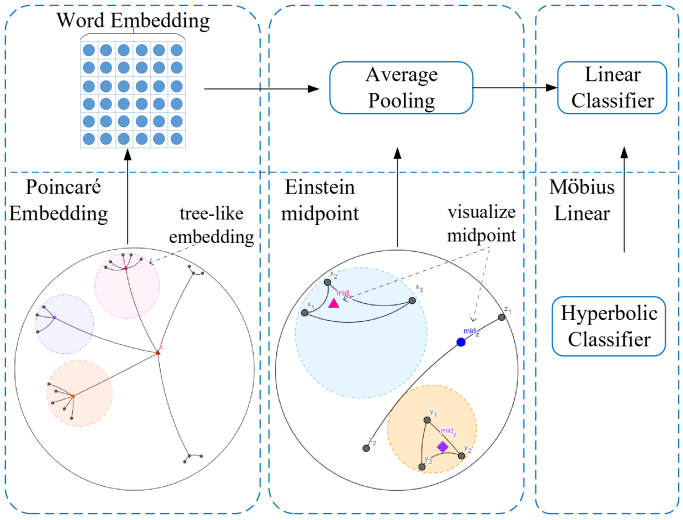


Figure 14 HyperText architecture for text classification [339]. Word embeddings are learned in the Poincaré ball, combined by the Einstein midpoint to obtain a text representation, and passed to a hyperbolic or Möbius-linear classifier. The upper branch shows the corresponding Euclidean FastText pipeline.

undetermined. After embedding training, it centers and orients the embeddings using generic and specific word sets chosen from corpus frequency or optional WordNet information. Its product of two-dimensional hyperbolic factors can then be interpreted as Gaussian mean and scale parameters, from which differences of log scales provide a generality score.

Radial organization can also be examined after context learning or constructed directly from context statistics. Dhingra et al. [67] train word points on a co-occurrence graph and sentence encoders to predict neighboring text. They subsequently relate word norms to frequency and phrase norms to constituent-tree height, using these analyses to test the structure learned from unannotated text. Ji [122] instead derives coordinates from conditional-entropy statistics weighted by collocation diversity and conditional-probability rank. The resulting radial and angular quantities describe popularity or typicality and similarity within selected semantic categories, evaluated on color and kinship words. These category-level meanings differ from the general-to-specific ordering sought in lexical entailment.

The use of probability introduces a separate choice about the representation itself. Poincaré-GloVe's points parameterize Gaussian distributions over Euclidean variables; Nagano et al. [199] assign each word a distribution over hyperbolic points, with a location and tangent-space covariance. Their word-embedding application trains a KL-based margin objective on WordNet links, making the relation data and the distributional representation explicit parts of the model. The wrapped-family sampling and density construction connects to Section 3.7.1. JoSE [189] uses probability to model word–word and word–paragraph co-occurrence while learning unit-norm point embeddings. Its cosine-based objective and fixed-concentration generative model put directional similarity into training, supporting word similarity and document-level comparison. Hyperspherical constraints also support normalized Transformers [180], discussed in Section 3.6.2.

Other hyperbolic text-embedding applications include cross-lingual alignment [240] and entity-structure modeling [331].

For model-weight compression, pyramid vector quantization normalizes integer codewords onto the unit sphere to encode weight directions and quantizes their magnitudes separately [271].

4.1.3 Text Generation

A recurrent decoder can predict from preceding words while making little use of a sentence's latent code. Matching that code to a prior introduces an additional training pressure. RNF-WAE [283] matches the aggregate posterior of Gaussian base codes to the prior through maximum mean discrepancy, then transforms samples through cluster-regularized planar flows before recurrent decoding. Its training also retains weighted KL and flow regularization. APo-VAE [61] retains each sentence's posterior-to-prior KL term and estimates it through an adversarial dual network. This sample-based objective permits an implicit tangent-noise posterior mapped into a Poincaré ball. Its decoder reads hyperbolic features of the code, and a mixture of posteriors conditioned on learned pseudo-inputs provides an optional data-driven prior.

With a uniform spherical prior, fixing the posterior concentration removes the KL gradient. Xu and Durrett [295] use this property of the von Mises–Fisher distribution to keep KL constant while reconstruction trains the mean direction, as described in Section 3.7.2. The code's informativeness is evaluated separately: RNF-WAE and APo-VAE estimate its mutual information with the input. The spherical study also varies the decoder's access to previous words and reports better perplexity than its Gaussian comparator in both settings. In its inputless word-order probe, adjacent-word swaps produce larger changes in the vMF code.

In the related task of answer selection, HyperQA [262] uses hyperbolic distances to score question–answer pairs; its output is a relevance score rather than generated text.

4.1.4 Language Modeling

Language modeling introduces geometry through backbone pretraining, low-rank adaptation, or optimization of the adapter parameters. Hyp-PLM [46] changes representations during pretraining without increasing model size. HypLoRA [308] and HoRA [306] instead adapt a pretrained model through hyperbolic low-rank transformations. HoRA emphasizes operations that avoid repeated exponential and logarithmic maps, connecting its geometric construction to adaptation cost. Riemannian LoRA [327] acts on the geometry of low-rank optimization; this differs from assigning negative curvature to language representations. RAFFT [337] extends Riemannian low-rank optimization to federated fine-tuning through parameter matching and rank-adaptive training with fixed-rank client optimization within each round, addressing aggregation noise and rank drift.

Expert models make the allocation of curvature an additional design choice. HELM-MiCE [106] routes tokens among Lorentz experts with different negative curvatures, allowing hierarchical representations at multiple scales within a hyperbolic model. MoSLoRA [325] uses sparse hyperbolic, spherical, and Euclidean LoRA experts during adaptation, combining curvature signs as well as learning their magnitudes. The former learns geometric experts during language-model training, whereas the latter adds geometric choices to parameter-efficient adaptation of an existing model.

Summary. For lexical entailment, the training relations and the chosen readout connect geometric coordinates to semantic generality. Direction-based similarity and normalized computation motivate spherical constraints. In language-model adaptation, the practical choice also concerns where geometry enters: the representation, the adapter transformation, or the optimization of low-rank parameters. Expert models further require a rule for allocating tokens across geometric components.

4.2 Computer Vision

Computer vision uses hyperbolic geometry for visual-semantic hierarchies and spherical geometry for omnidirectional media. These priors support image classification, segmentation, retrieval, gener-

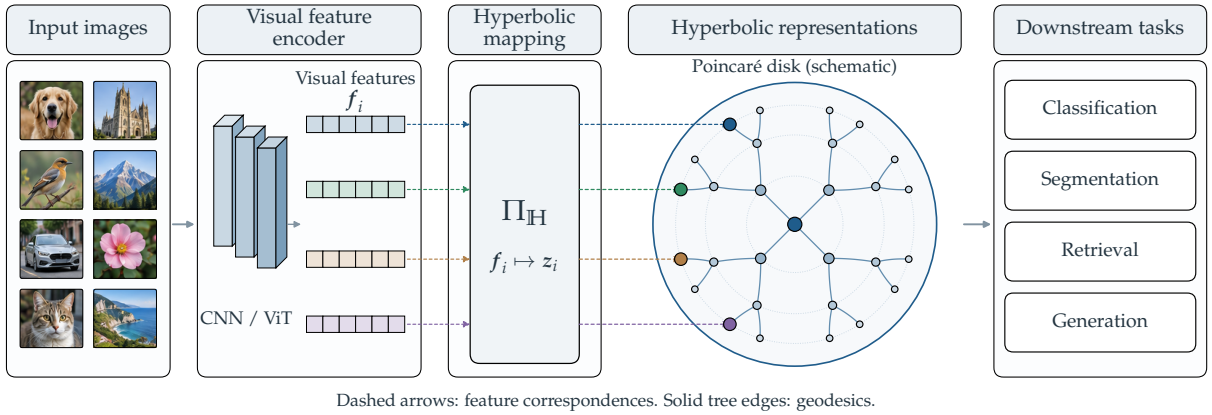


Figure 15 An illustrative pipeline for hyperbolic visual representations. A visual encoder produces feature vectors f_i, which a model-specific map $\Pi_{\mathbb{H}}$ sends to hyperbolic representations z_i. Dashed arrows associate features with their embeddings. The disk illustrates a hierarchy with broader nodes near the origin and more specific descendants farther out; solid tree edges are Poincaré geodesics. The final column lists example downstream tasks.

ation, and panoramic understanding. An example hyperbolic computer-vision pipeline is shown in Figure 15.

4.2.1 Image Classification

Class labels group images, while a taxonomy also relates their categories. For existing hyperbolic features, large-margin classification [50] fits geodesic separating hyperplanes with the feature coordinates held fixed, providing a foundation for geometric decision rules. Hyperbolic image embeddings [129] train conventional CNNs with a Poincaré output representation. Multiclass logistic regression provides a classification head, while few-shot learning uses hyperbolic support-set averages and query-to-prototype distances. These image models learn from class labels or labeled episodes without requiring a taxonomy. Their Omniglot comparison keeps a 64-dimensional four-block encoder architecture and changes the output map, averaging, and distance: one-shot results improve, while five-shot results tie or decrease slightly.

A given taxonomy supports a different training target. Entailment-cone classification [66] adds images as leaves of a label hierarchy and trains embeddings to respect label–label and label–image ancestry. During joint embedding, a CNN previously trained on ETHEC provides frozen features; the image projection and label embeddings are learned. Prediction selects the label with the lowest order-violation energy at each hierarchy level. In the reported 100-dimensional comparison, initialized hyperbolic cones have slightly higher image-classification F1 than Euclidean cones, while Euclidean cones reconstruct the label graph better. A hierarchy-aware CNN marginalization classifier achieves higher image F1 than either cone model, separating the benefit of the label relations from the choice to encode them as geometric order constraints.

HCNN [17] extends geometry through trainable convolution, normalization, and residual operations in the Lorentz model (Section 3.3.1). Its ResNet-18 study trains head-only, fully hyperbolic, and selected-block variants with common class supervision and training schedules. HECNN replaces blocks 1 and 3, selected through feature-hyperbolicity measurements, and uses a Lorentz classification head. On CIFAR-100, this partial replacement achieves higher mean accuracy than both the head-only Lorentz model and full HCNN. The placement of hyperbolic blocks within the encoder therefore matters, in addition to the choice of decision head.

Hyperbolic semantic hashing [7] aligns structural cues with semantic supervision for compact retrieval codes. Ge et al. [85] use contrastive learning to represent scene–object hypernymy.

4.2.2 Image Segmentation and Generation

A pixel near an object boundary can combine evidence from several classes, making confidence useful alongside its class prediction. Hyperbolic Image Segmentation [10] classifies mapped pixel features with a memory-efficient gyroplane classifier and examines their Poincaré norms as a confidence cue. Its boundary analyses associate smaller norms with less confident pixels. A hierarchical-softmax variant additionally incorporates parent–child class relations. Hyperbolic Uncertainty Loss (HyperUL) [41] turns a related radial cue into a training signal: smaller-radius outputs receive greater relative weight in pixel-wise cross-entropy, and radius-based selection determines which pixel losses enter the training objective. The reported uncertainty readout divides each pixel's origin-distance by the maximum in the same image and subtracts the result from one. It therefore provides an image-relative uncertainty map from a single prediction pass.

For fisheye images, the geometric intervention can instead target spatial sampling under projection distortion. FisheyeHDK [4] uses a hyperbolic branch to predict deformable kernel positions, which are consumed by Euclidean convolutions to compute image features. Pixel-label supervision trains these positions through the segmentation objective, adapting the encoder's receptive fields to the distorted input.

Image generation uses a different geometric object: an editable code between an image inverter and a generator. Hyperbolic Attribute Editing (HAE) [154] learns this code between a pretrained pSp inverter and StyleGAN2 generator, using category classification together with image and latent reconstruction losses. The inverter and generator are retained while the intervening layers are trained. During generation, HAE edits the code at selected radial levels and maps it back for image synthesis. Its radius experiments relate larger radii to finer within-category variation and smaller radii to broader semantic changes, including changes of category.

4.2.3 Image Recognition and Retrieval

Recognition and retrieval differ in whether the hyperbolic metric is applied to existing features or learned jointly with the encoder. Hyperbolic face recognition [268] replaces Euclidean distances in k-nearest neighbors and large-margin nearest-neighbor classifiers, with feature preprocessing such as edge filtering. Hyperbolic ViT metric learning [74] trains image embeddings with a modified pairwise cross-entropy loss for retrieval. A metric replacement changes the matching rule, while joint training also shapes the features to fit that rule.

4.2.4 Omnidirectional Vision and 360° Media

A ceiling light near a pole can occupy a wide horizontal strip in an equirectangular panorama. If it crosses the panorama seam, its image is also split between the left and right edges. A kernel must gather neighboring viewing directions despite this changing image footprint. SphConv [255] learns equirectangular filters to approximate a pretrained perspective CNN evaluated on corresponding tangent views. Kernel shapes are chosen according to latitude, and weights are learned separately for different rows while shared along longitude. Spherical DNNs [296] instead share a spherical-crown kernel across locations, rotating sampled positions back to its reference region and interpolating the kernel values. Circular padding reconnects the horizontal image boundaries. Crown-based pooling uses the same angular neighborhoods, while a solid-angle-weighted loss accounts for unequal pixel

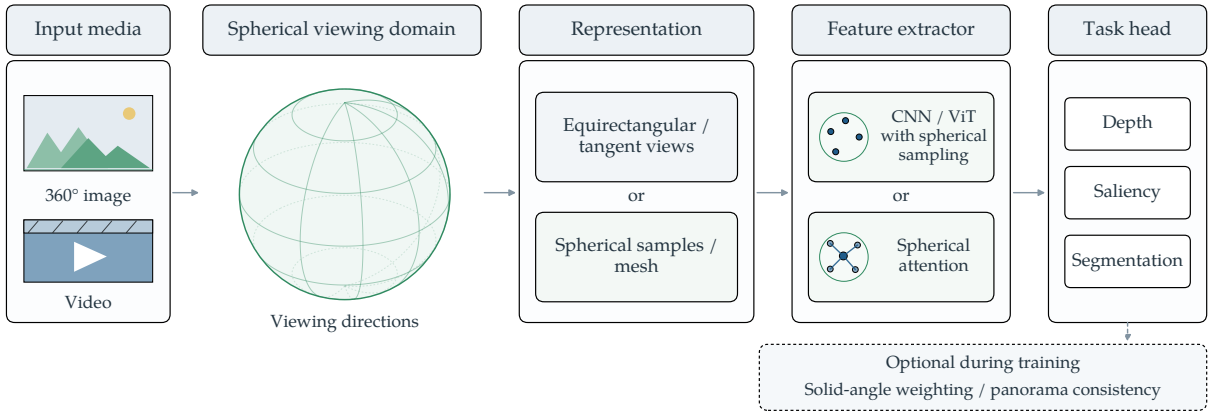


Figure 16 General pipeline for spherical computer vision. A 360° image or video samples a spherical viewing domain. Methods use equirectangular or tangent views, or samples on the sphere, before geometry-aware feature extraction and task-specific prediction. Solid-angle weighting and panorama-consistency objectives are optional during training.

areas. Perspective-teacher outputs supervise SphConv, while the shared spherical operators support task-trained U-Net and ConvLSTM models.

For dense depth prediction, PanoFormer [242] retains equirectangular feature maps and learns which samples contribute to each pixel. Each attention head uses nine positions initialized through a spherical tangent construction, then adjusts them with learned offsets. Attention weights combine these sampled features, while hierarchical stages expand the effective context. SalViT360 [56] uses tangent images at a different scale for video saliency: a frozen planar encoder extracts viewport features, and angular coordinates provide positional information. Temporal attention combines the same viewport across a video window, followed by spatial attention among viewports in each frame. The decoder predicts each viewport separately before inverse projection to an equirectangular saliency map. A consistency objective encourages agreement between the reprojected equirectangular saliency maps from two viewport configurations during training. Inference can use one configuration.

SphereFusion [299] combines two representations of the same panorama before depth decoding. An image encoder extracts equirectangular features, while a mesh encoder processes neighboring faces of an icosahedral sphere. Equirectangular features are transferred to the mesh domain, where gates select contributions from both branches and a mesh decoder predicts depth. This keeps planar appearance features available to a decoder organized by spherical adjacency. Face connectivity is cached and reused across layers at the same mesh resolution. The study reports an average inference time of 17.4 ms over 100 panoramas of size 512×1024 on one RTX 3090. These representation choices sit alongside the feature extractor, task head, and optional objectives in Figure 16.

SGFormer [328] uses a spherical prior for panorama depth, while SGAT4PASS [160] combines spherical geometry and deformable embeddings for panoramic segmentation. Spherical coordinates serve different input structures in SphereFormer [141], which groups LiDAR points through radial-window attention, and SUGAR [232], which uses spherical graph attention for cortical surface registration. These point and surface representations organize 3D observations or correspondences, whereas panorama methods account for projections of a camera's viewing sphere.

Summary. Vision methods apply geometry either to semantics or to the observation domain. Hyperbolic methods shape feature separation and, with suitable objectives, semantic hierarchy, while spherical operations preserve angular neighborhoods in panoramas and surfaces. Model selection

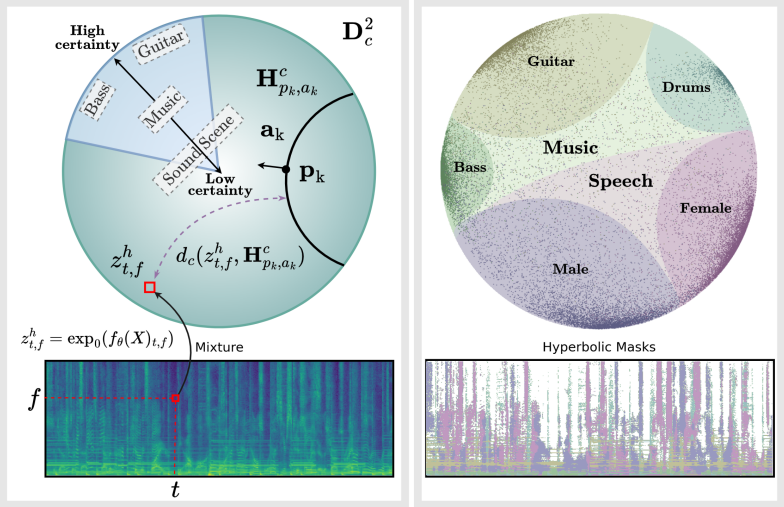


Figure 17 Hyperbolic source separation [221]. In the source notation, $\mathbb{D}_c^2$ is the two-dimensional Poincaré disk, $z^h_{t,f}$ is the hyperbolic embedding of time–frequency bin (t, f), and $H^c_{\boldsymbol{p}_k,\boldsymbol{a}_k}$ is the decision gyroplane for source k. The gyroplane partitions the embedding disk into source regions, and distances to the decision boundary determine the time–frequency masks shown on the right.

then concerns the point of intervention: a matching metric, an output embedding, or the feature extractor itself.

4.3 Speech and Audio Processing

Audio methods use hyperbolic coordinates to assign time–frequency features to sources, represent timbral relations, or estimate confidence. These roles share a representation space but require different training objectives, so radial position takes its meaning from the task and the loss.

4.3.1 Speech Separation

Hyperbolic source-separation methods differ in the hierarchy used to construct source masks. Petermann et al. [221] project time–frequency bins into the Poincaré ball and estimate masks with hyperbolic softmax layers, using geometric decision boundaries to assign acoustic features to sources. The separation objective balances interference reduction against artifacts introduced by masking. Petermann & Kim [220] extend this formulation to microphone-distance hierarchies: near and far source groups contain individual speakers as child classes. Hyperbolic multinomial logistic regression estimates masks at these levels, while radial certainty is analyzed in relation to acoustic properties. The distinction is the target hierarchy, with the later model jointly representing source identity and distance groups. Figure 17 illustrates the geometric mask estimation.

4.3.2 Sound Synthesis and Detection

Hyperbolic synthesis and anomaly detection assign different meanings to the latent radius. The hyperbolic VAE of Nakashima et al. [201] uses a hyperbolic timbre latent distribution to organize musical-instrument timbre and synthesize sounds. Its comparison with a Euclidean VAE connects low-dimensional timbre representations to instrument-classification accuracy and hierarchical organization. The anomalous-sound detector of Germain et al. [86] instead trains a surrogate classifier on normal sounds and uses radial confidence for unsupervised anomaly detection. Confident inlier



Figure 18 Hyperbolic image-text representations [64]. To explicitly capture visual-semantic hierarchies, MERU embeds image-text pairs within the hyperbolic Lorentz model.

embeddings tend toward the Poincaré-ball boundary, whereas outliers tend to lie nearer the origin. The reported inlier–outlier separation follows from normal-sound training and the use of negative radius as an anomaly score.

4.3.3 Phoneme Decoding

Phoneme decoding uses geometry to distinguish related classes in neural recordings. Tan et al. [261] combine a hyperbolic feed-forward network and multinomial logistic regression to decode Mandarin phonemes from intracortical signals, using phoneme hierarchies to organize classes with overlapping neural patterns. The geometric object is therefore a phoneme-class representation, rather than the waveform or a source-separation mask.

Summary. In audio, each application pairs geometry with its prediction target: source assignments and confidence for separation, latent timbral organization for synthesis, and related phoneme classes for neural decoding. Radial confidence is most informative when the objective connects certainty to distance from the origin.

4.4 Multimodal Learning

Multimodal models must align corresponding observations while retaining differences in semantic specificity. A caption can describe a general concept, an image can depict a particular instance, and a region can add a finer detail. We compare how image–text, vision–language, and audio–visual methods impose these relations through losses, feature transformations, and fusion.

4.4.1 Image-Text Models

Image–text learning distinguishes symmetric matching from directed entailment. Order embeddings [276] motivate explicit partial-order constraints, while CLIP [229] learns paired similarity. MERU [64] (Figure 18) combines contrastive alignment with hyperbolic entailment in the Lorentz model, placing generic concepts at smaller radii and specific instances at larger radii. Hierarchical reasoning also emerges in Euclidean CLIP representations [6]; explicit geometric ordering makes the general-to-specific relation part of the training objective.

Entailment can change either the alignment loss or the selection of training data. The angle-based

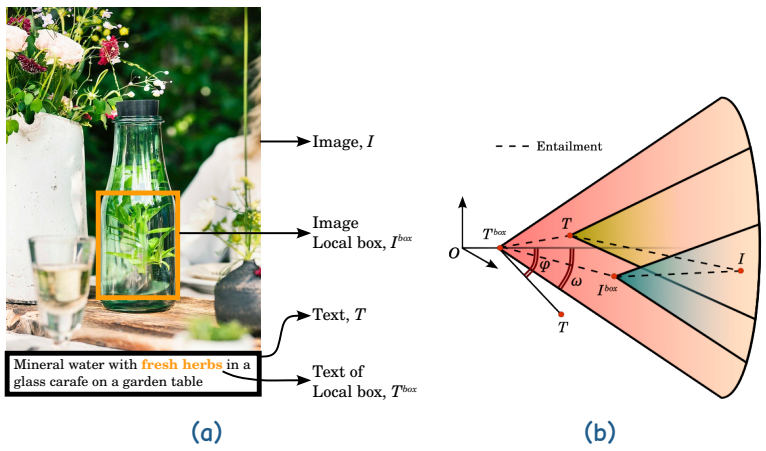


Figure 19 HyCoCLIP compositional entailment [209]. (a) Global pairs (I, T) provide context for local regions (I^{box}, T^{box}). (b) Hyperbolic entailment cones define the desired hierarchical global–local alignment during training.

loss of Ramasinghe et al. [230] aligns related concepts along shared geodesics, accommodating one-to-many relations while retaining intra-modal hierarchy. HYPE [131] instead combines Euclidean similarity and Lorentzian entailment to filter image–text pairs. The former changes how selected pairs shape the embedding space; the latter changes which pairs enter training.

4.4.2 Vision-language Models

The alignment mechanisms in Section 3.8.4 support different tests of visual understanding. Analyses of hyperbolic representations examine spatial reasoning and ambiguity [118]. Region-sensitive evaluation tests recognition of localized content, as illustrated by the compositions in Figure 19. HyCoCLIP tests recognition of ground-truth object boxes and discrimination between correct captions and object- or attribute-altered alternatives [209]. Among its single-term loss ablations on grounded CC3M, removing the contrast between image crops and complete captions yields the largest reported decrease in mean classification accuracy and in both retrieval directions. Retrieval measures a different consequence: MERU's own entailment-removal ablation retains or improves the reported COCO retrieval recall while reducing image-classification accuracy [64]. A well-organized containment hierarchy and a strong retrieval ranking therefore need separate evidence.

For language generation, the learned visual information must remain useful after passing through the language-model interface. The hyperbolic BLIP-2 study evaluates retrieval after its first training stage and captioning after the second stage and task fine-tuning [186]. Its Euclidean controls also benefit from Random Query Selection and Random Text Pruning, showing that these training choices contribute beyond the change of geometry. With both strategies, the reported hyperbolic model has slightly lower retrieval metrics and several higher captioning metrics than the corresponding Euclidean model. Its radius visualizations additionally examine variation with scene content, text corruption, and embedding dimension. Within MLLM question answering, HyperET's ScienceQA experiments instead compare structured vision-weight adjustments and matched added-parameter alternatives on the same MemVP backbone [217]. In this setup, block-diagonal and banded adjustments give small accuracy gains over diagonal scaling, while larger blocks or wider bands do not improve accuracy monotonically.

Open-vocabulary segmentation changes the prediction unit and the supervision available during adaptation. HyperCLIP uses dense COCO-Stuff segmentation annotations and category text, then

evaluates pixel labeling with text-defined classes across other segmentation benchmarks [218]. In its CLIP/decoder setup, adapting both modality branches gives higher segmentation scores than adapting only the image or only the text branch. This comparison tests whether both sides of pixel–class matching should adjust while the pretrained CLIP weights remain fixed. The radius-target control further shows that segmentation quality peaks at an intermediate target mean radius and decreases for both larger and smaller targets.

HySAC [223] applies hyperbolic entailment to safety-aware alignment; its content-control role is discussed in Section 4.9.2.

4.4.3 Audio-visual Models

The supervision attached to audio–visual data determines which relation the model must learn. Zero-shot recognition uses audiovisual examples from seen classes and semantic class words to support recognition of unseen classes [112]. HyperVD [336] instead learns to locate violent events from video-level presence labels, treating each video as a bag of snippets under multiple-instance learning. SHMamba [313] receives a question alongside the audiovisual sequence and predicts a word from a predefined answer vocabulary. Semantic class words define the candidates for class transfer, whereas a question conditions the answer sought from a particular sequence.

Agreement between the streams can be learned before their features are fused. Hong et al. [112] add an auxiliary loss before the AVCA cross-attention module: audio and visual features are projected into the Poincaré ball, mapped to the tangent space at the origin, and compared through their normalized within-modality cosine-similarity matrices. The loss asks whether relations among clips agree across the two modalities.[2] The adaptive variants obtain one or several negative curvatures from concatenated audio and visual features. In the paper's UCF-GZSL and ActivityNet-GZSL placement controls, aligning these features before cross-attention gives higher harmonic-mean and zero-shot accuracy than aligning the post-attention or final projected features. SHMamba [313] also couples an alignment objective to a downstream prediction network. Its journal formulation uses a curvature hyperparameter and introduces hyperbolic-distance dynamic time warping for sequence comparisons with timing variations.

Temporal context can be processed after fusion or retained separately within each stream. HyperVD's detour fusion learns visual features during fusion, then maps the joint audiovisual embeddings into the Lorentz model [336]. Two graph branches use fully hyperbolic layers [45] to propagate information through feature similarities and temporal relations among snippets, and a hyperbolic classifier produces the violence scores. SHMamba [313] retains separate audio and visual sequences through selective state-space updates before gated exchange. Each output stream has its own gate for both state-space outputs, and the fused representation subsequently combines with the question embedding for answer prediction. This temporal processing supplements the alignment objective: in the MUSIC-AVQA ablations, removing the Mamba blocks or replacing cross-fusion with concatenation reduces average answer accuracy, as does removing the alignment module.

Summary. Multimodal methods differ in the relation they supervise and the stage at which they impose it. Pairwise alignment connects modalities, entailment adds directed specificity, and local supervision represents composition. Data filtering applies these relations before training; radius-aware adapters and geometric fusion apply them within the model.

[2] For nonzero features where the maps are finite and real, radial projection and the origin logarithm preserve cosine similarities. The displayed ideal loss therefore matches angular relations and can still train the feature directions.

Table 12 Geometric mechanisms in static knowledge graph completion. Hyperbolic methods differ in their relation transformations; mixed-geometry methods differ in how they assign or combine component spaces.

Method	Geometry	Main operation	Geometric design choice
ATTH [40]	Poincaré ball	Attention over rotations and reflections, followed by Möbius translation	Relation-specific curvature; attention mixture and translation can change radius
HAQE [165]	Poincaré ball and its tangent space	Möbius translation followed by tangent-space unit-quaternion rotation	Relation-specific curvature; unit-Hamilton rotation couples four-coordinate blocks
FHRE [164]	Lorentz model	Paired spatial rotations with a Lorentz chord-based score	Fixed negative curvature; relation rotation preserves the head's radius
M^2GNN [284]	Product of constant-curvature spaces	Factorwise relational attention; weighted component distances	Learn curvatures and global space weights; retain factorwise features
DGS [121]	Spherical instances; hyperbolic ontology	Couple instance and ontology objectives through bridge entities	Assign geometry from given node and link roles
GIE [36]	Euclidean, hyperbolic, and spherical	Mix relation-transformed endpoints before two-direction distance scoring	Query-dependent interaction across geometric features
MGTCA [241]	Euclidean states; hyperbolic and spherical message branches	Fuse geometric messages before neighbor aggregation	Combine feature fusion with geometric distance scoring

4.5 Knowledge Graph Completion

Knowledge graph completion (KGC) predicts missing facts from observed triples. The same graph can contain hierarchical relations, symmetric associations, and compositional paths, making relation structure central to the geometric choice. These representations support question answering [49, 116] and recommendation [100, 172, 258]. We distinguish static completion, which scores time-invariant facts, from temporal completion, which also models when a fact holds.

4.5.1 Static Knowledge Graph Completion

Graph-based and multimodal approaches differ in the evidence used to construct entity embeddings. HyperKA [259] uses hyperbolic graph convolution [39] to aggregate relational neighborhoods for knowledge association across graphs, a task closely related to completion. HyME [166] instead incorporates visual and textual evidence, projecting features from VGG [247] and Word2Vec [192] into the Lorentz model. It then transforms relation-conditioned embeddings and uses hyperbolic distance to score triples. Neighborhood propagation emphasizes structural context, while multimodal encoding adds evidence from entity attributes.

Metric-based KGC constructs a query from a head entity h and relation r, then ranks candidate tails by their discrepancy from that query. Origin-fixing rotations and reflections preserve a point's radius, whereas their attention-weighted combinations and translations can change it. FHRE [164] applies paired rotations to the spatial coordinates of a Lorentz embedding, preserving its time coordinate and therefore its radius. This rotation can be evaluated directly without logarithmic or exponential feature maps. ATTH [40] and HAQE [165] add Möbius translation, which allows the relation to move the query between radial levels. Their ordering differs: ATTH translates an attention-weighted combination of rotated and reflected head embeddings; HAQE translates the head first, applies a

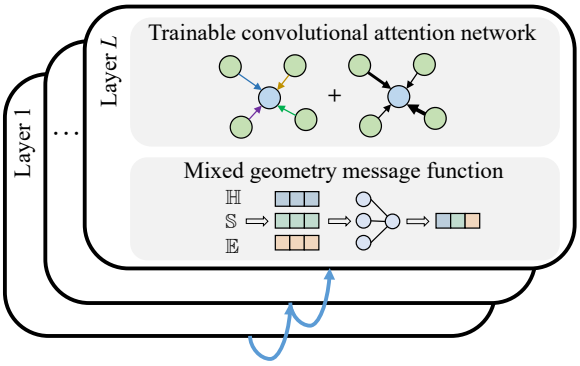


Figure 20 Mixed-geometry message passing in MGTCA [241]. Each layer constructs hyperbolic (H), spherical (S), and Euclidean (E) messages from relational neighbors, fuses them into Euclidean messages, and aggregates neighbors through trainable convolutional attention. The resulting entity states feed a scorer combining distances from the three geometries.

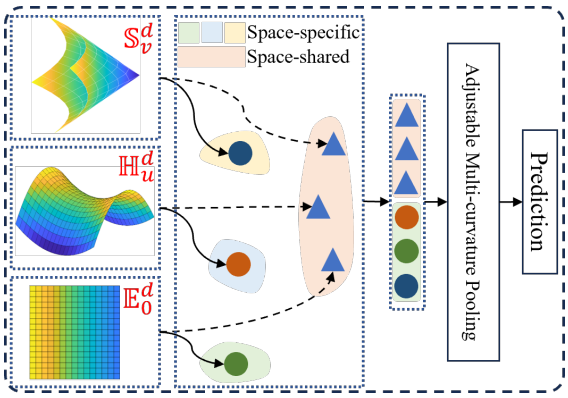


Figure 21 Mixed-curvature query representations in IME [280]. Hyperbolic (H), spherical (S), and Euclidean (E) features enter both shared-parameter and geometry-specific gates. Adjustable Multi-curvature Pooling combines the resulting channels through learned weights over coordinatewise sorted feature ranks before candidate prediction.

unit-quaternion Hamilton product to the resulting tangent coordinates, and maps the query back to the ball. ATTH thus learns a head-dependent mixture of candidate transformations, while HAQE applies a unit-Hamilton rotation to each group of four tangent coordinates.

Curvature and scoring further distinguish these query constructions. ATTH and HAQE associate curvature parameters with relations while sharing entity parameters, so the same entity participates in different relation-specific geometric spaces. FHRE uses a shared space of unit negative curvature. The scoring functions also differ: ATTH uses squared geodesic distance, HAQE uses geodesic distance, and FHRE uses a Lorentz chord-based score related to Equation (11). The choice of score sets the scale of the discrepancies entering the training loss, alongside the learned entity biases.

Ablations in the original papers compare these query designs. ATTH's YAGO3-10 relation analysis finds reflections effective for symmetric relations and rotations effective for several directed relations, with attention combining the two transformation families [40]. In HAQE's WN18RR ablation, the full model obtains an MRR of 0.496, compared with 0.489 for the authors' Euclidean-space translation variant and 0.487 for their Givens-rotation variant [165].

HBE [210] addresses a separate numerical concern through polar coordinates and an extended Poincaré-ball parameterization for embeddings near the boundary.

An entity can participate in factual relations and also have a type in an ontology. DGS [121] uses these roles to assign geometry: instance entities occupy a spherical space and ontology concepts a hyperbolic space. Entities that connect the two views receive bridge constraints and contribute to both objectives during alternating updates. The instance and ontology representations are therefore linked through the entities that carry evidence between them. This design requires the graph to identify entities, concepts, and their cross-view links. M^{2}GNN [284] instead gives entities and relations components in a chosen product of constant-curvature spaces. The factor family is shared across the graph, while component curvatures are learned together with scalar space weights shared across the graph. DGS thus assigns geometry from the node and link roles, whereas each M^{2}GNN representation carries the selected factor family.

For a relation query, neighboring facts can affect the representation before the candidate is scored.

M^2GNN forms and attends to relational messages within each geometric factor, retaining factorwise features through the graph update [284]. MGTCA [241] changes the point at which those features meet. Its layers maintain Euclidean entity states, construct hyperbolic, spherical, and Euclidean messages from each relational neighbor, then log-map the curved messages and fuse the three branches into a Euclidean message. Trainable convolutional attention subsequently aggregates these fused neighbor messages (Figure 20). Thus, M^2GNN can weight neighboring evidence inside each factor, while MGTCA's neighbor aggregation acts on messages that already combine geometric branches. The product representation already includes relational attention; the distinction is when features from different spaces are combined.

The final score also depends on where geometric components are combined. M^2GNN combines squared component distances using the same learned space weights for all entity pairs [284]. GIE [36] makes its geometric interaction specific to the queried endpoints: it transforms the head under the relation and the tail under its inverse in three geometries, log-maps the curved features for attention with the Euclidean features, and maps each mixture into the interaction geometry before evaluating two directional distance terms. Here the mixture weights depend on the transformed endpoint features, rather than providing a graph-neighbor update. MGTCA first predicts a tail representation from the encoded head and relation, then uses a scorer containing distances from the three geometries [241]. Its feature fusion and geometric scoring therefore coexist. The paper's geometry-removal ablation affects both stages; its separate message-function and scorer substitutions provide more specific comparisons of their roles.

Table 12 summarizes where each method assigns geometry and combines representations.

4.5.2 Temporal Knowledge Graph Completion

A person may be affiliated with an organization at one date and leave it at another. Temporal completion must allow this relation's score to change while reusing what is known about both entities. DyERNIE [104] ties an entity's representations across dates through an initial point and a learned velocity in each chosen manifold factor. It adds the timestamp-scaled velocity in the origin tangent space before mapping back, and then applies the relation transformation for scoring. The same entity motion therefore participates in queries about different relations. HERCULES [198] shares temporal information differently: it keeps persistent Euclidean entity vectors and derives the scoring curvature from a relation parameter and a timestamp parameter. Changing the date changes the projections and distance geometry for every entity pair queried under that relation. Facts at different dates thus train a shared entity trajectory in DyERNIE, whereas HERCULES reuses entity and relation parameters while learning a scalar for each date.

The temporal curvature rule leaves room for different relational interactions. HERCULES combines rotations and reflections of the subject through attention within one hyperbolic space. HyIE [330] retains the relation–timestamp curvature construction and changes how both endpoints enter the score. Drawing on the Householder transformations used in HousE [156], it first applies relation-specific modified projections that can change entity distances, followed by rotations in Euclidean, hyperbolic, and spherical representations. It maps the curved features to tangent coordinates, mixes them with Euclidean features through endpoint-dependent attention, and maps the mixtures into the interaction geometry. The score then combines two distances: the mixed head to the projected tail, and the mixed tail to the projected head. For the same affiliation relation and date, the mixture can therefore depend on the entities' transformed features as well as the shared temporal curvature.

Timestamped neighboring facts provide additional context for a query. HyGNet [161] composes neighboring entity features with their relations and uses the associated timestamp embeddings to gate

their messages before tangent-space aggregation. Its decoder then applies hyperbolic convolution to the encoded head, query relation, and query timestamp, and compares the result with a candidate tail. Changing the query date alters the decoder input; changing the available timestamped neighbors alters the encoded context. An additional consistency loss links embeddings of adjacent dates, imposing a smoothness assumption on temporal information. IME [280] instead begins by exchanging information among the subject, relation, and timestamp fields of the query within each geometry. A gated distributor collects their feature differences and returns messages to those fields. Changing the date therefore changes their joint encoding even when the starting entity parameters are reused. IME thus exchanges information within the query, whereas HyGNet aggregates neighboring facts.

IME further controls how the resulting geometric representations contribute to prediction [280]. Both its space-shared and space-specific gates read the concatenated hyperbolic, spherical, and Euclidean features of a query field. The former share gating parameters across geometries; the latter use separate parameters. Similarity, difference, and cross-space structural objectives encourage common information, complementary features, and agreement in relational structure. As illustrated in Figure 21, both channel types enter the final pooling stage. Adjustable Multi-curvature Pooling sorts each feature coordinate across the collected representations and applies learned weights to the sorted ranks. These weights can emphasize different parts of the feature-value distribution across fields and geometries. The pooled query vector is then compared with the candidate object through an inner product, completing a different path to the score from HyIE's two endpoint-distance terms.[3]

The experiments in these papers test whether a time-dependent score makes effective use of the date. Under HERCULES's training setup on two ICEWS benchmarks, the static AttH scorer remains competitive; its ICEWS14 timestamp-replacement probe also changes completion metrics only slightly [198]. HyIE's ablation with the timestamp removed from curvature instead has lower MRR under that paper's evaluation setup [330]. The shared relation–timestamp curvature construction therefore permits different observed contributions from time when combined with different operators and training procedures. HyGNet's separate experiment withholding dates inside the observed time range tests another consequence of temporal modeling: recovering facts at gaps between known dates [161]. Together, these controls test sensitivity to the query date and interpolation across missing dates.

Summary. KGC requires both a geometry and a relation operator. A hyperbolic space provides hierarchical capacity, while rotations, reflections, and translations determine the scored relation patterns. Mixed-curvature methods add component allocation and interaction for heterogeneous relations. For temporal graphs, the allocation of time-dependent parameters and the known facts available to the query determine how observations at different dates influence prediction.

4.6 Recommender Systems

Recommenders use curved representations to organize user–item interactions and auxiliary social, temporal, or knowledge-graph structure. Hyperbolic models associate branching neighborhoods and long-tail interactions with radial organization. Mixed-curvature models assign heterogeneous relations to additional components. Within each setting, we compare changes to the encoder, the training signal, and the information available to the model.

[3] Further temporal mixed-curvature designs include MADE [281].

Table 13 Where geometry changes collaborative filtering. All rows model user–item interactions; the columns give the operation, its intended role, and the additional computation or design choice.

Method	Geometric operation	Role in recommendation	Computation or design choice
HGCF [256]	Tangent-space graph aggregation and hyperbolic scoring	Aggregate user–item neighborhoods	Logarithmic and exponential maps around the graph encoder
HGFormer [309]	Fully hyperbolic local convolution and cross-attention	Combine local evidence with global interactions	Kernel approximation for linear attention computation
HRCF [302]	Root alignment and origin-aware penalty	Counter aggregation-induced embedding collapse	Geometric regularizer and its weight
HICF [301]	Geometry-aware margins and negative sampling	Account for head and tail items during ranking	Negative pool size and selection nearest the positive item
HyperCL [225]	Hyperbolic graph contrastive learning	Supply self-supervised embedding consistency	Two edge-dropped views, tangent and centroid encoders, and InfoNCE

4.6.1 Collaborative Filtering

Repeated neighborhood aggregation produces the user and item states used for candidate ranking, but it can also reduce their separation. HGCF [256] uses tangent-space aggregation with skip connections before hyperbolic distance scoring. HRCF [302] adds a constraint on the resulting configuration: it subtracts the mean of the user and item tangent vectors and penalizes a small root-mean-square norm of the centered vectors. This encourages overall radial spread after the exponential map, helping preserve distinctions as aggregation deepens.

The ranking triple determines which distinction a user state learns. HICF [301] adapts the margin using the user and positive item's distances to the origin and each other. From a sampled pool of unobserved items, it selects the negative nearest to the positive item in hyperbolic distance. The competing item therefore follows the positive item's learned location. HyperCL [225] adds a comparison of node identity: two edge-dropped graphs pass through tangent and Lorentz-centroid encoders, and their hyperboloid outputs are contrasted. The same user across views forms a positive pair, with other users as negatives; the item loss analogously compares items with items. These identity comparisons supplement user–item ranking, while the original tangent-aggregation backbone continues to produce recommendation scores.

Global message access further changes the state available for ranking. HGFormer [309] combines fully hyperbolic local centroid aggregation with cross-attention over all user–item pairs, including unobserved pairs. In its ablation study, adding the global branch improves HGCF, and removing the local branch from HGFormer produces the largest decline. The observed interaction graph therefore remains useful alongside global messages. Random-feature approximation makes the cross-attention aggregation linear in user and item counts for fixed feature dimensions and head count. Table 13 records the encoding and training requirements of these interventions.

HARec [182] extends beyond interaction-only training by aligning a collaborative-filtering graph with an LLM through a hyperbolic tree, connecting the geometric representation to exploration and exploitation.

4.6.2 Social and Sequential Recommendation

Social links influence recommendation through the user representation passed to item scoring. HSR [151] learns Poincaré user and item embeddings and aggregates a user's social neighbors, allowing social information to shape preferences alongside observed user–item interactions. HGSR [310] incorporates social information through two connected paths. Social-link pretraining first produces fixed user features, which are fused with trainable preference embeddings. A heterogeneous graph encoder then propagates both social and interaction edges: users receive information from social neighbors and items they have interacted with, while items receive information from their interacting users. A global weight balances the two message sources before the resulting Lorentz embeddings determine candidate rankings by proximity. The social graph therefore contributes both structural features and an explicit propagation route during preference learning.

The contribution of social propagation depends on how much interaction evidence a user already has. In HGSR's Ciao and Epinions comparisons grouped by training-interaction count, adding social propagation without pretraining improves most sparse groups but slightly reduces performance in the densest group; the full model with pretrained social features improves over the interaction-only backbone in each shown group [310]. The social-weight study also finds that increasing the contribution of social messages does not improve ranking monotonically.

Sequential recommendation. Two histories of the same user can end with the same item yet suggest different next choices. Whether a predictor can distinguish them depends on which earlier observations enter its current state. HTSR [322] scores candidates from a persistent user translation and the latest item. It adds a global translation and a user-specific offset to the latest-item embedding in tangent coordinates, maps the result into a Poincaré or Lorentz model, and scores a candidate by its popularity bias minus its distance from that translated point. Earlier interactions shape the learned user and item parameters; with those parameters fixed, histories sharing the user and latest item receive identical candidate scores. H^2SeqRec [162] gives the predictor a richer history. Monthly, quarterly, and yearly hypergraphs enrich item features with calendar-specific co-occurrence, while groups formed from overlapping item histories within the same month add further context. Hyperbolic graph features are mapped to Euclidean coordinates and fused before a Transformer processes the ordered item sequence. A candidate MLP then combines the sequence representation with dynamic item features. Thus, earlier items and their calendar or group context can influence prediction even when the latest item is unchanged; hypergraph encoding and self-attentive history processing operate together.

An output-layer intervention changes how the retained history is separated into candidate decisions. HSASRec [79] keeps the Euclidean SASRec sequence encoder, maps its output into the Poincaré ball, and produces logits from signed distances to candidate-specific hyperplanes. This classifier learns decision surfaces over a sequence state, while HTSR's point-distance score compares a personalized translated item with each candidate. The geometric head is trained jointly with the encoder and can therefore influence representation learning. Once parameters are fixed, however, two histories mapped to the same encoder state still receive the same candidate scores. HMamba-Full [329] places the proposed Lorentz-valued processing within a selective state-space backbone, allowing the ordered history to shape the state before Lorentz-distance ranking. The paper also provides an HMamba-Half variant with dot-product scoring. These matching rules differ in whether prediction evaluates geometric distances to candidate embeddings or their inner products with the sequence output, alongside the choice of sequence-processing machinery.

The candidate objective also affects what is learned from a geometric decision surface. The HSASRec study crosses two geometries with two objectives: Euclidean SASRec and hyperbolic HSASRec each use either sampled binary cross-entropy or full-catalog softmax cross-entropy, with the latter

variants denoted by a plus sign [79]. On MovieLens-1M, for example, the hyperbolic variant has lower mean reciprocal rank than the Euclidean variant under sampled binary cross-entropy, but higher mean reciprocal rank when both use full-catalog cross-entropy. With full-catalog cross-entropy, the Euclidean variant has higher mean reciprocal rank on Digital Music and Arts Crafts and Sewing. A comparison against the original SASRec alone would therefore combine changes in the objective and geometry. Full-catalog learning makes every candidate participate in the softmax normalization, adding computation as well as changing the competition that shapes the classifier. The geometric head's reported benefit consequently depends on the candidate objective applied to the encoded history.

4.6.3 Session-based and Knowledge Graph-enhanced Recommendation

Session-based recommendation. A short session can contain clicks motivated by several interests, yet the recommender must produce a next-item prediction. How those clicks are pooled determines which interests the hyperbolic representation carries into prediction. HCGR [99] and TA-HGAT [159] both encode consecutive clicks in directed session graphs and give the final item a privileged role in the readout. HCGR gates its contextualized final-item representation with that item's graph representation; TA-HGAT uses the final item to weight the other session items. MIHRN [176] broadens both the propagation structure and the readout: each session becomes a hyperedge in a shared item hypergraph, and several affinity matrices extract different weighted interests from the current session. Shared items connect evidence across session hyperedges, while the affinity matrices allow different subsets of the current session to influence different interests. These interest vectors are subsequently fused with a weighted session representation for prediction.

The distinction between co-occurrence, order, and elapsed time determines what these representations can tell the recommender. An undirected session hyperedge records which items occur together; MIHRN encodes order separately through reverse-position embeddings before extracting interests. TA-HGAT additionally embeds elapsed intervals and combines them with neighboring item features through Möbius operations [159]. Thus, identical item sequences with different time gaps can produce different messages. Its evolutionary model projects session and item embeddings toward a given future time and regularizes their change, assuming that intent changes smoothly between nearby interactions. At test time, the evaluation provides the interval from the final observed click to the target interaction, which makes this interval an additional input to prediction.

The training objective further determines which distinctions survive the session readout. HCGR supplements next-item cross-entropy with a Lorentz-distance margin that separates positive and negative item pairs [99]. This geometric ranking constraint acts on candidate discrimination. MIHRN instead adds agreement between two representations of the same session, obtained by repeating interest extraction and readout with independent dropout masks; other sessions in the mini-batch provide negative examples [176]. It trains this cosine-based agreement alongside next-item cross-entropy. Multiple-interest extraction and contrastive supervision therefore operate together: the affinity patterns determine how intent is summarized, and dropout agreement encourages consistency between independently masked session readouts. TA-HGAT's temporal regularization instead links representation changes to an explicit interval and to consistency among recent interactions.

Knowledge-graph-enhanced recommendation. A knowledge graph adds semantic relations that can inform preference beyond observed user–item interactions. A mixed-curvature recommender must decide which relational neighbors matter before combining their evidence into a candidate score. In MCKG [323], user–relation compatibility weights sampled knowledge-graph neighbors when constructing item representations in several spaces with learnable curvature. CurvRec [282] builds a unified user–item–entity graph and propagates both user and item neighborhoods through prescribed

hyperbolic, Euclidean, and spherical branches. Its attention combines relation-specific compatibility with a Ricci-derived local structural coefficient. Relation semantics and local geometry consequently influence the same messages. The edge coefficient adjusts neighbor contributions within these branches, whereas learning a component's curvature changes the geometry used throughout that component.

Where the geometric components meet also changes the final matching rule. MCKG first concatenates the mean of the component embeddings with each component embedding, then combines component geodesic distances through attention weights that depend on the user and candidate item. It trains this fused distance with a geometry-dependent ranking margin [323]. A candidate pair can therefore draw more strongly on some spaces than others even after information has been shared between their representations. CurvRec maps its hyperbolic and spherical representations to tangent coordinates, averages them with the Euclidean branch, and applies a dot-product prediction to the fused user and item vectors [282]. Here, the final score compares common-vector representations. Comparing these designs requires specifying both the geometry of each branch and how representation fusion and distance fusion enter the prediction.

Agreement objectives connect this fusion decision to the information learned in each component. CurvRec trains its recommendation loss jointly with hyperbolic–Euclidean and spherical–Euclidean interaction objectives [282]. The Euclidean branch thus participates in both cross-manifold agreement and the final representation average, so its local-curvature attention and its agreement mechanism serve complementary parts of the same model. MKGCL [334] introduces variation in the graph evidence as well: it combines user–item interaction views and knowledge-augmented semantic views with product-manifold representations, using knowledge enhancement and perturbation in view construction. Its supervised auxiliary and contrastive graph-structure tasks train these views. This makes view construction part of the modeling assumption, since changing the interaction and semantic evidence changes the information that agreement between representations is intended to retain.

Summary. Recommendation design begins with the available evidence: interactions alone, social links, ordered histories, sessions, or a knowledge graph. Within that setting, a curved representation can change neighborhood propagation, ranking and sampling, or the separation of multiple interests. Mixed-curvature models are useful when distinct relation types motivate separate geometric components, with component fusion becoming an additional modeling decision.

4.7 Robotics

Robotics uses geometry for both physical variables and learned task structure. Orientations, stiffness and damping matrices, and manipulability measures have manifold constraints [32, 316]; probabilistic control and movement models operate on those variables through geometric distributions and maps [1, 2]. Hyperbolic robot-learning methods serve a different role when they encode a hierarchy of designs, object parts, or body joints. We compare these uses in design, manipulation, localization, navigation, and action recognition.

4.7.1 Robot Design and Manipulation

Robot design uses geometry to organize candidate structures, while manipulation uses it to respect the orientation variable. HERD [69] embeds a coarse-to-fine design tree in the Poincaré ball, with parent designs at smaller radii and more specific descendants at larger radii (Figure 22). A Cross-Entropy search evaluates layouts with a shared reinforcement-learning policy. G-RL [5] instead

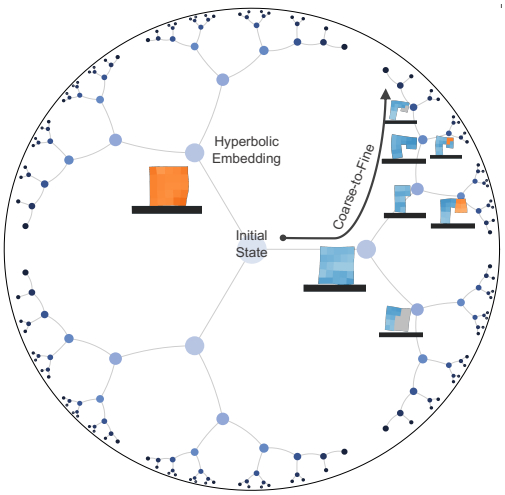


Figure 22 Coarse-to-fine robot designs in the Poincaré ball [69]. The initial design is placed near the center, while successive component replacements generate increasingly specific descendants toward the boundary, organizing the design space as a hierarchy.

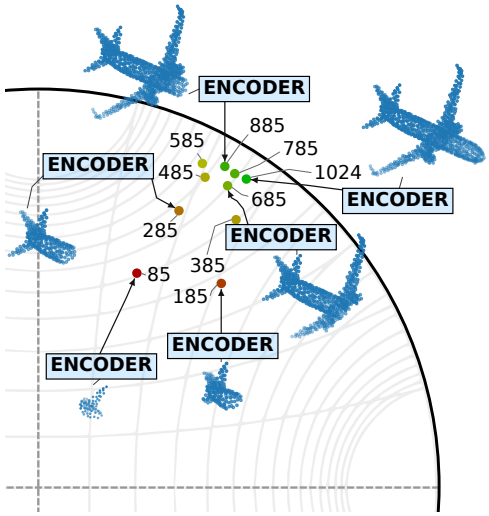


Figure 23 Hierarchical part-to-whole point-cloud embeddings in HyCoRe [197], shown in two dimensions with hyperbolic UMAP. Numbers indicate the point count of each encoded part, with 1,024 points for the complete object.

represents rotational states by unit quaternions on $\mathbb{S}^3$ and uses geometric distances for orientation-policy learning. HERD's hierarchy is a prior over the search space, whereas G-RL's sphere follows from the parameterization of rotations.

4.7.2 Localization and Navigation

Point-cloud methods distinguish semantic part hierarchy from spatial neighborhood geometry. HyCoRe [197] embeds compositional point-cloud representations in the Poincaré ball to organize part-to-whole relations (Figure 23). HypLiLoc [285] combines 3D point features and 2D spherical projections across Euclidean and hyperbolic representations, fusing them with attention for six-degree-of-freedom LiDAR relocalization. A discrete spherical convolution kernel [150] instead partitions local neighborhoods by azimuth, elevation, and radius to process point clouds. The first approach provides semantic organization, the second combines complementary feature views for pose regression, and the third encodes local spatial coordinates in the convolution itself.

4.7.3 Action Recognition

Skeleton-based action recognition combines a spatial joint graph with a temporal sequence. HyLiFormer [163] maps Euclidean skeleton coordinates into the Poincaré ball to represent the joint hierarchy, then uses hyperbolic linear attention for spatiotemporal interactions (Figure 24). The hyperbolic representation provides the structural prior, while linear attention avoids explicitly constructing dense quadratic attention maps.

Summary. In robotics, the geometry of a physical variable and the geometry of a learned hierarchy are independent choices. Unit quaternions constrain orientation, spherical coordinates organize local spatial neighborhoods, and hyperbolic embeddings organize design, object-part, or joint relations. Fusion is useful when a task such as relocalization needs both geometric observations and learned semantic features.

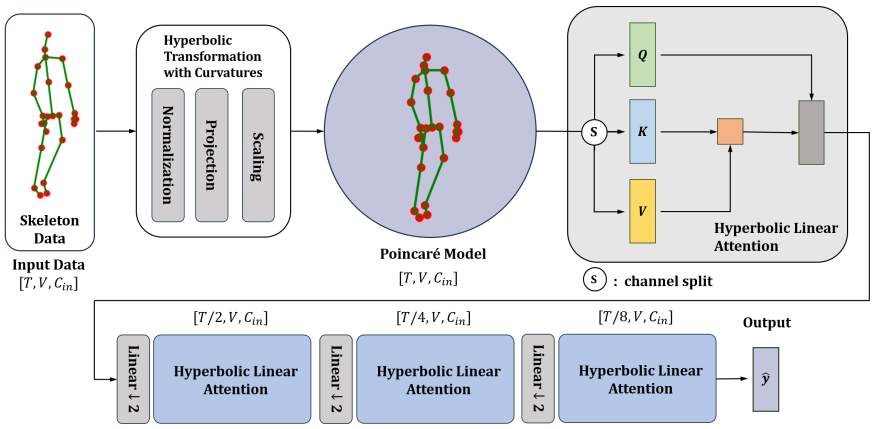


Figure 24 Skeleton-based action recognition in HyLiFormer [163]. Sequential Euclidean skeleton coordinates are transformed into the Poincaré ball, split into query, key, and value channels, and processed by stacked hyperbolic linear-attention layers before action classification.

4.8 AI for Science

Scientific machine learning uses learned representations to support prediction and discovery in settings such as materials modeling and drug discovery [30, 254]. The geometric choice depends on the relation being modeled: evolutionary distance, molecular taxonomy, cellular organization, or brain connectivity. We compare these targets and distinguish embeddings of scientific relations from the physical coordinate space of the observations.

4.8.1 Biological Structure and Molecular Modeling

Biological representations differ in whether geometry encodes relations among image regions, sequences, or molecules. Hyperbolic VAEs for 3D biomedical imagery [113] learn latent relationships between subvolumes for unsupervised segmentation. For sequences, the given relation determines the embedding objective. NeuroSEED [57] trains encoders to match embedding distances to computed edit distances between sequences, evaluating the approximation on randomly held-out sequences and using the learned distances for clustering and multiple sequence alignment. The encoder and its distance metric are separate choices within this framework. A biological taxonomy provides a different target: Gong et al. [89] jointly train image, DNA-barcode, and taxonomic-text encoders with contrastive alignment and stacked entailment in a shared Lorentz space. Parent–child constraints connect successive taxonomic ranks, and specimen embeddings align with their taxonomic labels. Their BIOSCAN-1M experiments train on the seen-species split with complete labels and evaluate retrieval for both seen and unseen taxa. For macro top-1 retrieval, stacked-entailment variants give the highest unseen-species DNA-to-DNA retrieval accuracy, while image-to-image retrieval favors the Euclidean baseline.

Molecular embeddings can incorporate an explicit drug hierarchy alongside molecular information. The semi-supervised hyperbolic VAE of Yu et al. [317] supports drug repositioning and side-effect prediction; the extension of Yu et al. [318] incorporates Anatomical Therapeutic Chemical (ATC) classification and evaluates drug repositioning on a chronological repoDB split. Comparing embeddings with and without ATC information separates the contribution of taxonomy from the molecular representation. At the architectural level, Joseph et al. [123] relate hierarchical and scale-free brain-network organization to hyperbolic neural networks, using biological structure as an inductive bias for artificial models.

4.8.2 Cellular and Genomic Analysis

A known hierarchy can guide language-model pretraining, while functional labels can directly train a geometric sequence encoder. HyperHELM [274] uses the known codon–amino-acid hierarchy to construct fixed hyperbolic prototypes for masked-codon prediction. A Euclidean Transformer provides contextual token features, which pass through a Poincaré projection and hyperbolic linear layer before prototype classification. The pretrained backbone is then frozen, and task-specific heads are trained with labeled data for mRNA property prediction and antibody-coding mRNA region annotation. It is compared with hierarchical Euclidean HELM, which shares the backbone and pretraining data and differs in the pretraining head. In the *P. pastoris* evaluation, both prototype variants improve prediction for sequences of 2,000–3,000 nucleotides, longer than the sequences used in pretraining. A GC-stratified analysis on the COVID-19 degradation dataset shows that the entailment variant improves the medium- and high-GC results and matches HELM in the low-GC group. Khan et al. [128] instead train Lorentz CNNs directly from DNA sequences and downstream class labels, placing geometric operations in convolutional feature extraction and classification. Their transposable-element benchmark holds out chromosomes within the studied species. A simulated experiment tests a tree-versus-background classifier on sequences generated from a new tree, evaluating the architectural prior under a change in evolutionary generating structure.

Cellular and spatial analyses instead use relations among cells or between images and gene expression. TopoLa [335] uses latent hyperbolic distances to encode intercellular topology in single-cell RNA sequencing and spatial transcriptomics. DELST [47] uses dual entailment learning to align gene-expression patterns and tissue images for multimodal spatial transcriptomics. TopoLa emphasizes topology among cellular representations; DELST adds a cross-modal hierarchical alignment objective and therefore uses paired molecular and visual evidence.

4.8.3 Neuroscience

A connectome can be compared through its overall geometry or through the neighborhood of a particular brain region. Cacciola et al. [31] derive low-dimensional hyperbolic coordinates from weighted structural connectomes and test the angular arrangement against hemisphere and lobe labels. Their Parkinson's disease/control analysis finds significant differences in global averages of hyperbolic distances or hyperbolic shortest-path lengths for several embeddings, whereas mean edge weight has no significant group difference. Longhena et al. [179] instead embed separate structural diffusion-MRI and functional resting-state fMRI networks, retaining and binarizing their strongest connections. Neighborhood similarity guides angular order, while degree rank determines radius. In their Alzheimer's disease/control study, a local distortion score compares distances to the same control-defined neighbors in patient and control embeddings. The score highlights memory-related and frontal differences that remain visible across parcellation scales.

Predictive and alignment models use additional targets during representation learning. Hyperbolic Kernel Graph Fusion (HKGF) [300] trains graph encoders and structural–functional coupling to distinguish participants with significant memory complaints or HIV-associated asymptomatic neurocognitive impairment from controls in separate cohort comparisons. HEEGNet [157] instead decodes electroencephalography (EEG) epochs using Euclidean signal encoders and Lorentz layers. Its cross-subject and within-subject cross-session evaluations adapt domain-specific moments using unlabeled target signals while keeping the fitted feature extractor and classifier fixed. Multi-level Neuroimaging Meta-analysis (MNM) [14] uses a different pairing: article text and activation maps reconstructed from published peak coordinates. It maps both into Lorentz space and trains contrastive

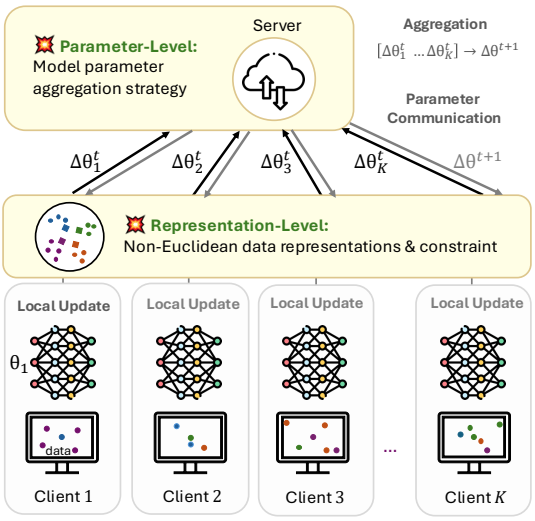


Figure 25 Non-Euclidean geometry in federated learning. At the representation level, clients align data embeddings or classifier prototypes under shared manifold constraints. At the parameter level, geometric designs determine which parameters are shared or retained locally before server aggregation of updates $\Delta\theta_i$.

correspondence on matched article–map pairs. A centroid constraint encodes the assumption that activation maps provide broader context than their texts, while an additional loss orders brain maps by the number of activated atlas regions. Bidirectional retrieval evaluates the learned correspondence, and activation-count analyses test the imposed broad-to-local ordering.

Summary. Scientific applications make the supervision behind a geometric representation especially consequential. Evolutionary distances and taxonomies provide explicit relational targets; cell, image, and brain-network models can also infer organization from observations. We therefore compare what relation the embedding is trained to preserve and how it enters the downstream task, alongside the choice of curvature.

4.9 Trustworthiness

Trustworthiness applications use geometry to align heterogeneous clients, distinguish harmful content, and recognize unfamiliar or perturbed inputs. Geometry provides a different signal in each case: shared class structure, content-specific entailment, or uncertainty. Hyperbolic hierarchy also affects how attacks propagate, making the representation part of the threat model.

4.9.1 Federated Learning

Federated learning (FL) trains across clients without centralizing their raw training data [126, 188]. Geometric FL methods differ in whether they align representations or organize parameter sharing (Figure 25). Representation-level methods constrain features or class prototypes, while parameter-level methods regularize or decouple client updates.

Representation-level. Shared class targets require consistent class identifiers across clients, even when a client lacks examples of some classes. SphereFed [70] constructs orthonormal classifier directions and trains normalized features against these fixed targets. After representation training, it calibrates the classifier from aggregated feature second moments and feature–label cross moments, keeping the extractor frozen. HyperFed [168] optimizes separated prototype directions, contracts them to a common radius inside the Poincaré ball, and fixes them across clients. Local labeled examples train

mapped features against positive and negative class prototypes through a triplet objective. Clients exchange model updates while the prototypes remain fixed. These constructions separate classes without requiring a parent–child taxonomy.

HyperFLAW [233] addresses source-free adaptation after pretraining on labeled synthetic data. Its source and target tasks share a semantic class set, with drone images covering a subset. Teacher pseudo-labels group target pixel features into hyperbolic class centroids. These centroids evolve during local self-training and are aggregated alongside model updates and learned curvature, providing common class references across car and drone viewpoints and weather conditions.

Other representation-level methods change the information transmitted or selected at clients. The hyperbolic classification method of Prakash et al. [224] aggregates protected geometric summaries of client points for server-side convex SVM classification. FedHGCN [72] combines FedAvg with hyperbolic graph learning and client node selection. The former constructs a server-side classifier from geometric data summaries, whereas the latter trains a shared graph model through selected local information.

FedMRUR [8] uses the received global model as a reference, aligning the feature representations produced by the local and global models on the same local data. It maps those features into the Lorentz model and regularizes their geometric agreement, then uses normalized model-update aggregation at the server. Its geometric regularizer acts on the models' outputs, linking representation alignment to the federated training procedure.

Parameter sharing. FlatLand [174] personalizes graph models through locally learned curvature and a private/shared split of Lorentz-layer weights. Graph statistics initialize the local geometry. Clients upload weights acting on spatial coordinates while retaining the time-coordinate weights and curvature; the averaged spatial block is combined with these local components each round. The resulting representations remain in each client's Lorentz space.

Summary. Shared classifiers are a direct choice when the main problem is inconsistent class representations. Feature regularization aligns local and global outputs, while parameter decoupling supports client-specific components. Table 14 compares the geometric object and sharing mechanism of these methods.

4.9.2 Safety and Harmful Content

Geometric safety methods intervene either in vision–language alignment or in the prompt representation. HySAC [223] learns a safety and modality hierarchy through entailment and contrastive objectives, then uses radial position for unsafe-content detection and controlled retrieval. Harmful-prompt detection and sanitization [185] instead treats harmful inputs as anomalies, using radial information for detection and geometric movement for sanitization. The first changes how a multimodal model relates safety concepts and content; the second adds an input-level detection and correction mechanism. Their safety interpretation is learned from the task's content labels and objectives.

4.9.3 Robustness and Out-of-Distribution Detection

Out-of-distribution detection and adversarial defense use different geometric signals. Balanced hyperbolic embeddings [127] use radial position for detection without outlier exposure, connecting confidence to the organization induced by training. HyperDefender [184] instead addresses adversarial perturbations and noise in hyperbolic graph networks. Radial detection scores unfamiliar

Table 14 Geometric interventions in federated learning. Representation-level methods align or summarize client data; geometry-guided parameter sharing separates shared and local model components. Rows identify the object constrained by geometry and the corresponding communication or learning mechanism.

Method	Geometric object	Client-side operation	Shared information or constraint
Representation-level geometry			
Prakash et al. [224]	Poincaré data points	Construct geometric data summaries	Secure convex-hull aggregation for server classification
FedHGCN [72]	Hyperbolic graph features	Node selection and graph-model training	FedAvg model aggregation
SphereFed [70]	Spherical classifier directions	Train features against a fixed classifier	Common directional class targets
HyperFed [168]	Hyperbolic class prototypes	Train mapped features against fixed class prototypes	Common hyperbolic class targets
HyperFLAW [233]	Hyperbolic class prototypes	Source-free adaptation for driving images	Aggregated class centroids and curvature
FedMRUR [8]	Lorentz feature representations	Align local and global features on the same data	Normalized model-update aggregation
Geometry-guided parameter sharing			
FlatLand [174]	Client representations in Lorentz spaces	Decouple weights acting on time and spatial coordinates	Share spatial-coordinate weights; keep time-coordinate weights and curvature local

inputs, while a robustness method changes the processing of corrupted inputs; these objectives call for different evaluations.

Hierarchical structure can also amplify an attack's reach. Geometry-aware backdoor attacks [15] exploit high-level regions of hyperbolic embeddings so that a trigger influences descendant classes. This places the location and propagation of an attack within the geometric threat model, alongside ordinary prediction accuracy and robustness.

Summary. Trustworthiness methods control different behaviors: client alignment, safety-aware content relations, unfamiliar-input detection, or resistance to corrupted data. A radial coordinate supports several of these mechanisms, but its meaning follows from the supervision and objective. Hierarchical organization additionally determines which classes an attack can reach.

Cross-domain synthesis. Across applications, geometry acts on three recurring objects: data relations, learned representations, and model parameters. Hyperbolic radius supports specificity when the objective trains an order, and confidence when the objective links radius to certainty. Spherical models use angular structure for directional similarity or manifold-valued observations. Mixed-curvature models combine component spaces for heterogeneous relations, adding choices about allocation and information sharing.

4.10 Tools and Resources

Tool selection depends on whether a project needs geometric primitives, ready-made neural layers, or complete model modules. Geoopt [137] provides manifold-aware parameter types and Riemannian optimizers for PyTorch models; Geomstats [194] emphasizes geometric statistics and estimators across a broad manifold collection. Manifolds.jl [11] provides differential-geometric operations in Julia and connects to the surrounding optimization ecosystem. These libraries give flexibility over

Table 15 Open-source libraries for geometric learning, grouped by their role in model construction: manifold optimization and geometry, hyperbolic neural layers, and foundation-model modules. The columns describe the primary focus of each library, the manifolds it exposes, its support for Riemannian optimization and ready-made neural modules, and its host ecosystem. A dash denotes functionality outside the library's stated focus.

Library	Primary focus	Manifold support	Optimization support	Neural-module support	Ecosystem
Geoopt [137]	Riemannian optimization primitives	Poincaré ball, Lorentz model, sphere, stereographic and product manifolds	Riemannian SGD and Adam over manifold-constrained parameters	Manifold parameter and tensor types for user-defined layers	PyTorch
Geomstats [194]	Riemannian geometry and statistics	Broad manifold collection, including hyperbolic, spherical, and matrix manifolds	Riemannian gradient descent and geometric estimators	Geometric learning utilities and estimators	Python with interchangeable array backends
Manifolds.jl [11]	Differential-geometric operations on manifolds	Extensible manifold collection, including hyperbolic, spherical, and product manifolds	Interfaces to Riemannian solvers in the surrounding ecosystem	—	Julia
HypLL [273]	Hyperbolic deep learning components	Poincaré ball	Riemannian optimizers for hyperbolic parameters	Hyperbolic layers and network building blocks	PyTorch
Hyperlib [117]	Hyperbolic layers and embedding utilities	Poincaré ball	Riemannian optimizers for hyperbolic parameters	Hyperbolic layers and embedding routines	TensorFlow and Keras
HyperCore [109]	Modules for hyperbolic foundation models	Lorentz model and Poincaré ball	Riemannian optimization utilities	Hyperbolic Transformer, graph, and vision modules	PyTorch

the model design and leave more layer construction to the user.

HypLL [273] and Hyperlib [117] provide hyperbolic neural components in PyTorch and TensorFlow/Keras, respectively. HyperCore [109] provides modules for hyperbolic foundation models across modalities. Their narrower geometric interfaces make standard layers and architectures easier to assemble. NetworkX [103] complements these packages with graph-input analysis. Table 15 compares the six geometric libraries by manifold coverage, optimization, neural modules, and ecosystem. Combining packages requires agreement on curvature conventions, coordinates, and tensor interfaces, motivating the unified framework in Section 5.5.

5 Challenges and Discussion

The method comparisons reveal three recurring design choices: how much geometric structure to encode, where to perform the computation, and how much computation the task can afford. Allowing embeddings to lie farther from the origin at fixed curvature provides room for deeper hierarchies but demands more numerical precision. A product space represents several kinds of relations but introduces choices about factor dimensions, curvature, and interaction. Geometric backbones apply these decisions throughout a model, while heads and adapters concentrate them in smaller modules. We discuss how these choices affect implementation, evaluation, and model selection.

5.1 Numerical Instability

Coordinate precision and optimization. Poincaré and Lorentz coordinates describe the same hyperbolic geometry, but they encounter different numerical problems. At fixed curvature, Poincaré coordinates approach a finite boundary as distance from the origin increases; Lorentz coordinates grow in magnitude. In the Poincaré ball, boundary terms in distances and maps become sensitive to rounding [45, 60, 227], while gradients and updates can become difficult to resolve [82, 101]. The distinction between representing a point accurately and optimizing its location matters here. Under the 64-bit analysis of Mishne et al. [195], the Poincaré model represents a larger range of radii correctly, whereas the Lorentz model has advantages for optimization. Model selection therefore depends on the coordinates, arithmetic precision, and operations used together.

Stable operators. Exponential and logarithmic maps involve hyperbolic functions and inverse functions whose evaluation requires care near domain boundaries [319]. Distances create another difficulty: the derivative of arcosh is singular at one, the argument reached by coincident Lorentz points. Squared distances and suitable reformulations can improve numerical behavior near such points [235]. The Lorentz chord score $\widetilde{d}^2_{\mathbb{L},c}$ in Equation (11) avoids arcosh and supports closed-form centroid computations [144]. It also rescales distances nonlinearly, changing loss gradients and attention weights. Choosing this score is consequently part of the model design. Accurate representations at large radii remain an active numerical problem [320].

These comparisons suggest evaluating numerical behavior at the level of the operations a model repeats. Useful quantities include embedding radii, violations of manifold constraints, the size of updates, and the frequency of non-finite values. A stable implementation should preserve both the intended geometry and usable gradients through those operations.

5.2 Optimization Cost

Pairwise computation and repeated maps. In hyperbolic learning, the main costs come from the number of point pairs evaluated, repeated geometric maps, and arithmetic precision. Dense distance objectives require pairwise work that can grow quadratically with the number of points; acceleration methods target this cost [251]. Higher-precision representations address rounding at an additional arithmetic cost [321]. Neural architectures make a separate choice between reusing Euclidean operations through tangent maps and implementing transformations directly on the manifold. Initialization also matters: HNN and HNN++ use controlled scales to manage the behavior of geometric transformations [82, 243]. We recommend comparing these choices over the complete training or inference computation, since a cheaper individual operation can still be called more often.

Spatial resolution and geometric factors. For spherical images, tangent-plane processing renders local views and applies planar operations, making repeated sampling part of the cost [255]. Harmonic spherical CNNs share computation through spherical transforms and provide rotation-equivariant operations, with costs that depend on resolution and spectral representation [53]. In mixed-curvature attention, several factors contribute geometric scores or values, so factor count and width affect both computation and memory. Kernelized attention, as used by FPS-T, addresses pairwise attention cost within this setting [51]. The practical question is how much additional structure each factor contributes at a given compute budget. This connects efficient implementation to learning factor dimensions and avoiding unnecessary repeated projections.

5.3 Comparing Geometric Models

Separating geometry from architecture. A comparison at a fixed backbone and representation dimension helps identify the effect of changing geometry and its operators. A complementary comparison at a fixed total parameter or compute budget tests whether the geometric design uses resources effectively. Both are useful because a Lorentz representation includes a constrained ambient coordinate, and product models distribute dimensions across several factors. Reporting the intrinsic representation dimension, trainable parameters, and training cost makes these alternatives easier to interpret.

Connecting structure to task quality. Downstream accuracy, ranking quality, or generation quality answers whether a model helps the task. Distortion, learned curvature, and neighborhood structure help explain what changed in the representation. Evaluating both connects the structural motivation to the observed result. Hierarchical, directional, cyclic, and heterogeneous examples provide different tests of a geometric prior. Numerical behavior, memory, and throughput then show the cost of obtaining that result. For applications involving uncertainty or distribution shift, calibration and robustness offer additional task-relevant comparisons.

5.4 Geometric Model Selection

Choose the structural assumption before the coordinates. Hyperbolic geometry is a useful candidate when distances should preserve a branching hierarchy or a graph with hierarchical organization [139, 203, 213]. Within that geometry, Poincaré coordinates offer a bounded visualization and established neural operations [82, 243]; large radii require attention to clipping and precision [38]. Lorentz coordinates support simple formulas for transformations and aggregation [45, 205]. Their practical value depends on the sequence of operations, as illustrated by graph networks [175] and temporal embeddings [303]. Klein coordinates make geodesics straight and simplify Einstein-midpoint aggregation [96]. These are coordinate and operator choices within one geometry. Their role also depends on whether the geometric component is a full backbone or a smaller module in a multimodal model [186].

Spherical representations suit objectives based on direction or angular similarity, as in vision embeddings [291] and text representations [189]. Positive-curvature factors also support some cyclic distance patterns in product representations [94, 250] and relation-specific knowledge graph models [284]. When different relations call for different structures, mixed-curvature models allocate them to multiple geometric components [120, 212]. This makes factor allocation and the rule for combining factors central design decisions.

Use diagnostics to narrow the candidates. Gromov hyperbolicity measures metric tree-likeness through thin triangles or four-point distances. Graph curvature and resistance summarize complementary properties of connectivity, while angular statistics describe directional concentration. As discussed in Section 2.4.4, these quantities help identify candidate geometries but do not uniquely determine one. Task quality, representation distortion, and computation then distinguish the candidates. For adaptive models, inspecting how curvature and factor use vary across relations or inputs can reveal which geometric components the task actually uses.

5.5 Unified Geometric Framework

Geometric software supports different levels of model construction. Geoopt offers Riemannian optimization primitives [137], HypLL provides hyperbolic learning components [273], and HyperCore targets modules for hyperbolic foundation models [109]; table 15 compares their roles. Combining

these tools requires compatible conventions for curvature signs, coordinate order, tangent basepoints, and normalization. An operation that accepts a manifold point must also distinguish it from a tangent vector and retain the information needed for subsequent mappings.

Mixed-curvature models extend this requirement to collections of factors. A useful interface would track each factor's dimension and curvature, define how features move between factors, and expose the resulting cost to the training system. Shared implementations of these operations would make architectural comparisons easier to reproduce and allow efficient kernels to benefit several model families. This provides a concrete software direction for the adaptive models discussed throughout the survey.

6 Conclusion

We have compared hyperbolic, spherical, and mixed-curvature learning through the relationships they represent and the operations they require. Hyperbolic models use radial and angular structure to organize hierarchies, spherical representations support directional similarity, and product or adaptive constructions combine geometric components for heterogeneous relations. Across model families, the most useful distinctions concern where geometry enters the computation, how features are transformed and aggregated, and how the resulting objective serves the task. These choices explain the differences between tangent-space and manifold operations, geometric backbones and adapters, and a single learned curvature and several interacting factors. The application comparisons connect those mechanisms to language, perception, relational data, science, and trustworthiness. Further progress depends on making the same choices practical: stable computations at useful embedding radii, efficient geometric operators, and ways to learn component dimensions and curvature together with the task. We hope the comparisons and resources in this survey help readers understand existing methods and build on them.

References

[1] Fares J Abu-Dakka and Ville Kyrki. Geometry-aware dynamic movement primitives. In *ICRA*, pp. 4421–4426. IEEE, 2020.

[2] Fares J Abu-Dakka, Yanlong Huang, João Silvério, and Ville Kyrki. A probabilistic framework for learning geometry-based robot manipulation skills. *Robot. Auton. Syst.*, 141:103761, 2021.

[3] Aaron B Adcock, Blair D Sullivan, and Michael W Mahoney. Tree-like structure in large social and information networks. In *ICDM*, pp. 1–10. IEEE, 2013.

[4] Ola Ahmad and Freddy Lecue. FisheyeHDK: Hyperbolic deformable kernel learning for ultra-wide field-of-view image recognition. In *AAAI*, volume 36, pp. 5968–5975, 2022.

[5] Naseem Alhousani, Matteo Saveriano, Ibrahim Sevinc, Talha Abdulkuddus, Hatice Kose, and Fares J Abu-Dakka. Geometric reinforcement learning for robotic manipulation. *IEEE Access*, 11:111492–111505, 2023.

[6] Morris Alper and Hadar Averbuch-Elor. Emergent visual-semantic hierarchies in image-text representations. In *ECCV*, pp. 220–238. Springer, 2024.

[7] Fazail Amin, Arijit Mondal, and Jimson Mathew. Deep semantic hashing with structure-semantic disagreement correction via hyperbolic metric learning. In *MMSP*, pp. 1–6. IEEE, 2022.

[8] Xuming An, Li Shen, Han Hu, and Yong Luo. Federated learning with manifold regularization and normalized update reaggregation. In *NeurIPS*, volume 36, 2023.

[9] Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *IJRR*, 39(1):3–20, 2020.

[10] Mina Ghadimi Atigh, Julian Schoep, Erman Acar, Nanne Van Noord, and Pascal Mettes. Hyperbolic image segmentation. In *CVPR*, pp. 4453–4462, 2022.

[11] Seth D Axen, Mateusz Baran, Ronny Bergmann, and Krzysztof Rzecki. Manifolds. jl: an extensible Julia framework for data analysis on manifolds. *ACM TOMS*, 49(4):1–23, 2023.

[12] Darko Babić, Douglas J Klein, István Lukovits, Sonja Nikolić, and Nenad Trinajstić. Resistance-distance matrix: A computational algorithm and its application. *Int. J. Quantum Chem.*, 90(1):166–176, 2002.

[13] Gregor Bachmann, Gary Bécigneul, and Octavian Ganea. Constant curvature graph convolutional networks. In *ICML*, pp. 486–496. PMLR, 2020.

[14] Seunghun Baek, Jaejin Lee, Jaeyoon Sim, Minjae Jeong, and Won Hwa Kim. MNM: Multi-level neuroimaging meta-analysis with hyperbolic brain-text representations. In *MICCAI*, pp. 397–406. Springer, 2025.

[15] Ali Baheri. Geometry-aware backdoor attacks: Leveraging curvature in hyperbolic embeddings. *arXiv preprint arXiv:2510.06397*, 2025.

[16] Ivana Balazevic, Carl Allen, and Timothy Hospedales. Multi-relational Poincaré graph embeddings. In *NeurIPS*, volume 32, pp. 4465–4475, 2019.

[17] Ahmad Bdeir, Kristian Schwethelm, and Niels Landwehr. Fully hyperbolic convolutional neural networks for computer vision. In *ICLR*, 2024.

[18] Gary Bécigneul and Octavian-Eugen Ganea. Riemannian adaptive optimization methods. In *ICLR*, 2019.

[19] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE TPAMI*, 35(8):1798–1828, 2013.

[20] Marián Boguñá, Fragkiskos Papadopoulos, and Dmitri Krioukov. Sustaining the Internet with hyperbolic mapping. *Nature Communications*, 1(1):62, 2010.

[21] Rishi Bommasani, Drew A. Hudson, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[22] Mario Bonk and Oded Schramm. Embeddings of Gromov hyperbolic spaces. *GAFA*, 10(2):266–306, 2000.

[23] Silvere Bonnabel. Stochastic gradient descent on Riemannian manifolds. *IEEE TAC*, 58(9):2217–2229, 2013.

[24] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, volume 26, 2013.

[25] J. Bourgain. The metrical interpretation of superreflexivity in Banach spaces. *Israel Journal of Mathematics*, 56(2):222–230, 1986. doi: 10.1007/BF02766125.

[26] Jean Bourgain. On Lipschitz embedding of finite metric spaces in Hilbert space. *Israel J. Math.*, 52(1–2): 46–52, 1985.

[27] Martin R Bridson and André Haefliger. *Metric spaces of non-positive curvature*, volume 319 of *Grundlehren der mathematischen Wissenschaften*. Springer, 1999.

[28] Michael M Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: going beyond Euclidean data. *IEEE SPM*, 34(4):18–42, 2017.

[29] Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.

[30] Keith T Butler, Daniel W Davies, Hugh Cartwright, Olexandr Isayev, and Aron Walsh. Machine learning for molecular and materials science. *Nature*, 559(7715):547–555, 2018.

[31] Alberto Cacciola, Alessandro Muscoloni, Vaibhav Narula, Alessandro Calamuneri, Salvatore Nigro, Emeran A. Mayer, Jennifer S. Labus, Giuseppe Anastasi, Aldo Quattrone, Angelo Quartarone, Demetrio Milardi, and Carlo Vittorio Cannistraci. Coalescent embedding in the hyperbolic space unsupervisedly discloses the hidden geometry of the brain. *arXiv preprint arXiv:1705.04192*, 2017.

[32] Sylvain Calinon. Gaussians on Riemannian manifolds: Applications for robot learning and adaptive control. *IEEE RA-M*, 27(2):33–45, 2020.

[33] James W. Cannon, William J. Floyd, Richard Kenyon, and Walter R. Parry. Hyperbolic geometry. In Silvio Levy (ed.), *Flavors of Geometry*, volume 31 of *MSRI Publications*, pp. 59–115. Cambridge University Press, 1997.

[34] Jiahang Cao, Jinyuan Fang, Zaiqiao Meng, and Shangsong Liang. Knowledge graph embedding: A survey from the perspective of representation spaces. *ACM CSUR*, 56(6):1–42, 2024.

[35] Linxiao Cao, Ruitao Wang, Jindong Li, Zhipeng Zhou, and Menglin Yang. HyperbolicRAG: Enhancing retrieval-augmented generation with hyperbolic representations. *arXiv preprint arXiv:2511.18808*, 2025.

[36] Zongsheng Cao, Qianqian Xu, Zhiyong Yang, Xiaochun Cao, and Qingming Huang. Geometry interaction knowledge graph embeddings. In *AAAI*, volume 36, pp. 5521–5529, 2022.

[37] Dorota Celińska-Kopczyńska and Eryk Kopczyński. Numerical aspects of hyperbolic geometry. In *ICCS*, pp. 115–130. Springer, 2024.

[38] Edoardo Cetin, Benjamin Paul Chamberlain, Michael M. Bronstein, and Jonathan J. Hunt. Hyperbolic deep reinforcement learning. In *ICLR*, 2023.

[39] Ines Chami, Zhitao Ying, Christopher Ré, and Jure Leskovec. Hyperbolic graph convolutional neural networks. In *NeurIPS*, volume 32, pp. 4868–4879, 2019.

[40] Ines Chami, Adva Wolf, Da-Cheng Juan, Frederic Sala, Sujith Ravi, and Christopher Ré. Low-dimensional hyperbolic knowledge graph embeddings. In *ACL*, pp. 6901–6914, 2020.

[41] Bike Chen, Wei Peng, Xiaofeng Cao, and Juha Röning. Hyperbolic uncertainty aware semantic segmentation. *IEEE TITS*, 25(2):1275–1290, 2023.

[42] Boli Chen, Yao Fu, Guangwei Xu, Pengjun Xie, Chuanqi Tan, Mosha Chen, and Liping Jing. Probing BERT in hyperbolic spaces. In *ICLR*, 2021.

[43] Nan Chen, Xiangdong Su, and Feilong Bao. Hyperbolic representations for prompt learning. In *LREC-COLING*, pp. 8487–8492, 2024.

[44] Tianyi Chen, Yunfei Zhang, Xiaoyang Huo, Si Wu, Yong Xu, and Hau San Wong. SphericGAN: Semi-supervised hyper-spherical generative adversarial networks for fine-grained image synthesis. In *CVPR*, pp. 10001–10010, 2022.

[45] Weize Chen, Xu Han, Yankai Lin, Hexu Zhao, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. Fully hyperbolic neural networks. In *ACL*, pp. 5672–5686, 2022.

[46] Weize Chen, Xu Han, Yankai Lin, Kaichen He, Ruobing Xie, Jie Zhou, and Zhiyuan Liu. Hyperbolic pre-trained language model. *IEEE/ACM TASLP*, 32:3101–3112, 2024.

[47] Xulin Chen and Junzhou Huang. DELST: Dual entailment learning for hyperbolic image-gene pretraining in spatial transcriptomics. *arXiv preprint arXiv:2503.00804*, 2025.

[48] Yankai Chen, Menglin Yang, Yingxue Zhang, Mengchen Zhao, Ziqiao Meng, Jianye Hao, and Irwin King. Modeling scale-free graphs with hyperbolic geometry for knowledge-aware recommendation. In *WSDM*, pp. 94–102, 2022.

[49] Ziyang Chen, Jinzhi Liao, and Xiang Zhao. Multi-granularity temporal question answering over knowledge graphs. In *ACL*, pp. 11378–11392, 2023.

[50] Hyunghoon Cho, Benjamin DeMeo, Jian Peng, and Bonnie Berger. Large-margin classification in hyperbolic space. In *AISTATS*, pp. 1832–1840. PMLR, 2019.

[51] Sungjun Cho, Seunghyuk Cho, Sungwoo Park, Hankook Lee, Honglak Lee, and Moontae Lee. Curve your attention: Mixed-curvature transformers for graph representation learning. *arXiv preprint arXiv:2309.04082*, 2023.

[52] Aaron Clauset, Cristopher Moore, and Mark EJ Newman. Hierarchical structure and the prediction of missing links in networks. *Nature*, 453(7191):98–101, 2008.

[53] Oliver J Cobb, Christopher GR Wallis, Augustine N Mavor-Parker, Augustin Marignier, Matthew A Price, Mayeul d'Avezac, and Jason D McEwen. Efficient generalized spherical CNNs. In *ICLR*, 2021.

[54] Andy Coenen, Emily Reif, Ann Yuan, Been Kim, Adam Pearce, Fernanda Viégas, and Martin Wattenberg. Visualizing and measuring the geometry of BERT. In *NeurIPS*, 2019.

[55] Taco S Cohen, Mario Geiger, Jonas Köhler, and Max Welling. Spherical CNNs. In *ICLR*, 2018.

[56] Mert Cokelek, Nevrez Imamoglu, Cagri Ozcinar, Erkut Erdem, and Aykut Erdem. Spherical vision transformer for 360-degree video saliency prediction. In *BMVC*, 2023.

[57] Gabriele Corso, Zhitao Ying, Michal Pándy, Petar Veličković, Jure Leskovec, and Pietro Liò. Neural distance embeddings for biological sequences. In *NeurIPS*, volume 34, pp. 18539–18551, 2021.

[58] Sueli IR Costa, Sandra A Santos, and Joao E Strapasson. Fisher information distance: A geometrical reading. *Discrete Appl. Math.*, 197:59–69, 2015.

[59] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *NeurIPS*, volume 26, pp. 2292–2300, 2013.

[60] Jindou Dai, Yuwei Wu, Zhi Gao, and Yunde Jia. A hyperbolic-to-hyperbolic graph convolutional network. In *CVPR*, pp. 154–163, 2021.

[61] Shuyang Dai, Zhe Gan, Yu Cheng, Chenyang Tao, Lawrence Carin, and Jingjing Liu. APo-VAE: Text generation in hyperbolic space. In *NAACL-HLT*, pp. 416–431, 2021.

[62] Tim R Davidson, Luca Falorsi, Nicola De Cao, Thomas Kipf, and Jakub M Tomczak. Hyperspherical variational auto-encoders. In *UAI*, pp. 856–865, 2018.

[63] Michaël Defferrard, Martino Milani, Frédérick Gusset, and Nathanaël Perraudin. DeepSphere: A graph-based spherical CNN. In *ICLR*, 2020.

[64] Karan Desai, Maximilian Nickel, Tanmay Rajpurohit, Justin Johnson, and Shanmukha Ramakrishna Vedantam. Hyperbolic image-text representations. In *ICML*, pp. 7694–7731. PMLR, 2023.

[65] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, pp. 4171–4186, 2019.

[66] Ankit Dhall, Anastasia Makarova, Octavian Ganea, Dario Pavllo, Michael Greeff, and Andreas Krause. Hierarchical image classification using entailment cone embeddings. In *CVPRW*, pp. 836–837, 2020.

[67] Bhuwan Dhingra, Christopher J Shallue, Mohammad Norouzi, Andrew M Dai, and George E Dahl. Embedding text in hyperbolic spaces. In *TextGraphs at NAACL-HLT*, pp. 59–69, 2018.

[68] M.P. do Carmo. *Riemannian Geometry*. Mathematics (Birkhäuser) theory. Birkhäuser Boston, 1992. ISBN 9780817634902.

[69] Heng Dong, Junyu Zhang, and Chongjie Zhang. Leveraging hyperbolic embeddings for coarse-to-fine robot design. In *ICLR*, 2024.

[70] Xin Dong, Sai Qian Zhang, Ang Li, and HT Kung. SphereFed: Hyperspherical federated learning. In *ECCV*, pp. 165–184. Springer, 2022.

[71] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[72] Haizhou Du, Conghao Liu, Haotian Liu, Xiaoyu Ding, and Huan Huo. An efficient federated learning framework for graph learning in hyperbolic space. *KBS*, 289:111438, 2024.

[73] Wendy Ellens. *Effective resistance and other graph measures for network robustness*. PhD thesis, Leiden University, 2011.

[74] Aleksandr Ermolov, Leyla Mirvakhabova, Valentin Khrulkov, Nicu Sebe, and Ivan Oseledets. Hyperbolic vision transformers: Combining improvements in metric learning. In *CVPR*, pp. 7409–7419, 2022.

[75] Xiran Fan, Chun-Hao Yang, and Baba C Vemuri. Nested hyperbolic spaces for dimensionality reduction and hyperbolic nn design. In *CVPR*, pp. 356–365, 2022.

[76] Pengfei Fang, Mehrtash Harandi, Trung Le, and Dinh Phung. Hyperbolic geometry in computer vision: A survey. *arXiv preprint arXiv:2304.10764*, 2023.

[77] Hamza Farooq, Yongxin Chen, Tryphon T Georgiou, Allen Tannenbaum, and Christophe Lenglet. Network curvature as a hallmark of brain structural connectivity. *Nat. Commun.*, 10(1):1–11, 2019.

[78] Edward Fish and Richard Bowden. Geo-Sign: Hyperbolic contrastive regularisation for geometrically aware sign language translation. In *NeurIPS*, 2025.

[79] Evgeny Frolov, Tatyana Matveeva, Leyla Mirvakhabova, and Ivan Oseledets. Self-attentive sequential recommendations with hyperbolic representations. In *RecSys*, pp. 981–986, 2024.

[80] Xingcheng Fu, Jianxin Li, Jia Wu, Qingyun Sun, Cheng Ji, Senzhang Wang, Jiajun Tan, Hao Peng, and Philip S Yu. ACE-HGNN: Adaptive curvature exploration hyperbolic graph neural network. In *ICDM*, pp. 111–120. IEEE, 2021.

[81] Octavian Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic entailment cones for learning hierarchical embeddings. In *ICML*, pp. 1646–1655, 2018.

[82] Octavian Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. In *NeurIPS*, volume 31, 2018.

[83] Chen Gao, Xiang Wang, Xiangnan He, and Yong Li. Graph neural networks for recommender system. In *ICDM*, pp. 1623–1625, 2022.

[84] Johannes Gasteiger, Janek Groß, and Stephan Günnemann. Directional message passing for molecular graphs. In *ICLR*, 2020.

[85] Songwei Ge, Shlok Mishra, Simon Kornblith, Chun-Liang Li, and David Jacobs. Hyperbolic contrastive learning for visual representations beyond objects. In *CVPR*, pp. 6840–6849, 2023.

[86] François G Germain, Gordon Wichern, and Jonathan Le Roux. Hyperbolic unsupervised anomalous sound detection. In *WASPAA*, pp. 1–5. IEEE, 2023.

[87] Mina Ghadimi Atigh, Martin Keller-Ressel, and Pascal Mettes. Hyperbolic Busemann learning with ideal prototypes. In *NeurIPS*, volume 34, 2021.

[88] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *ICML*, pp. 1263–1272. PMLR, 2017.

[89] ZeMing Gong, Chuanqi Tang, Xiaoliang Huo, Nicholas Pellegrino, Austin Wang, Graham W Taylor, Angel X Chang, Scott C Lowe, and Joakim Bruslund Haurum. Hyperbolic multimodal representation learning for biological taxonomies. In *NeurIPS Imageomics Workshop*, 2025.

[90] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *NeurIPS*, volume 27, 2014.

[91] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv:2407.21783*, 2024.

[92] Mikhael Gromov. Hyperbolic groups. In *Essays in group theory*, pp. 75–263. Springer, 1987.

[93] Karish Grover, Haiyang Yu, Xiang Song, Qi Zhu, Han Xie, Vassilis N. Ioannidis, and Christos Faloutsos. Spectro-Riemannian graph neural networks. In *ICLR*, 2025.

[94] Albert Gu, Frederic Sala, Beliz Gunel, and Christopher Ré. Learning mixed-curvature representations in product spaces. In *ICLR*, 2019.

[95] Xiang Gu, Jian Sun, and Zongben Xu. Spherical space domain adaptation with robust pseudo-label loss. In *CVPR*, pp. 9101–9110, 2020.

[96] Caglar Gulcehre, Misha Denil, Mateusz Malinowski, Ali Razavi, Razvan Pascanu, Karl Moritz Hermann, Peter Battaglia, Victor Bapst, David Raposo, Adam Santoro, and Nando de Freitas. Hyperbolic attention networks. In *ICLR*, 2019.

[97] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025.

[98] Naicheng Guo, Xiaolei Liu, Shaoshuai Li, Qiongxu Ma, Yunan Zhao, Bing Han, Lin Zheng, Kaixin Gao, and Xiaobo Guo. HCGR: Hyperbolic contrastive graph representation learning for session-based recommendation. *arXiv preprint arXiv:2107.05366*, 2021.

[99] Naicheng Guo, Xiaolei Liu, Shaoshuai Li, Mingming Ha, Qiongxu Ma, Binfeng Wang, Yunan Zhao, Linxun Chen, and Xiaobo Guo. Hyperbolic contrastive graph representation learning for session-based recommendation. *IEEE TKDE*, 2023.

[100] Qingyu Guo, Fuzhen Zhuang, Chuan Qin, Hengshu Zhu, Xing Xie, Hui Xiong, and Qing He. A survey on knowledge graph-based recommender systems. *IEEE TKDE*, 34(8):3549–3568, 2020.

[101] Yunhui Guo, Xudong Wang, Yubei Chen, and Stella X Yu. Clipped hyperbolic classifiers are super-hyperbolic classifiers. In *CVPR*, pp. 11–20, 2022.

[102] Zihao Guo, Qingyun Sun, Haonan Yuan, Xingcheng Fu, Min Zhou, Yisen Gao, and Jianxin Li. GraphMoRE: Mitigating topological heterogeneity via mixture of Riemannian experts. In *AAAI*, volume 39, pp. 11754–11762, 2025.

[103] Aric Hagberg, Pieter J Swart, and Daniel A Schult. Exploring network structure, dynamics, and function using NetworkX. Technical report, Los Alamos National Laboratory (LANL), Los Alamos, NM (United States), 2008.

[104] Zhen Han, Peng Chen, Yunpu Ma, and Volker Tresp. DyERNIE: Dynamic evolution of Riemannian manifold embeddings for temporal knowledge graph completion. In *EMNLP*, pp. 7301–7316, 2020.

[105] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pp. 770–778, 2016.

[106] Neil He, Rishabh Anand, Hiren Madhu, Ali Maatouk, Smita Krishnaswamy, Leandros Tassiulas, Menglin Yang, and Rex Ying. HELM: Hyperbolic large language models via mixture-of-curvature experts. In *NeurIPS*, 2025.

[107] Neil He, Jiahong Liu, Buze Zhang, Ngoc Bui, Ali Maatouk, Menglin Yang, Irwin King, Melanie Weber, and Rex Ying. Position: Beyond Euclidean–foundation models should embrace non-Euclidean geometries. In *LoG*, 2025.

[108] Neil He, Hiren Madhu, Ngoc Bui, Menglin Yang, and Rex Ying. Hyperbolic deep learning for foundation models: A survey. In *KDD*, 2025.

[109] Neil He, Menglin Yang, and Rex Ying. HyperCore: The core framework for building hyperbolic foundation models with comprehensive modules. *arXiv preprint arXiv:2504.08912*, 2025.

[110] Neil He, Menglin Yang, and Rex Ying. Lorentzian residual neural networks. In *KDD*, 2025.

[111] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, volume 33, pp. 6840–6851, 2020.

[112] Jie Hong, Zeeshan Hayder, Junlin Han, Pengfei Fang, Mehrtash Harandi, and Lars Petersson. Hyperbolic audio-visual zero-shot learning. In *ICCV*, pp. 7873–7883, 2023.

[113] Joy Hsu, Jeffrey Gu, Gong Wu, Wah Chiu, and Serena Yeung. Capturing implicit hierarchical structure in 3D biomedical images with self-supervised hyperbolic representations. In *NeurIPS*, volume 34, pp. 5112–5123, 2021.

[114] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

[115] Yunzhe Hu, Difan Zou, and Dong Xu. Hyper-SET: Designing transformers via hyperspherical energy minimization. In *ICLR*, 2026.

[116] Xiao Huang, Jingyuan Zhang, Dingcheng Li, and Ping Li. Knowledge graph embedding based question answering. In *ICDM*, pp. 105–113, 2019.

[117] HyperLib Contributors. HyperLib: Deep learning in the hyperbolic space. Software, version 0.0.6, 2022.

[118] Sarah Ibrahimi, Mina Ghadimi Atigh, Nanne Van Noord, Pascal Mettes, and Marcel Worring. Intriguing properties of hyperbolic embeddings in vision-language models. *TMLR*, 2024.

[119] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, pp. 448–456. pmlr, 2015.

[120] Roshni Iyer, Yewen Wang, Wei Wang, and Yizhou Sun. Non-Euclidean mixture model for social network embedding. In *NeurIPS*, volume 37, pp. 111464–111488, 2024.

[121] Roshni G Iyer, Yunsheng Bai, Wei Wang, and Yizhou Sun. Dual-geometric space embedding model for two-view knowledge graphs. In *KDD*, pp. 676–686, 2022.

[122] Eugene Yu Ji. A cognitively driven weighted-entropy model for embedding semantic categories in hyperbolic geometry. *arXiv preprint arXiv:2112.06876*, 2021.

[123] Alexander Joseph, Nathan Francis, and Meijke Balay. Brain-inspired AI with hyperbolic geometry. *arXiv preprint arXiv:2409.12990*, 2024.

[124] Jürgen Jost and Shiping Liu. Ollivier's Ricci curvature, local clustering and curvature-dimension inequalities on graphs. *Discrete Comput. Geom.*, 51(2):300–322, 2014.

[125] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. Bag of tricks for efficient text classification. In *EACL*, pp. 427–431, 2017.

[126] Peter Kairouz et al. Advances and open problems in federated learning. *Found. Trends Mach. Learn.*, 14 (1–2):1–210, 2021.

[127] Tejaswi Kasarla, Max van Spengler, and Pascal Mettes. Balanced hyperbolic embeddings are natural out-of-distribution detectors. *arXiv preprint arXiv:2506.10146*, 2025.

[128] Raiyan R Khan, Philippe Chlenski, and Itsik Pe'er. Hyperbolic genome embeddings. In *ICLR*, 2025.

[129] Valentin Khrulkov, Leyla Mirvakhabova, Evgeniya Ustinova, Ivan Oseledets, and Victor Lempitsky. Hyperbolic image embeddings. In *CVPR*, pp. 6418–6428, 2020.

[130] Sungyeon Kim, Boseung Jeong, and Suha Kwak. HIER: Metric learning beyond class labels via hierarchical regularization. In *CVPR*, pp. 19903–19912, 2023.

[131] Wonjae Kim, Sanghyuk Chun, Taekyung Kim, Dongyoon Han, and Sangdoo Yun. HYPE: Hyperbolic entailment filtering for underspecified images and texts. In *ECCV*, pp. 247–265. Springer, 2024.

[132] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *ICLR*, 2014.

[133] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

[134] B Ravi Kiran, Ibrahim Sobh, Victor Talpaert, Patrick Mannion, Ahmad A Al Sallab, Senthil Yogamani, and Patrick Pérez. Deep reinforcement learning for autonomous driving: A survey. *IEEE TITS*, 23(6): 4909–4926, 2021.

[135] Anna Klimovskaia, David Lopez-Paz, Léon Bottou, and Maximilian Nickel. Poincaré maps for analyzing complex hierarchies in single-cell data. *Nat. Commun.*, 11(1):1–9, 2020.

[136] Reinmar J Kobler, Jun-ichiro Hirayama, and Motoaki Kawanabe. Controlling the Fréchet variance improves batch normalization on the symmetric positive definite manifold. In *ICASSP*, pp. 3863–3867. IEEE, 2022.

[137] Max Kochurov, Rasul Karimov, and Serge Kozlukov. Geoopt: Riemannian optimization in PyTorch, 2020.

[138] Dimitrios Konstantinidis, Ilias Papastratis, Kosmas Dimitropoulos, and Petros Daras. Multi-manifold attention for vision transformers. *IEEE Access*, 11:123433–123444, 2023.

[139] Dmitri Krioukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat, and Marián Boguná. Hyperbolic geometry of complex networks. *Phys. Rev. E*, 82(3):036106, 2010.

[140] Brian Kulis. Metric learning: A survey. *Found. Trends Mach. Learn.*, 5(4):287–364, 2013.

[141] Xin Lai, Yukang Chen, Fanbin Lu, Jianhui Liu, and Jiaya Jia. Spherical transformer for LiDAR-based 3D recognition. In *CVPR*, pp. 17545–17555, 2023.

[142] Marc Law. Ultrahyperbolic neural networks. In *NeurIPS*, volume 34, 2021.

[143] Marc Law and Jos Stam. Ultrahyperbolic representation learning. In *NeurIPS*, volume 33, pp. 1668–1678, 2020.

[144] Marc Law, Renjie Liao, Jake Snell, and Richard Zemel. Lorentzian distance learning for hyperbolic representations. In *ICML*, pp. 3672–3681, 2019.

[145] Diego Lazcano, Nicolás Fredes Franco, and Werner Creixell. HGAN: Hyperbolic generative adversarial network. *IEEE Access*, 9:96309–96320, 2021.

[146] Binh Minh Le and Simon S Woo. SEE: Spherical embedding expansion for improving deep metric learning. In *PAKDD*, pp. 131–143. Springer, 2024.

[147] Hojoon Lee, Youngdo Lee, Takuma Seno, Donghu Kim, Peter Stone, and Jaegul Choo. Hyperspherical normalization for scalable deep reinforcement learning. In *ICML*, volume 267, pp. 33352–33403, 2025.

[148] John M. Lee. *Smooth Manifolds*, pp. 1–29. Springer New York, New York, NY, 2003. ISBN 978-0-387-21752-9.

[149] John M Lee. *Riemannian manifolds: an introduction to curvature*, volume 176. Springer Science & Business Media, 2006.

[150] Huan Lei, Naveed Akhtar, and Ajmal Mian. Spherical kernel for efficient graph convolution on 3D point clouds. *IEEE TPAMI*, 43(10):3664–3680, 2020.

[151] Anchen Li, Bo Yang, Farookh Khadeer Hussain, and Huan Huo. HSR: Hyperbolic social recommender. *Inf. Sci.*, 585:275–288, 2022.

[152] Jun Li, Jinpeng Wang, Chaolei Tan, Niu Lian, Long Chen, Yaowei Wang, Min Zhang, Shu-Tao Xia, and Bin Chen. HLFormer: Enhancing partially relevant video retrieval with hyperbolic learning. In *ICCV*, 2025.

[153] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, pp. 19730–19742. PMLR, 2023.

[154] Lingxiao Li, Yi Zhang, and Shuhui Wang. The Euclidean space is evil: Hyperbolic attribute editing for few-shot image generation. In *ICCV*, pp. 22714–22724, 2023.

[155] Pengxiang Li, Yuwei Wu, Zhi Gao, Xiaomeng Fan, Wei Wu, Zhipeng Lu, Yunde Jia, and Mehrtash Harandi. Geometry-aware distance measure for diverse hierarchical structures in hyperbolic spaces. *arXiv preprint arXiv:2506.18533*, 2025.

[156] Rui Li, Jianan Zhao, Chaozhuo Li, Di He, Yiqi Wang, Yuming Liu, Hao Sun, Senzhang Wang, Weiwei Deng, Yanming Shen, Xing Xie, and Qi Zhang. HousE: Knowledge graph embedding with householder parameterization. In *ICML*, pp. 13209–13224. PMLR, 2022.

[157] Shanglin Li, Shiwen Chu, Okan Koç, Yi Ding, Qibin Zhao, Motoaki Kawanabe, and Ziheng Chen. HEEGNet: Hyperbolic embeddings for EEG. In *ICLR*, 2026.

[158] Wenrui Li, Yidan Lu, Yeyu Chai, Rui Zhao, Hengyu Man, and Xiaopeng Fan. Hyperbolic hierarchical alignment reasoning network for text-3D retrieval. In *AAAI*, 2026.

[159] Xiaohan Li, Yuqing Liu, Zheng Liu, and Philip S Yu. Time-aware hyperbolic graph attention network for session-based recommendation. In *IEEE Big Data*, pp. 626–635. IEEE, 2022.

[160] Xuewei Li, Tao Wu, Zhongang Qi, Gaoang Wang, Ying Shan, and Xi Li. SGAT4PASS: Spherical geometry-aware transformer for PAnoramic semantic segmentation. In *IJCAI*, pp. 1125–1133, 2023.

[161] Yancong Li, Xiaoming Zhang, Ying Cui, and Shuai Ma. Hyperbolic graph neural network for temporal knowledge graph completion. In *LREC-COLING*, pp. 8474–8486, 2024.

[162] Yicong Li, Hongxu Chen, Xiangguo Sun, Zhenchao Sun, Lin Li, Lizhen Cui, Philip S Yu, and Guandong Xu. Hyperbolic hypergraphs for sequential recommendation. In *CIKM*, pp. 988–997, 2021.

[163] Yue Li, Haoxuan Qu, Mengyuan Liu, Jun Liu, and Yujun Cai. HyLiFormer: Hyperbolic linear attention for skeleton-based human action recognition. *arXiv preprint arXiv:2502.05869*, 2025.

[164] Qiuyu Liang, Weihua Wang, Feilong Bao, and Guanglai Gao. Fully hyperbolic rotation for knowledge graph embedding. In *ECAI*, pp. 1615–1622, 2024.

[165] Qiuyu Liang, Weihua Wang, Jie Yu, and Feilong Bao. Hierarchy-aware quaternion embedding for knowledge graph completion. In *IJCNN*, pp. 1–8. IEEE, 2024.

[166] Qiuyu Liang, Weihua Wang, Cunda Wang, Feilong Bao, and Jie Yu. Hyperbolic multimodal knowledge graph embedding. In *ICASSP*, pp. 1–5. IEEE, 2025.

[167] Shuai Liao, Efstratios Gavves, and Cees GM Snoek. Spherical regression: Learning viewpoints, surface normals and 3D rotations on n-spheres. In *CVPR*, pp. 9759–9767, 2019.

[168] Xinting Liao, Weiming Liu, Chaochao Chen, Pengyang Zhou, Huabin Zhu, Yanchao Tan, Jun Wang, and Yue Qi. HyperFed: hyperbolic prototypes exploration with consistent aggregation for non-IID data in federated learning. In *IJCAI*, pp. 3957–3965, 2023.

[169] Fangfei Lin, Bing Bai, Kun Bai, Yazhou Ren, Peng Zhao, and Zenglin Xu. Contrastive multi-view hyperbolic hierarchical clustering. In *IJCAI*, 2022.

[170] Mengqi Lin, Qing Liao, Yan Jia, and Ye Wang. Survey on knowledge graph embedding based on hyperbolic geometry. In *DSC*, pp. 152–158, 2021.

[171] Nathan Linial, Eran London, and Yuri Rabinovich. The geometry of graphs and some of its algorithmic applications. *Combinatorica*, 15(2):215–245, 1995.

[172] Hai Liu, Chao Zheng, Duantengchuan Li, Xiaoxuan Shen, Ke Lin, Jiazhang Wang, Zhen Zhang, Zhaoli Zhang, and Neal N Xiong. EDMF: Efficient deep matrix factorization with review feature learning for industrial recommender system. *IEEE TII*, 18(7):4361–4371, 2021.

[173] Jiahong Liu, Menglin Yang, Min Zhou, Shanshan Feng, and Philippe Fournier-Viger. Enhancing hyperbolic graph embeddings via contrastive learning. In *NeurIPS SSL Workshop*, 2022.

[174] Jiahong Liu, Ram Samarth B B, Xinyu Fu, Menglin Yang, Weixi Zhang, Rex Ying, and Irwin King. FlatLand: Personalized graph federated learning via tailored Lorentz space. In *ICML*, volume 306, 2026.

[175] Qi Liu, Maximilian Nickel, and Douwe Kiela. Hyperbolic graph neural networks. In *NeurIPS*, volume 32, 2019.

[176] Tongcun Liu, Xukai Bao, Jiaxin Zhang, Kai Fang, and Hailin Feng. Enhancing session-based recommendation with multi-interest hyperbolic representation networks. *IEEE TNNLS*, 2024.

[177] Weiyang Liu, Yan-Ming Zhang, Xingguo Li, Zhiding Yu, Bo Dai, Tuo Zhao, and Le Song. Deep hyperspherical learning. In *NeurIPS*, volume 30, 2017.

[178] Yi Liu, Limei Wang, Meng Liu, Yuchao Lin, Xuan Zhang, Bora Oztekin, and Shuiwang Ji. Spherical message passing for 3D molecular graphs. In *ICLR*, 2022.

[179] Alice Longhena, Martin Guillemaud, Fabrizio De Vico Fallani, Raffaella Migliaccio, and Mario Chavez. Hyperbolic embedding of brain networks detects regions disrupted by neurodegeneration in Alzheimer's disease. *Phys. Rev. E*, 111(4):044402, 2025.

[180] Ilya Loshchilov, Cheng-Ping Hsieh, Simeng Sun, and Boris Ginsburg. nGPT: Normalized transformer with representation learning on the hypersphere. In *ICLR*, 2025.

[181] Aaron Lou, Isay Katsman, Qingxuan Jiang, Serge Belongie, Ser-Nam Lim, and Christopher De Sa. Differentiating through the Fréchet mean. In *ICML*, pp. 6393–6403. PMLR, 2020.

[182] Qiyao Ma, Menglin Yang, Mingxuan Ju, Tong Zhao, Neil Shah, and Rex Ying. HARec: Hyperbolic graph-LLM alignment for exploration and exploitation in recommender systems. *arXiv preprint arXiv:2411.13865*, 2024.

[183] Hiren Madhu, Ngoc Bui, Ali Maatouk, Leandros Tassiulas, Smita Krishnaswamy, Menglin Yang, Sukanta Ganguly, Kiran Srinivasan, and Rex Ying. HypRAG: Hyperbolic dense retrieval for retrieval augmented generation. *arXiv preprint arXiv:2602.07739*, 2026.

[184] Nikita Malik, Rahul Gupta, and Sandeep Kumar. HyperDefender: A robust framework for hyperbolic GNNs. In *AAAI*, volume 39, pp. 19396–19404, 2025.

[185] Igor Maljkovic, Maria Rosaria Briglia, Iacopo Masi, Antonio Emanuele Cinà, and Fabio Roli. Harnessing hyperbolic geometry for harmful prompt detection and sanitization. In *ICLR*, 2026.

[186] Paolo Mandica, Luca Franco, Konstantinos Kallidromitis, Suzanne Petryk, and Fabio Galasso. Hyperbolic learning with multimodal large language models. In *ECCV*, pp. 382–398. Springer, 2024.

[187] Emile Mathieu, Charline Le Lan, Chris J Maddison, Ryota Tomioka, and Yee Whye Teh. Continuous hierarchical representations with Poincaré variational auto-encoders. In *NeurIPS*, volume 32, 2019.

[188] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, pp. 1273–1282. PMLR, 2017.

[189] Yu Meng, Jiaxin Huang, Guangyuan Wang, Chao Zhang, Honglei Zhuang, Lance Kaplan, and Jiawei Han. Spherical text embedding. In *NeurIPS*, volume 32, 2019.

[190] Pascal Mettes, Mina Ghadimi Atigh, Martin Keller-Ressel, Jeffrey Gu, and Serena Yeung. Hyperbolic deep learning in computer vision: A survey. *IJCV*, 132(9):3484–3508, 2024.

[191] Marko Valentin Micic and Hugo Chu. Hyperbolic deep learning for Chinese natural language understanding. *arXiv preprint arXiv:1812.10408*, 2018.

[192] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *ICLR*, 2013.

[193] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *NeurIPS*, volume 26, 2013.

[194] Nina Miolane, Nicolas Guigui, Alice Le Brigant, Johan Mathe, Benjamin Hou, Yann Thanwerdas, Stefan Heyder, Olivier Peltre, Niklas Koep, Hadi Zaatiti, Hatem Hajri, Yann Cabanes, Thomas Gerald, Paul Chauchat, Christian Shewmake, Daniel Brooks, Bernhard Kainz, Claire Donnat, Susan Holmes, and Xavier Pennec. Geomstats: A Python package for Riemannian geometry in machine learning. *JMLR*, 21 (223):1–9, 2020.

[195] Gal Mishne, Zhengchao Wan, Yusu Wang, and Sheng Yang. The numerical stability of hyperbolic representation learning. In *ICML*, pp. 24925–24949. PMLR, 2023.

[196] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[197] Antonio Montanaro, Diego Valsesia, and Enrico Magli. Rethinking the compositionality of point clouds through regularization in the hyperbolic space. In *NeurIPS*, volume 35, pp. 33741–33753, 2022.

[198] Sebastien Montella, Lina M Rojas Barahona, and Johannes Heinecke. Hyperbolic temporal knowledge graph embeddings with relational and time curvatures. In *Findings of ACL*, pp. 3296–3308, 2021.

[199] Yoshihiro Nagano, Shoichiro Yamaguchi, Yasuhiro Fujita, and Masanori Koyama. A wrapped normal distribution on hyperbolic space for gradient-based learning. In *ICML*, pp. 4693–4702. PMLR, 2019.

[200] Laurent Najman and Pascal Romon. *Modern approaches to discrete curvature*, volume 2184. Springer, 2017.

[201] Futa Nakashima, Tomohiko Nakamura, Norihiro Takamune, Satoru Fukayama, and Hiroshi Saruwatari. Hyperbolic timbre embedding for musical instrument sound synthesis based on variational autoencoders. In *APSIPA ASC*, pp. 735–742. IEEE, 2022.

[202] Viet Dung Nguyen, Minh Hoang Nguyen, Luc Nguyen, Rachel S. Y. Teo, Tan Minh Nguyen, and Linh Duy Tran. CAMEx: Curvature-aware merging of experts. In *ICLR*, 2025.

[203] Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *NeurIPS*, volume 30, 2017.

[204] Maximilian Nickel, Volker Tresp, and Hans-Peter Kriegel. A three-way model for collective learning on multi-relational data. In *ICML*, pp. 809–816, 2011.

[205] Maximillian Nickel and Douwe Kiela. Learning continuous hierarchies in the Lorentz model of hyperbolic geometry. In *ICML*, pp. 3779–3788. PMLR, 2018.

[206] Yann Ollivier. Ricci curvature of Markov chains on metric spaces. *J. Funct. Anal.*, 256(3):810–864, 2009.

[207] Yann Ollivier. A visual introduction to Riemannian curvatures and some discrete generalizations. In *Analysis and Geometry of Metric Measure Spaces*, CRM Proceedings and Lecture Notes, pp. 197–220. American Mathematical Society, 2013. doi: 10.1090/crmp/056/08.

[208] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[209] Avik Pal, Max van Spengler, Guido D'Amely di Melendugno, Alessandro Flaborea, Fabio Galasso, and Pascal Mettes. Compositional entailment learning for hyperbolic vision-language models. In *ICLR*, 2025.

[210] Zhe Pan and Peng Wang. Hyperbolic hierarchy-aware knowledge graph embedding for link prediction. In *Findings of EMNLP*, pp. 2941–2948, 2021.

[211] Fragkiskos Papadopoulos, Maksim Kitsak, M. Ángeles Serrano, Marián Boguñá, and Dmitri Krioukov. Popularity versus similarity in growing networks. *Nature*, 489(7417):537–540, 2012.

[212] Marios Papamichalis, Kathryn Turnbull, Simon Lunagomez, and Edoardo Airoldi. Latent space network modelling with hyperbolic and spherical geometries. *arXiv preprint arXiv:2109.03343*, 2021.

[213] Jongmin Park, Seunghoon Han, Soohwan Jeong, and Sungsu Lim. Hyperbolic heterogeneous graph attention networks. In *WebConf Companion*, pp. 561–564, 2024.

[214] Sung Woo Park and Junseok Kwon. Sphere generative adversarial network based on geometric moment matching. In *CVPR*, pp. 4292–4301, 2019.

[215] Wei Peng, Tuomas Varanka, Abdelrahman Mostafa, Henglin Shi, and Guoying Zhao. Hyperbolic deep neural networks: A survey. *IEEE TPAMI*, 44(12):10023–10044, 2022.

[216] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *ICRA*, pp. 3803–3810. IEEE, 2018.

[217] Zelin Peng, Zhengqin Xu, Qingyang Liu, Xiaokang Yang, and Wei Shen. HyperET: Efficient training in hyperbolic space for multi-modal large language models. In *NeurIPS*, 2025.

[218] Zelin Peng, Zhengqin Xu, Zhilin Zeng, Changsong Wen, Yu Huang, Menglin Yang, Feilong Tang, and Wei Shen. Understanding fine-tuning CLIP for open-vocabulary semantic segmentation in hyperbolic space. In *CVPR*, pp. 4562–4572, 2025.

[219] Jeffrey Pennington, Richard Socher, and Christopher D Manning. GloVe: Global vectors for word representation. In *EMNLP*, pp. 1532–1543, 2014.

[220] Darius Petermann and Minje Kim. Hyperbolic distance-based speech separation. In *ICASSP*, pp. 1191–1195. IEEE, 2024.

[221] Darius Petermann, Gordon Wichern, Aswin Subramanian, and Jonathan Le Roux. Hyperbolic audio source separation. In *ICASSP*, pp. 1–5. IEEE, 2023.

[222] Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Found. Trends Mach. Learn.*, 11(5–6): 355–607, 2019.

[223] Tobia Poppi, Tejaswi Kasarla, Pascal Mettes, Lorenzo Baraldi, and Rita Cucchiara. Hyperbolic safety-aware vision-language models. In *CVPR*, pp. 4222–4232, 2025.

[224] Saurav Prakash, Jin Sima, Chao Pan, Eli Chien, and Olgica Milenkovic. Federated classification in hyperbolic spaces via secure aggregation of convex hulls. *TMLR*, 2024.

[225] Zhida Qin, Wentao Cheng, Wenxing Ding, and Gangyi Ding. Hyperbolic graph contrastive learning for collaborative filtering. *IEEE TKDE*, 2024.

[226] Zexuan Qiu, Jiahong Liu, Yankai Chen, and Irwin King. HiHPQ: Hierarchical hyperbolic product quantization for unsupervised image retrieval. In *AAAI*, volume 38, pp. 4614–4622, 2024.

[227] Eric Qu and Dongmian Zou. Lorentz direct concatenation for stable training in hyperbolic neural networks. In *NeurReps Workshop*, 2022.

[228] Eric Qu and Dongmian Zou. Autoencoding hyperbolic representation for adversarial generation. *TMLR*, 2024.

[229] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, volume 139, pp. 8748–8763, 2021.

[230] Sameera Ramasinghe, Violetta Shevchenko, Gil Avraham, and Ajanthan Thalaiyasingam. Accept the modality gap: An exploration in the hyperbolic space. In *CVPR*, pp. 27263–27272, 2024.

[231] John G. Ratcliffe. *Foundations of Hyperbolic Manifolds*, volume 149 of *Graduate Texts in Mathematics*. Springer, 1994.

[232] Jianxun Ren, Ning An, Youjia Zhang, Danyang Wang, Zhenyu Sun, Cong Lin, Weigang Cui, Weiwei Wang, Ying Zhou, Wei Zhang, Qingyu Hu, Ping Zhang, Dan Hu, Danhong Wang, and Hesheng Liu. SUGAR: Spherical ultrafast graph attention framework for cortical surface registration. *Med. Image Anal.*, 94:103122, 2024.

[233] Giulia Rizzoli, Matteo Caligiuri, Donald Shenaj, Francesco Barbato, and Pietro Zanuttigh. When cars meet drones: Hyperbolic federated learning for source-free domain adaptation in adverse weather. In *WACV*, pp. 1587–1596, 2025.

[234] Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.

[235] Frederic Sala, Christopher De Sa, Albert Gu, and Christopher Ré. Representation tradeoffs for hyperbolic embeddings. In *ICML*, volume 80, pp. 4460–4469, 2018.

[236] Ahmad EL Sallab, Mohammed Abdou, Etienne Perot, and Senthil Yogamani. Deep reinforcement learning framework for autonomous driving. In *IS&T Electronic Imaging*, 2017.

[237] Areejit Samal, RP Sreejith, Jiao Gu, Shiping Liu, Emil Saucan, and Jürgen Jost. Comparative analysis of two discretizations of Ricci curvature for complex networks. *Sci. Rep.*, 8(1):8650, 2018.

[238] Rik Sarkar. Low distortion delaunay embedding of trees in hyperbolic plane. In *GD*, pp. 355–366. Springer, 2011.

[239] Ramit Sawhney, Shivam Agarwal, Megh Thakkar, Arnav Wadhwa, and Rajiv Ratn Shah. Hyperbolic online time stream modeling. In *SIGIR*, pp. 1682–1686, 2021.

[240] Chandni Saxena, Mudit Chaudhary, and Helen Meng. Cross-lingual word embeddings in hyperbolic space. *arXiv preprint arXiv:2205.01907*, 2022.

[241] Bin Shang, Yinliang Zhao, Jun Liu, and Di Wang. Mixed geometry message and trainable convolutional attention network for knowledge graph completion. In *AAAI*, volume 38, pp. 8966–8974, 2024.

[242] Zhijie Shen, Chunyu Lin, Kang Liao, Lang Nie, Zishuo Zheng, and Yao Zhao. PanoFormer: panorama transformer for indoor 360 depth estimation. In *ECCV*, pp. 195–211. Springer, 2022.

[243] Ryohei Shimizu, Yusuke Mukuta, and Tatsuya Harada. Hyperbolic neural networks++. In *ICLR*, 2021.

[244] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[245] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[246] Anish K Simhal, Kimberly LH Carpenter, Saad Nadeem, Joanne Kurtzberg, Allen Song, Allen Tannenbaum, Guillermo Sapiro, and Geraldine Dawson. Measuring robustness of brain networks in autism spectrum disorder with Ricci curvature. *Sci. Rep.*, 10(1):1–8, 2020.

[247] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[248] Richard Sinkhorn and Paul Knopp. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific J. Math.*, 21(2):343–348, 1967.

[249] Andrii Skliar and Maurice Weiler. Hyperbolic convolutional neural networks. *arXiv preprint arXiv:2308.15639*, 2023.

[250] Ondrej Skopek, Octavian-Eugen Ganea, and Gary Bécigneul. Mixed-curvature variational autoencoders. In *ICLR*, 2020.

[251] Martin Skrodzki, Hunter van Geffen, Nicolas F Chaves-de Plaza, Thomas Höllt, Elmar Eisemann, and Klaus Hildebrandt. Accelerating hyperbolic t-SNE. *IEEE TVCG*, 30(7):4403–4415, 2024.

[252] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.

[253] RP Sreejith, Karthikeyan Mohanraj, Jürgen Jost, Emil Saucan, and Areejit Samal. Forman curvature for complex networks. *J. Stat. Mech.*, 2016(6):063206, 2016.

[254] Jonathan M. Stokes, Kevin Yang, Kyle Swanson, Wengong Jin, Andres Cubillos-Ruiz, Nina M. Donghia, Craig R. MacNair, Shawn French, Lindsey A. Carfrae, Zohar Bloom-Ackermann, Victoria M. Tran, Anush Chiappino-Pepe, Ahmed H. Badran, Ian W. Andrews, Emma J. Chory, George M. Church, Eric D. Brown, Tommi S. Jaakkola, Regina Barzilay, and James J. Collins. A deep learning approach to antibiotic discovery. *Cell*, 180(4):688–702, 2020.

[255] Yu-Chuan Su and Kristen Grauman. Learning spherical convolution for fast features from 360 imagery. In *NeurIPS*, volume 30, 2017.

[256] Jianing Sun, Zhaoyue Cheng, Saba Zuberi, Felipe Pérez, and Maksims Volkovs. HGCF: Hyperbolic graph convolution networks for collaborative filtering. In *The WebConf*, pp. 593–601, 2021.

[257] Li Sun, Zhongbao Zhang, Junda Ye, Hao Peng, Jiawei Zhang, Sen Su, and Philip S Yu. A self-supervised mixed-curvature graph neural network. In *AAAI*, volume 36, pp. 4146–4155, 2022.

[258] Rui Sun, Xuezhi Cao, Yan Zhao, Junchen Wan, Kun Zhou, Fuzheng Zhang, Zhongyuan Wang, and Kai Zheng. Multi-modal knowledge graphs for recommender systems. In *CIKM*, pp. 1405–1414, 2020.

[259] Zequn Sun, Muhao Chen, Wei Hu, Chengming Wang, Jian Dai, and Wei Zhang. Knowledge association with hyperbolic knowledge graph embeddings. In *EMNLP*, pp. 5704–5716, 2020.

[260] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.

[261] Xianhan Tan, Junming Zhu, Jianmin Zhang, Yueming Wang, and Yu Qi. Decoding Chinese phonemes from intracortical brain signals with hyperbolic-space neural representations. *arXiv preprint arXiv:2305.08354*, 2023.

[262] Yi Tay, Luu Anh Tuan, and Siu Cheung Hui. Hyperbolic representation learning for fast and efficient neural question answering. In *WSDM*, pp. 583–591, 2018.

[263] Gemma Team. Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[264] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

[265] Yu Tian, Zachary Lubberts, and Melanie Weber. Curvature-based clustering on graphs. *JMLR*, 26(52): 1–67, 2025.

[266] Alexandru Tifrea, Gary Bécigneul, and Octavian-Eugen Ganea. Poincaré GloVe: Hyperbolic word embeddings. In *ICLR*, 2019.

[267] Lucas Vinh Tran, Yi Tay, Shuai Zhang, Gao Cong, and Xiaoli Li. HyperML: A boosting metric learning approach in hyperbolic space for recommender systems. In *WSDM*, pp. 609–617, 2020.

[268] Alenka Trpin and BM Boshkoska. Face recognition with a hyperbolic metric classification model. In *MIPRO*, pp. 317–320. IEEE, 2022.

[269] Abraham A Ungar. Hyperbolic trigonometry and its application in the Poincaré ball model of hyperbolic geometry. *Comput. Math. Appl.*, 41(1-2):135–147, 2001.

[270] Abraham Albert Ungar. *A Gyrovector Space Approach to Hyperbolic Geometry*. Springer International Publishing, 2009.

[271] Tycho FA van der Ouderaa, Maximilian L Croci, Agrin Hilmkil, and James Hensman. Pyramid vector quantization for LLMs. *arXiv preprint arXiv:2410.16926*, 2024.

[272] Max Van Spengler, Erwin Berkhout, and Pascal Mettes. Poincaré ResNet. In *ICCV*, pp. 5419–5428, 2023.

[273] Max van Spengler, Philipp Wirth, and Pascal Mettes. HypLL: The hyperbolic learning library. In *ACM MM*, pp. 9676–9679, 2023.

[274] Max van Spengler, Artem Moskalev, Tommaso Mansi, Mangal Prakash, and Rui Liao. HyperHELM: Hyperbolic hierarchy encoding for mRNA language modeling. *arXiv preprint arXiv:2509.24655*, 2025.

[275] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, pp. 5998–6008, 2017.

[276] Ivan Vendrov, Ryan Kiros, Sanja Fidler, and Raquel Urtasun. Order-embeddings of images and language. In *ICLR*, 2016.

[277] Kevin Verbeek and Subhash Suri. Metric embedding, hyperbolic space, and social networks. In *SoCG*, pp. 501–510, 2014.

[278] Li Wan, Matthew Zeiler, Sixin Zhang, Yann Le Cun, and Rob Fergus. Regularization of neural networks using dropconnect. In *ICML*, pp. 1058–1066. PMLR, 2013.

[279] Feng Wang, Xiang Xiang, Jian Cheng, and Alan Loddon Yuille. NormFace: L2 hypersphere embedding for face verification. In *ACM MM*, pp. 1041–1049, 2017.

[280] Jiapu Wang, Zheng Cui, Boyue Wang, Shirui Pan, Junbin Gao, Baocai Yin, and Wen Gao. IME: Integrating multi-curvature shared and specific embedding for temporal knowledge graph completion. In *The WebConf*, pp. 1954–1962, 2024.

[281] Jiapu Wang, Boyue Wang, Junbin Gao, Shirui Pan, Tengfei Liu, Baocai Yin, and Wen Gao. MADE: Multicurvature adaptive embedding for temporal knowledge graph completion. *IEEE TCYB*, 2024.

[282] Jihu Wang, Yuliang Shi, Han Yu, Xinjun Wang, Zhongmin Yan, and Fanyu Kong. Mixed-curvature manifolds interaction learning for knowledge graph-aware recommendation. In *SIGIR*, pp. 372–382, 2023.

[283] Prince Zizhuang Wang and William Yang Wang. Riemannian normalizing flow on variational Wasserstein autoencoder for text modeling. In *NAACL-HLT*, pp. 284–294, 2019.

[284] Shen Wang, Xiaokai Wei, Cicero Nogueira Nogueira dos Santos, Zhiguo Wang, Ramesh Nallapati, Andrew Arnold, Bing Xiang, Philip S Yu, and Isabel F Cruz. Mixed-curvature multi-relational graph neural network for knowledge graph completion. In *The WebConf*, pp. 1761–1771, 2021.

[285] Sijie Wang, Qiyu Kang, Rui She, Wei Wang, Kai Zhao, Yang Song, and Wee Peng Tay. HypLiLoc: Towards effective LiDAR pose regression with hyperbolic fusion. In *CVPR*, pp. 5176–5185, 2023.

[286] Tongzhou Wang and Phillip Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *ICML*, volume 119, pp. 9929–9939, 2020.

[287] Yujie Wang, Shuo Zhang, Junda Ye, Hao Peng, and Li Sun. A mixed-curvature graph diffusion model. In *CIKM*, pp. 2482–2492, 2024.

[288] Melanie Weber, Emil Saucan, and Jürgen Jost. Characterizing complex networks with Forman-Ricci curvature and associated geometric flows. *J. Complex Netw.*, 5(4):527–550, 2017.

[289] Simon Welz, Lucie Flek, and Akbar Karimi. Multi-hop reasoning for question answering with hyperbolic representations. In *Findings of ACL*, pp. 17667–17679, 2025.

[290] Lingfeng Wen, Xuan Tang, Mingjie Ouyang, Xiangxiang Shen, Jian Yang, Daxin Zhu, Mingsong Chen, and Xian Wei. Hyperbolic graph diffusion model. In *AAAI*, pp. 15823–15831, 2024.

[291] Richard C Wilson, Edwin R Hancock, Elżbieta Pekalska, and Robert PW Duin. Spherical and hyperbolic embeddings of data. *IEEE TPAMI*, 36(11):2255–2269, 2014.

[292] Bo Xiong, Shichao Zhu, Mojtaba Nayyeri, Chengjin Xu, Shirui Pan, Chuan Zhou, and Steffen Staab. Ultrahyperbolic knowledge graph embeddings. In *KDD*, pp. 2130–2139, 2022.

[293] Bo Xiong, Shichao Zhu, Nico Potyka, Shirui Pan, Chuan Zhou, and Steffen Staab. Pseudo-Riemannian graph convolutional networks. In *NeurIPS*, 2022.

[294] Bo Xiong, Mojtaba Nayyeri, Ming Jin, Yunjie He, Michael Cochez, Shirui Pan, and Steffen Staab. Geometric relational embeddings: A survey. *arXiv preprint arXiv:2304.11949*, 2023.

[295] Jiacheng Xu and Greg Durrett. Spherical latent spaces for stable variational autoencoders. In *EMNLP*, pp. 4503–4513, 2018.

[296] Yanyu Xu, Ziheng Zhang, and Shenghua Gao. Spherical DNNs and their applications in 360 images and videos. *IEEE TPAMI*, 44(10):7235–7252, 2021.

[297] Zhirong Xu, Shiyang Wen, Junshan Wang, Guojun Liu, Liang Wang, Zhi Yang, Lei Ding, Yan Zhang, Di Zhang, Jian Xu, and Bo Zheng. AMCAD: adaptive mixed-curvature representation based advertisement retrieval system. In *ICDE*, pp. 3439–3452. IEEE, 2022.

[298] Jiexi Yan, Lei Luo, Cheng Deng, and Heng Huang. Unsupervised hyperbolic metric learning. In *CVPR*, pp. 12465–12474, 2021.

[299] Qingsong Yan, Qiang Wang, Kaiyong Zhao, Jie Chen, Bo Li, Xiaowen Chu, and Fei Deng. SphereFusion: Efficient panorama depth estimation via gated fusion. In *3DV*, 2025.

[300] Meimei Yang, Yongheng Sun, Qianqian Wang, Andrea Bozoki, Maureen Kohi, and Mingxia Liu. Hyperbolic kernel graph neural networks for neurocognitive decline analysis from multimodal brain imaging. *IEEE TPAMI*, 2025.

[301] Menglin Yang, Zhihao Li, Min Zhou, Jiahong Liu, and Irwin King. HICF: Hyperbolic informative collaborative filtering. In *KDD*, pp. 2212–2221, 2022.

[302] Menglin Yang, Min Zhou, Jiahong Liu, Defu Lian, and Irwin King. HRCF: Enhancing collaborative filtering via hyperbolic geometric regularization. In *The WebConf*, pp. 2462–2471, 2022.

[303] Menglin Yang, Min Zhou, Hui Xiong, and Irwin King. Hyperbolic temporal network embedding. *IEEE TKDE*, 35(11):11489–11502, 2022.

[304] Menglin Yang, Min Zhou, Tong Zhang, Jiahong Liu, Zhihao Li, Lujia Pan, Hui Xiong, and Irwin King. Hyperbolic graph neural networks: A review of methods and applications. *arXiv preprint arXiv:2202.13852*, 2022.

[305] Menglin Yang, Min Zhou, Rex Ying, Yankai Chen, and Irwin King. Hyperbolic representation learning: Revisiting and advancing. In *ICML*, volume 202, pp. 39639–39659. PMLR, 2023.

[306] Menglin Yang, Aosong Feng, Bo Xiong, Jiahong Liu, Irwin King, and Rex Ying. Enhancing LLM complex reasoning capability through hyperbolic geometry. In *ICML LLMs and Cognition Workshop*, 2024.

[307] Menglin Yang, Harshit Verma, Delvin Ce Zhang, Jiahong Liu, Irwin King, and Rex Ying. Hypformer: Exploring efficient transformer fully in hyperbolic space. In *KDD*, pp. 3770–3781, 2024.

[308] Menglin Yang, Ram Samarth B B, Aosong Feng, Bo Xiong, Jiahong Liu, Irwin King, and Rex Ying. Hyperbolic fine-tuning for large language models. In *NeurIPS*, 2025.

[309] Xin Yang, Xingrun Li, Heng Chang, Yang Jinze, Xihong Yang, Shengyu Tao, Maiko Shigeno, Ningkang Chang, Junfeng Wang, Dawei Yin, and Erxue Min. HGFormer: Hyperbolic graph transformer for collaborative filtering. In *ICML*, volume 267, pp. 70813–70832, 2025.

[310] Yonghui Yang, Le Wu, Kun Zhang, Richang Hong, Hailin Zhou, Zhiqiang Zhang, Jun Zhou, and Meng Wang. Hyperbolic graph learning for social recommendation. *IEEE TKDE*, 36(12):8488–8501, 2023.

[311] Yujun Yang. Resistance distances and the global cyclicity index of fullerene graphs. *Dig. J. Nanomater. Biostruct.*, 7:593–598, 2012.

[312] Yujun Yang. On a new cyclicity measure of graphs—the global cyclicity index. *Discrete Appl. Math.*, 172: 88–97, 2014.

[313] Zhe Yang, Wenrui Li, and Guanghui Cheng. SHMamba: Structured hyperbolic state space model for audio-visual question answering. *IEEE/ACM TASLP*, 33:3582–3593, 2025.

[314] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *KDD*, pp. 974–983, 2018.

[315] Daiki Yoshikawa and Takashi Matsubara. PHyCLIP: ℓ_1-product of hyperbolic factors unifies hierarchy and compositionality in vision-language representation learning. In *ICLR*, 2026.

[316] Tsuneo Yoshikawa. Manipulability of robotic mechanisms. *IJRR*, 4(2):3–9, 1985.

[317] Ke Yu, Shyam Visweswaran, and Kayhan Batmanghelich. Semi-supervised hierarchical drug embedding in hyperbolic space. *J. Chem. Inf. Model.*, 60(12):5647–5657, 2020.

[318] Ke Yu, Shyam Visweswaran, and Kayhan Batmanghelich. Hyperbolic molecular representation learning for drug repositioning. *arXiv preprint arXiv:2208.06361*, 2022.

[319] Tao Yu and Christopher De Sa. Random laplacian features for learning with hyperbolic space. In *ICLR*, 2023.

[320] Tao Yu and Christopher M De Sa. Numerically accurate hyperbolic embeddings using tiling-based models. In *NeurIPS*, volume 32, 2019.

[321] Tao Yu and Christopher M De Sa. Representing hyperbolic space accurately using multi-component floats. In *NeurIPS*, volume 34, pp. 15570–15581, 2021.

[322] Yonghong Yu, Aoran Zhang, Li Zhang, Rong Gao, Shang Gao, and Hongzhi Yin. Hyperbolic translation-based sequential recommendation. *IEEE TCSS*, 2024.

[323] Meng Yuan, Fuzhen Zhuang, Zhao Zhang, Deqing Wang, and Jin Dong. Knowledge-based multiple adaptive spaces fusion for recommendation. In *RecSys*, pp. 565–575, 2023.

[324] Yun Yue, Fangzhou Lin, Kazunori D Yamada, and Ziming Zhang. Hyperbolic contrastive learning. *arXiv preprint arXiv:2302.01409*, 2023.

[325] Buze Zhang, Jinkai Tao, Zilang Zeng, Neil He, Ali Maatouk, Menglin Yang, and Rex Ying. Parameter-efficient fine-tuning of LLMs with mixture of space experts. *arXiv preprint arXiv:2602.14490*, 2026.

[326] Dingyi Zhang, Yingming Li, and Zhongfei Zhang. Deep metric learning with spherical embedding. In *NeurIPS*, volume 33, pp. 18772–18783, 2020.

[327] Fangzhao Zhang and Mert Pilanci. Riemannian preconditioned LoRA for fine-tuning foundation models. In *ICML*, volume 235, pp. 59641–59669, 2024.

[328] Junsong Zhang, Zisong Chen, Chunyu Lin, Zhijie Shen, Lang Nie, Kang Liao, and Yao Zhao. SGFormer: Spherical geometry transformer for 360 depth estimation. *IEEE TCSVT*, 2025.

[329] Qianru Zhang, Honggang Wen, Wei Yuan, Crystal Chen, Menglin Yang, Siu-Ming Yiu, and Hongzhi Yin. HMamba: Hyperbolic mamba for sequential recommendation. *ACM TOIS*, 44(5):1–27, 2026.

[330] Sensen Zhang, Xun Liang, Hui Tang, and Zhenyu Guan. Hybrid interaction temporal knowledge graph embedding based on householder transformations. In *ACM MM*, pp. 8954–8962, 2023.

[331] Taolin Zhang, Ruyao Xu, Chengyu Wang, Zhongjie Duan, Cen Chen, Minghui Qiu, Dawei Cheng, Xiaofeng He, and Weining Qian. Learning knowledge-enhanced contextual language representations for domain natural language understanding. In *EMNLP*, pp. 15663–15676, 2023.

[332] Yiding Zhang, Xiao Wang, Chuan Shi, Nian Liu, and Guojie Song. Lorentzian graph convolutional networks. In *The WebConf*, pp. 1249–1261, 2021.

[333] Yifei Zhang, Hao Zhu, Menglin Yang, Jiahong Liu, Rex Ying, Irwin King, and Piotr Koniusz. Understanding and mitigating hyperbolic dimensional collapse in graph contrastive learning. In *KDD*, 2025.

[334] Yihao Zhang, Junlin Zhu, Ruizhen Chen, Weiwen Liao, Yulin Wang, and Wei Zhou. Mixed-curvature knowledge-enhanced graph contrastive learning for recommendation. *Expert Syst. Appl.*, 237:121569, 2024.

[335] Kai Zheng, Shaokai Wang, Yunpei Xu, Qiming Lei, Qichang Zhao, Xiao Liang, Qilong Feng, Yaohang Li, Min Li, Jinhui Xu, and Jianxin Wang. TopoLa: A universal framework to enhance cell representations for single-cell and spatial omics through topology-encoded latent hyperbolic geometry. *bioRxiv*, pp. 2025–07, 2025.

[336] Xiao Zhou, Xiaogang Peng, Hao Wen, Yikai Luo, Keyang Yu, Ping Yang, and Zizhao Wu. Learning weakly supervised audio-visual violence detection in hyperbolic space. *Image Vis. Comput.*, 151:105286, 2024.

[337] Zihan Zhou, Yang Zhou, Tianshi Che, Zeru Zhang, Jiaxiang Ren, Da Yan, Zhe Jiang, Yelong Shen, Ruoming Jin, and Jianfeng Gao. Noise-free dynamic rank-adaptation via Riemannian methods in federated fine-tuning. In *AISTATS*, 2026.

[338] Shichao Zhu, Shirui Pan, Chuan Zhou, Jia Wu, Yanan Cao, and Bin Wang. Graph geometry interaction learning. In *NeurIPS*, volume 33, pp. 7548–7558, 2020.

[339] Yudong Zhu, Di Zhou, Jinghui Xiao, Xin Jiang, Xiao Chen, and Qun Liu. HyperText: Endowing FastText with hyperbolic geometry. In *Findings of EMNLP*, pp. 1166–1171, 2020.

A Technical constructions

This appendix gives the complete constructions accompanying the comparisons in Sections 2.3.3 and 3.8.2. We use the signed-curvature notation and geometric maps defined in Section 2.2.

A.1 Hyperbolic batch normalization

Here $c < 0$, $\boldsymbol{o}_c = (1/\sqrt{-c}, 0, \ldots, 0)^\top$ is the Lorentz basepoint, and $\lambda_x^c = 2/(1 + c\|x\|^2)$ is the Poincaré conformal factor. The logarithmic, exponential and parallel-transport maps use the same curvature and coordinate conventions as the main text.

Euclidean batch normalization (BN) centers and rescales each channel using mini-batch statistics [119]. Geometric variants replace coordinatewise means with a manifold center and use geodesic distances to measure spread. Bdeir et al. [17] implement Lorentz BN through tangent-space centering, transport, and rescaling. Given a batch $\mathcal{B} = \{x_i\}_{i=1}^m \subset \mathbb{L}_c^n$, let $S_\mathcal{B} = \sum_{i=1}^m \nu_i x_i$ with $\nu_i = 1/m$. The Lorentz batch centroid is

$$\mu_\mathcal{B} = \frac{S_\mathcal{B}}{\sqrt{c\langle S_\mathcal{B}, S_\mathcal{B}\rangle_\mathcal{L}}}, \qquad S_\mathcal{B} = \sum_{i=1}^m \nu_i x_i, \quad \nu_i = \frac{1}{m}. \tag{A.1}$$

The denominator is real because positive-weight sums of upper-sheet Lorentz points are timelike. The squared-distance spread around this centroid is [136]

$$\sigma_\mathcal{B}^2 = \frac{1}{m}\sum_{i=1}^m \left(d_\mathbb{L}^c(x_i, \mu_\mathcal{B})\right)^2. \tag{A.2}$$

The layer maps each feature to the tangent space at $\mu_\mathcal{B}$, transports it through $\boldsymbol{o}_c$, rescales it, and maps it to a learned center β:

$$\mathrm{BN}_\mathbb{L}(x) = \exp_\beta^c\left(\mathrm{PT}_{\boldsymbol{o}_c\to\beta}^c\left(\gamma \cdot \frac{\mathrm{PT}_{\mu_\mathcal{B}\to\boldsymbol{o}_c}^c\left(\log_{\mu_\mathcal{B}}^c(x)\right)}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}}\right)\right), \tag{A.3}$$

where $\gamma \in \mathbb{R}$ and $\beta \in \mathbb{L}_c^n$ are trainable parameters, and $\epsilon > 0$ stabilizes the denominator. Skliar & Weiler [249] define batch normalization on the Poincaré ball $\mathbb{P}_c^n$. To compute the batch mean $\mu_\mathcal{B}$, a closed-form gyromidpoint surrogate for the Fréchet mean is used for efficiency:

$$\mu_\mathcal{B} = \frac{1}{2} \otimes_c \left(\frac{\sum_{i=1}^m 2\gamma_{x_i}^2 x_i}{\sum_{i=1}^m (2\gamma_{x_i}^2 - 1)}\right), \tag{A.4}$$

where $x_i \in \mathbb{P}_c^n$ and $\gamma_{x_i} = 1/\sqrt{1 + c\|x_i\|^2}$. This closed-form center provides an efficient alternative to minimizing the sum of squared geodesic distances, which defines the Fréchet mean. Its corresponding squared-distance spread is

$$\sigma_\mathcal{B}^2 = \frac{1}{m}\sum_{i=1}^m \left(d_\mathbb{P}^c(x_i, \mu_\mathcal{B})\right)^2. \tag{A.5}$$

The normalization process then proceeds in two steps. First, hyperbolic features are centered and rescaled using Möbius operations:

$$\hat{x}_i = \frac{1}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \otimes_c \left((-\mu_\mathcal{B}) \oplus_c x_i\right). \tag{A.6}$$

Second, the features undergo an affine transformation:

$$\mathrm{BN}_{\mathbb{P}}(x_i) = \beta \oplus_c (\gamma \otimes_c \hat{x}_i), \tag{A.7}$$

where $\gamma \in \mathbb{R}$, $\beta \in \mathbb{P}_c^n$ are trainable parameters. Lou et al. [181] introduce an iterative approximation of the Fréchet mean on $\mathbb{P}_c^n$. Van Spengler et al. [272] use the Poincaré midpoint [270], written for N inputs as

$$\mu = \frac{1}{2} \otimes_c \frac{\sum_{i=1}^N \lambda_{\boldsymbol{x_i}}^c \boldsymbol{x_i}}{\sum_{i=1}^N (\lambda_{\boldsymbol{x_i}}^c - 1)}. \tag{A.8}$$

Since $\lambda_{\boldsymbol{x_i}}^c = 2\gamma_{x_i}^2$, this is the same equal-weight gyromidpoint formula above. The main choice is therefore between an iterative Fréchet mean and a closed-form midpoint surrogate.

A.2 MoSLoRA coordinate construction

MoSLoRA [325] uses the following coordinate construction for its geometric experts.

The core Mixture of Space (MoS) scheme unifies the manifold constraints by operating in an $(n+1)$-dimensional ambient space $\mathbb{R}^{n+1}$ with coordinates $(\xi; s^\top)^\top$. Under the signed-curvature convention, the upper hyperboloid sheet and the nonnegative-first-coordinate hemisphere can be constructed by completing a spatial vector s as

$$G_c(s) = \begin{cases} \left[\sqrt{\|s\|_2^2 - 1/c},\, s^T\right]^T, & c < 0, \\ \left[\sqrt{1/c - \|s\|_2^2},\, s^T\right]^T, & c > 0,\ \|s\|_2^2 \le 1/c, \\ \left[0,\, s^T\right]^T, & c = 0. \end{cases} \tag{A.9}$$

MoSLoRA uses stereographic projection to avoid repeated exp / log maps. In the projection formulas below, x denotes the scaled projected coordinate and $(\xi; s)$ its ambient image. Before the inverse projection, the method scales the input features; it rescales the projected outputs afterward. For $c < 0$, the scaled coordinate x must satisfy $\|x\|_2^2 < -1/c$, which makes the denominator positive and selects the upper Lorentz sheet. The inverse projection is:

$$\rho_c^{-1}(x) = \begin{cases} \left(\dfrac{1}{\sqrt{|c|}} \dfrac{1 - c\|x\|_2^2}{1 + c\|x\|_2^2},\ \dfrac{2x}{1 + c\|x\|_2^2}\right)^\top, & c \neq 0, \\ \left[0, x^T\right]^T, & c = 0, \end{cases} = \begin{bmatrix} \xi \\ s \end{bmatrix}, \tag{A.10}$$

The $c = 0$ branch above is a separately defined Euclidean embedding rather than the limit of the nonzero-curvature ambient coordinates, whose radius diverges as $c \to 0$. The method applies model transformations in the ambient space and projects back by

$$\rho_c\left(\begin{bmatrix} \xi' \\ s' \end{bmatrix}\right) = \begin{cases} \dfrac{s'}{1 + \sqrt{|c|}\,\xi'}, & c \neq 0, \\ s', & c = 0. \end{cases} \tag{A.11}$$

For $c \neq 0$, the output projection is evaluated only where $1 + \sqrt{|c|}\xi' \neq 0$; at $c = 0$ it returns the spatial coordinates directly. The main text uses this projection in the expert-fusion rule in Section 3.8.2.